NPTEL Deep Learning (IIT Ropar): Complete Exam Notes

This file contains the full set of notes for all 15 lectures (CS7015 slides by Mitesh M. Khapra), written for an MCQ exam that tests concepts, theory and numericals. It has been audited against the source PDFs: every concept, formula and worked example was checked, and every numerical answer was recomputed.

Contents:

- **Part I:** Lecture Notes
- **Part II:** High-Yield Revision
- **Part III:** Formula Sheet
- **Part IV:** Practice MCQs
- **Appendix:** Slide Errors

Week → lecture map

Week	Lecture(s)	Topic
1	L1, L2	History of DL; MP neuron, perceptron, PLA and its convergence, MLPs, representation power
2	L3	Sigmoid neurons, gradient descent, FFN representation power (UAT)
3	L4	Feedforward networks, backpropagation
4	L5	GD, momentum, NAG, SGD and mini-batch, AdaGrad, RMSProp, Adam
5	L6	Eigenvalues and eigenvectors, EVD, PCA, SVD *(assumed: the week map you gave skips Week 5)*
6	L7	Autoencoders (relation to PCA; denoising, sparse and contractive)
7	L8	Regularization: bias–variance, L2, augmentation, noise, early stopping, ensembles, dropout
8	L9	Greedy pre-training, activations, initialization, batch norm
9	L10	Word representations: SVD, CBOW, skip-gram, negative sampling, GloVe
10	L11, L12	CNNs and architectures; visualizing CNNs, guided backprop, Deep Dream, Deep Art, fooling
11	L13, L14	RNNs, BPTT, vanishing and exploding gradients; LSTM, GRU
12	L15	Encoder–decoder models, attention

How each lecture is structured

Each lecture follows its slide modules. Within a module, the subsections run A to G:

Subsection	Contents
A	Core concept
B	Terminology
C	Properties and insights
D	Formulas
E	Numerical problem-solving
F	MCQ traps
G	Quick revision summary

Labels used in the notes:

- **"(Additional clarification)"** marks standard deep-learning knowledge that is *not* on the slides and was added to explain a point.
- ⚠️ **Slide note** marks a typo, inconsistency or ambiguity found on a slide. These are flagged, not silently corrected.

Table of Contents

- Part I · Lecture Notes (Weeks 1–12)
- Week 1 · Lecture 1: (Partial/Brief) History of Deep Learning
  - 1.1 Biological neurons (Chapter 1)
  - 1.2 From spring to winter of AI (Chapter 2)
  - 1.3 The deep revival (Chapter 3)
  - 1.4 From cats to CNNs (Chapter 4)
  - 1.5 Faster, higher, stronger: better optimization (Chapter 5)
  - 1.6 The curious case of sequences (Chapter 6)
  - 1.7 Beating humans at their own game (Chapter 7)
  - 1.8 The madness (2013–) (Chapter 8)
  - 1.9 (Need for) sanity (Chapter 9)
- Week 1 · Lecture 2: McCulloch–Pitts Neuron, Perceptron, PLA, Convergence, MLPs
  - Module 2.1: Biological neurons
  - Module 2.2: McCulloch–Pitts (MP) neuron
  - Module 2.3: Perceptron
  - Module 2.4: Errors and error surfaces
  - Module 2.5: Perceptron Learning Algorithm (PLA)
  - Module 2.6: Proof of convergence
  - Module 2.7: Linearly separable boolean functions
  - Module 2.8: Representation power of a network of perceptrons
- Week 2 · Lecture 3: Sigmoid Neurons, Gradient Descent, Feedforward Networks
  - Module 3.1: Sigmoid neuron
  - Module 3.2: A typical supervised machine learning set-up
  - Module 3.3: Learning parameters by (infeasible) guess work
  - Module 3.4: Learning parameters with gradient descent
  - Module 3.5: Representation power of a multilayer network of sigmoid neurons
- Week 3 · Lecture 4: Feedforward Neural Networks, Backpropagation
  - Module 4.1: Feedforward Neural Networks (multilayered network of neurons)
  - Module 4.2: Learning Parameters of FFNNs (Intuition)
  - Module 4.3: Output Functions and Loss Functions
  - Module 4.4: Backpropagation (Intuition)
  - Module 4.5: Gradients w.r.t. Output Units
  - Module 4.6: Gradients w.r.t. Hidden Units
  - Module 4.7: Gradients w.r.t. Parameters
  - Module 4.8: Backpropagation — Pseudocode
  - Module 4.9: Derivatives of Activation Functions
  - Lecture 4 — G. Quick Revision Summary
- Week 4 · Lecture 5: Gradient Descent and Its Variants (Momentum, NAG, SGD, AdaGrad, RMSProp, Adam)
  - Module 5.1: Learning Parameters — Guess Work (Infeasible)
  - Module 5.2: Gradient Descent (recap — derivation is Lecture 3)

    - Module 5.3: Contours
    - Module 5.4: Momentum-based Gradient Descent
    - Module 5.5: Nesterov Accelerated Gradient Descent (NAG)
    - Module 5.6: Stochastic and Mini-Batch Gradient Descent
    - Module 5.7: Tips for Adjusting Learning Rate and Momentum
    - Module 5.8: Line Search
    - Module 5.9: Gradient Descent with Adaptive Learning Rate
    - Lecture 5 — G. Quick Revision Summary
- Week 5 · Lecture 6: Eigenvalues, Eigenvectors, EVD, PCA, SVD
    - 6.1 Eigenvalues and Eigenvectors
    - 6.2 Linear Algebra: Basic Definitions
    - 6.3 Eigenvalue Decomposition (EVD)
    - 6.4 PCA: Interpretation 1 (decorrelate the dimensions)
    - 6.5 PCA: Interpretation 2 (minimise reconstruction error)
    - 6.6 PCA: Interpretation 3 (maximise variance)
    - 6.7 PCA: Practical Example (Eigenfaces)
    - 6.8 Singular Value Decomposition (SVD)
- Week 6 · Lecture 7: Autoencoders
    - 7.1 Introduction to Autoencoders
    - 7.2 Link between PCA and Autoencoders
    - 7.3 Regularization in Autoencoders
    - 7.4 Denoising Autoencoders (DAE)
    - 7.5 Sparse Autoencoders
    - 7.6 Contractive Autoencoders (CAE)
    - 7.7 Summary and Comparison Table
- Week 7 · Lecture 8: Regularization
    - 8.1 Bias and Variance
    - 8.2 Train error vs Test error
    - 8.2 (cont.) True error vs empirical error · 8.3 True error and Model complexity
    - 8.4 l_2 regularization (weight decay)
    - 8.5 Dataset augmentation
    - 8.6 Parameter sharing and tying
    - 8.7 Adding noise to the inputs
    - 8.8 Adding noise to the outputs (soft targets / label smoothing)
    - 8.9 Early stopping
    - 8.10 Ensemble methods
    - 8.11 Dropout
    - G. Quick Revision Summary — Lecture 8
- Week 8 · Lecture 9: Greedy Layerwise Pre-training, Better Activations, Better Initialization, Batch Normalization
    - 9.1 Recap: training deep neural networks
    - 9.2 Unsupervised (greedy layerwise) pre-training
    - 9.3 Better activation functions
    - 9.4 Better initialization strategies
    - 9.5 Batch Normalization
    - G. Quick Revision Summary — Lecture 9
- Week 9 · Lecture 10: Learning Vectorial Representations of Words

    - 10.1 One-hot representations
    - 10.2 Distributed representations: co-occurrence matrix, PMI, PPMI
    - 10.3 SVD for learning word representations
    - 10.4 Continuous bag of words (CBOW)
    - 10.5 Skip-gram model (and negative sampling)
    - 10.6 Contrastive estimation
    - 10.7 Hierarchical softmax
    - 10.8 GloVe representations
    - 10.9 Evaluating word representations
    - 10.10 Relation between SVD and word2vec
    - G. Lecture 10 – Quick Revision Summary
- Week 10 · Lecture 11: Convolutional Neural Networks, LeNet, AlexNet, ZF-Net, VGGNet, GoogLeNet, ResNet
    - 11.1 The convolution operation
    - 11.2 Relation between input size, output size and filter size
    - 11.3 Convolutional Neural Networks
    - 11.4 CNNs: success stories on ImageNet (AlexNet, ZF-Net, VGGNet)
    - 11.5 GoogLeNet and ResNet
    - G. Lecture 11 – Quick Revision Summary
- Week 10 · Lecture 12 — Visualizing CNNs, Guided Backprop, Deep Dream, Deep Art, Fooling CNNs
    - 12.0 Setup: the CNN used in the slides & the receptive field of a neuron
    - 12.1 Visualizing patches which maximally activate a neuron
    - 12.2 Visualizing filters of a CNN
    - 12.3 Occlusion experiments
    - 12.4 Finding influence of input pixels using backpropagation
    - 12.5 Guided backpropagation (Springenberg et al.)
    - 12.6 Optimization over images
    - 12.7 Creating images from embeddings
    - 12.8 Deep Dream
    - 12.9 Deep Art (Neural style transfer)
    - 12.10 Fooling deep convolutional neural networks
    - L12 — Comparison of visualization / input-optimization techniques
    - G. Quick Revision Summary — Lecture 12
- Week 11 · Lecture 13 — Sequence Learning Problems, RNNs, BPTT, Vanishing/Exploding Gradients
    - 13.1 Sequence learning problems
    - 13.2 Recurrent Neural Networks
    - 13.3 Backpropagation through time (BPTT)
    - 13.4 The problem of exploding and vanishing gradients
    - 13.5 Some gory details
    - G. Quick Revision Summary — Lecture 13
- Week 11 · Lecture 14: Long Short-Term Memory Cells (LSTMs), Gated Recurrent Units (GRUs)
    - 14.1 Selective Read, Selective Write, Selective Forget: The Whiteboard Analogy
    - 14.2 LSTM and GRU
    - 14.3 How LSTMs Avoid the Problem of Vanishing Gradients
- Week 12 · Lecture 15: Encoder–Decoder Models, Attention Mechanism
    - 15.1 Introduction to Encoder–Decoder Models
    - 15.2 Applications of Encoder–Decoder Models

    - 15.3 Attention Mechanism
    - Interpreting attention (heat maps)
    - 15.4 Attention over Images
    - 15.5 Hierarchical Attention
- Part II · High-Yield Revision
    - 1. Week 1–2: MP neuron → Perceptron → Sigmoid neuron
    - 2. Week 3: FFN and Backpropagation
    - 3. Week 4: Optimizers (comparison)
    - 4. Week 5: Linear algebra, PCA, SVD
    - 5. Week 6: Autoencoders
    - 6. Week 7: Regularization (comparison)
    - 7. Week 8: Training deep nets better
    - 8. Week 9: Word representations
    - 9. Week 10: CNNs and visualization
    - 10. Week 11: RNN, BPTT, LSTM, GRU
    - 11. Week 12: Encoder–decoder and attention
    - Top cross-course MCQ traps
- Part III · Formula Sheet
    - Quick formula finder: question cue → formula
    - Weeks 1–2 · Lectures 1–3
    - Weeks 3–4 · Lectures 4–5
    - Weeks 5–6 · Lectures 6–7
    - Weeks 7–8 · Lectures 8–9
    - Weeks 9–10 · Lectures 10–11
    - Weeks 10–11 · Lectures 12–13
    - Weeks 11–12 · Lectures 14–15
- Part IV · Practice MCQs
    - Weeks 1–2 · Lectures 1–3
    - Weeks 3–4 · Lectures 4–5
    - Weeks 5–6 · Lectures 6–7
    - Weeks 7–8 · Lectures 8–9
    - Weeks 9–10 · Lectures 10–11
    - Weeks 10–11 · Lectures 12–13
    - Weeks 11–12 · Lectures 14–15
- Appendix · Slide Errors and Ambiguities

Part I · Lecture Notes (Weeks 1–12)

Each lecture follows its slide modules, and each module is organised into sections A–G (see the legend above).

Overview: Lectures 1–3: History, MP Neurons and Perceptrons, Sigmoid Neurons and Gradient Descent

Lectures covered / modules:

- **Lecture 1: (Partial/Brief) History of Deep Learning.** Ch.1 Biological neurons · Ch.2 From spring to winter of AI · Ch.3 The deep revival · Ch.4 From cats to CNNs · Ch.5 Faster, higher, stronger (optimizers) · Ch.6 The curious case of sequences · Ch.7 Beating humans at their own game · Ch.8 The madness (2013–) · Ch.9 (Need for) sanity
- **Lecture 2: MP Neuron, Perceptron, PLA, Convergence, MLP.** 2.1 Biological neurons · 2.2 McCulloch–Pitts neuron · 2.3 Perceptron · 2.4 Errors and error surfaces · 2.5 Perceptron learning algorithm · 2.6 Proof of convergence · 2.7 Linearly separable boolean functions · 2.8 Representation power of a network of perceptrons
- **Lecture 3: Sigmoid Neurons, Gradient Descent, Feedforward Networks.** 3.1 Sigmoid neuron · 3.2 A typical supervised ML setup · 3.3 Learning parameters: (infeasible) guess work · 3.4 Learning parameters: gradient descent · 3.5 Representation power of a multilayer network of sigmoid neurons (UAT, tower functions)

Week 1 · Lecture 1: (Partial/Brief) History of Deep Learning

Based mostly on Schmidhuber's article "*Deep Learning in Neural Networks: An Overview*". The facts to know are names, years and contributions.

1.1 Biological neurons (Chapter 1)

Year	Who	What
1871–1873	**Joseph von Gerlach**	**Reticular theory**: the nervous system is a *single continuous network*, not many discrete cells
1871–1873	**Camillo Golgi**	Developed a chemical **staining technique** for examining nervous tissue in detail. He **supported** the reticular theory
1888–1891	**Santiago Ramón y Cajal**	Used Golgi's technique and proposed the **Neuron Doctrine**: the nervous system is made of *discrete individual cells* that form a network
~1891	**Heinrich Wilhelm Gottfried von Waldeyer-Hartz**	**Coined the term "neuron"** and consolidated the Neuron Doctrine
1906	Golgi and Cajal	**Shared** the Nobel Prize in Physiology or Medicine, although their views conflicted (reticular theory vs neuron doctrine)
1950s	Electron microscopy	**Confirmed the neuron doctrine**: nerve cells are individual cells connected through **synapses**

1.2 From spring to winter of AI (Chapter 2)

Year	Milestone	Key fact
1847	**Gradient descent**	**Cauchy**, who needed it to compute the orbits of heavenly bodies
1943	**MP neuron**	**McCulloch** (neuroscientist) and **Pitts** (logician): a highly simplified model of the neuron
1957–1958	**Perceptron**	**Frank Rosenblatt**: "may eventually be able to learn, make decisions, and translate languages". The NYT called it "the embryo of an electronic computer that the Navy expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence"
1965–1968	**First-generation MLPs**	**Ivakhnenko** et al.
1969	**Limitations of perceptrons**	**Minsky and Papert**, book "*Perceptrons*"
1969–1986	**AI winter of connectionism**	Connectionist AI was almost abandoned
1960s–70s, **1982**, **1986**	**Backpropagation**	Discovered and rediscovered several times in the 1960s–70s. **Werbos (1982)** first used it for ANNs. **Rumelhart et al. (1986)** popularized it
1989	**Universal Approximation Theorem**	A multilayered network with a **single hidden layer** can approximate **any continuous function** to any desired precision (Cybenko 1989; Hornik 1991, cited in L3)

1.3 The deep revival (Chapter 3)

Year	Milestone	Key fact
1991–1993	**"Very Deep Learner"** (J. Schmidhuber)	Unsupervised pre-training first used here, well before 2006
2006	**Unsupervised pre-training**	**Hinton and Salakhutdinov**: an effective way to **initialize weights** so that deep **autoencoders** learn low-dimensional representations
2007–2009	Further work	Studies of *why* unsupervised pre-training is effective
2009	**Handwriting**	**Graves et al.** beat all entries in an international **Arabic handwriting** recognition competition
2010	**Speech**	**Dahl et al.**: relative error reductions of **16.0% and 23.2%** over a state-of-the-art system
2010	**MNIST record**	**Ciresan et al.**: plain backpropagation on **GPUs**. This marks GPUs entering the field
2011	**First superhuman visual pattern recognition**	**D. C. Ciresan et al.**: **0.56% error** in the **IJCNN Traffic Sign Recognition** competition
2012–2016	**Success on ImageNet**	See the table below

ImageNet winners (error, number of layers). The error falls as depth grows:

Network	Error	Layers
AlexNet (2012)	16.0%	8
ZFNet	11.2%	8
VGGNet	7.3%	19
GoogLeNet	6.7%	22
MS ResNet	3.6%	**152**

1.4 From cats to CNNs (Chapter 4)

Year	Milestone	Key fact
1959	**Hubel and Wiesel** (experiments on cats)	Each neuron has a **fixed receptive field**: it fires only for stimuli in a specific region of visual space
1980	**Neocognitron** (Fukushima)	Handwritten character and pattern recognition
1989	**CNN** (LeCun et al.)	Handwritten digit recognition using **backprop over a CNN**
1998	**LeNet-5** (LeCun et al.)	Introduced the **MNIST** dataset

The lecture's quip: "*An algorithm inspired by an experiment on cats is today used to detect cats in videos.*"

1.5 Faster, higher, stronger: better optimization (Chapter 5)

Better optimizers give faster convergence and better accuracy: **Nesterov (1983)** → **Adagrad (2011)** → **RMSProp (2012)** → **Adam (2015)** (and **BatchNorm, 2015**) → **Eve (2016)** → **"Beyond Adam" (2018)**. These are covered in L5.

1.6 The curious case of sequences (Chapter 6)

Sequences (time series, speech, music, text, video) need models that capture how units in the sequence interact.

Year	Model / event	Key fact
1982	**Hopfield network**	Content-addressable memory that stores and retrieves patterns
1986	**Jordan network**	The **output** state of each time step is fed to the next time step
1990	**Elman network**	The **hidden** state of each time step is fed to the next time step
1991–1994	**RNN drawbacks**	**Hochreiter et al.** and **Bengio et al.**: RNNs are hard to train because of **exploding and vanishing gradients**
1997	**LSTMs**	Solved complex **long time-lag** tasks that no earlier model could
2014	**Seq2Seq and Attention**	First large-scale seq2seq successes with RNNs/LSTMs. **Attention** was introduced here
1991	**RL for attention**	**Schmidhuber and Huber**: RNNs that use reinforcement learning to decide *where to look*

⚠ **Jordan vs Elman:** Jordan feeds back the **output**; Elman feeds back the **hidden state**.

1.7 Beating humans at their own game (Chapter 7)

Year	Event
2015	**DQNs**: human-level control on **Atari games** using deep RL
2015	**AlphaGo** / **AlphaGo Zero**: best Go player ever. Go is harder than chess because of the number of possible moves, and there is no brute-force backtracking as in earlier chess agents
2016	**Poker (DeepStack)**: defeated 11 professional poker players, with only one outside the margin of statistical significance
2017	**Dota 2** (Defense of the Ancients): the bot was undefeated against many top professionals (complex strategies, large visual space)

1.8 The madness (2013–) (Chapter 8)

Deep learning spread to: language modeling (Mikolov 2010), speech recognition (Hinton 2012, Graves 2013), machine translation (Sutskever 2014, Bahdanau 2015, Cho 2014), conversation modeling, QA, object detection (Faster R-CNN 2015, YOLO9000 2016, Mask R-CNN 2017), visual tracking, image/video captioning, visual and video QA, video summarization, and generation. Generative milestones to know: **VAE (Kingma, 2013)**, **GAN (Goodfellow, 2014)**, **Progressive GANs (Karras, 2017)**, **WaveNet (Oord, 2016)** for raw audio, and **PixelRNN (Oord, 2016)**.

1.9 (Need for) sanity (Chapter 9)

Paradox of deep learning: deep learning works well despite

- **high capacity**, which makes it susceptible to overfitting
- **numerical instability** (vanishing and exploding gradients)
- **sharp minima**, which lead to overfitting
- **non-robustness**: the slide's adversarial example turns "panda" (57.7% confidence) into "gibbon" (99.3% confidence) after adding imperceptible noise

There are no clear answers yet, but there is a growing emphasis on **explainability and theoretical justification**.

G. Quick revision: Lecture 1

- 1847 Cauchy GD → 1943 MP → 1958 Rosenblatt perceptron → 1969 Minsky–Papert → AI winter 1969–86 → 1986 backprop (Rumelhart) → 1989 UAT → 2006 Hinton–Salakhutdinov pre-training → 2012 AlexNet.
- Pre-training dates back to Schmidhuber (1991–93), not 2006. Backprop was first used for ANNs by Werbos in 1982 and popularized in 1986.
- Hubel–Wiesel 1959 (receptive field) → Neocognitron 1980 → LeCun CNN 1989 → LeNet-5/MNIST 1998.
- Hopfield 1982, Jordan 1986 (output fed back), Elman 1990 (hidden state fed back), LSTM 1997, attention 2014.
- ResNet: 152 layers, 3.6% error. AlexNet: 8 layers, 16.0% error.

Week 1 · Lecture 2: McCulloch–Pitts Neuron, Perceptron, PLA, Convergence, MLPs

Module 2.1: Biological neurons

A. Core concept

The **artificial neuron** is the basic unit of a deep network. It is inspired by the biological neuron (neural cell, or neural processing unit).

B. Terminology

Part	Role
Dendrite	Receives signals from other neurons
Synapse	Point of connection to other neurons
Soma	Processes the information
Axon	Transmits this neuron's output

C. Insights

- The brain has about $\mathbf{10^{11}}$ **(100 billion) neurons**. It is a **massively parallel interconnected network** with a **division of work**: each neuron has a role or responds to a certain stimulus.
- **Cartoon of a single neuron:** sense organs interact with the world and relay information to neurons, which (may) get activated and produce a response (the slide's example: laughter). In reality the sense organs feed the **lowest layer** of a network; some neurons fire and relay to the neurons they connect to, and so on until the response.
- Neurons are arranged in a **hierarchy**. In the visual cortex, information goes from the retina through **V1** → **V2** → **...** → **AIT**, from simple visual forms (edges, corners) through intermediate forms (feature groups) up to high-level objects (faces, objects). This is the motivation for layered and deep networks.
- **Sample illustration of hierarchical processing** (idea from Hugo Larochelle): **Layer 1** detects edges and corners → **Layer 2** forms feature groups (nose, mouth, eyes) → **Layer 3** detects high-level objects such as faces.
- The instructor's disclaimer: this is an *overly simplified* picture of the brain, but it is enough for this course.

Module 2.2: McCulloch–Pitts (MP) neuron

A. Core concept

Proposed in **1943** by McCulloch (neuroscientist) and Pitts (logician). It is a highly simplified computational model of a neuron:

- **Inputs** $x_i \in \{0, 1\}$ and **output** $y \in \{0, 1\}$ are boolean.
- g **aggregates** the inputs and f **makes a decision** based on that aggregate.
- Inputs are either **excitatory** or **inhibitory**.

D. Formulas

$$g(x_1, \ldots, x_n) = g(\mathbf{x}) = \sum_{i=1}^{n} x_i, \qquad y = f(g(\mathbf{x})) = \begin{cases} 1 & g(\mathbf{x}) \geq \theta \\ 0 & g(\mathbf{x}) < \theta \end{cases}$$

- **Inhibitory rule:** $y = 0$ if any inhibitory input is 1. This overrides everything else.
- **θ** is the **thresholding parameter**, and the scheme is called **thresholding logic**.
- There are **no weights**: every excitatory input counts equally (weight 1). θ is **hand-coded**, not learned.
- **Common mistake:** treating inhibitory inputs as "weight −1". They are not. Any active inhibitory input forces $y = 0$.

E. Boolean functions with an MP unit (slide figures)

Function	Inputs	θ	Notes
AND (3 inputs)	x_1, x_2, x_3	**3**	Fires only if all inputs are 1
OR (3 inputs)	x_1, x_2, x_3	**1**	Fires if at least one input is 1
x_1 AND $\neg x_2$	x_1 excitatory, x_2 inhibitory	**1**	$x_2 = 1$ forces 0
NOR (2 inputs)	both **inhibitory**	**0**	Fires only when both are 0
NOT	x_1 **inhibitory**	**0**	Sum $0 \geq 0$ fires when $x_1 = 0$
AND (2 inputs)	x_1, x_2	**2**	
Tautology (always ON)	x_1, x_2	**0**	

Shortcut: with n excitatory inputs, θ = k means "fire when at least k inputs are 1". So θ = 1 is OR, θ = n is AND, and θ = 0 is always ON.

C. Geometric interpretation

- One MP neuron **splits the input points into two halves** using the line (or plane, or hyperplane) $\sum_i x_i - \theta = 0$. Points with output 1 lie on or above it ($\sum x_i \geq \theta$). Points with output 0 lie below it.
- 2 inputs: 4 points, separated by a **line**. 3 inputs: 8 points, separated by a **plane**. For OR with 3 inputs, the plane $x_1 + x_2 + x_3 = 1$ puts (0,0,0) on one side and the other 7 points on the other.
- **Result:** a single MP neuron can represent **only linearly separable** boolean functions.
- **Linear separability (for boolean functions):** there is a line (or plane) such that all inputs that produce 1 lie on one side and all inputs that produce 0 lie on the other.
- ⚠ With an MP neuron the separating line always has the form $x_1 + x_2 + \cdots = \theta$, with all coefficients equal to 1. Its orientation is fixed and only its offset (θ) can change. Inhibitory inputs add some extra flexibility.

Module 2.3: Perceptron

A. Core concept

- Proposed by **Frank Rosenblatt (1958)**, an American psychologist. It is a more general model than MP neurons.
- **Main differences from MP:** inputs have **numerical weights**, and there is a **mechanism for learning those weights**. Inputs are **no longer limited to booleans** and can be real-valued.
- The model used in this course is the one refined and analyzed by **Minsky and Papert (1969)**.

D. Formulas

$$y = \begin{cases} 1 & \sum_{i=1}^{n} w_i x_i \geq \theta \\ 0 & \sum_{i=1}^{n} w_i x_i < \theta \end{cases} \iff y = \begin{cases} 1 & \sum_{i=0}^{n} w_i x_i \geq 0 \\ 0 & \sum_{i=0}^{n} w_i x_i < 0 \end{cases}, \quad x_0 = 1,\ w_0 = -\theta$$

- **Purpose:** absorbing the threshold into an extra input $x_0 = 1$ with weight $w_0 = -\theta$ means the threshold can be **learned like any other weight**.
- w_0 **is called the bias** because it represents the **prior (prejudice)**.
- **Interpretation (movie example):** inputs are isActorDamon, isGenreThriller and isDirectorNolan. A **movie buff** has a very low threshold (**θ = 0**) and watches anything. A **selective viewer** watches only thrillers starring Damon and directed by Nolan (**θ = 3**). A high weight on isDirectorNolan lets that feature alone cross the threshold.
- Weights $(w_1..w_n)$ and the bias w_0 are learned **from data** (the viewer's history).
- **Common mistakes:** sign confusion. **θ = −w₀**, and "fires if $\sum w_i x_i \geq -w_0$". The comparison is **≥ 0 → 1** and **< 0 → 0**, so a point exactly on the boundary is classified as **1**.

C. Properties

- A perceptron **also separates the input space into two halves**, so a single perceptron can only implement **linearly separable** functions, just like an MP neuron.
- **What actually differs from MP:** weights, including the threshold, **can be learned**, and inputs **can be real-valued**.
- ⚠ **Slide note:** the comparison slide writes the MP rule (assuming no inhibitory inputs) as $y = 1$ iff $\sum_{i=0}^{n} x_i \geq 0$. Starting the sum at $i = 0$ only makes sense if the threshold is absorbed as an extra term $x_0 \cdot w_0$ with $w_0 = -\theta$; read it as $\sum_{i=1}^{n} x_i - \theta \geq 0$. The perceptron row, $\sum_{i=0}^{n} w_i x_i \geq 0$, differs only by the weights w_i.
- **Example: OR with a perceptron.** The truth table gives four inequalities: $w_0 < 0$, $w_2 \geq -w_0$, $w_1 \geq -w_0$, $w_1 + w_2 \geq -w_0$. One solution is $w_0 = -1, w_1 = 1.1, w_2 = 1.1$ (many others exist). A similar system can be solved for an MP neuron's θ.

F. MP neuron vs perceptron

Aspect	MP neuron (1943)	Perceptron (1958, Minsky–Papert 1969)
Inputs	Boolean {0,1}	Real-valued allowed
Weights	None (all equal); excitatory or inhibitory	Real weights w_i
Threshold	Hand-coded θ	Learned (bias $w_0 = -\theta$)
Learning	None	Perceptron Learning Algorithm
Can represent	Linearly separable functions only	Linearly separable functions only
Inhibitory input	Absolute veto	No such concept; negative weights instead

Module 2.4: Errors and error surfaces

C. Insights (OR function, fixed threshold $-w_0 = 1$, i.e. $w_0 = -1$)

The slide's table, verified by computation:

w_1	w_2	Errors (out of 4)
−1	−1	**3**

w_1	w_2	**Errors (out of 4)**
1.5	0	**1** (misclassifies (0,1))
0.45	0.45	**3** (only (0,0) is correct)
1.1	1.1	0

- An **error** occurs if $(x_1, x_2) = (0, 0)$ but $w_0 + w_1 x_1 + w_2 x_2 \geq 0$, or if $(x_1, x_2) \neq (0, 0)$ but the sum is < 0.
- The **error surface** is the number of errors plotted over (w_1, w_2) with w_0 fixed. The goal is an **algorithm** that finds weights with minimum (zero) error.
- ⚠ **Slide note:** the bullet "*Say $w_1 = -1, w_2 = -1$ … we make an error on 1 out of the 4 inputs*" contradicts the table on the same slide, which gives **3 errors** for (−1,−1). Direct computation also gives 3 errors. The "1 error" fits $(1.5, 0)$.
- ⚠ **Slide note:** one slide writes the quantity as "$-w_0 + w_1 x_1 + w_2 x_2$" while also fixing $w_0 = -1$. The consistent form is $w_0 + w_1 x_1 + w_2 x_2$, i.e. $-1 + w_1 x_1 + w_2 x_2$.

Module 2.5: Perceptron Learning Algorithm (PLA)

A. Core concept

The main use of a perceptron is as a **binary classifier**. In the movie example, each of m movies has n features (some boolean, some real, e.g. imdbRating scaled to [0,1]) and a like/dislike label. **Assume the data is linearly separable.** The perceptron must find the **separating plane** $\mathbf{w}^T \mathbf{x} = 0$.

D. The algorithm

```
P ← inputs with label 1;  N ← inputs with label 0
Initialize w randomly
while !convergence:
    pick random x ∈ P ∪ N
    if x ∈ P and w·x < 0:  w = w + x
    if x ∈ N and w·x ≥ 0:  w = w − x
# converges when all inputs are classified correctly
```

- $\mathbf{w} = [w_0, w_1, \ldots, w_n]$, $\mathbf{x} = [1, x_1, \ldots, x_n]$, $\mathbf{w} \cdot \mathbf{x} = \mathbf{w}^T \mathbf{x} = \sum_{i=0}^{n} w_i x_i$.
- **Updates happen only on mistakes.** No update is made for a correctly classified point. There is **no learning rate**; the update is just $\pm \mathbf{x}$.

C. Geometric intuition (why it works)

- Every point $\mathbf{x}$ on the line $\mathbf{w}^T \mathbf{x} = 0$ makes an angle of **90°** with $\mathbf{w}$, because $\cos \alpha = \frac{\mathbf{w}^T \mathbf{x}}{\|\mathbf{w}\| \|\mathbf{x}\|} = 0$. So $\mathbf{w}$ **is perpendicular (normal) to the decision boundary**.
- Points in the **positive half-space** ($\mathbf{w}^T \mathbf{x} \geq 0$) make an angle **< 90°** with $\mathbf{w}$. Points in the **negative half-space** make an angle **> 90°**.
- **Misclassified positive point** ($\mathbf{x} \in P$, $\mathbf{w}^T \mathbf{x} < 0$, so α > 90°): the update $\mathbf{w}_{new} = \mathbf{w} + \mathbf{x}$ gives $\mathbf{w}_{new}^T \mathbf{x} = \mathbf{w}^T \mathbf{x} + \mathbf{x}^T \mathbf{x}$, which is larger. The angle **decreases**, which is what we want.
- **Misclassified negative point** ($\mathbf{x} \in N$, $\mathbf{w}^T \mathbf{x} \geq 0$, so α < 90°): the update $\mathbf{w}_{new} = \mathbf{w} - \mathbf{x}$ gives $\mathbf{w}^T \mathbf{x} - \mathbf{x}^T \mathbf{x}$, which is smaller. The angle **increases**.
- ⚠ **Slide note:** the slides write $\cos(\alpha_{new}) \propto \cos \alpha + \mathbf{x}^T \mathbf{x}$. This is loose, because $\|\mathbf{w}_{new}\| \neq \|\mathbf{w}\|$. What holds exactly is that the dot product $\mathbf{w}^T \mathbf{x}$ moves in the right direction by $\|\mathbf{x}\|^2$. A single update does not guarantee that the point becomes correctly classified, or that other points stay correct.

- **Toy run (slides):** with a random initial **w**, all positives are at > 90° and all negatives at < 90°, the exact opposite of what is wanted. After corrections on p1, p2, n1 and p3 (none needed for n3 and n2), a full pass finds no errors and the algorithm has **converged**.

E. Worked numerical (verified with python)

OR with PLA. $\mathbf{x} = [1, x_1, x_2]$, $\mathbf{w}_{init} = [0, 0, 0]$, points visited cyclically in the order (0,0), (0,1), (1,0), (1,1):

Step	Point (label)	$\mathbf{w}^T\mathbf{x}$	Action	New w
1	(0,0) (0)	0 ≥ 0 → wrong	$\mathbf{w} - \mathbf{x}$	[−1,0,0]
2	(0,1) (1)	−1 < 0 → wrong	$\mathbf{w} + \mathbf{x}$	[0,0,1]
–	(1,0), (1,1)	0, 1 ≥ 0 → correct	–	–
3	(0,0) (0)	0 ≥ 0 → wrong	–	[−1,0,1]
4	(1,0) (1)	−1 → wrong	+	[0,1,1]
5	(0,0) (0)	0 → wrong	–	**[−1,1,1]**

After 5 updates the boundary is $-1 + x_1 + x_2 = 0$ and all four points are correct. **Trap:** because "≥ 0" means class 1, $\mathbf{w} = 0$ classifies everything as 1. A negative point therefore always triggers an update when $\mathbf{w} = 0$.

Module 2.6: Proof of convergence

D. Theorem (as on the slides)

- **Definition (absolutely linearly separable):** two sets P and N in n-dimensional space are absolutely linearly separable if $n + 1$ reals $w_0, \ldots, w_n$ exist such that every point in P satisfies $\sum_{i=1}^{n} w_i x_i > w_0$ and every point in N satisfies $\sum_{i=1}^{n} w_i x_i < w_0$. Both inequalities are **strict**.
- **Proposition:** if P and N are **finite** and **linearly separable**, the PLA updates the weight vector $\mathbf{w}_t$ **a finite number of times**. Equivalently, if the vectors in P and N are tested **cyclically**, a separating $\mathbf{w}_t$ is found after a **finite number of steps** t.

C. Proof set-up and key steps (results only)

1. **Negate the negatives.** If $\mathbf{x} \in N$, then $\mathbf{w}^T\mathbf{x} < 0 \iff \mathbf{w}^T(-\mathbf{x}) > 0$. Define $P' = P \cup N^-$, where N^- holds the negations of the points in N. Now the only goal is $\mathbf{w}^T\mathbf{p} \geq 0$ for all $\mathbf{p} \in P'$, and only **one update rule** is needed: if $\mathbf{w} \cdot \mathbf{p} < 0$, set $\mathbf{w} = \mathbf{w} + \mathbf{p}$.
2. **Normalize** every $\mathbf{p}$ so that $\|\mathbf{p}\| = 1$. This does not change the solution, since the sign of $\mathbf{w}^T\mathbf{p}$ is unchanged by positive scaling.
3. Let $\mathbf{w}^*$ be a **normalized solution vector**. It exists because the data is linearly separable, but its value is unknown.
4. Let $\delta = \min_i \mathbf{w}^* \cdot \mathbf{p}_i$. This is > 0 because of strict separability.
5. Let β be the angle between $\mathbf{w}^*$ and $\mathbf{w}_{t+1}$, so $\cos\beta = \dfrac{\mathbf{w}^* \cdot \mathbf{w}_{t+1}}{\|\mathbf{w}_{t+1}\|}$ (using $\|\mathbf{w}^*\| = 1$).
6. **Numerator:** each correction adds at least δ, so after k corrections (with $k \leq t$), $\mathbf{w}^* \cdot \mathbf{w}_{t+1} \geq \mathbf{w}^* \cdot \mathbf{w}_0 + k\delta$. This grows **linearly in k**.
7. **Denominator:** $\|\mathbf{w}_{t+1}\|^2 = \|\mathbf{w}_t\|^2 + 2\mathbf{w}_t \cdot \mathbf{p}_i + \|\mathbf{p}_i\|^2 \leq \|\mathbf{w}_t\|^2 + 1$, because the correction happened ($\mathbf{w}_t \cdot \mathbf{p}_i \leq 0$) and $\|\mathbf{p}_i\| = 1$. So $\|\mathbf{w}_{t+1}\|^2 \leq \|\mathbf{w}_0\|^2 + k$. The squared norm grows **at most linearly**, so the norm grows like $\sqrt{k}$.
8. **Combining the two:**

$$\cos\beta \;\geq\; \frac{\mathbf{w}^* \cdot \mathbf{w}_0 + k\delta}{\sqrt{\|\mathbf{w}_0\|^2 + k}} \;\propto\; \sqrt{k}$$

The lower bound grows like $\sqrt{k}$, but $\cos\beta \leq 1$. So k **must be bounded**: there can only be a **finite number of corrections**, and the algorithm **converges**.

- **What the proof establishes:** the number of *corrections* k is finite. It does not bound the number of time steps t when no correction happens. It does not show that the solution found is unique or optimal (any separating **w** counts). It needs **finite, linearly separable** data.
- **(Additional clarification)** With $\mathbf{w}_0 = \mathbf{0}$, the bound becomes $k\delta/\sqrt{k} \leq 1$, i.e. $k \leq 1/\delta^2$. A larger margin δ means fewer updates. This explicit bound is not on the slides.
- ⚠ **Slide note:** the algorithm corrects when $\mathbf{w} \cdot \mathbf{p} < 0$, while the proof slides say a correction is made "if $\mathbf{w}^T \cdot \mathbf{p}_i \leq 0$". The two conditions differ only for points exactly on the boundary.
- ⚠ **Slide note:** the Definition is of *absolutely* linearly separable, while the Proposition says just "linearly separable". The proof needs strictness ($\delta > 0$).

F. Traps

- "The PLA converges for any data": **False**. It is guaranteed only for **linearly separable** data. For XOR it never converges, and **w** keeps cycling.
- "The PLA minimizes squared error" or "uses a learning rate": **False** (for this course's PLA). It makes mistake-driven $\pm\mathbf{x}$ updates.
- "$\mathbf{w}$ is parallel to the decision boundary": **False**. It is **perpendicular** (normal) to it.

Coming back to the motivating questions (slide summary)

Question	Answer
Non-boolean (real) inputs?	Allowed in the perceptron
Must the threshold be hand-coded?	No, it can be learned
Can some inputs matter more?	Yes, through weights
Non-linearly separable functions?	Not possible with a single perceptron; a **network** of perceptrons is needed

Module 2.7: Linearly separable boolean functions

C. XOR is not linearly separable

x_1	x_2	XOR	Condition
0	0	0	$w_0 < 0$
0	1	1	$w_2 \geq -w_0$
1	0	1	$w_1 \geq -w_0$
1	1	0	$w_1 + w_2 < -w_0$

Adding conditions 2 and 3 gives $w_1 + w_2 \geq -2w_0 > -w_0$ (since $w_0 < 0$), which **contradicts condition 4**. So no single line separates XOR's 1s from its 0s.

- Most real-world data is **not linearly separable** and contains outliers. Sometimes data is inseparable even **without** outliers. A single perceptron cannot handle such data, but a **network of perceptrons** can.

D/E. Counting boolean functions

- Number of boolean functions of n inputs: $\boxed{2^{2^n}}$. There are 2^n rows in the truth table, and each row's output can be 0 or 1.
  - n = 1 → 4, **n = 2 → 16**, n = 3 → 256, n = 4 → 65 536.
- For **n = 2**, all 16 functions ($f_1..f_{16}$ on the slide) are linearly separable **except XOR (f_7: 0,1,1,0) and !XOR/XNOR (f_{10}: 1,0,0,1)**. That makes **14 of 16** linearly separable.
- The slide only says that "at least some" of the 2^{2^n} functions are not linearly separable and leaves the exact count as an exercise.
- **(Additional clarification, verified by brute force)** linearly separable counts: n = 1: 4 of 4; n = 2: 14 of 16; n = 3: **104 of 256**. The non-separable fraction grows quickly with n.

Module 2.8: Representation power of a network of perceptrons

A. Construction (2 inputs)

- **Convention for this construction:** True = **+1** and False = **−1**.
- **2 inputs → 4 hidden perceptrons**, each with bias $w_0 = -2$. Each fires only if its weighted input sum is ≥ 2. Hidden-layer weights are **±1** (red edge = −1, blue edge = +1), chosen so that each hidden perceptron's weight pattern matches exactly **one** input combination.
  - h_1 fires only for (−1,−1), h_2 only for (−1,1), h_3 only for (1,−1), and h_4 only for (1,1). **No two hidden perceptrons fire for the same input.**
- The **output perceptron** has weights $w_1..w_4$ (learned) and bias w_0. It fires if $\sum_{i=1}^{4} w_i h_i \geq w_0$.
- **Terminology:** the network has **3 layers**: an **input layer** (x_1, x_2), a **hidden layer** (4 perceptrons, outputs $h_1..h_4$) and an **output layer** (1 neuron). The red and blue edges are the **layer-1 weights**, and $w_1..w_4$ are the **layer-2 weights**.

C. Why it works (XOR example)

Exactly one h_i is 1 for each input, so $\sum_i w_i h_i = w_i$ for the active unit. XOR needs $w_1 < w_0$, $w_2 \geq w_0$, $w_3 \geq w_0$, $w_4 < w_0$. There is **no contradiction**: each w_i is responsible for exactly one input row and can be set independently. The same network represents **all 16** boolean functions of 2 inputs, each with a different set of consistent inequalities.

- ⚠ **Slide note:** the XOR table on this slide writes the inputs as 0/1, although the construction uses ±1 (0 ↔ −1). Also, "w_0" for the output neuron here is a **threshold** (fire if $\sum w_i h_i \geq w_0$), not a bias added to the sum as in the earlier convention $w_0 = -\theta$.

D. Theorem (representation power of an MLP of perceptrons)

> Any boolean function of n inputs can be represented **exactly** by a network of perceptrons containing **1 hidden layer with 2^n perceptrons** and **one output layer containing 1 perceptron**.

- 3 inputs → **8** hidden perceptrons, each with bias **−3** (slide). Each of the 8 second-layer weights is responsible for one of the 8 inputs. **(Additional clarification)** In general, 2^n **hidden perceptrons with bias** $-n$ (the slides show only n = 2 and n = 3), each firing for exactly one of the 2^n inputs. The total is $2^n + 1$ perceptrons.
- **Note:** $2^n + 1$ perceptrons is **sufficient but not necessary**. AND, for example, needs just 1 perceptron.

- **Catch:** the number of hidden perceptrons grows **exponentially** with n.
- **Link to classification:** given movie data (points p_i with $y = 1$ and n_j with $y = 0$), linearly separable **or not**, there exists a network of perceptrons whose weights can be learned to reproduce every label.
- **Naming:** a network with an input layer, an output layer and one or more hidden layers is a **Multilayer Perceptron (MLP)**. "Multilayered Network of Perceptrons" would be the more accurate name. An MLP with a **single hidden layer can represent any boolean function**.

E. Numericals

- **Example:** $n = 5$ needs $2^5 = \mathbf{32}$ hidden perceptrons, each with bias **−5**. Total neurons: 33. Number of boolean functions: $2^{32} = 4\,294\,967\,296$.
- **Example:** in the 2-input network, input (1,−1) fed to the hidden unit with weights (+1,−1): sum = 2, and 2 − 2 = 0 ≥ 0, so it fires. Fed to the unit with weights (+1,+1): sum = 0, and 0 − 2 < 0, so it does not fire.
- **Recognizing the question:** "how many hidden neurons for any boolean function of n inputs" → 2^n. "How many boolean functions" → 2^{2^n}. "How many rows in the truth table" → 2^n.

F. MCQ traps (Lecture 2)

Statement	True/False
A single perceptron can represent XOR	**F**
An MLP with one hidden layer of 2^n perceptrons can represent **any** boolean function of n inputs	**T**
2^n hidden perceptrons are **necessary** for every boolean function	**F** (sufficient, not necessary)
MP neurons have learnable weights	**F**
The perceptron allows real-valued inputs	**T**
Perceptrons can represent more functions than MP neurons (single unit)	Careful: both are limited to **linearly separable** functions. The perceptron adds learnable weights and real inputs
PLA convergence requires linearly separable, finite data	**T**
Of the 16 two-input boolean functions, 2 are not linearly separable	**T** (XOR, XNOR)
The weight vector is orthogonal to the decision boundary	**T**
In the construction, True/False are encoded as 1/0	**F**: +1/−1 (the slide's XOR table uses 0/1 for readability)

G. Quick revision: Lecture 2

- MP: $y = 1$ iff $\sum x_i \geq \theta$ and no inhibitory input is on. Boolean inputs, no weights, hand-set θ.
- Perceptron: $y = 1$ iff $\sum_{i=0}^{n} w_i x_i \geq 0$ with $x_0 = 1$ and $w_0 = -\theta$ (the bias, or prior).
- PLA: $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}$ on a misclassified positive, $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{x}$ on a misclassified negative. It converges after finitely many corrections when the data is linearly separable ($\cos\beta$ bound grows like $\sqrt{k}$ but must stay ≤ 1).
- 2^{2^n} boolean functions. For n = 2, 14 are linearly separable (all but XOR and XNOR).
- One hidden layer with 2^n perceptrons (bias −n, ±1 weights) plus one output perceptron represents any boolean function. The size is exponential in n.

Week 2 · Lecture 3: Sigmoid Neurons, Gradient Descent, Feedforward Networks

Module 3.1: Sigmoid neuron

A. Core concept

- **New goal:** represent arbitrary functions $y = f(\mathbf{x})$ with $\mathbf{x} \in \mathbb{R}^n$ (not $\{0,1\}^n$) and $y \in \mathbb{R}$ (not $\{0,1\}$). This requires moving from perceptrons to **sigmoid neurons**.
- **The perceptron is "harsh".** With a single input criticsRating $x_1 \in [0,1]$, $w_1 = 1$ and $w_0 = -0.5$ (threshold 0.5): rating **0.51 → like** and **0.49 → dislike**. This is not specific to the example. The perceptron is a **step function**, so its decision always changes abruptly when $\sum w_i x_i$ crosses the threshold $-w_0$. Real applications want a **smooth** decision that moves gradually from 0 to 1.

D. Formula: logistic (sigmoid) neuron

$$y = \frac{1}{1 + e^{-(w_0 + \sum_{i=1}^{n} w_i x_i)}} = \frac{1}{1 + e^{-\sum_{i=0}^{n} w_i x_i}} = \sigma(\mathbf{w}^T \mathbf{x})$$

- **Symbols:** $x_0 = 1$, $w_0 = -\theta$ (from L3.3 on, written as **b**, the bias). σ is the sigmoid; the **logistic** function is *one form* of the sigmoid.
- **Output:** a **real value in (0,1)**, interpretable as a **probability** (for example, the probability of liking the movie) instead of a hard like/dislike.
- **Properties:** smooth, continuous and differentiable. The perceptron is **not smooth, not continuous (at $-w_0$), and not differentiable**. (⚠ Slide note: the slide text says "not continuous (at w0)", but its own plot puts the jump at $z = -w_0$, the threshold.)
- **Limits:** $\sigma(0) = 0.5$; as $z \to +\infty$, $\sigma \to 1$; as $z \to -\infty$, $\sigma \to 0$.
- **Effect of w and b in $\sigma(wx + b)$:** the transition (σ = 0.5) is at $x = -b/w$. Larger $|w|$ gives a **steeper** transition, and as $w \to \infty$ σ **recovers the step function**. Changing b **shifts the transition point** along the x-axis.
- **(Additional clarification)** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, with a maximum of 0.25 at $z = 0$. This is derived in Module 3.4.

E. Numerical

Movie example as a sigmoid with $w_1 = 1$, $w_0 = -0.5$: $\sigma(0.01) = 0.5025$ and $\sigma(-0.01) = 0.4975$. These are nearly equal probabilities, unlike the perceptron's hard 1 versus 0.

F. Perceptron vs sigmoid neuron

	Perceptron	Sigmoid neuron
Output	{0,1}	(0,1), real-valued
Function	Step at $-w_0$	Smooth S-curve
Continuous / differentiable	No / No	Yes / Yes
Learning	PLA	Gradient descent
Interpretation	Hard decision	Probability

Module 3.2: A typical supervised machine learning set-up

B. Components (definitions)

Component	Meaning	Example
Data	$\{\mathbf{x}_i, y_i\}_{i=1}^n$	movie, like/dislike
Model	Our **approximation** of the relation between x and y	$\hat{y} = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$, $\hat{y} = \mathbf{w}^T\mathbf{x}$, $\hat{y} = \mathbf{x}^T W\mathbf{x}$, or almost any function
Parameters	What must be learned from data	$\mathbf{w}$
Learning algorithm	Algorithm that learns the parameters	Perceptron learning algorithm, gradient descent
Objective / loss / error function	Guides learning; the algorithm aims to **minimize** it	$\mathscr{L}(\mathbf{w}) = \sum_{i=1}^n (\hat{y}_i - y_i)^2$ (squared error)

C. Insights

- Even when the data is not linearly separable, a perceptron line that misclassifies 3 blue and 3 red points may be acceptable. **From here on, the aim is the minimum possible error, not necessarily zero error.**

Module 3.3: Learning parameters by (infeasible) guess work

A. Toy set-up

- One input, model $f(x) = \sigma(wx + b) = \frac{1}{1+e^{-(wx+b)}}$. From here on w_0 **is called** b (the bias), to match the literature.
- **Task:** predict criticsRating (y) from imdbRating (x).
- **Training objective:**

$$\min_{w,b} \mathscr{L}(w, b) = \frac{1}{2}\sum_{i=1}^{N}(y_i - f(x_i))^2$$

- **Data:** $(x, y) = (0.5, 0.2)$ and $(2.5, 0.9)$. Training means finding w^*, b^* with $f(0.5) \to 0.2$ and $f(2.5) \to 0.9$, i.e. a sigmoid that **passes through both points**.

E. Worked numerical: loss values (recomputed with python)

At $w = 0.5, b = 0$: $f(0.5) = \sigma(0.25) = 0.5622$ and $f(2.5) = \sigma(1.25) = 0.7773$, so

$$\mathscr{L} = \tfrac{1}{2}\left[(0.9 - 0.7773)^2 + (0.2 - 0.5622)^2\right] = 0.0731 \approx \mathbf{0.073}$$

w	b	$\mathscr{L}(w, b)$ **(slide)**	Comment
0.50	0.00	0.0730	Random guess
−0.10	0.00	0.1481	Worse: the sigmoid now *decreases* with x
0.94	−0.94	0.0214	Better: push w up and b down
1.42	−1.73	0.0028	
1.65	−2.08	0.0003	
1.78	−2.27	0.0000	

- The exact fit solves $wx + b = \text{logit}(y)$ at both points, giving $w^* \approx 1.79$ **and** $b^* \approx -2.28$.

- ⚠ **Slide note:** some table entries and the figure legend differ slightly: the figure shows (0.94, −0.95), (1.43, −1.74) and (1.79, −2.28). The tabulated losses match the legend values exactly (0.0214, 0.0028); the table's own (w, b) give 0.0217 and 0.0029. This is rounding only.
- ⚠ **Slide note:** the loss is written $\sum(\hat{y}-y)^2$ (Module 3.2), $\sum(y_i - f(x_i))^2$ without ½ (the training-objective slide), and with ½ (the loss computation and the gradient derivation). The constant ½ does not change the minimizer. It only cancels the 2 when differentiating. Use the version the question states.

C. Why guess work is infeasible

- With 2 points and 2 parameters, $\mathscr{L}(w,b)$ can be plotted and its minimum picked. Even then the slides plot only $(w,b) \in (-6,6)$, not $(-\infty,\infty)$.
- This becomes **intractable** with many data points and parameters. Geometrically, the guess-work algorithm is a **"random search on the error surface"** (slide title): it jumps between points with no principled direction.

Module 3.4: Learning parameters with gradient descent

A. Goal

Traverse the error surface so that the minimum is reached **quickly without brute-force search**, in a principled way.

D. Set-up and Taylor-series argument

- $\boldsymbol{\theta} = [w,b]$ is the parameter vector, initialized randomly. $\Delta\boldsymbol{\theta} = [\Delta w, \Delta b]$ is the change. The move is conservative, scaled by a small **η** (the learning rate):

$$\boldsymbol{\theta}_{new} = \boldsymbol{\theta} + \eta \cdot \Delta\boldsymbol{\theta}$$

- The right $\Delta\boldsymbol{\theta}$ comes from the **Taylor series**. Let $\Delta\boldsymbol{\theta} = \mathbf{u}$:

$$\mathscr{L}(\boldsymbol{\theta}+\eta\mathbf{u}) = \mathscr{L}(\boldsymbol{\theta}) + \eta\,\mathbf{u}^T\nabla_{\boldsymbol{\theta}}\mathscr{L}(\boldsymbol{\theta}) + \frac{\eta^2}{2!}\mathbf{u}^T\nabla^2\mathscr{L}(\boldsymbol{\theta})\mathbf{u} + \frac{\eta^3}{3!}\cdots \;\approx\; \mathscr{L}(\boldsymbol{\theta}) + \eta\,\mathbf{u}^T\nabla_{\boldsymbol{\theta}}\mathscr{L}(\boldsymbol{\theta})$$

The higher-order terms can be dropped because **η is small**, so $\eta^2, \eta^3, \cdots \to 0$.

- **A move is favourable** if $\mathscr{L}(\boldsymbol{\theta}+\eta\mathbf{u}) - \mathscr{L}(\boldsymbol{\theta}) < 0$, which requires $\mathbf{u}^T\nabla_{\boldsymbol{\theta}}\mathscr{L}(\boldsymbol{\theta}) < 0$.
- **Range of the term.** Let β be the angle between $\mathbf{u}$ and $\nabla\mathscr{L}$ and $k = \|\mathbf{u}\|\,\|\nabla\mathscr{L}\|$. Then $-k \le k\cos\beta = \mathbf{u}^T\nabla\mathscr{L} \le k$. The change is **most negative when** $\cos\beta = -1$**, i.e. β = 180°**.
- ⚠ **Slide note:** the slide writes $\mathscr{L}(\boldsymbol{\theta}+\eta\mathbf{u}) - \mathscr{L}(\boldsymbol{\theta}) = \mathbf{u}^T\nabla\mathscr{L} = k\cos\beta$, dropping the factor η. Strictly the change is $\approx \eta\,k\cos\beta$; the conclusion (β = 180°) is unaffected because η > 0.

D. Gradient descent rule

> **Move in the direction opposite to the gradient** (180° from it).

$$w_{t+1} = w_t - \eta\nabla w_t, \qquad b_{t+1} = b_t - \eta\nabla b_t, \qquad \nabla w_t = \left.\frac{\partial\mathscr{L}(w,b)}{\partial w}\right|_{w=w_t,b=b_t},\ \nabla b_t = \left.\frac{\partial\mathscr{L}(w,b)}{\partial b}\right|_{w=w_t,b=b_t}$$

```
Algorithm gradient_descent():
  t ← 0; max_iterations ← 1000
  while t < max_iterations:
      w_{t+1} ← w_t − η∇w_t
      b_{t+1} ← b_t − η∇b_t
      t ← t + 1
```

- **Common mistakes:** using $+\eta\nabla$ (that is gradient *ascent*); evaluating the gradient at the *new* point instead of at (w_t, b_t); forgetting that **both w and b** are updated using the **old** values.

D. Gradients for the sigmoid neuron with squared error

For **one point**, with $\mathscr{L} = \frac{1}{2}(f(x) - y)^2$ and $f(x) = \sigma(wx + b)$:

$$\frac{\partial f(x)}{\partial w} = f(x)\,(1 - f(x))\,x \qquad (\text{sigmoid derivative times the chain-rule factor } x)$$

$$\nabla w = (f(x) - y)\,f(x)\,(1 - f(x))\,x, \qquad \nabla b = (f(x) - y)\,f(x)\,(1 - f(x))$$

For **N points** (the slides use 2), the gradients **sum** over the points:

$$\nabla w = \sum_i (f(x_i) - y_i)\,f(x_i)(1 - f(x_i))\,x_i, \qquad \nabla b = \sum_i (f(x_i) - y_i)\,f(x_i)(1 - f(x_i))$$

- **Interpretation:** each term is (prediction error) × (sigmoid slope) × (input). ∇b is the same expression without the x factor, since $\partial(wx + b)/\partial b = 1$.
- **Sign check:** if the prediction is too high ($f > y$) and $x > 0$, then $\nabla w > 0$ and GD *decreases* w.
- **Saturation (follows from the formula):** if $f(x) \approx 0$ or 1, then $f(1 - f) \approx 0$, the gradient is tiny and learning is slow even when the error is large.

E. Worked numericals (verified with python)

(1) One GD step, single point. $x = 2$, $y = 0$, $w = 1$, $b = 0$, η = 0.5.

- $f = \sigma(2) = 0.8808$
- $f(1 - f) = 0.1050$
- $\nabla w = (0.8808 - 0)(0.8808)(0.1192)(2) = \mathbf{0.1850}$
- $\nabla b = \mathbf{0.0925}$
- $w_{new} = 1 - 0.5(0.1850) = \mathbf{0.9075}$ and $b_{new} = 0 - 0.5(0.0925) = -\mathbf{0.0462}$
- Loss before the step: $\frac{1}{2}(0.8808)^2 = 0.388$

(2) From zero, single point. $x = 1$, $y = 1$, $w = b = 0$.

- $f = 0.5$, so $\nabla w = \nabla b = (0.5 - 1)(0.5)(0.5)(1) = -0.125$
- With η = 1: $w = b = 0.125$. A negative gradient means the parameter increases.

(3) The slides' GD demo (code shown on the slide): data X = [0.5, 2.5], Y = [0.2, 0.9], initial $w = -2$, $b = -2$, **η = 1.0**, max_epochs = 1000, batch gradients summed over both points.

- At the start, $\mathscr{L} = 0.416$, $\nabla w = -0.0055$ and $\nabla b = -0.0077$. These are tiny because the start lies on a **flat region** of the surface.
- After 1000 epochs: $w \approx 1.79$, $b \approx -2.28$, and $\mathscr{L} \approx 0$. This is the same solution the guess work found.

Recognizing the question:

- "Direction of steepest decrease" → $-\nabla\mathscr{L}$.
- "Angle between the update and the gradient" → 180°.
- "Why drop higher-order Taylor terms" → η is small.
- Effect of η: if η is too small, progress is slow. **(Additional clarification)** If η is too large, the steps can overshoot. GD variants are covered in L5.

Module 3.5: Representation power of a multilayer network of sigmoid neurons

C. Universal approximation theorem (UAT)

Multilayer network of **perceptrons**	Multilayer network of **sigmoid neurons**
A single hidden layer can **represent** any **boolean** function **precisely (no errors)**	A single hidden layer can **approximate** any **continuous** function **to any desired precision**

- **Formally:** for any $f(\mathbf{x}) : \mathbb{R}^n \to \mathbb{R}^m$, there is always a neural network (**1 hidden layer with enough neurons**) whose output $g(\mathbf{x})$ satisfies $|g(\mathbf{x}) - f(\mathbf{x})| < \epsilon$.
- **References:** [Cybenko, 1989] and [Hornik, 1991]. The illustrative proof follows Nielsen's online book, Ch. 4 (neuralnetworksanddeeplearning.com).
- ⚠ The UAT is an **existence** guarantee. It does not say how many neurons are needed (only "enough"), nor that GD will find the weights.

A. Illustrative proof via "tower functions" (1-D input)

1. Any arbitrary function can be approximated by a **sum of "tower" functions** (rectangular bumps). **More towers give a better approximation.**
2. All towers are alike and differ only in **height** and **position** on the x-axis. A black box ("tower maker") builds them from x, and a simple network adds them up.
3. **Step from sigmoid:** in $\sigma(wx + b)$, a **very high w** recovers the **step function** (the slides animate w = 0, 1, ..., 41 and then w = 50). **b controls where** the transition from 0 to 1 happens (the slides use w = 50 with b = 1...35). The transition is at $x = -b/w$.
4. **Tower = difference of two steps:** $\sigma(w_1 x + b_1) - \sigma(w_2 x + b_2)$ with the same large w and different b gives a tower.
5. **Network for one tower (1-D):** two hidden sigmoid neurons h_{11} (with w_1, b_1) and h_{12} (with w_2, b_2) feed h_{21} with weights **+1 and −1**. So **2 neurons per tower** for 1-D input. The output layer sums the towers, weighting each by its height.

A. 2-D input (oil example: salinity x_1 and pressure x_2 → oil yes/no)

- $y = \sigma(w_1 x_1 + w_2 x_2 + b)$ is a 2-D sigmoid. Setting $w_2 = 0$ and making w_1 large gives a 2-D step along x_1. Changing b moves it.
- Subtracting two such steps (different b) gives a tower **open on two sides**: a ridge or wall, not yet a tower.
- Doing the same with $w_1 = 0$ and w_2 large gives an open tower with the **other orientation**.
- **Adding the two open towers** gives a **tower standing on an elevated base**. Passing that through **another sigmoid**, which thresholds away the base, gives the **desired closed tower**.
- **Network (slide figure):** first hidden layer of **4 sigmoids**, then two "−" units (weights +1, −1), then a "+" unit (weights +1, +1), then a final sigmoid. Slide values: $w_1 = 100, w_2 = 1, b = \pm 200$ (and the mirror

image); final sigmoid $w = 50$, $b = -100$. A smoother version uses $w = 2.5$, $b = \pm 5$ with a final $w = 3$, $b = -6$.
- **Counting:** 1-D input needs **2 neurons** per tower and 2-D input needs **4 neurons** per tower. The slide asks how many are needed in n dimensions and leaves it as an exercise. **(Additional clarification)** Following the pattern, $2n$ first-layer sigmoids per tower, plus the combining or thresholding unit.

C. Linking back to classification

A single sigmoid neuron that separates blue from red points makes errors. The true decision function is a complex surface, and it can be approximated by a **sum of towers**, which is a network with **two hidden layers** in the 2-D construction. So a neural network can **exactly separate** the blue points from the red points.

- **Trap:** the UAT says *one* hidden layer suffices for approximation in principle. The illustrative 2-D *construction* uses two hidden layers. These do not contradict each other: the construction is just a convenient proof device.
- **(Additional clarification)** In the 2-D slide construction (sharp steps), the base reaches height 1 and the tower region reaches about 2. The final sigmoid's transition point $-b/w$ should therefore lie between 1 and 2. With $w = 50, b = -100$ it sits at exactly 2, which is on the edge of that range: at the tower centre the sum is 2, so the output is $\sigma(50 \cdot 2 - 100) = \sigma(0) = 0.5$, while on the base it is $\sigma(50 \cdot 1 - 100) = \sigma(-50) \approx 0$.

F. MCQ traps (Lecture 3)

Statement	Verdict
Sigmoid output is binary	**F**: real value in (0,1)
The perceptron is differentiable	**F**: not smooth, not continuous, not differentiable
GD moves along the gradient	**F**: opposite to it (180°)
GD's Taylor argument relies on a small η	**T**
Any $\mathbf{u}$ with $\mathbf{u}^T \nabla \mathscr{L} < 0$ (to first order) decreases the loss	**T** (for small η). The **most** negative change is at β = 180°
∇b has the factor x	**F**: only ∇w has x
UAT: a single hidden layer can approximate any continuous function to arbitrary precision	**T**
UAT: a single hidden layer can approximate *any* function *exactly*	**F**: "approximate" and "continuous"
Larger w in σ(wx+b) gives a smoother curve	**F**: steeper, approaching a step
b changes the steepness of the sigmoid	**F**: b shifts the transition, located at $-b/w$
More towers give a better approximation	**T**
Subtracting two 2-D steps already gives a closed tower	**F**: it is open on two sides

G. Quick revision: Lecture 3

- $\sigma(z) = 1/(1 + e^{-z})$ with $z = \mathbf{w}^T \mathbf{x} + b$. Smooth, differentiable, output in (0,1), interpretable as a probability.
- ML set-up: data, model, parameters, learning algorithm, loss.
- Taylor: $\mathscr{L}(\boldsymbol{\theta} + \eta \mathbf{u}) \approx \mathscr{L}(\boldsymbol{\theta}) + \eta \mathbf{u}^T \nabla \mathscr{L}$. We need $\mathbf{u}^T \nabla \mathscr{L} < 0$, and the best choice is $\mathbf{u} = -\nabla \mathscr{L}$ (β = 180°).
- Update: $w \leftarrow w - \eta \nabla w$ and $b \leftarrow b - \eta \nabla b$, with $\nabla w = \sum (f - y) f (1 - f) x$ and $\nabla b = \sum (f - y) f (1 - f)$.

- UAT (Cybenko 1989, Hornik 1991): one hidden layer with enough sigmoid neurons approximates any continuous $\mathbb{R}^n \to \mathbb{R}^m$ function within ε.
- Towers: large w gives a step, b sets the position, step minus step gives a tower (2 neurons in 1-D, 4 in 2-D), and summing towers approximates any function.

Overview: Lectures 4–5: Feedforward Networks, Backpropagation & Optimization Algorithms

Lectures covered / modules:

- **Lecture 4 — Feedforward Neural Networks, Backpropagation**: 4.1 Feedforward networks (notation, parameter counting) · 4.2 Learning parameters (intuition) · 4.3 Output & loss functions · 4.4 Backpropagation (intuition) · 4.5 Gradients w.r.t. output units · 4.6 Gradients w.r.t. hidden units · 4.7 Gradients w.r.t. parameters · 4.8 Pseudocode · 4.9 Derivatives of activation functions
- **Lecture 5 — GD, Momentum, NAG, SGD, AdaGrad, RMSProp, Adam**: 5.1 Guess work · 5.2 Gradient descent (recap) · 5.3 Contours · 5.4 Momentum GD · 5.5 Nesterov accelerated GD · 5.6 Stochastic & mini-batch GD · 5.7 Tips for learning rate & momentum · 5.8 Line search · 5.9 Adaptive learning rates (AdaGrad, RMSProp, Adam, bias correction)

> Notation used throughout (Khapra's): $\mathbf{x} = \mathbf{h}_0$ input; $\mathbf{a}_i$ pre-activation, $\mathbf{h}_i$ activation of layer i; $W_i, \mathbf{b}_i$ = weights/bias between layer $i-1$ and i; L = number of layers (hidden + output, input layer is layer 0); g = hidden activation; O = output activation; ℓ = true class label; η = learning rate; $\mathscr{L}(\theta)$ = loss.

Week 3 · Lecture 4: Feedforward Neural Networks, Backpropagation

Module 4.1: Feedforward Neural Networks (multilayered network of neurons)

A. Core Concept

- Input: an n-dimensional vector $\mathbf{x}$ (= layer 0).
- $L-1$ **hidden layers**, each (on the slide) with n neurons; one **output layer** (= layer L) with k neurons (e.g. k classes). Slide example: $L=3$ (2 hidden layers + output).
- Every hidden/output neuron has **two parts**: **pre-activation** a (weighted sum + bias) and **activation** h (non-linearity applied). $\mathbf{a}_i, \mathbf{h}_i$ are vectors.
- Full model (for $L=3$):

$$\hat{\mathbf{y}} = f(\mathbf{x}) = O\big(W_3\, g(W_2\, g(W_1\mathbf{x}+\mathbf{b}_1)+\mathbf{b}_2)+\mathbf{b}_3\big)$$

- Parameters: $\theta = [W_1, \ldots, W_L,\ \mathbf{b}_1, \ldots, \mathbf{b}_L]$. Algorithm: gradient descent with backpropagation. Data: $\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N$.

B. Terminology

Term	Meaning
Pre-activation $\mathbf{a}_i$	$\mathbf{a}_i = \mathbf{b}_i + W_i\mathbf{h}_{i-1}$ (linear part)
Activation $\mathbf{h}_i$	$\mathbf{h}_i = g(\mathbf{a}_i)$, g applied element-wise (logistic, tanh, linear, ...)
Output activation O	$f(\mathbf{x}) = \mathbf{h}_L = O(\mathbf{a}_L)$ (softmax, linear, ...) — **different from** g
Layer 0 / layer L	input layer / output layer
$W_{k,ij}$	weight connecting unit j of layer $k-1$ to unit i of layer k (consistent with $a_{ki} = b_{ki} + \sum_j W_{kij} h_{k-1,j}$)

C. Properties / examinable statements

- Slide shapes: $W_i \in \mathbb{R}^{n\times n}$, $\mathbf{b}_i \in \mathbb{R}^n$ for $0 < i < L$; $W_L \in \mathbb{R}^{n\times k}$, $\mathbf{b}_L \in \mathbb{R}^k$.
- ⚠ **Slide note:** with $\mathbf{a}_L = W_L\mathbf{h}_{L-1} + \mathbf{b}_L$ and $\mathbf{a}_L \in \mathbb{R}^k$, W_L must actually be $k \times n$ (rows = units of the current layer). The slide writes $n \times k$ (and $\nabla W_L \in \mathbb{R}^{n\times k}$). The **number of entries (nk) is the same either way**, so parameter counts are unaffected; but for "shape" questions use (rows = #units in layer k, cols = #units in layer $k-1$), which is what $\nabla_{W_k} = \nabla_{\mathbf{a}_k}\mathbf{h}_{k-1}^T$ produces.
- Biases: one per neuron of every hidden **and output** layer; **no biases on the input layer**.

D. Formulas

Formula	Notes
$\mathbf{a}_i = \mathbf{b}_i + W_i\mathbf{h}_{i-1}$	pre-activation; $\mathbf{h}_0 = \mathbf{x}$
$\mathbf{h}_i = g(\mathbf{a}_i)$	$1 \le i \le L-1$
$\hat{\mathbf{y}} = f(\mathbf{x}) = \mathbf{h}_L = O(\mathbf{a}_L)$	output layer uses O, not g

Formula	Notes
#params (layer i, sizes $d_{i-1} \to d_i$) $= d_{i-1}d_i + d_i$	weights + biases
Slide network total $= (L-1)(n^2+n) + nk + k$	n inputs, $L-1$ hidden layers of width n, k outputs

E. Numerical: parameter counting

- Recipe: for each consecutive pair of layers multiply sizes (weights), then add the size of every non-input layer (biases).
- Example 1: 100 → 50 → 50 → 10. Weights $= 100 \cdot 50 + 50 \cdot 50 + 50 \cdot 10 = 5000 + 2500 + 500 = 8000$; biases $= 50 + 50 + 10 = 110$; **total 8110**.
- Example 2 (slide-style): $n = 10$, $L = 3$, $k = 4$: $2(100 + 10) + 40 + 4 =$ **264**.
- Example 3: 784 → 128 → 64 → 10: weights 109184, biases 202, total **109386**.
- Traps: forgetting output-layer biases; counting input "biases"; using L as number of hidden layers (there are $L-1$ hidden layers); counting neurons instead of connections.

Module 4.2: Learning Parameters of FFNNs (Intuition)

A. Core Concept

- Same gradient descent algorithm as before, but now $\theta = [W_1, \ldots, W_L, \mathbf{b}_1, \ldots, \mathbf{b}_L]$:

$$\theta_{t+1} \leftarrow \theta_t - \eta \nabla\theta_t, \qquad \nabla\theta_t = \left[\frac{\partial \mathscr{L}(\theta)}{\partial W_{1,t}}, \ldots, \frac{\partial \mathscr{L}(\theta)}{\partial W_{L,t}}, \frac{\partial \mathscr{L}(\theta)}{\partial \mathbf{b}_{1,t}}, \ldots, \frac{\partial \mathscr{L}(\theta)}{\partial \mathbf{b}_{L,t}}\right]^T$$

- $\nabla\theta$ ("much more nasty") is composed of $\nabla W_1, \ldots, \nabla W_{L-1} \in \mathbb{R}^{n \times n}$, $\nabla W_L \in \mathbb{R}^{n \times k}$ (slide shape, see ⚠ above), $\nabla \mathbf{b}_1, \ldots, \nabla \mathbf{b}_{L-1} \in \mathbb{R}^n$, $\nabla \mathbf{b}_L \in \mathbb{R}^k$.
- **Key fact:** every gradient has the **same shape as the parameter it corresponds to**.
- Two questions the rest of the lecture answers: (1) how to choose $\mathscr{L}(\theta)$? (2) how to compute $\nabla\theta$ efficiently (→ backpropagation)?

Module 4.3: Output Functions and Loss Functions

A. Core Concept

The choice of loss (and output activation) **depends on the problem at hand**.

Case 1 — real-valued outputs (e.g. predict movie ratings from IMDb, Critics, RT: $\mathbf{y}_i \in \mathbb{R}^3$, e.g. $[7.5, 8.2, 7.7]$):

- Loss: **squared error** $\mathscr{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{3}(\hat{y}_{ij} - y_{ij})^2$.
- Output function: **linear**, $f(\mathbf{x}) = \mathbf{h}_L = O(\mathbf{a}_L) = W_O\mathbf{a}_L + \mathbf{b}_O$. **Not logistic**, because logistic restricts $\hat{y}$ to $(0, 1)$ but we want $\hat{y} \in \mathbb{R}$ (unbounded).

Case 2 — classification into 1 of k classes (e.g. Apple/Mango/Orange/Banana; $\mathbf{y} = [1, 0, 0, 0]$ one-hot):

- $\mathbf{y}$ is a **probability distribution**, so $\hat{\mathbf{y}}$ should also be one → output activation = **softmax**:

$$\hat{y}_j = O(\mathbf{a}_L)_j = \frac{e^{a_{L,j}}}{\sum_{i=1}^{k} e^{a_{L,i}}}$$

- Loss capturing the difference between two distributions = **cross-entropy**:

$$\mathscr{L}(\theta) = -\sum_{c=1}^{k} y_c \log \hat{y}_c = -\log \hat{y}_\ell \quad (\text{since } y_c = 1 \text{ only for } c = \ell)$$

- Objective: $\min_\theta -\log \hat{y}_\ell \iff \max_\theta \log \hat{y}_\ell$. $\hat{y}_\ell$ = probability that $\mathbf{x}$ belongs to class ℓ (push it towards 1); $\log \hat{y}_\ell$ is the **log-likelihood** of the data. $\hat{y}_\ell$ **is** a function of θ: $\hat{y}_\ell = [O(W_3 g(W_2 g(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3)]_\ell$.

C. Choice table (slide)

Outputs	Output activation	Loss function
Real values	Linear	Squared error
Probabilities	Softmax	Cross entropy

- Other losses possible; these two are the most common. Rest of Lecture 4 assumes **softmax + cross-entropy**.

D/F. Formulas & traps

- Softmax outputs are positive and sum to 1. (Additional clarification: softmax is unchanged if a constant is added to all $a_{L,j}$; larger $a_{L,j}$ → larger $\hat{y}_j$.)
- Cross-entropy for one-hot $\mathbf{y}$ depends **only on the predicted probability of the true class**; range $[0, \infty)$; $= 0$ iff $\hat{y}_\ell = 1$.
- Trap: "Squared error cannot be used for classification" — false; it *can* be used (slide: "we could use the squared error loss"), cross-entropy is just better suited.
- Trap: logistic/sigmoid output for unbounded regression — wrong (bounded to (0,1)).

E. Numerical

- $\mathbf{a}_L = [2, 1, 0.1] \to \hat{\mathbf{y}} = [0.659, 0.242, 0.099]$; if $\ell = 1$: $\mathscr{L} = -\ln 0.659 = 0.417$.
- $\mathbf{a}_L = [1, 2, 3] \to \hat{\mathbf{y}} = [0.090, 0.245, 0.665]$; $\ell = 3$: $\mathscr{L} = 0.408$; if instead $\ell = 1$: $\mathscr{L} = 2.408$ (wrong-class confidence ⇒ large loss).

Module 4.4: Backpropagation (Intuition)

A. Core Concept

- Goal: $\partial \mathscr{L} / \partial W_{112}$ (any single weight) for gradient descent/SGD.
- **Deep but thin network** (one neuron per layer): plain chain rule,

$$\frac{\partial \mathscr{L}}{\partial W_{111}} = \frac{\partial \mathscr{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{L11}} \frac{\partial a_{L11}}{\partial h_{21}} \frac{\partial h_{21}}{\partial a_{21}} \frac{\partial a_{21}}{\partial h_{11}} \frac{\partial h_{11}}{\partial a_{11}} \frac{\partial a_{11}}{\partial W_{111}}$$

which can be **compressed**: $\frac{\partial \mathscr{L}}{\partial W_{111}} = \frac{\partial \mathscr{L}}{\partial h_{11}} \frac{\partial h_{11}}{\partial W_{111}}$, $\frac{\partial \mathscr{L}}{\partial W_{211}} = \frac{\partial \mathscr{L}}{\partial h_{21}} \frac{\partial h_{21}}{\partial W_{211}}$, $\frac{\partial \mathscr{L}}{\partial W_{L11}} = \frac{\partial \mathscr{L}}{\partial a_{L1}} \frac{\partial a_{L1}}{\partial W_{L11}}$ → **reuse** of already computed partial products.

- **"Responsibility" story**: loss at output → output layer says "I'm only as good as the hidden layer and weights below me" ($\hat{y} = O(W_L \mathbf{h}_{L-1} + \mathbf{b}_L)$) → $W_L, \mathbf{b}_L$ take full responsibility, $\mathbf{h}_{L-1}$ blames its pre-activation, which blames the layer and weights below … ⇒ responsibility lies with **all** parameters. Instead of talking to weights directly, talk to them **through the output and hidden layers** — exactly what the chain rule does:

$$\underbrace{\frac{\partial \mathscr{L}}{\partial W_{111}}}_{\text{weight directly}} = \underbrace{\frac{\partial \mathscr{L}}{\partial \hat{\mathbf{y}}}}_{\text{output layer}} \underbrace{\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{a}_3} \frac{\partial \mathbf{a}_3}{\partial \mathbf{h}_2}}_{\text{prev. hidden}} \underbrace{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1}}_{\text{prev. hidden}} \underbrace{\frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W_{111}}}_{\text{weights}}$$

- **Roadmap**: gradient w.r.t. (1) output units, (2) hidden units, (3) weights and biases. Focus: **cross-entropy loss + softmax output**.

Module 4.5: Gradients w.r.t. Output Units

D. Results

1. W.r.t. output $\hat{y}_i$ (with $\mathscr{L} = -\log \hat{y}_\ell$):

$$\frac{\partial \mathscr{L}}{\partial \hat{y}_i} = -\frac{1_{(i=\ell)}}{\hat{y}_\ell} \qquad \Rightarrow \qquad \nabla_{\hat{\mathbf{y}}} \mathscr{L}(\theta) = -\frac{1}{\hat{y}_\ell}\, \mathbf{e}(\ell)$$

$\mathbf{e}(\ell)$ = k-dim one-hot vector with 1 at position ℓ. Only the true-class output gets a nonzero gradient. 2. W.r.t. pre-activation $a_{L,i}$ — $\hat{y}_\ell$ depends on **every** $a_{L,i}$ through the softmax denominator. Using the quotient rule $\frac{\partial}{\partial x}\frac{g(x)}{h(x)} = \frac{\partial g}{\partial x}\frac{1}{h} - \frac{g}{h^2}\frac{\partial h}{\partial x}$:

$$\frac{\partial \mathscr{L}}{\partial a_{L,i}} = -\frac{1}{\hat{y}_\ell}\left(1_{(\ell=i)}\hat{y}_\ell - \hat{y}_\ell \hat{y}_i\right) = -\left(1_{(\ell=i)} - \hat{y}_i\right)$$

$$\boxed{\nabla_{\mathbf{a}_L} \mathscr{L}(\theta) = -\left(\mathbf{e}(\ell) - \hat{\mathbf{y}}\right) = \hat{\mathbf{y}} - \mathbf{e}(\ell)} \quad \in \mathbb{R}^k$$

- ⚠ Slide note: in the third line of the derivation (slide 36) the softmax denominator is printed as $\sum_{i'} \exp(\mathbf{a}_L)_\ell$; it should be $\sum_{i'} \exp(\mathbf{a}_L)_{i'}$ (the next lines use the correct form).

C. Insights

- Intermediate property: $\frac{\partial \hat{y}_\ell}{\partial a_{L,i}} = \hat{y}_\ell(1_{(\ell=i)} - \hat{y}_i)$ (softmax derivative). (Additional clarification) The same formula holds for any output j: $\frac{\partial \hat{y}_j}{\partial a_{L,i}} = \hat{y}_j(\delta_{ij} - \hat{y}_i)$. The diagonal entries $\hat{y}_i(1-\hat{y}_i)$ are positive and the off-diagonal entries $-\hat{y}_i\hat{y}_j$ are negative, so the Jacobian is symmetric. E.g. $\hat{\mathbf{y}} = [0.2, 0.3, 0.5]$: $\partial \hat{y}_1/\partial a_{L,1} = 0.16$, $\partial \hat{y}_1/\partial a_{L,2} = -0.06$.
- True-class component: $\hat{y}_\ell - 1 \leq 0$ (negative ⇒ gradient descent **increases** $a_{L,\ell}$); other components $\hat{y}_i \geq 0$ (descent **decreases** wrong-class scores).
- (Additional clarification) Components of $\nabla_{\mathbf{a}_L}\mathscr{L}$ sum to 0; magnitude is bounded (each entry in $[-1, 1]$), unlike $\nabla_{\hat{\mathbf{y}}}$ which blows up as $\hat{y}_\ell \to 0$.
- **Numerical**: $\mathbf{a}_L = [1, 2, 3]$, $\ell = 3$: $\nabla_{\mathbf{a}_L} = [0.090,\ 0.245,\ 0.665 - 1] = [0.090, 0.245, -0.335]$.
- Trap: $\nabla_{\mathbf{a}_L}\mathscr{L} = \mathbf{e}(\ell) - \hat{\mathbf{y}}$ (sign flipped) is **wrong**; correct is $-(\mathbf{e}(\ell) - \hat{\mathbf{y}})$.

Module 4.6: Gradients w.r.t. Hidden Units

A. Chain rule along multiple paths

If $p(z)$ can be written via intermediate results $q_m(z)$:

$$\frac{\partial p(z)}{\partial z} = \sum_m \frac{\partial p(z)}{\partial q_m(z)} \frac{\partial q_m(z)}{\partial z}$$

Here $p = \mathscr{L}(\theta)$, $z = h_{ij}$, $q_m = a_{i+1,m}$ (each h_{ij} feeds **all** units of the next layer).

D. Results

$$\frac{\partial \mathscr{L}}{\partial h_{ij}} = \sum_{m=1}^{k} \frac{\partial \mathscr{L}}{\partial a_{i+1,m}} W_{i+1,m,j} = (W_{i+1,\cdot,j})^T \nabla_{\mathbf{a}_{i+1}} \mathscr{L}$$

($W_{i+1,\cdot,j}$ = j-th **column** of W_{i+1}.) Stacking over j:

$$\boxed{\nabla_{\mathbf{h}_i} \mathscr{L}(\theta) = (W_{i+1})^T \nabla_{\mathbf{a}_{i+1}} \mathscr{L}(\theta)}$$

Then through the activation ($h_{ij} = g(a_{ij})$, element-wise):

$$\frac{\partial \mathscr{L}}{\partial a_{ij}} = \frac{\partial \mathscr{L}}{\partial h_{ij}} g'(a_{ij}) \qquad \Rightarrow \qquad \boxed{\nabla_{\mathbf{a}_i} \mathscr{L}(\theta) = \nabla_{\mathbf{h}_i} \mathscr{L}(\theta) \odot [\dots, g'(a_{ij}), \dots]}$$

- $\odot$ = element-wise (Hadamard) product. $\nabla_{\mathbf{a}_{i+1}}$ for $i < L-1$ is obtained **recursively** from the layer above (this is why we go backwards).
- ⚠ Slide note: the sum is written $\sum_{m=1}^{k}$ — here k just means "number of units in layer $i+1$" (it is n for a hidden layer $i+1$; it is k only when $i+1 = L$). The slide also writes $\mathbf{a}_{i+1} = W_{i+1} h_{ij} + \mathbf{b}_{i+1}$ loosely; it means $\mathbf{a}_{i+1} = W_{i+1} \mathbf{h}_i + \mathbf{b}_{i+1}$.

F. Traps

- Uses W_{i+1}^T (transpose of the weight **above** layer i), not W_i.
- $\nabla_{\mathbf{a}_i}$ needs g' evaluated at the **pre-activation** $\mathbf{a}_i$ (stored from the forward pass), and it is an element-wise product, **not** a matrix product.
- Shapes: $\nabla_{\mathbf{h}_i}, \nabla_{\mathbf{a}_i}$ have the size of layer i.

Module 4.7: Gradients w.r.t. Parameters

D. Results

Since $\mathbf{a}_k = \mathbf{b}_k + W_k \mathbf{h}_{k-1}$, $\frac{\partial a_{ki}}{\partial W_{kij}} = h_{k-1,j}$, so

$$\frac{\partial \mathscr{L}}{\partial W_{kij}} = \frac{\partial \mathscr{L}}{\partial a_{ki}} h_{k-1,j} \qquad \Rightarrow \qquad \boxed{\nabla_{W_k} \mathscr{L}(\theta) = \nabla_{\mathbf{a}_k} \mathscr{L}(\theta) \cdot \mathbf{h}_{k-1}^T} \text{ (outer product)}$$

$$\frac{\partial \mathscr{L}}{\partial b_{ki}} = \frac{\partial \mathscr{L}}{\partial a_{ki}} \qquad \Rightarrow \qquad \boxed{\nabla_{\mathbf{b}_k} \mathscr{L}(\theta) = \nabla_{\mathbf{a}_k} \mathscr{L}(\theta)}$$

- Slide shows the 3×3 case: entry (i, j) of ∇_{W_k} is $\frac{\partial \mathscr{L}}{\partial a_{ki}} h_{k-1,j}$.
- ⚠ Slide note: the bias-gradient vector on the slide is written with denominators "a_{k1}" (missing ∂) — typo for ∂a_{k1}.

C. Insights

- Gradient of a weight = (error signal at its **destination** unit) × (activation at its **source** unit). If the source activation $h_{k-1,j} = 0$, all weights leaving that unit get zero gradient (same idea as sparse features in L5).
- ∇_{W_k} is **rank-1** (outer product) for a single example. (Additional clarification)

Module 4.8: Backpropagation — Pseudocode

Algorithm (slides)

```
gradient_descent():
  t <- 0; max_iterations <- 1000; initialize θ0 = [W1..WL, b1..bL]
  while t++ < max_iterations:
      h1..h(L-1), a1..aL, ŷ = forward_propagation(θt)
      ∇θt = backward_propagation(h1..h(L-1), a1..aL, y, ŷ)
      θ(t+1) <- θt − η ∇θt

forward_propagation(θ):
  for k = 1 to L-1:  a_k = b_k + W_k h_(k-1);  h_k = g(a_k)
  a_L = b_L + W_L h_(L-1);  ŷ = O(a_L)

back_propagation(h1..h(L-1), a1..aL, y, ŷ):
  ∇aL L = -(e(y) − ŷ)                          // output gradient
  for k = L to 1:
      ∇Wk L = ∇ak L · h(k-1)^T                  // params
      ∇bk L = ∇ak L
      ∇h(k-1) L = Wk^T (∇ak L)                  // layer below
      ∇a(k-1) L = ∇h(k-1) L ⊙ [..., g'(a(k-1,j)), ...]   // pre-activation below
```

- The forward pass must come first: backward pass **reuses stored** $\mathbf{h}_i, \mathbf{a}_i, \hat{\mathbf{y}}$.
- ⚠ Slide note: pseudocode writes $\mathbf{e}(y)$ where the derivation used $\mathbf{e}(\ell)$ — same thing (one-hot of the true label).
- (Additional clarification) At $k = 1$ the last two lines compute gradients w.r.t. $\mathbf{h}_0 = \mathbf{x}$, which are not needed for learning, and $\mathbf{a}_0$ does not exist. Also, $\nabla_{W_1} = \nabla_{\mathbf{a}_1}\mathbf{h}_0^T$ needs $\mathbf{h}_0 = \mathbf{x}$, although the argument list only shows $\mathbf{h}_1, \ldots, \mathbf{h}_{L-1}$.
- Summary of the pieces (slide 52): $\nabla_{\mathbf{a}_L}$ (output layer); $\nabla_{\mathbf{h}_k}, \nabla_{\mathbf{a}_k}$ for hidden layers $1 \le k < L$; $\nabla_{W_k}, \nabla_{\mathbf{b}_k}$ for all layers $1 \le k \le L$.

Shapes of every quantity (layer k has d_k units; $d_0 = n$, $d_L = k$; slide: $d_1 = \cdots = d_{L-1} = n$)

Quantity	Shape	Formula
$\mathbf{a}_k, \mathbf{h}_k, \nabla_{\mathbf{a}_k}, \nabla_{\mathbf{h}_k}$	$d_k \times 1$	—
$\nabla_{\hat{\mathbf{y}}}$, $\nabla_{\mathbf{a}_L}$	$k \times 1$	$-\frac{1}{\hat{y}_\ell}\mathbf{e}(\ell)$, $\hat{\mathbf{y}} - \mathbf{e}(\ell)$
W_k, ∇_{W_k}	$d_k \times d_{k-1}$ (slide writes W_L as $n \times k$, see ⚠ 4.1)	$\nabla_{\mathbf{a}_k}\mathbf{h}_{k-1}^T$: $(d_k \times 1)(1 \times d_{k-1})$
$\mathbf{b}_k$, $\nabla_{\mathbf{b}_k}$	$d_k \times 1$	$\nabla_{\mathbf{a}_k}$
$\nabla_{\mathbf{h}_{k-1}}$	$d_{k-1} \times 1$	$W_k^T\nabla_{\mathbf{a}_k}$: $(d_{k-1} \times d_k)(d_k \times 1)$

E. Worked example (verified with python, incl. finite-difference check)

$\mathbf{x} = [1, 1]^T$, $W_1 = \begin{bmatrix} 0.5 & -0.5 \\ 1 & 0 \end{bmatrix}$, $W_2 = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}$, biases 0, $g = \sigma$, O=softmax, true class $\ell = 1$.

- Forward: $\mathbf{a}_1 = [0, 1]$, $\mathbf{h}_1 = [0.5, 0.7311]$, $\mathbf{a}_2 = [-0.2311, 1]$, $\hat{\mathbf{y}} = [0.226, 0.774]$, $\mathscr{L} = -\ln 0.226 = 1.487$.
- $\nabla_{\mathbf{a}_2} = \hat{\mathbf{y}} - \mathbf{e}(1) = [-0.774, 0.774]$
- $\nabla_{W_2} = \nabla_{\mathbf{a}_2}\mathbf{h}_1^T = \begin{bmatrix} -0.387 & -0.566 \\ 0.387 & 0.566 \end{bmatrix}$, $\nabla_{\mathbf{b}_2} = [-0.774, 0.774]$
- $\nabla_{\mathbf{h}_1} = W_2^T\nabla_{\mathbf{a}_2} = [0.774, 0.774]$; $g'(\mathbf{a}_1) = \sigma(1-\sigma) = [0.25, 0.1966]$

- $\nabla_{\mathbf{a}_1} = [0.1935, 0.1522]$; $\nabla_{W_1} = \nabla_{\mathbf{a}_1}\mathbf{x}^T = \begin{bmatrix} 0.1935 & 0.1935 \\ 0.1522 & 0.1522 \end{bmatrix}$ ✓ matches numerical gradient.

Module 4.9: Derivatives of Activation Functions

$g(z)$	**Definition**	$g'(z)$	**(Additional clarification) range of g'**
Logistic	$\sigma(z) = \frac{1}{1+e^{-z}}$	$g(z)\,(1 - g(z))$	$(0, 0.25]$, max 0.25 at $z = 0$
tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - (g(z))^2$	$(0, 1]$, max 1 at $z = 0$

- Both derivatives are expressed **in terms of the function value itself** → cheap: reuse $\mathbf{h}$ stored in forward pass.
- Numericals: $\sigma(z) = 0.8 \Rightarrow \sigma' = 0.16$; $\sigma(2) = 0.881,\ \sigma'(2) = 0.105$; $\tanh(z) = 0.5 \Rightarrow \tanh' = 0.75$; $\tanh(1) = 0.762,\ \tanh'(1) = 0.420$.
- Trap: $\sigma'(z) = \sigma(z)(1 + \sigma(z))$ or $\tanh' = 1 - \tanh$ — both wrong. (Additional clarification: small g' when saturated ⇒ gradients shrink as they are multiplied backwards — vanishing gradients, discussed in later lectures.)

Lecture 4 — G. Quick Revision Summary

- $\mathbf{a}_i = \mathbf{b}_i + W_i\mathbf{h}_{i-1}$, $\mathbf{h}_i = g(\mathbf{a}_i)$, $\hat{\mathbf{y}} = O(\mathbf{a}_L)$; $L - 1$ hidden layers; input = layer 0.
- Params per layer $= d_{i-1}d_i + d_i$; slide net: $(L-1)(n^2 + n) + nk + k$.
- Real outputs → linear + squared error; class probabilities → softmax + cross-entropy ($= -\log \hat{y}_\ell$, minimizing it = maximizing log-likelihood).
- Backprop equations:
  - $\nabla_{\mathbf{a}_L} = -(\mathbf{e}(\ell) - \hat{\mathbf{y}})$
  - $\nabla_{W_k} = \nabla_{\mathbf{a}_k}\mathbf{h}_{k-1}^T$, $\nabla_{\mathbf{b}_k} = \nabla_{\mathbf{a}_k}$
  - $\nabla_{\mathbf{h}_{k-1}} = W_k^T\nabla_{\mathbf{a}_k}$, $\nabla_{\mathbf{a}_{k-1}} = \nabla_{\mathbf{h}_{k-1}} \odot g'(\mathbf{a}_{k-1})$
- $\sigma' = \sigma(1 - \sigma)$, $\tanh' = 1 - \tanh^2$.
- **Likely MCQ takeaways**: (1) gradient shapes = parameter shapes; (2) softmax+CE output gradient is $\hat{\mathbf{y}} - \mathbf{e}(\ell)$; (3) hidden gradient uses W^T and element-wise g'; (4) multi-path chain rule = **sum** over paths; (5) bias gradient = pre-activation gradient.

Week 4 · Lecture 5: Gradient Descent and Its Variants (Momentum, NAG, SGD, AdaGrad, RMSProp, Adam)

Running toy example (whole lecture): single sigmoid neuron $f(x) = \frac{1}{1+e^{-(w \cdot x + b)}}$, two training points $(x, y) = (0.5, 0.2)$ and $(2.5, 0.9)$; loss $\mathscr{L}(w, b) = \frac{1}{2}\sum_i (y_i - f(x_i))^2$. Training = find w^*, b^* such that the sigmoid passes through both points.

Module 5.1: Learning Parameters — Guess Work (Infeasible)

- Random guess $w = 0.5, b = 0$: $\mathscr{L} = \frac{1}{2}[(0.9 - f(2.5))^2 + (0.2 - f(0.5))^2] = 0.073$. Want $\mathscr{L} \to 0$.
- Slide's guess table: $(-0.10, 0) \to 0.1481$ (worse); $(0.94, -0.94) \to 0.0214$; $(1.42, -1.73) \to 0.0028$; $(1.65, -2.08) \to 0.0003$; $(1.78, -2.27) \to 0.0000$ — i.e. **increase** w**, decrease** b.
- Plotting $\mathscr{L}(w, b)$ (the **error surface**) and picking the minimum works for 2 params/2 points, but is **intractable** with many points/parameters; and even the plot covers only a finite range ($(-6, 6)$, not $(-\infty, \infty)$). Guess work = random traversal of the error surface → need a principled method.
- ⚠ Slide note: the first slide defines $\mathscr{L} = \sum (y_i - f(x_i))^2$ (no $\frac{1}{2}$), the computation uses $\frac{1}{2}\sum$. Recomputed value for $(0.94, -0.94)$ is 0.0217 (slide 0.0214 — rounding of the displayed w, b).

Module 5.2: Gradient Descent (recap — derivation is Lecture 3)

- Set-up: $\theta = [w, b]$, $\theta_{new} = \theta + \eta \cdot \Delta\theta$. What is the right $\Delta\theta = u$? Taylor series: $\mathscr{L}(\theta + \eta u) = \mathscr{L}(\theta) + \eta\, u^T \nabla \mathscr{L}(\theta) + \frac{\eta^2}{2!} u^T \nabla^2 \mathscr{L}(\theta) u + \ldots$; the $\eta^2, \eta^3, \ldots$ terms are dropped because η is small.
- **Recap:** first-order Taylor: $\mathscr{L}(\theta + \eta u) \approx \mathscr{L}(\theta) + \eta\, u^T \nabla \mathscr{L}(\theta)$ (η small). A move helps iff $u^T \nabla \mathscr{L} < 0$; since $u^T \nabla \mathscr{L} = \|u\| \|\nabla \mathscr{L}\| \cos\beta$, it is most negative at $\beta = 180^\circ$ ⇒ move **opposite to the gradient**:

$$w_{t+1} = w_t - \eta \nabla w_t, \qquad b_{t+1} = b_t - \eta \nabla b_t$$

- Toy-network gradients (one point): $\nabla w = (f(x) - y)\, f(x)(1 - f(x))\, x$, $\nabla b = (f(x) - y)\, f(x)(1 - f(x))$; for multiple points **sum** over points.
- Code on slides: start $w = b = -2$, $\eta = 1.0$, max_epochs $= 1000$; `error()` uses $0.5\,(f(x) - y)^2$ summed over points; vanilla (batch) GD accumulates `dw, db` over **all** points then does one update per epoch.
- **Key behaviour**: update $\propto$ gradient. Steep region (e.g. $f(x) = x^2 + 1$ far from 0) → large gradient → big steps; **gentle region → tiny gradient → tiny steps**. "Irrespective of where we start from, once we hit a surface which has a gentle slope, the progress slows down" (plateaus are the problem).
- E (verified): at $(w, b) = (0.5, 0)$ on the toy data: $\nabla w = -0.0085$, $\nabla b = 0.0679$; one GD step with $\eta = 1$ → $(0.5085, -0.0679)$, loss $0.0731 \to 0.0686$.

Module 5.3: Contours

- **Contour map** = top view of horizontal slices of the 3-D error surface taken at regular intervals of the error; each contour line = points of equal loss.
- **Small distance between contours ⇒ steep slope** along that direction; **large distance ⇒ gentle slope**.
- GD on contours: steps are short where contours are far apart (gentle) and long where contours are dense (steep). (Additional clarification) GD steps are perpendicular to contour lines.
- Trap: dense contours ≠ "low error" — density signals steepness, not loss value.

- The slides include "Guess the 3d surface" exercises, which pair contour plots with their 3-D surfaces. The slides then redraw the same GD trajectory (start $(w, b) = (-2, -2)$) on the toy problem's contour plot, so the 3-D story can be read in 2-D. Steps are small wherever the contours are far apart (gentle slope).

Module 5.4: Momentum-based Gradient Descent

A. Core Concept

- Problem: GD takes a long time on gentle slopes (small gradients).
- Intuition: if repeatedly asked to move in the same direction, gain confidence and take bigger steps — like a **ball gaining momentum** rolling down a slope. Look at the **history of updates** in addition to the current gradient.

D. Update rule

$$\text{update}_t = \gamma \cdot \text{update}_{t-1} + \eta \nabla w_t, \qquad w_{t+1} = w_t - \text{update}_t, \qquad \text{update}_0 = 0$$

Unrolled (exponentially weighted sum of past gradients):

$$\text{update}_t = \gamma^{t-1}\eta \nabla w_1 + \gamma^{t-2}\eta \nabla w_2 + \cdots + \eta \nabla w_t$$

- Symbols: γ = momentum coefficient, (Additional clarification) normally $0 \leq \gamma < 1$. Code on slides: $\gamma = 0.9$, $\eta = 1.0$, `prev_v_w = prev_v_b = 0`. The same rule applies to b.
- $\gamma = 0 \Rightarrow$ vanilla GD. Older gradients are weighted by smaller powers of γ.
- ⚠ Slide note: the slide's general expression writes "$\gamma^{t-2} \cdot \eta \nabla w_1$" for the second term; it should be $\gamma^{t-2} \cdot \eta \nabla w_2$.

C. Properties

- Even in gentle regions momentum takes large steps — "momentum carries it along".
- **Flip side**: on a surface where error is high on either side of a narrow minima valley, momentum **overshoots — oscillates in and out of the valley, many u-turns** before converging.
- Despite u-turns it **still converges faster than vanilla GD**: after 100 iterations momentum reached error 0.00001 while vanilla GD was stuck at 0.36.
- The slides also show this in 3-D (slide 43, a figure). The trajectory overshoots across the narrow valley of the error surface. A second plot shows the sigmoid fitted at successive iterations swinging back and forth across the data points before it settles. The modified data points on that plot (one is labelled $(0.35, 0.5)$; the labels of the others overlap) could not be read reliably.

E. Numericals (verified)

- Constant gradient $\nabla w = 1$, $\eta = 0.1$, $\gamma = 0.9$: updates $0.1, 0.19, 0.271, 0.3439, 0.4095, \cdots \to \frac{\eta g}{1-\gamma} = 1.0$ (Additional clarification: steady-state step is $1/(1-\gamma)$ = 10× the GD step for $\gamma = 0.9$).
- $f(w) = w^2$ ($\nabla = 2w$), $w_0 = 1$, $\eta = 0.1, \gamma = 0.9$: GD w: $0.8, 0.64, 0.512$; momentum: update $0.2, 0.34, 0.398$ → w: $0.8, 0.46, 0.062$.

Module 5.5: Nesterov Accelerated Gradient Descent (NAG)

A. Core Concept

- Question: can we reduce momentum's oscillations? Intuition: **"Look before you leap."** We know we will move at least $\gamma \cdot \text{update}_{t-1}$, so compute the gradient at that partially-updated (look-ahead) point instead of at w_t.

D. Update rule

$$w_{\text{look ahead}} = w_t - \gamma \cdot \text{update}_{t-1}$$

$$\text{update}_t = \gamma \cdot \text{update}_{t-1} + \eta \nabla w_{\text{look ahead}}, \qquad w_{t+1} = w_t - \text{update}_t$$

(similar for b_t). Only difference from momentum: **where the gradient is evaluated**.

C. Properties

- Looking ahead lets NAG **correct its course quicker** than momentum ⇒ **smaller oscillations / shorter u-turns**, smaller chance of escaping the minima valley.
- Code: partial update with $\gamma\cdot$prev_v, compute grads there, then full update ($\gamma = 0.9$, $\eta = 1.0$).

E. Numerical (verified) — $f(w) = w^2$, $w_0 = 1$, $\eta = 0.1$, $\gamma = 0.9$

step	momentum: grad at	update	w	NAG: look-ahead w	grad	update	w
1	1.0	0.2	0.8	1.0	2.0	0.2	0.8
2	0.8	0.34	0.46	$0.8 - 0.9(0.2) = 0.62$	1.24	0.304	0.496
3	0.46	0.398	0.062	$0.496 - 0.9(0.304) = 0.2224$	0.4448	0.3181	0.1779

- Step 1 is identical (update$_0=0$). NAG's updates are smaller as w approaches the minimum (it "sees" the decreasing gradient ahead) → less overshoot.

Module 5.6: Stochastic and Mini-Batch Gradient Descent

A. Core Concept

Variant	Gradient used per update	Properties
Vanilla (batch) GD	sum over **all** N points	True gradient, **no approximation** ⇒ theoretical guarantees hold (each step decreases loss). Flip side: 1 update needs N (e.g. a million) computations ⇒ very slow
Stochastic GD	**one** data point	Updates after every point (a million updates per epoch). Flip side: **approximate (stochastic) gradient — no guarantee each step decreases the loss**; many **oscillations** because of greedy, locally-favourable decisions (points "compete": an update good for one point may harm others). "Like tossing a coin once to estimate P(heads)"
Mini-batch GD	B **(= k) points**	Better stochastic estimates ⇒ fewer oscillations (even $k = 2$ helps; coin tossed k times). Higher k ⇒ more accurate estimates. Typical k = **16, 32, 64**. Oscillations remain as long as the gradient is approximate

- **1 epoch** = one pass over the entire data; **1 step** = one parameter update.
- Slide code: SGD is the same as batch GD except that `w, b` are updated inside the loop over points ($w = b = -2$, $\eta = 1.0$). Mini-batch code: `mini_batch_size = 2`, $\eta = 1.0$. It keeps a `num_points_seen` counter, updates when `num_points_seen % mini_batch_size == 0`, then resets `dw, db` to 0.
- (Additional clarification) The batch-GD "guarantee" that each step decreases the loss relies on the first-order Taylor argument, so it needs a small enough η. With a large η (e.g. 10, Module 5.7) even true-gradient steps overshoot.

Algorithm	# steps in 1 epoch
Vanilla (batch) GD	1
Stochastic GD	N
Mini-batch GD	N/B

- Stochastic (and mini-batch) versions of **momentum** and **NAG** exist. Stochastic momentum (red) and NAG (blue) still oscillate, but NAG's relative advantage holds (shorter u-turns), and **both are much faster than plain SGD** (after 60 steps SGD still has high error, momentum/NAG close to convergence).

E. Numericals (verified)

- $N = 1000$, $B = 50$: steps/epoch = batch 1, SGD 1000, mini-batch 20; for 10 epochs: 10, 10000, 200.
- $N = 60000$, $B = 64$: $N/B = 937.5$ — the slide formula is simply N/B; (Additional clarification) when it is not an integer the answer depends on the last partial batch: $\lceil N/B \rceil = 938$ if it is used, $\lfloor N/B \rfloor = 937$ if dropped. Exam questions normally pick divisible numbers.
- Traps: "epoch" ≠ "step"; SGD does **more** updates per epoch than batch GD, but each is noisier; mini-batch with $B = N$ = batch GD, $B = 1$ = SGD.

Module 5.7: Tips for Adjusting Learning Rate and Momentum

C. Learning-rate observations

- Why not just set a **large** η to blow up small gradients on gentle slopes? With $\eta = 10$, in steep regions the already large gradient **blows up further** (overshoots). Want a learning rate that **adapts to the gradient** → adaptive methods (5.9).

Tips

- **Initial learning rate**: try values on a **log scale** $0.0001, 0.001, 0.01, 0.1, 1.0$; run a few epochs with each; then **finer search** around the best (e.g. best 0.1 → try $0.05, 0.2, 0.3$). Disclaimer: heuristics, no clear winner strategy.
- **Annealing (decaying) the learning rate**:

Schedule	Rule	Notes
Step decay	Halve η every 5 epochs, **or** halve η after an epoch if validation error is more than at the end of previous epoch	piecewise constant
Exponential decay	slide: $\eta = \eta_0^{-kt}$	η_0, k hyperparameters, t = step number
$1/t$ decay	$\eta = \frac{\eta_0}{1 + kt}$	η_0, k hyperparameters, t = step number

- ⚠️ **Slide note (exponential decay):** the slide prints $\eta = \eta_0^{-kt}$. Read literally with $\eta_0 < 1$ this *increases* η (e.g. $\eta_0 = 0.1, k = 0.1, t = 10$ gives $0.1^{-1} = 10$). The standard form is $\eta = \eta_0 e^{-kt}$ ($\rightarrow 0.1e^{-1} = 0.0368$). Use $\eta_0 e^{-kt}$ unless an exam question clearly uses the slide's form.
- $1/t$ example: $\eta_0 = 0.1, k = 0.1, t = 10 \Rightarrow \eta = 0.1/2 = 0.05$.
- **Momentum schedule (Sutskever et al., 2013)**:

$$\gamma_t = \min\left(1 - 2^{-1-\log_2(\lfloor t/250 \rfloor + 1)},\ \gamma_{max}\right), \quad \gamma_{max} \in \{0.999, 0.995, 0.99, 0.9, 0\}$$

Since $2^{-1-\log_2 m} = \frac{1}{2m}$: $\gamma_t = 1 - \frac{1}{2(\lfloor t/250 \rfloor + 1)}$ (capped). Values (verified): $t < 250$: 0.5; $t = 250$: 0.75; 500: 0.833; 750: 0.875; 1750: 0.9375 — momentum **increases** over time towards γ_{max}.

Module 5.8: Line Search

- At each step, update w using **several values of** η (code: $[0.1, 0.5, 1.0, 5.0, 10.0]$) and **retain the updated** w **giving the lowest loss** — i.e. use the best available η at every step.
- **Flip side**: many more computations per step (one loss evaluation per candidate η). (Revisited with second-order methods.)
- In action: **converges faster than vanilla GD**; shows some oscillations, but **different** in nature from momentum/NAG oscillations.

Module 5.9: Gradient Descent with Adaptive Learning Rate

A. Motivation: sparse features

- Neuron with 4 inputs: $\nabla w_i = (f(\mathbf{x}) - y) f(\mathbf{x})(1 - f(\mathbf{x}))\, x_i$. If feature x_2 is **sparse** (0 for most inputs), $\nabla w_2 = 0$ for most inputs $\Rightarrow$ w_2 gets **too few updates**. If x_2 is sparse **and important**, we want to take its updates more seriously $\Rightarrow$ **a different learning rate per parameter**, depending on feature frequency.
- Toy experiment: 100 random (x, y) pairs, x set to 0 for ~80% $\Rightarrow$ the feature for w is sparse (b's "input" is always 1, so it can't be made sparse). GD (black), momentum (red), NAG (blue) initially move **mainly along the vertical** b**-axis**, very little along w: b is dense, w is sparse. Such sparsity is common in large networks with 1000s of input features.

AdaGrad

- Intuition: **decay the learning rate for each parameter in proportion to its update history** (more updates $\Rightarrow$ more decay).

$$v_t = v_{t-1} + (\nabla w_t)^2, \qquad w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t$$

(similar for b_t; code: $\eta = 0.1$, $\epsilon = 10^{-8}$, $v_0 = 0$).

- Effects: sparse w has small v_t $\Rightarrow$ **higher effective learning rate** $\Rightarrow$ larger updates despite sparsity; dense b accumulates large v_t $\Rightarrow$ its effective learning rate decreases (growing denominator).
- Slide: "in practice, this does not work so well if we **remove the square root** from the denominator."
- **Flip side**: v_t only grows (sum of squares, monotonic) $\Rightarrow$ the effective learning rate for b **decays so much that** b **stops updating** — AdaGrad **got stuck near convergence**, unable to move in the vertical (b) direction.

RMSProp

- Intuition: AdaGrad decays the learning rate too aggressively; **decay the denominator** (exponentially weighted average instead of sum) to prevent its rapid growth.

$$v_t = \beta\, v_{t-1} + (1-\beta)(\nabla w_t)^2, \qquad w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}}\,\nabla w_t$$

(code: $\eta = 0.1$, $\epsilon = 10^{-8}$, $\beta = 0.9$). Overcomes AdaGrad's stuck-near-convergence problem by being **less aggressive on the decay**.

Adam (Adaptive Moments)

- Intuition: do everything RMSProp does (fix AdaGrad's decay) **plus** use a **cumulative history of the gradients** (momentum-like first moment).

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1)\nabla w_t & v_t &= \beta_2 v_{t-1} + (1-\beta_2)(\nabla w_t)^2 \\ \hat{m}_t &= \frac{m_t}{1-\beta_1^t} & \hat{v}_t &= \frac{v_t}{1-\beta_2^t} \\ w_{t+1} &= w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}}\,\hat{m}_t \end{aligned}$$

- In practice $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ (code: $\eta = 0.1$). Taking a cumulative history gives a **speed-up** over RMSProp on the toy problem.
- (Additional clarification) The slides put ϵ **inside** the square root, $\sqrt{v_t + \epsilon}$; many references (incl. the original Adam paper) write $\sqrt{v_t} + \epsilon$. Both only guard against division by zero. With $\epsilon = 10^{-8}$ inside the root the floor is effectively $\sqrt{\epsilon} = 10^{-4}$, which matters only when v_t is tiny.

Why bias correction? (derivation result)

- m_t is an exponentially moving average meant to estimate $\mathbb{E}[\nabla w_t]$ (rely on the overall behaviour of gradients, not one point estimate). With $m_0 = 0$ and $g_t = \nabla w_t$:

$$m_t = (1-\beta)\sum_{i=1}^{t} \beta^{t-i} g_i$$

- **Assumption:** all g_i come from the same distribution, $\mathbb{E}[g_i] = \mathbb{E}[g]$. Then (GP sum with ratio β):

$$\mathbb{E}[m_t] = \mathbb{E}[g](1-\beta^t) \quad \Rightarrow \quad \mathbb{E}\Big[\frac{m_t}{1-\beta^t}\Big] = \mathbb{E}[\hat{m}_t] = \mathbb{E}[g]$$

- Meaning: because $m_0 = 0$, m_t is **biased towards 0** early on (at $t = 1$, $\beta_1 = 0.9$: $\mathbb{E}[m_1] = 0.1\,\mathbb{E}[g]$). Dividing by $1-\beta^t$ removes the bias; as $t \to \infty$, $1-\beta^t \to 1$ and the correction vanishes. Same argument for v_t with β_2 ($1 - 0.999^t$: 0.001 at $t = 1$, 0.095 at $t = 100$, 0.632 at $t = 1000$ — the v correction matters for much longer).

E. Numericals (verified with python)

AdaGrad vs RMSProp on sparse vs dense gradients ($\eta = 0.1$, $\epsilon = 10^{-8}$; $\nabla w = [0, 0, 1, 0]$ sparse, $\nabla b = [1, 1, 1, 1]$ dense):

t	AdaGrad v_b	AdaGrad step b	AdaGrad step w	RMSProp ($\beta = 0.9$) v_b	RMSProp step b	RMSProp step w
1	1	0.1	0	0.1	0.316	0
2	2	0.0707	0	0.19	0.229	0

t	**AdaGrad v_b**	**AdaGrad step b**	**AdaGrad step w**	**RMSProp ($\beta = 0.9$) v_b**	**RMSProp step b**	**RMSProp step w**
3	3	0.0577	**0.1** ($v_w = 1$)	0.271	0.192	**0.316** ($v_w = 0.1$)
4	4	0.05	0	0.344	0.171	0

- AdaGrad: the rare w gradient at $t = 3$ gets the full $\eta = 0.1$ step while dense b's step has decayed to 0.0577. After 100 unit gradients, b's effective rate $= 0.1/\sqrt{100} = 0.01$ and keeps falling ($\propto 1/\sqrt{t}$).
- RMSProp: $v_b \to 1$ (for constant $g = 1$), so b's effective rate settles at $\eta = 0.1$ instead of decaying to 0. (Additional clarification: early RMSProp steps exceed η because v_t starts at 0 and has no bias correction — the same issue Adam corrects.)

Adam, two steps ($w_0 = 1$, $\eta = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, gradients $g_1 = 0.5$, $g_2 = 0.3$):

t	m_t	v_t	$\hat{m}_t$	$\hat{v}_t$	**step $\eta\hat{m}_t/\sqrt{\hat{v}_t}$**	w_t
1	0.05	0.00025	0.5	0.25	0.1	0.9
2	0.075	0.00033975	0.3947	0.1700	0.0957	0.8043

- Shortcut: at $t = 1$, $\hat{m}_1 = g_1$, $\hat{v}_1 = g_1^2 \Rightarrow$ **first step** $= \eta \cdot \text{sign}(g_1)$**, independent of gradient scale**. Without bias correction the steps would have been 0.316 and 0.407 (since $m/\sqrt{v}$ is distorted by the different $(1 - \beta_1)$ vs $\sqrt{1 - \beta_2}$ scalings).

Which algorithm to use in practice? (slide)

- **Adam** seems to be more or less the **default choice** ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$).
- Supposed to be robust to initial learning rate, but for **sequence generation** problems $\eta = 0.001, 0.0001$ works best.
- Many papers report **SGD with momentum (Nesterov or classical) + simple annealing schedule** also works well (typically starting $\eta = 0.001, 0.0001$ for sequence generation).
- "Adam might just be the best choice overall!!" — but some recent work suggests Adam **may not converge in some cases**. (Additional clarification: Reddi et al., 2018, "On the convergence of Adam and beyond" → AMSGrad.)

F. Comparison table & traps

Method	State kept	Update	Fixes	Weakness
GD	—	$w - \eta\nabla w$	—	slow on gentle slopes; batch = slow per step
Momentum	update$_{t-1}$	$\gamma u_{t-1} + \eta\nabla w_t$	gentle slopes	overshoots, oscillates in valleys
NAG	update$_{t-1}$	gradient at $w_t - \gamma u_{t-1}$	momentum's oscillations	still some oscillation
Line search	—	best η from a set each step	fixed-η problems	many loss evaluations per step
AdaGrad	$v_t = \sum g^2$	$\eta g/\sqrt{v_t + \epsilon}$	sparse features (per-parameter lr)	lr decays monotonically → stalls (frequent params stop updating)
RMSProp	EMA of g^2	$\eta g/\sqrt{v_t + \epsilon}$	AdaGrad's aggressive decay	no cumulative gradient history (what Adam adds)
Adam	EMA of g and g^2 + bias correction	$\eta\hat{m}_t/\sqrt{\hat{v}_t + \epsilon}$	combines momentum + RMSProp	may fail to converge in some cases

- AdaGrad's v_t is a **sum** (never decreases); RMSProp/Adam's v_t is an **exponential moving average** (can decrease).
- Only **Adam** (in these slides) has bias correction; RMSProp has none.
- In Adam, β_1 goes with the **gradient** (m_t), β_2 with the **squared gradient** (v_t). Swapping them is a common distractor.
- Adaptive methods give **per-parameter** learning rates; momentum/NAG use one global η.
- Bias correction makes $\mathbb{E}[\hat{m}_t] = \mathbb{E}[g]$ under the assumption that gradients are identically distributed; it matters most for **small** t.
- Momentum/NAG can be combined with SGD or mini-batch; oscillations from noise remain as long as the gradient is approximate.

Lecture 5 — G. Quick Revision Summary

- GD: $w_{t+1} = w_t - \eta \nabla w_t$; step $\propto$ gradient ⇒ slow on plateaus. Contours: dense = steep.
- Momentum: $u_t = \gamma u_{t-1} + \eta \nabla w_t$, $w_{t+1} = w_t - u_t$; NAG: gradient at $w_t - \gamma u_{t-1}$ ⇒ fewer/shorter u-turns.
- Steps per epoch: batch 1, SGD N, mini-batch N/B (typical B 16/32/64).
- Annealing: step (halve every 5 epochs / when val error rises), exponential, $1/t$: $\eta_0/(1+kt)$; initial η via log-scale search.
- AdaGrad $v_t = v_{t-1} + g_t^2$; RMSProp $v_t = \beta v_{t-1} + (1-\beta) g_t^2$; Adam m, v EMAs + bias correction ($\hat{m}_t = m_t/(1-\beta_1^t)$, $\hat{v}_t = v_t/(1-\beta_2^t)$), $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$.
- **Likely MCQ takeaways**: (1) SGD gives no per-step guarantee of loss decrease, batch GD does (true gradient); (2) AdaGrad helps sparse features but its lr decays to ~0; RMSProp fixes this with an EMA; (3) Adam = RMSProp + momentum + bias correction; (4) Adam's first bias-corrected step has magnitude ≈ η; (5) momentum still converges faster than vanilla GD despite oscillations.

Overview: Lectures 6–7: Linear Algebra, PCA, SVD, Autoencoders

Lectures covered / modules:

- **Lecture 6**: Eigenvalues, eigenvectors, eigenvalue decomposition, PCA, SVD
  - 6.1 Eigenvalues and eigenvectors (incl. dominant eigenvalue, stochastic matrices, the behaviour of $A^n x_0$)
  - 6.2 Linear algebra: basic definitions (basis, linear independence, orthonormal basis)
  - 6.3 Eigenvalue decomposition (EVD, diagonalisation, symmetric case, the $\max x^T A x$ theorem)
  - 6.4 PCA, Interpretation 1 (decorrelate: diagonalise the covariance)
  - 6.5 PCA, Interpretation 2 (minimise reconstruction error)
  - 6.6 PCA, Interpretation 3 (maximise variance)
  - 6.7 PCA practical example (eigenfaces)
  - 6.8 Singular Value Decomposition
- **Lecture 7**: Autoencoders
  - 7.1 Introduction (encoder/decoder, under-/over-complete, choice of f, g and the loss, gradients)
  - 7.2 Link between PCA and autoencoders
  - 7.3 Regularization in autoencoders (motivation, L2, tied weights)
  - 7.4 Denoising autoencoders (+ visualising filters, MNIST application)
  - 7.5 Sparse autoencoders
  - 7.6 Contractive autoencoders
  - 7.7 Summary

> Cross-reference: general regularization (L2, noise injection, dataset augmentation, early stopping, etc.) is in **Lecture 8** notes. The denoising AE and the AE regularizers here are special cases of those ideas.

Week 5 · Lecture 6: Eigenvalues, Eigenvectors, EVD, PCA, SVD

6.1 Eigenvalues and Eigenvectors

A. Core Concept

- When a matrix "hits" a vector, the vector is **transformed**: its direction changes ("it strays from its path") and it may be scaled.
  - Slide example: $A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$, $x = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$ gives $Ax = \begin{bmatrix} 7 \\ 5 \end{bmatrix}$, so the direction changes.
- **Eigenvectors** are special vectors of a *square* matrix that "refuse to stray from their path". They are only scaled:

$$Ax = \lambda x$$

- Slide example: $x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ gives $Ax = \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3x$, so $\lambda = 3$.
- Why eigenvectors matter: many properties of a matrix can be analysed through its eigenvalues (e.g. spectral graph theory). In this course they are used for:
  1. the long-run behaviour of $A^n x_0$ (steady state, explode, vanish);
  2. a convenient basis, used in PCA.

B. Terminology

Term	Meaning
Eigenvector x / eigenvalue λ	$Ax = \lambda x$, with $x \neq 0$ and A square
Dominant eigenvalue	λ_1 with $\lvert\lambda_1\rvert \geq \lvert\lambda_i\rvert$ for $i = 2, \ldots, n$ (largest **in absolute value**)
Dominant eigenvector e_d	The eigenvector of the dominant eigenvalue λ_d
Stochastic matrix (slide definition)	All entries positive, and **each column sums to 1**

⚠ Slide note (p. 8): the definition says "Let $\lambda_1, \ldots, \lambda_n$ be the **eigenvectors**". This should read *eigenvalues*.

C. Properties and theoretical insights

- **Restaurant (Markov chain) example.** On day 0, k_1 students eat Chinese and k_2 eat Mexican. Each day:
  - a fraction p of the Chinese eaters stay with Chinese, and $1 - p$ switch to Mexican;
  - a fraction q of the Mexican eaters stay with Mexican, and $1 - q$ switch to Chinese.

  This gives

$$v^{(1)} = \begin{bmatrix} p & 1-q \\ 1-p & q \end{bmatrix} \begin{bmatrix} k_1 \\ k_2 \end{bmatrix} = Mv^{(0)}, \qquad v^{(n)} = M^n v^{(0)}$$

M is a stochastic matrix because its columns sum to 1.

- **Theorem:** the largest (dominant) eigenvalue of a stochastic matrix is **1**.

- **Theorem (stated loosely, as the slide admits):** if A ($n \times n$) has a dominant eigenvalue, then $Av_0, A^2v_0, \ldots$ approaches **a multiple of the dominant eigenvector**.
- Consequence for the restaurants: once $v^{(n)} = ke_d$,

$$v^{(n+1)} = M(ke_d) = k\lambda_d e_d = ke_d \quad (\lambda_d = 1)$$

So the populations become **constant**: a steady state.

- **General square A.** Once $A^p x_0 = ke_d$, we have $A^{p+n}x_0 = k(\lambda_d)^n e_d$. Therefore:

$\lvert\lambda_d\rvert$	**Behaviour of $x_0, Ax_0, A^2x_0, \ldots$**
> 1	**explodes**
< 1	**vanishes**
$= 1$	reaches a **steady state**

The slide says "we will use this in the course at some point". (Additional clarification: this is the basis of the exploding/vanishing gradient analysis of repeated multiplication by the same matrix, e.g. in RNNs.)

- (Additional clarification) The convergence theorem needs a *strictly* dominant eigenvalue, and v_0 must have a non-zero component along e_d. Repeated application of A (normalising at each step) is the "power iteration" method for finding the dominant eigenvector.

E. Numerical problem-solving

- **Two-state stochastic matrix.** The eigenvalues are 1 and $p + q - 1$. The steady state is proportional to $[1 - q,\ 1 - p]^T$, and the total $k_1 + k_2$ is conserved.
  - Worked example (verified): $p = 0.7$, $q = 0.6$, $v^{(0)} = [100, 0]^T$.
  - Day 1 is $[70, 30]$, day 2 is $[61, 39]$, and the sequence converges to $[57.14,\ 42.86]$, i.e. $100 \cdot [4/7,\ 3/7]$.
  - The second eigenvalue is 0.3.
- **Growth by the dominant eigenvalue.** With $\lambda_d = 0.5$, after 10 more steps the vector is scaled by $0.5^{10} \approx 0.001$ (vanishes). With $\lambda_d = 2$, it is scaled by $2^{10} = 1024$ (explodes).
- **Eigenvalues of** $\begin{bmatrix} a & b \\ b & a \end{bmatrix}$ are $a + b$ (eigenvector $[1, 1]$) and $a - b$ (eigenvector $[1, -1]$). For $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ this gives 3 and -1, so the dominant eigenvalue is 3.

F. MCQ traps

- "Dominant" means largest **absolute** value. Eigenvalues -5 and 3 have dominant eigenvalue -5.
- The slide's stochastic matrix sums to 1 along **columns**, not rows, because it multiplies column vectors from the left.
- A steady state requires $|\lambda_d| = 1$. If $|\lambda_d| < 1$, everything vanishes to zero rather than reaching a non-zero steady state.

6.2 Linear Algebra: Basic Definitions

B. Terminology

- **Linearly independent:** no vector in the set is a linear combination of the others. Equivalently, the only solution of $c_1v_1 + \cdots + c_nv_n = 0$ is $c_1 = \cdots = c_n = 0$.

- **Basis of $\mathbb{R}^n$:** a set of vectors that is linearly independent **and** spans $\mathbb{R}^n$ (every vector is a linear combination of them).
- **Orthonormal basis:** $u_i^T u_j = 0$ for $i \neq j$, and $u_i^T u_i = \|u_i\|^2 = 1$.

C. Properties

- The standard basis $[1,0]^T, [0,1]^T$ is not sacred. *Any* n linearly independent vectors form a basis. Slide example: $[2,3]^T$ and $[5,7]^T$.
- **General basis.** To write $z = \alpha_1 u_1 + \cdots + \alpha_n u_n$, i.e. $z = U\alpha$ with the u_i as columns of U, you solve a linear system by Gaussian elimination. Cost: $O(n^3)$.
- **Orthonormal basis.** Each coefficient is a single dot product:

$$\alpha_i = z^T u_i \ (= |z| \cos\theta)$$

Each costs $O(n)$, so all of them together cost $O(n^2)$. Hence: "An orthogonal basis is the most convenient basis one can hope for."

- **Theorem 1:** eigenvectors of $A \in \mathbb{R}^{n \times n}$ with **distinct eigenvalues** are linearly independent.
- **Theorem 2:** eigenvectors of a **square symmetric** matrix are **orthogonal**. So eigenvectors can form a basis, and for symmetric matrices that basis is orthogonal, which is especially convenient.
- (Additional clarification) A real symmetric matrix also has **real eigenvalues**. Strictly speaking, eigenvectors for *distinct* eigenvalues are orthogonal. If an eigenvalue repeats, orthogonal eigenvectors can still be *chosen*.

E. Numerical example

- Express $[1,1]^T$ in the basis $\{[2,3]^T, [5,7]^T\}$. Solve $2x_1 + 5x_2 = 1$ and $3x_1 + 7x_2 = 1$, which gives $x_1 = -2$, $x_2 = 1$ (verified).
- With an orthonormal basis $u_1 = \frac{1}{\sqrt{2}}[1,1]$ and $u_2 = \frac{1}{\sqrt{2}}[-1,1]$, the point $z = [3.3, 3]$ has $\alpha_1 = z^T u_1 = 6.3/\sqrt{2}$ and $\alpha_2 = -0.3/\sqrt{2}$. These come straight from dot products; no system needs solving.

F. Traps

- $\alpha_i = z^T u_i$ holds **only** when the basis is orthonormal. For a general basis you must solve $U\alpha = z$.
- Distinct eigenvalues are **sufficient** for linear independence of the eigenvectors, but not necessary.

6.3 Eigenvalue Decomposition (EVD)

A. Core Concept

Put the eigenvectors $u_1, \ldots, u_n$ as the columns of U, and let $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$. Then

$$AU = U\Lambda$$

D. Formulas

Formula	Conditions	Interpretation
$A = U\Lambda U^{-1}$ (EVD)	U^{-1} exists, i.e. A has n **linearly independent eigenvectors** (a *sufficient* condition is n distinct eigenvalues)	Factorises A into eigenvectors and eigenvalues
$U^{-1}AU = \Lambda$	same	**Diagonalisation** of A

Formula	Conditions	Interpretation
$A = U\Lambda U^T$ with $U^TU = I$	A **symmetric**, eigenvectors normalised ($u_i^T u_i = 1$)	The inverse is just the transpose, so it is cheap to compute
$\max_x x^T Ax$ s.t. $\|x\| = 1$	A square symmetric	Solution is the eigenvector of the **largest** eigenvalue; the max value is $\lambda_{\max}$
$\min_x x^T Ax$ s.t. $\|x\| = 1$	A square symmetric	Solution is the eigenvector of the **smallest** eigenvalue; the min value is $\lambda_{\min}$

- **Proof idea (Lagrange multipliers).** Use $L = x^TAx - \lambda(x^Tx - 1)$.
  - Setting $\partial L/\partial x = 2Ax - 2\lambda x = 0$ gives $Ax = \lambda x$.
  - Multiplying by x^T gives $x^TAx = \lambda$.
  - So every critical point is an eigenvector, and the objective value there is its eigenvalue.
- ⚠ Slide note (p. 27): "the critical points of this constrained problem are the eigenvalues of A". The critical points are the **eigenvectors**; the objective **values** at them are the eigenvalues.
- ⚠ Slide note (p. 25): "Given the EVD, $A = U\Sigma U^T$" uses Σ for the eigenvalue matrix (elsewhere Λ). The exercise asks about $x_0, Ax_0, \ldots$. Since $A^n = U\Lambda^n U^T$, you reach the same conclusion as before: explode, vanish or steady state depending on $|\lambda_d|$.

C. "The story so far" (examinable statements)

- Eigenvectors for different eigenvalues are linearly independent.
- Eigenvectors of a square symmetric matrix are orthogonal, so they form a convenient (orthonormal) basis.
- In an eigenbasis, matrix multiplication becomes scalar multiplication (see 6.8):

$$x = \sum \alpha_i v_i \;\Rightarrow\; Ax = \sum \alpha_i \lambda_i v_i$$

E. Numerical example (verified)

- $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ has eigenvalues 3 and 1, with eigenvectors $\frac{1}{\sqrt{2}}[1, 1]$ and $\frac{1}{\sqrt{2}}[1, -1]$.
- $\max_{\|x\|=1} x^TAx = 3$, attained at $\frac{1}{\sqrt{2}}[1, 1]$. For comparison, $x = [1, 0]$ gives only 2.
- The minimum is 1.

F. Traps

- Not every square matrix has an EVD. You need n independent eigenvectors.
- $U^{-1} = U^T$ needs **symmetric** A **plus normalised eigenvectors**.
- The max/min theorem is stated for symmetric A under the **unit-norm** constraint. Without the constraint, x^TAx is unbounded when $\lambda_{\max} > 0$.

6.4 PCA: Interpretation 1 (decorrelate the dimensions)

A. Core Concept

- Data in $\mathbb{R}^2$ may have almost no spread along some direction u_2 (essentially noise). It can then be represented in $\mathbb{R}^1$ (along u_1) by choosing a smarter basis.
- **Wish-list for the new basis:** dimensions with

    1. **high variance**: low-variance directions don't help, e.g. a classifier can't distinguish points along them;
    2. **low covariance / uncorrelated** (non-redundant): slide example of column z being almost linearly dependent on y (correlation $\rho_{yz} \approx 0.997$, verified), so z adds no new information;
    3. ideally **orthogonal**, which is the most convenient basis.
- **Correlation coefficient** used on the slide (p. 35) to show the redundancy of z:

$$\rho_{yz} = \frac{\sum_{i=1}^{n}(y_i - \bar{y})(z_i - \bar{z})}{\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}\sqrt{\sum_{i=1}^{n}(z_i - \bar{z})^2}}$$

It lies in $[-1, 1]$; $|\rho| \approx 1$ means one column is (almost) a linear function of the other, so it carries no new information. (Here the sum runs over the data rows.)

B. Terminology / setup (slide notation)

- $X \in \mathbb{R}^{m \times n}$: the **rows** are the m data points $x_i \in \mathbb{R}^n$. The data is assumed **zero-mean** (the slide also says unit variance).
- $P \in \mathbb{R}^{n \times n}$: its columns $p_1, \ldots, p_n$ form an orthonormal basis.
- Coordinates in the new basis: $\alpha_{ij} = x_i^T p_j$. The transformed point is $\hat{x}_i = x_i^T P$, and the transformed data is $\hat{X} = XP$.

C. Properties / theorems

1. If the columns of X have zero mean, the columns of $\hat{X} = XP$ also have zero mean. Proof sketch: $1^T A$ is the row vector of **column sums** of any A, and $1^T \hat{X} = (1^T X)P = 0$.
2. $X^T X$ is **symmetric**, since $(X^T X)^T = X^T X$.
3. **Covariance matrix.** By definition (slide p. 40), with μ_i, μ_j the column means,

$$C_{ij} = \frac{1}{m}\sum_{k=1}^{m}(X_{ki} - \mu_i)(X_{kj} - \mu_j)$$

For zero-mean columns this becomes $C_{ij} = \frac{1}{m}\sum_k X_{ki}X_{kj} = \frac{1}{m}(X^T X)_{ij}$, i.e. $\Sigma = \frac{1}{m}X^T X$ ($n \times n$). Diagonal entries = variances of the features; off-diagonal = covariances. Note the slides divide by m (not $m - 1$). 4. The covariance of the transformed data is

$$\frac{1}{m}\hat{X}^T\hat{X} = P^T \Sigma P$$

- We want the off-diagonal entries to be 0 (no covariance) and the diagonal entries non-zero (variance).
- So we want $P^T \Sigma P = D$, a diagonal matrix.

5. The orthogonal matrix that diagonalises Σ is the matrix whose columns are **the eigenvectors of** Σ, by EVD. So the **PCA basis = eigenvectors of** $X^T X$.
    - These eigenvectors are linearly independent and orthogonal, because $X^T X$ is symmetric.
6. In practice you keep only the **top-k** dimensions, those with high variance.

⚠ Slide note (p. 42): it writes "eigenvectors of $\Sigma = X^T X$". Strictly $\Sigma = \frac{1}{m}X^T X$. The eigenvectors are identical; the eigenvalues differ by the factor $\frac{1}{m}$.

6.5 PCA: Interpretation 2 (minimise reconstruction error)

A. Core idea

Project the data onto a new basis, keep only k dimensions, and **minimise the error in reconstructing** x_i.

D. Formulas

- Exact representation: $x_i = \sum_{j=1}^{n} \alpha_{ij} p_j$. The top-$k$ approximation is $\hat{x}_i = \sum_{j=1}^{k} \alpha_{ij} p_j$.
- Reconstruction error:

$$e = \sum_{i=1}^{m} (x_i - \hat{x}_i)^T (x_i - \hat{x}_i) = \sum_{i=1}^{m} \sum_{j=k+1}^{n} \alpha_{ij}^2 = \sum_{j=k+1}^{n} p_j^T (mC) p_j, \qquad C = \frac{1}{m} X^T X = \frac{1}{m} \sum_i x_i x_i^T$$

The cross terms vanish because the basis is orthonormal.

- Minimise $\sum_{j=k+1}^{n} p_j^T m C \, p_j$ subject to $p_j^T p_j = 1$.
  - The solution is the eigenvectors of the **smallest** eigenvalues of C (by the min theorem of 6.3).
  - So **choose P = eigenvectors of C, and keep the top-k (discard $k+1, \ldots, n$)**.
- (Additional clarification — derived from the slide result) Since $p_j^T C p_j = \lambda_j(C)$, the minimum error is

$$e_{\min} = m \sum_{j>k} \lambda_j(C) = \sum_{j>k} \lambda_j(X^T X)$$

In words: the error is the sum of the **discarded** eigenvalues.

⚠ Slide notes (pp. 45–46):

- p. 45 writes $\hat{x}_i = \sum_{j=1}^{k} \alpha_{ik} p_k$. It should be $\alpha_{ij} p_j$.
- In the cross-term sum on p. 46 the index condition reads "$L = k+1, L \neq k$". It should be $L \neq j$.

E. Toy example (slide, verified)

- The basis is $u_1 = \frac{1}{\sqrt{2}}[1, 1]$ and $u_2 = \frac{1}{\sqrt{2}}[-1, 1]$. Take the point $x = [3.3, 3]$.
- Its coordinates are $\alpha_1 = 6.3/\sqrt{2} \approx 4.455$ and $\alpha_2 = -0.3/\sqrt{2} \approx -0.212$.
- Using both directions gives perfect reconstruction: $\alpha_1 u_1 + \alpha_2 u_2 = [3.3, 3]$.
- Using $k = 1$ gives $\hat{x} = \alpha_1 u_1 = [3.15, 3.15]$. The error is $\|x - \hat{x}\|^2 = 0.045 = \alpha_2^2$ (exactly the squared dropped coordinate).

C. Recap (slide)

- PCA represents the data in a new basis made of only the top-k eigenvectors.
- The $n - k$ discarded dimensions contribute very little to the reconstruction error.
- These are also the directions of **minimum variance**.

6.6 PCA: Interpretation 3 (maximise variance)

D. Formula

- The i-th transformed dimension is $\hat{X}_i = X p_i$. Its variance is

$$\frac{\hat{X}_i^T \hat{X}_i}{m} = \frac{1}{m} p_i^T X^T X p_i = \frac{\lambda_i}{m}$$

where λ_i is the eigenvalue of $X^T X$ belonging to p_i. Equivalently, it is the i-th eigenvalue of $C = \frac{1}{m} X^T X$.

- **The variance along the i-th principal direction equals its (scaled) eigenvalue.** So discarding the eigenvectors with small eigenvalues discards the low-variance directions. That is the right thing to do.

C. Summary: three interpretations of PCA (slide)

1. It **minimises the covariance** between the new dimensions (diagonalises the covariance).
2. It picks dimensions with **high variance**.
3. It represents the data with **fewer dimensions** at minimum reconstruction error.

E. Numerical worked example (verified)

- Take centred data $X = \begin{bmatrix} 2 & 1 \\ -2 & -1 \\ 1 & 2 \\ -1 & -2 \end{bmatrix}$, so $m = 4$ and the column means are 0.
    1. $C = \frac{1}{4} X^T X = \begin{bmatrix} 2.5 & 2 \\ 2 & 2.5 \end{bmatrix}$.
    2. The eigenvalues of C are 4.5 (direction $\frac{1}{\sqrt{2}}[1, 1]$) and 0.5 (direction $\frac{1}{\sqrt{2}}[1, -1]$). The eigenvalues of $X^T X$ are 18 and 2, i.e. 4 times larger, with the same eigenvectors.
    3. The variance explained by PC1 is $4.5/(4.5 + 0.5) = 90\%$.
    4. The projections onto PC1 are $\alpha = \pm 3/\sqrt{2} = \pm 2.121$, and their variance is $4.5 = \lambda_1(C)$.
    5. The reconstruction error with $k = 1$ is $2.0 = m \cdot \lambda_2(C) = \lambda_2(X^T X)$.
- **Choosing k (Additional clarification; not explicitly on the slides).** Keep the smallest k for which $\frac{\sum_{i<k} \lambda_i}{\sum_i \lambda_i}$ exceeds a target such as 90–95%.
    - The total variance equals $\mathrm{trace}(C)$.
    - Example: $C = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ has eigenvalues 3 and 1, so PC1 explains 75%. $C = \begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix}$ has eigenvalues 5 and 0, so PC1 explains 100% (the data is collinear).

F. MCQ traps (PCA overall)

- PCA's eigenvectors come from $X^T X$ ($n \times n$, feature space), **not** from $X X^T$ ($m \times m$), when rows are data points.
- Keep the **largest**-eigenvalue eigenvectors. The **smallest** ones are those that solve the "discarded directions" minimisation.
- Variance along PC i is λ_i / m for eigenvalues of $X^T X$, or λ_i for eigenvalues of C. Watch which matrix the question uses.
- PCA requires **centred (zero-mean)** data for $\frac{1}{m} X^T X$ to be the covariance.
- PCA is a **linear** method, and its principal directions are **orthogonal**.
- The new dimensions are uncorrelated (zero covariance). That is not the same as "statistically independent".

6.7 PCA: Practical Example (Eigenfaces)

- **Data.** m face images, each 100×100, giving $n = 10\text{K}$ dimensions, so $X \in \mathbb{R}^{m \times 10K}$ with one image per row.
- **Covariance-type matrix.** $X^T X$ is $10K \times 10K$, so each eigenvector is 10K-dimensional. Reshaping one into a 100×100 image gives an **eigenface**.
- **Representation.** Keep the top ~100 eigenvectors (the slides mention a goal of 50–200 dimensions). Any face is then approximated as $\sum_{j=1}^{k} \alpha_{ij} p_j$. Reconstruction improves as k grows (the slides show $k = 1, 2, 4, 8, 12, 16$).
- **Storage.** Store $p_1, \ldots, p_k$ **once**. For each image, store only k scalars $\alpha_{i1}, \ldots, \alpha_{ik}$. This cuts storage greatly "without much loss in image quality".
    - Worked example (verified): $m = 1000$, $k = 100$. The original needs 10^7 numbers. The compressed version needs $100 \times 10^4 + 1000 \times 100 = 1.1 \times 10^6$ numbers, about 9× less.

6.8 Singular Value Decomposition (SVD)

A. Core Concept

- In the eigenbasis of a square A, multiplication becomes scaling: $Ax = \sum \alpha_i \lambda_i v_i$.
- A **rectangular** $A_{m \times n}$ has no eigenvectors. $Ax = \lambda x$ is impossible because $Ax \in \mathbb{R}^m$ while $x \in \mathbb{R}^n$.
- SVD recovers the same convenience with **pairs** of vectors (v_i, u_i), where $v_i \in \mathbb{R}^n$ (input space) and $u_i \in \mathbb{R}^m$ (output space), such that

$$Av_i = \sigma_i u_i$$

- If $x = \sum_{i=1}^{k} \alpha_i v_i$, then $Ax = \sum_{i=1}^{k} \alpha_i \sigma_i u_i$, so multiplication again reduces to scalar multiplication.
- **Geometry.** $A : \mathbb{R}^n \to \mathbb{R}^m$.
    - The **row space** of A (in $\mathbb{R}^n$) and the **column space** (in $\mathbb{R}^m$) both have dimension $k = \text{rank}(A)$.
    - V is a basis for the inputs and U a basis for the outputs.
    - Only the k-dimensional part of the input produces non-zero output. The remaining $n - k$ directions map to 0. So k terms suffice for x.

D. Formulas

Formula	Shapes / conditions	Meaning
$A_{m \times n} V_{n \times k} = U_{m \times k} \Sigma_{k \times k}$	k orthogonal v_i	Compact form of $Av_i = \sigma_i u_i$
$A_{m \times n} V_{n \times n} = U_{m \times m} \Sigma_{m \times n}$	Complete V and U to full orthonormal bases (Gram–Schmidt)	Full form
$U^T A V = \Sigma$, i.e. $A = U \Sigma V^T$	$U^{-1} = U^T$, $V^{-1} = V^T$ (orthogonal)	**SVD**; Σ is diagonal with only the first k entries non-zero
$A^T A = V \Sigma^2 V^T$	—	EVD of $A^T A$, so V **= eigenvectors of** $A^T A$ (right singular vectors)
$A A^T = U \Sigma^2 U^T$	—	U **= eigenvectors of** $A A^T$ (left singular vectors)
$\sigma_i = \sqrt{\lambda_i}$	λ_i = eigenvalues of $A^T A$	**Singular values**, with $\Sigma^2 = \Lambda$
$A = \sum_{i=1}^{k} \sigma_i u_i v_i^T$	$U_{m \times k}\, \Sigma_{k \times k}\, V^T_{k \times n}$	Sum of rank-1 matrices

Formula	Shapes / conditions	Meaning
$\min_B \|A-B\|_F^2$ over rank-k B	—	Solution $B = U_{\cdot,k}\Sigma_{k,k}V^T_{k,\cdot} = \sum_{i=1}^{k}\sigma_i u_i v_i^T$: the **best rank-$k$ approximation** (minimises the reconstruction error of A)

- **Names:** σ_i = singular values; U = **left** singular matrix; V = **right** singular matrix.
- $\sigma_1 u_1 v_1^T$ is the best rank-1 approximation, and $\sum_{i=1}^{2}\sigma_i u_i v_i^T$ is the best rank-2 approximation.
- ⚠ Slide note (p. 70): the slide writes just "$\min\|A-B\|_F^2$" with no constraint. The constraint $\text{rank}(B) \le k$ is implied; without it the trivial answer is $B = A$.
- (Additional clarification) $\|A\|_F^2 = \sum_i \sigma_i^2$, and the best rank-$k$ error is $\sum_{i>k}\sigma_i^2$ (Eckart–Young). The number of non-zero σ_i equals $\text{rank}(A)$. A^TA ($n \times n$) and AA^T ($m \times m$) share their non-zero eigenvalues.

E. Numerical examples (verified)

1. $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$.
   - $A^TA = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$, with eigenvalues 45 and 5.
   - So $\sigma_1 = \sqrt{45} = 3\sqrt{5} \approx 6.708$ and $\sigma_2 = \sqrt{5} \approx 2.236$.
   - $\|A\|_F^2 = 9 + 16 + 25 = 50 = 45 + 5$.
   - The best rank-1 approximation is $\sigma_1 u_1 v_1^T = \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix}$, with error $\|A - A_1\|_F^2 = 5 = \sigma_2^2$.
2. $A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$ (2×3).
   - AA^T (2×2) has eigenvalues 3 and 1. A^TA (3×3) has eigenvalues 3, 1 and 0.
   - So $\sigma = \sqrt{3}$ and 1, and the rank is 2.
   - Full SVD shapes: U is 2×2, Σ is 2×3, V is 3×3.

F. MCQ traps (SVD)

- $\sigma_i = \sqrt{\lambda_i(A^TA)}$, **not** $\lambda_i(A)$. For a non-symmetric square A, the singular values differ from the eigenvalues. (For symmetric PSD A they coincide.)
- V belongs with A^TA (input space $\mathbb{R}^n$), and U belongs with AA^T (output space $\mathbb{R}^m$). Swapping them is the classic trap.
- SVD exists for **any** (rectangular) matrix. EVD needs a square matrix with n independent eigenvectors.
- Singular values are ≥ 0. Eigenvalues can be negative.
- In the full SVD, Σ is $m \times n$ (rectangular diagonal). In the compact SVD it is $k \times k$.

G. Quick Revision Summary: Lecture 6

- **Key facts**
  - $Ax = \lambda x$.
  - The dominant eigenvalue has the largest $|\lambda|$, and $A^n x_0 \to$ a multiple of e_d. It explodes, vanishes or stays steady for $|\lambda_d| > 1$, < 1 or $= 1$.
  - A stochastic matrix (column sums 1) has dominant eigenvalue 1, which gives a steady state.
- **Bases.**
  - An orthonormal basis gives coefficients $\alpha_i = z^T u_i$, costing $O(n^2)$ in total, against $O(n^3)$ by Gaussian elimination.
  - Distinct eigenvalues imply independent eigenvectors. A symmetric matrix has orthogonal eigenvectors.
- **EVD.**

- $A = U\Lambda U^{-1}$ needs n independent eigenvectors. For symmetric A, $A = U\Lambda U^T$.
- $\max_{\|x\|=1} x^T Ax = \lambda_{\max}$, attained at its eigenvector.

- **PCA.**
  - Centre the data, form $C = \frac{1}{m} X^T X$, take its eigenvectors P, then $\hat{X} = XP_{:,1:k}$.
  - The variance along p_i is $\lambda_i(C)$.
  - The reconstruction error is $\sum$ of the discarded eigenvalues.
  - Three views: decorrelate, maximise variance, minimise reconstruction error.
- **SVD.**
  - $A = U\Sigma V^T$, with V = eigvecs(A^TA), U = eigvecs$(AA^T),\sigma_i=\sqrt{\lambda_i}$.
  - $\sum_{i \le k} \sigma_i u_i v_i^T$ is the best rank-k approximation in Frobenius norm.
- **Likely MCQ takeaways**
  1. Which eigenvectors does PCA keep? Those of the largest eigenvalues of the covariance $X^T X$ (or $\frac{1}{m} X^T X$).
  2. % variance explained $= \lambda_1 / \sum \lambda_i$.
  3. Singular values are the square roots of the eigenvalues of $A^T A$.
  4. Steady state, explode or vanish depends on $|\lambda_d|$ against 1.
  5. Shapes: $U_{m\times m}$, $\Sigma_{m\times n}$, $V_{n\times n}$.

Week 6 · Lecture 7: Autoencoders

7.1 Introduction to Autoencoders

A. Core Concept

- An **autoencoder (AE)** is a special feed-forward network that **encodes** its input x_i into a hidden representation h and then **decodes** the input back from h:

$$h = g(Wx_i + b), \qquad \hat{x}_i = f(W^*h + c)$$

- It is trained to make $\hat{x}_i$ close to x_i, using backpropagation like any feed-forward network.
- **Undercomplete AE**, $\dim(h) < \dim(x_i)$:
  - If $\hat{x}_i$ is still reconstructed perfectly, then h is a **loss-free encoding** that captures all the important characteristics of x_i.
  - This is the analogy with PCA.
- **Overcomplete AE**, $\dim(h) \geq \dim(x_i)$:
  - It can learn a **trivial identity encoding**: copy x_i into h, then h into $\hat{x}_i$.
  - That is useless because it tells us nothing about the important characteristics of the data. Regularization is needed.

B. Terminology

Symbol	Meaning
W, b	Encoder weights and bias
W^*, c	Decoder weights and bias
g	Encoder activation, typically **sigmoid**
f	Decoder activation, chosen according to the input type
$h_0 = x_i$, a_1, h_1, a_2, $h_2 = \hat{x}_i$	Layer notation used for backpropagation

C. Choice of decoder f and loss (by input type)

Input type	Decoder f	Loss	Why
Binary, $x_{ij} \in \{0, 1\}$	**logistic (sigmoid)**	**Cross-entropy:** $-\sum_{j=1}^{n} \big(x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij})\big)$	Logistic restricts outputs to $(0, 1)$, which can be read as probabilities. The loss is minimised when $\hat{x}_{ij} = x_{ij}$
Real, $x_{ij} \in \mathbb{R}$	**linear:** $\hat{x}_i = W^*h + c$	**Squared error:** $\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{n}(\hat{x}_{ij} - x_{ij})^2 = \frac{1}{m}\sum_i(\hat{x}_i - x_i)^T(\hat{x}_i - x_i)$	Logistic or tanh would restrict $\hat{x}_i$ to $[0, 1]$ or $[-1, 1]$, but we need $\hat{x}_i \in \mathbb{R}^n$

- The options the slides consider for the decoder are tanh, linear and logistic. The answer is logistic for binary inputs and linear for real inputs.
- In both cases the encoder g is "typically" sigmoid.

D. Gradients (one training example)

- Chain rule:

$$\frac{\partial \mathscr{L}}{\partial W^*} = \frac{\partial \mathscr{L}}{\partial h_2}\frac{\partial h_2}{\partial a_2}\frac{\partial a_2}{\partial W^*}, \qquad \frac{\partial \mathscr{L}}{\partial W} = \frac{\partial \mathscr{L}}{\partial h_2}\frac{\partial h_2}{\partial a_2}\frac{\partial a_2}{\partial h_1}\frac{\partial h_1}{\partial a_1}\frac{\partial a_1}{\partial W}$$

The "boxed" factors are the same as in ordinary backpropagation (Lecture 4). On the squared-error slide everything after $\partial \mathscr{L}/\partial h_2$ is boxed; on the cross-entropy slide the boxes are $\partial a_2/\partial W^*$ and $\frac{\partial a_2}{\partial h_1}\frac{\partial h_1}{\partial a_1}\frac{\partial a_1}{\partial W}$, and the first two factors ($\partial \mathscr{L}/\partial h_2$, $\partial h_2/\partial a_2$) are computed explicitly. Only these first factors change with the loss/decoder.

- **Squared error.**

$$\frac{\partial \mathscr{L}}{\partial h_2} = \nabla_{\hat{x}_i}(\hat{x}_i - x_i)^T(\hat{x}_i - x_i) = 2(\hat{x}_i - x_i)$$

This is a vector in $\mathbb{R}^n$. (Additional clarification: with a linear decoder, $\partial h_2/\partial a_2 = I$.)

- **Cross-entropy with sigmoid decoder.** $\partial \mathscr{L}/\partial h_2$ is the vector of components

$$\frac{\partial \mathscr{L}}{\partial h_{2j}} = -\frac{x_{ij}}{\hat{x}_{ij}} + \frac{1 - x_{ij}}{1 - \hat{x}_{ij}}, \qquad \frac{\partial h_{2j}}{\partial a_{2j}} = \sigma(a_{2j})(1 - \sigma(a_{2j}))$$

- (Additional clarification) The product of the two simplifies to $\hat{x}_{ij} - x_{ij}$.

E. Numerical examples (verified)

1. **Cross-entropy gradient**, with $x_{ij} = 1$ and $\hat{x}_{ij} = 0.8$:
   - $\partial \mathscr{L}/\partial h_{2j} = -1/0.8 = -1.25$ and $\sigma' = 0.8 \times 0.2 = 0.16$.
   - The product is $-0.2 = \hat{x} - x$.
   - The loss contribution is $-\ln 0.8 = 0.223$.
2. **Cross-entropy gradient**, with $x_{ij} = 0$ and $\hat{x}_{ij} = 0.3$: $\partial \mathscr{L}/\partial h_{2j} = 1/0.7 = 1.4286$ and $\sigma' = 0.21$, so the product is $0.3 = \hat{x} - x$.
3. **Parameter count**, for $n = 784$ (MNIST, 28×28) and $k = 100$:
   - Untied: W $(100 \times 784) + b$ $(100) + W^*$ $(784 \times 100) + c$ $(784) = 157{,}684$.
   - Tied ($W^* = W^T$): $78{,}400 + 100 + 784 = 79{,}284$.
4. **Shapes.**
   - $W \in \mathbb{R}^{k \times n}$ and $W^* \in \mathbb{R}^{n \times k}$.
   - $\partial \mathscr{L}/\partial W$ has the same shape as W, and $\partial \mathscr{L}/\partial h_2 \in \mathbb{R}^n$.

F. Traps

- An **overcomplete** AE ($\dim h \geq \dim x$) risks learning the identity. An **undercomplete** AE ($\dim h < \dim x$) is forced to compress. Note the slides define overcomplete with $\geq$, so equal dimensions count as overcomplete.
- A sigmoid decoder for real-valued (unbounded) inputs is **wrong**, because outputs can't exceed 1. Tanh is wrong for both binary and real inputs on the slides.
- Squared-error gradient: $2(\hat{x} - x)$, **not** $(x - \hat{x})$. Watch the sign and the factor 2.

7.2 Link between PCA and Autoencoders

C. The equivalence (core examinable statement)

The encoder of a linear autoencoder is equivalent to PCA if we:

1. use a **linear encoder**;
2. use a **linear decoder**;
3. use a **squared-error loss**;
4. **normalise the inputs** to

$$\hat{x}_{ij} = \frac{1}{\sqrt{m}}\Big(x_{ij} - \frac{1}{m}\sum_{k=1}^{m} x_{kj}\Big)$$

- **Why the normalisation.** The bracket makes each column (dimension j) zero-mean. Call the result X'; then $X = \frac{1}{\sqrt{m}}X'$, so

$$X^T X = \frac{1}{m} X'^T X'$$

which is **exactly the covariance matrix**.

D. Proof outline (results kept, algebra dropped)

1. With a linear decoder and squared error, ignoring biases, minimising $\sum_i \sum_j (x_{ij} - \hat{x}_{ij})^2$ is equivalent to $\min_{W^*,H} \|X - HW^*\|_F^2$.
   - Here H ($m \times k$) is the matrix of hidden codes and W^* is $k \times n$.
   - $\|A\|_F = \sqrt{\sum_i \sum_j a_{ij}^2}$.
2. By SVD (best rank-k approximation), the optimum is $HW^* = U_{\cdot,\leq k}\Sigma_{k,k}V^T_{\cdot,\leq k}$. One solution is $H = U_{\cdot,\leq k}\Sigma_{k,k}$ and $W^* = V^T_{\cdot,\leq k}$.
3. Using $X = U\Sigma V^T$, one can show $H = XV_{\cdot,\leq k}$. So H is a **linear function of** X, and the encoder weights are $W = V_{\cdot,\leq k}$.
   - In words: with a linear decoder and squared error, the optimum is achieved by a linear encoder.
4. From SVD, V = eigenvectors of $X^T X$. From PCA, P = eigenvectors of the covariance matrix. With the normalisation, $X^T X$ *is* the covariance matrix. So $W = P$: the encoder and the PCA projection can be the same.

- ⚠ Slide note (p. 20): the derivation writes $H = (XX^T)(XX^T)^{-1}U_{\cdot,\leq k}\Sigma_{k,k}$, which implicitly needs XX^T ($m \times m$) to be invertible. (Additional clarification: that requires $\text{rank}(X) = m$. The final result $H = XV_{\cdot,\leq k}$ follows more generally from $XV = U\Sigma$.)
- **Notation shift.** Here the data points are **rows**: $H = XW$, with $W \in \mathbb{R}^{n\times k}$ ($= V_{\cdot,\leq k}$). In 7.1 the convention is $h = Wx$, with $W \in \mathbb{R}^{k\times n}$.
- (Additional observation) In this optimum the decoder is $W^* = V^T_{\cdot,\leq k} = W^T$, i.e. the weights are **tied**.

F. Traps

- All four conditions are needed. A **sigmoid** encoder or decoder, or a cross-entropy loss, breaks the exact equivalence.
- The normalisation is **mean-centring plus scaling by** $1/\sqrt{m}$. It is not division by the standard deviation.
- It is the **encoder** of the linear AE that corresponds to the PCA projection P, taking the top-k eigenvectors.

7.3 Regularization in Autoencoders

C. Motivation and methods

- Poor generalisation can happen in undercomplete AEs. It is **even more serious for overcomplete AEs**, which can simply copy $x_i \to h \to \hat{x}_i$. Regularization is needed.
- **L2 regularization (weight decay):**

$$\min_{\theta, W, W^*, b, c} \frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{n} (\hat{x}_{ij} - x_{ij})^2 + \lambda \|\theta\|^2$$

It is easy to implement: it just adds a term λW to $\partial \mathscr{L} / \partial W$, and similarly for the other parameters.

- ⚠ Slide note: the gradient of $\lambda\|\theta\|^2$ is strictly $2\lambda W$. The slide's "λW" corresponds to the convention $\frac{\lambda}{2}\|\theta\|^2$, or the 2 absorbed into λ.
- **Tied weights**, $W^* = W^T$. This reduces the capacity of the AE and so **acts as a regularizer**. It roughly halves the weight parameters (see the parameter count in 7.1).
- More regularizers follow: denoising (7.4), sparse (7.5), contractive (7.6).

7.4 Denoising Autoencoders (DAE)

A. Core Concept

- A DAE **corrupts the input** with a probabilistic process $P(\tilde{x}_{ij} \mid x_{ij})$ before feeding it to the network.
- **Key point:** the objective is still to reconstruct the **original, uncorrupted** x_i:

$$\arg\min_{\theta} \frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{n} (\hat{x}_{ij} - x_{ij})^2$$

- Copying the corrupted $\tilde{x}_i$ through $h(\tilde{x}_i)$ to $\hat{x}_i$ no longer minimises the loss. The model must learn to reconstruct a corrupted x_{ij} **from its interactions with the other elements of** x_i, i.e. capture the true characteristics of the data.

D. Corruption processes

Type	Definition
Masking / "flip to 0"	$P(\tilde{x}_{ij} = 0 \mid x_{ij}) = q$ and $P(\tilde{x}_{ij} = x_{ij} \mid x_{ij}) = 1 - q$. With probability q the input is set to 0; otherwise it is kept
Additive Gaussian	$\tilde{x}_{ij} = x_{ij} + \mathcal{N}(0, 1)$

C. Visualising what neurons learn

- Treat each hidden neuron as a **filter**, e.g. $h_1 = \sigma(W_1^T x_i)$, ignoring the bias.
- Assume normalised inputs, $\|x_i\| = 1$. The input that maximally activates the neuron solves

$$\max_{x_i} W_1^T x_i \;\text{ s.t. }\; x_i^T x_i = 1 \quad \Rightarrow \quad x_i = \frac{W_1}{\sqrt{W_1^T W_1}}$$

This is the weight vector, normalised. Plotting $W_l / \|W_l\|$ for each neuron shows its filter.

- **MNIST observations** (vanilla AE vs DAE with $q = 0.25$ and $q = 0.5$):
  - The vanilla AE does not learn many meaningful patterns.
  - DAE neurons act like **pen-stroke detectors**, e.g. a stroke common to 0, 2, 3, 8 and 9.
  - **As the noise increases, the filters become wider**, because a neuron must rely on more adjacent pixels to be confident about a stroke.
- **Gaussian-noise DAE on a different dataset** (the figure shows natural-image patches): the hidden neurons behave like **edge detectors**. **PCA does not give such edge detectors.** The slide also shows "weight decay filters" for comparison.
- **Application (MNIST, $|x_i| = 784 = 28 \times 28$):** instead of feeding raw pixels to a classifier, first learn $h \in \mathbb{R}^d$ with an AE, then train the classifier on h.

E. Numericals

- With $q = 0.25$ on a 784-dim input, the expected number of entries forced to 0 is $784 \times 0.25 = 196$ (verified).
  - Entries that were already 0 stay 0, so the expected number that actually *change* is smaller.
- With $q = 0$ the DAE reduces to a vanilla AE. Larger q means more corruption and wider filters.

F. Traps

- The target is the **clean** x_i, **not** $\tilde{x}_i$. "The DAE reconstructs the corrupted input" is false.
- Corruption is applied to the **input** only (during training). It is not applied to the target or to the weights.
- q is the probability of **zeroing** an entry, not of keeping it.
- (Cross-ref L8) Adding noise to the inputs is a form of regularization / data augmentation.

7.5 Sparse Autoencoders

A. Core Concept

- A sigmoid hidden neuron outputs values in $(0, 1)$. It is "activated" when its output is near 1 and inactive when near 0.
- A **sparse AE** tries to keep each neuron **inactive most of the time**.

D. Formulas

Item	Formula	Notes
Average activation of neuron l	$\hat{\rho}_l = \frac{1}{m} \sum_{i=1}^{m} h(x_i)_l$	Over the m training examples. A sparse neuron has $\hat{\rho}_l \to 0$
Sparsity parameter	ρ	Typically very close to 0, e.g. 0.005. Enforce $\hat{\rho}_l = \rho$
Sparsity penalty	$\Omega(\theta) = \sum_{l=1}^{k} \left[\rho \log \frac{\rho}{\hat{\rho}_l} + (1 - \rho) \log \frac{1-\rho}{1-\hat{\rho}_l}\right]$	Sum over the k hidden neurons. Its **minimum is 0, at** $\hat{\rho}_l = \rho$. (Additional clarification: this is the KL divergence between Bernoulli(ρ) and Bernoulli($\hat{\rho}_l$); it is ≥ 0 and convex in $\hat{\rho}_l$)
Total loss	$\hat{\mathscr{L}}(\theta) = \mathscr{L}(\theta) + \Omega(\theta)$	$\mathscr{L}$ is the squared-error or cross-entropy loss
Gradient	$\frac{\partial \hat{\mathscr{L}}}{\partial W} = \frac{\partial \mathscr{L}}{\partial W} + \frac{\partial \Omega}{\partial W}$, with $\frac{\partial \Omega}{\partial W} = \frac{\partial \Omega}{\partial \hat{\rho}} \cdot \frac{\partial \hat{\rho}}{\partial W}$	Chain rule
	$\frac{\partial \Omega}{\partial \hat{\rho}_l} = -\frac{\rho}{\hat{\rho}_l} + \frac{1-\rho}{1-\hat{\rho}_l}$	$\frac{\partial \Omega}{\partial \hat{\rho}} = [\partial\Omega/\partial\hat{\rho}_1, \ldots, \partial\Omega/\partial\hat{\rho}_k]^T$

Item	Formula	Notes
	$\frac{\partial \hat{\rho}_l}{\partial W_{jl}} = \frac{1}{m}\sum_{i=1}^{m} g'(W_{:,l}^T x_i + b_l)\, x_{ij}$, i.e. in matrix form $\frac{\partial \hat{\rho}_l}{\partial W} = x_i\big(g'(W^T x_i + b)\big)^T$	Partial of a scalar w.r.t. a matrix is a matrix of the shape of W

- The penalty Ω itself is always ≥ 0. Its **gradient** $\partial\Omega/\partial\hat{\rho}_l$ is **positive when** $\hat{\rho}_l > \rho$ (gradient descent pushes the activation down) and **negative when** $\hat{\rho}_l < \rho$ (pushes it up). The slide's plot for $\rho = 0.2$ is a convex curve with minimum (value 0) at $\hat{\rho}_l = 0.2$.
- ⚠ Slide notes (p. 44–45):
  - The matrix form $x_i(g'(W^T x_i + b))^T$ drops the $\frac{1}{m}\sum_i$ that appears in the element-wise form.
  - That matrix form is an $n \times k$ outer product: it collects $\partial\hat{\rho}_l/\partial W_{jl}$ for **all** l (column l belongs to $\hat{\rho}_l$), although the slide labels it $\partial\hat{\rho}_l/\partial W$. For a single $\hat{\rho}_l$ only column l of W matters.
  - It also uses the convention $W \in \mathbb{R}^{n\times k}$ ($h = g(W^T x + b)$), unlike 7.1 ($h = g(Wx + b)$).
  - The slide adds Ω with no weighting coefficient. (Additional clarification: in practice a weight β multiplies Ω.)

E. Numerical examples (verified, natural log)

ρ	$\hat{\rho}_l$	Ω_l	$\partial\Omega/\partial\hat{\rho}_l$
0.2	0.2	**0**	0
0.2	0.5	$0.2\ln 0.4 + 0.8\ln 1.6 = 0.1927$	$-0.4 + 1.6 = 1.2$ (positive, so $\hat{\rho}$ is pushed down)
0.2	0.1	0.0444	$-2 + 0.889 = -1.111$ (negative, so $\hat{\rho}$ is pushed up)
0.05	0.1	0.0167	0.556

F. Traps

- $\hat{\rho}_l$ is averaged over **training examples** for a fixed neuron. It is not averaged over neurons for a fixed example.
- The penalty is minimised (equal to 0) at $\hat{\rho}_l = \rho$, **not** at $\hat{\rho}_l = 0$.
- Sparse AE regularises the **activations** (hidden representation). L2 / weight decay regularises the **weights**.

7.6 Contractive Autoencoders (CAE)

A. Core Concept

- A CAE also prevents an overcomplete AE from learning the identity. It does so by adding a penalty on the **Jacobian of the encoder**:

$$\Omega(\theta) = \|J_x(h)\|_F^2 = \sum_{j=1}^{n}\sum_{l=1}^{k}\left(\frac{\partial h_l}{\partial x_j}\right)^2$$

- $J_x(h)$ is $k \times n$ (hidden dimension × input dimension). Entry (l, j) is $\partial h_l/\partial x_j$: how much the output of neuron l varies with a small variation in input j.

C. Intuition (examinable)

- $\partial h_1/\partial x_1 = 0$ means neuron 1 is **insensitive** to variations in x_1.

- This *contradicts* the reconstruction loss $\mathscr{L}(\theta)$, which needs h to capture variations in the input. **That is the idea**:

Term	Role
$\mathscr{L}(\theta)$	capture the important variations in the data
$\Omega(\theta)$	do *not* capture variations in the data
Trade-off	capture **only the very important variations** seen in the training data

- **Illustration.** For data spread along u_1 (high variance) with small noise along u_2, the neuron should be sensitive to u_1 and insensitive to u_2. "What does this remind you of?" The slide leaves the question open; the intended answer is **PCA** (same u_1/u_2 picture: keep the high-variance direction, ignore the low-variance one).
- (Additional clarification) For $h = \sigma(Wx + b)$, $\frac{\partial h_l}{\partial x_j} = h_l(1 - h_l)W_{lj}$. So $\|J\|_F^2 = \sum_l \big(h_l(1 - h_l)\big)^2 \sum_j W_{lj}^2$.
  - Example (verified): $W = \begin{bmatrix} 1 & -2 \\ 0.5 & 1 \end{bmatrix}$, $x = [1, 1]$, $b = 0$.
  - This gives $h = [0.269, 0.818]$ and $J = \begin{bmatrix} 0.197 & -0.393 \\ 0.075 & 0.149 \end{bmatrix}$, so $\|J\|_F^2 = 0.221$.

F. Traps

- The Jacobian is of the **encoder** output h with respect to the **input** x. It is not taken with respect to the weights, and not of the decoder.
- The norm is the **squared Frobenius** norm, the sum of squares of all $k \cdot n$ entries.
- **CAE vs DAE.** A CAE makes h robust to small input perturbations *analytically*, through the Jacobian penalty. A DAE does it *stochastically*, by corrupting the input.

7.7 Summary and Comparison Table

Variant	What it adds	Formula	Prevents identity via
Linear AE ≡ PCA	linear encoder and decoder, squared error, normalised inputs	$\min_\theta \|X - \underbrace{HW^*}_{U\Sigma V^T \text{ (SVD)}}\|_F^2$	bottleneck ($k < n$)
Weight decay	penalty on the weights	$\Omega = \lambda\|\theta\|^2$	small weights
Tied weights	$W^* = W^T$	—	reduced capacity
Denoising	corrupted input, clean target	$P(\tilde{x}_{ij} \mid x_{ij})$	copying no longer minimises the loss
Sparse	average activation $\approx \rho$	$\Omega = \sum_l \mathrm{KL}(\rho \,\|\, \hat{\rho}_l)$	most neurons inactive
Contractive	insensitivity of h to x	$\Omega = \sum_j \sum_l (\partial h_l / \partial x_j)^2$	penalises the encoder's Jacobian

G. Quick Revision Summary: Lecture 7

- **Architecture:** $h = g(Wx_i + b)$ and $\hat{x}_i = f(W^*h + c)$.
  - Undercomplete: $\dim h < \dim x$, which gives compression.
  - Overcomplete: $\dim h \geq \dim x$, which risks the identity map, so regularize.
- **Choice of f and loss:** binary inputs use a sigmoid decoder with cross-entropy; real inputs use a linear decoder with squared error. The encoder g is typically sigmoid.

- **Gradients:**
  - Squared error: $\partial \mathscr{L} / \partial \hat{x} = 2(\hat{x} - x)$.
  - Cross-entropy: $\partial \mathscr{L} / \partial h_{2j} = -x/\hat{x} + (1-x)/(1-\hat{x})$, with $\partial h_{2j} / \partial a_{2j} = \sigma(1-\sigma)$.
- **AE ≡ PCA** under four conditions: linear encoder, linear decoder, squared error, inputs normalised by $\frac{1}{\sqrt{m}}(x - \bar{x})$. The encoder then equals $V_{\cdot, \leq k}$, the top eigenvectors of $X^T X$.
- **Regularizers:**
  - L2 (adds λW to the gradient) and tied weights.
  - DAE: masking with prob. q or Gaussian noise; the target is the clean input.
  - Sparse: $\hat{\rho}_l = \frac{1}{m} \sum_i h_l(x_i)$ with a KL penalty, minimum 0 at $\hat{\rho}_l = \rho$.
  - Contractive: $\|J_x(h)\|_F^2$, where J is $k \times n$.
- **Filter visualisation:** the maximising input is $x = W_l / \sqrt{W_l^T W_l}$. DAE filters are pen strokes or edges, and get wider with more noise. PCA doesn't give edge detectors.
- **Likely MCQ takeaways**
  1. The four conditions for AE ≡ PCA.
  2. Decoder and loss choice for binary vs real inputs.
  3. A DAE reconstructs the **clean** input.
  4. The KL sparsity term is 0 iff $\hat{\rho}_l = \rho$, and its gradient has the sign of $\hat{\rho}_l - \rho$.
  5. The CAE Jacobian is $k \times n$, and its penalty trades off against reconstruction to capture only important variations.

Overview: Lectures 8–9: Regularization, Pre-training, Activations, Initialization, Batch Norm

Lectures covered / modules:

- **Lecture 8 — Regularization**: 8.1 Bias & Variance · 8.2 Train error vs Test error · 8.3 True error & Model complexity · 8.4 l_2 regularization · 8.5 Dataset augmentation · 8.6 Parameter sharing & tying · 8.7 Adding noise to inputs · 8.8 Adding noise to outputs · 8.9 Early stopping · 8.10 Ensemble methods · 8.11 Dropout · Appendix (early-stopping induction proof)
- **Lecture 9 — Training deep nets better**: 9.1 Recap of training deep NNs · 9.2 Unsupervised (greedy layerwise) pre-training · 9.3 Better activation functions · 9.4 Better initialization strategies · 9.5 Batch Normalization

Notation (slides): f = true function, $\hat{f}$ = learned estimate, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ noise, n train points, m test points, $\mathscr{L}(w)$ unregularized loss, $\widetilde{\mathscr{L}}(w)$ regularized loss, w^* = unregularized optimum, $\tilde{w}$ = regularized optimum, H = Hessian, α = regularization strength, η = learning rate.

Week 7 · Lecture 8: Regularization

8.1 Bias and Variance

A. Core Concept

- Setup on slides: points drawn from a **sinusoidal** true $f(x)$; training data = 100 points. Two models:
  - **Simple** (degree 1): $\hat{f}(x) = w_1 x + w_0$
  - **Complex** (degree 25): $\hat{f}(x) = \sum_{i=1}^{25} w_i x^i + w_0$
- Both make an *assumption* about how y relates to x; we never know the true f.
- Experiment: sample **25 points** from the training data, train both models; repeat **k times** (each model sees a different sample).
- Observations:
  - Simple models trained on different samples **do not differ much from each other**, but are **far from the true sinusoid → underfitting**.
  - Complex models trained on different samples are **very different from each other → high variance** (but their *average* is close to the true curve).

B. Terminology

Term	Definition (slides)	Meaning
Bias	$\text{Bias}(\hat{f}(x)) = E[\hat{f}(x)] - f(x)$	How far the **average** model is from the truth
Variance	$\text{Variance}(\hat{f}(x)) = E[(\hat{f}(x) - E[\hat{f}(x)])^2]$	How much models trained on different samples **differ from each other**
$E[\hat{f}(x)]$	average (expected) value of the model over training samples	green line (simple) / blue curve (complex) in figure

C. Properties

- **Simple model: high bias, low variance** (underfit). **Complex model: low bias, high variance** (overfit).
- There is always a **trade-off** between bias and variance; both contribute to the mean-squared error.
- Expectations are over **different training samples** (different trained models), at a fixed x.

E. Worked example (verified)

True $f(x_0) = 2$. Four simple models predict $1.5, 1.7, 1.6, 1.8 \rightarrow E[\hat{f}] = 1.65$, Bias $= -0.35$, Bias$^2 = 0.1225$, Variance $= 0.0125$. Four complex models predict $1.0, 3.0, 2.5, 1.5 \rightarrow E[\hat{f}] = 2.0$, Bias $= 0$, Variance $= 0.625$. → low-bias/high-variance vs high-bias/low-variance. (Variance uses $1/k$ population form, as in $E[\cdot]$.)

8.2 Train error vs Test error

A. Core Concept

For a new unseen point (x, y), the expected squared error decomposes:

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2 \text{ (irreducible error)}$$

- Parameters of $\hat{f}$ are trained on $\{(x_i, y_i)\}_{i=1}^{n}$; at test time we care about a **validation (unseen) set**.

$$\text{train}_{err} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{f}(x_i))^2 \qquad \text{test}_{err} = \frac{1}{m}\sum_{i=n+1}^{n+m}(y_i - \hat{f}(x_i))^2$$

C. Properties

- Figure (error vs model complexity): **training error decreases monotonically** with complexity; **test (validation) error is U-shaped** — high on the left (**high bias**), high on the right (**high variance**), minimum at the **"sweet spot"** = ideal model complexity / perfect trade-off.
- As complexity increases, **train error becomes overly optimistic** (wrong picture of how close $\hat{f}$ is to f); **validation error gives the real picture**.
- σ^2 is irreducible: no model can remove it.

D. Formula notes

Formula	Notes
$E[(y - \hat{f})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$	**Bias is squared**, variance is not. σ^2 = noise variance. Mistake: writing "Bias + Variance".

E. Numerical

With the example above and $\sigma^2 = 0.04$: simple model expected test MSE $= 0.1225 + 0.0125 + 0.04 = 0.175$.

8.2 (cont.) True error vs empirical error · 8.3 True error and Model complexity

A. Core Concept

Model: $y_i = f(x_i) + \varepsilon_i$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$; $\hat{y}_i = \hat{f}(x_i)$ estimated using $T \subset D$. We want the **true error** $E[(\hat{f}(x_i) - f(x_i))^2]$ but can't compute it (unknown f). Expanding $E[(\hat{y}_i - y_i)^2]$ gives the key identity:

$$\underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2\,E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- Expectations are estimated empirically by averages (detour: expected goals $E[z] \approx \frac{1}{k}\sum z_i$).
- Last term $= \text{covariance}(\varepsilon_i,\ \hat{f}(x_i) - f(x_i))$ since $E[\varepsilon] = 0$ (Cov$(X,Y)=E[XY]when\mu_X=0$).

Case 1 — test observations:

$$\text{true error} = \underbrace{\tfrac{1}{m}\textstyle\sum_{i=n+1}^{n+m}(\hat{y}_i - y_i)^2}_{\text{empirical estimate}} - \underbrace{\tfrac{1}{m}\textstyle\sum_{i=n+1}^{n+m}\varepsilon_i^2}_{\text{small constant}} + 2\,\underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{=0}$$

Test points did not participate in estimating $\hat{f} \Rightarrow \varepsilon \perp (\hat{f} - f) \Rightarrow$ covariance $= E[\varepsilon]E[\hat{f} - f] = 0$. ⇒ **true error = empirical test error + small constant.** Hence **always use a validation set independent of the training set** to estimate error.

Case 2 — training observations: $\varepsilon \not\perp \hat{f}$ because ε was used to estimate $\hat{f}$'s parameters ⇒ covariance term $\neq 0$ ⇒ **empirical train error is smaller than the true error** (doesn't give the true picture).

Stein's Lemma result (module 8.3):

$$\frac{1}{n}\sum_{i=1}^{n}\varepsilon_i(\hat{f}(x_i)-f(x_i))=\frac{\sigma^2}{n}\sum_{i=1}^{n}\frac{\partial\hat{f}(x_i)}{\partial y_i}$$

- $\partial\hat{f}(x_i)/\partial y_i$ is high when a **small change in an observation causes a large change in the estimate** → **complex models are more sensitive** (figure: changing one data point barely moves the simple fit, but substantially changes the complex fit).

$$\text{true error}=\text{empirical train error}+\text{small constant}+\Omega(\text{model complexity})$$

Basis of regularization: instead of minimizing $\mathscr{L}_{train}(\theta)$, minimize

$$\min_{\theta}\ \mathscr{L}(\theta)=\mathscr{L}_{train}(\theta)+\Omega(\theta)$$

where $\Omega(\theta)$ is high for complex models, small for simple ones; it approximates $\frac{\sigma^2}{n}\sum_i\partial\hat{f}(x_i)/\partial y_i$ and should ensure the model has **reasonable complexity** (sweet spot). l_1, l_2, early stopping and input noise are all instances of this form.

- **Figure (slide 27):** on the error-vs-complexity plot, the **gap between the test (red) and train (blue) error curves is labelled** $\frac{\sigma^2}{n}\sum_i\partial\hat{f}(x_i)/\partial y_i$ — this optimism of the train error **grows with model complexity**; $\Omega(\theta)$ should keep the model near the sweet spot.

C. Properties / Insights

- Deep NNs are **highly complex** (many parameters, many non-linearities) → easy to overfit and drive **training error to 0** → need regularization.
- Forms covered: l_2, dataset augmentation, parameter sharing & tying, noise to inputs, noise to outputs, early stopping, ensembles, dropout.
- ⚠ Slide note: the "small constant" $\frac{1}{m}\sum\varepsilon_i^2$ ($\approx\sigma^2$) enters with a **minus** sign in the equation, although the slide summary says "true error = empirical test error **+** small constant". Read it as "differs by a small constant". Also the slide sets $\Omega(\theta)\approx\frac{\sigma^2}{n}\sum\partial\hat{f}/\partial y_i$, i.e. drops the factor 2 from the $2E[\cdot]$ term. Neither affects the conclusion.

F. MCQ Traps

- "Training error is an unbiased estimate of true error" → **False** (it's optimistic due to the non-zero covariance term).
- "Test error is a good estimate because test noise is independent of $\hat{f}$" → **True**.
- Sensitivity $\partial\hat{f}/\partial y_i$ large ↔ **complex** model ↔ high variance.

8.4 l_2 regularization (weight decay)

A. Core Concept

$$\widetilde{\mathscr{L}}(w)=\mathscr{L}(w)+\frac{\alpha}{2}\|w\|^2,\qquad\nabla\widetilde{\mathscr{L}}(w)=\nabla\mathscr{L}(w)+\alpha w$$

$$w_{t+1}=w_t-\eta\nabla\mathscr{L}(w_t)-\eta\alpha w_t$$

Very small code change. (Additional clarification: equivalently $w_{t+1}=(1-\eta\alpha)w_t-\eta\nabla\mathscr{L}(w_t)$ — weights "decay" each step.)

Geometric analysis (keep results, not algebra)

- Assume w^* = optimum of **unregularized** $\mathscr{L}$ ($\nabla\mathscr{L}(w^*) = 0$). 2nd-order Taylor around w^* with $u = w - w^*$:

$$\mathscr{L}(w) = \mathscr{L}(w^*) + \tfrac{1}{2}(w - w^*)^T H(w - w^*), \qquad \nabla\mathscr{L}(w) = H(w - w^*)$$

- Regularized optimum $\tilde{w}$ satisfies $H(\tilde{w} - w^*) + \alpha\tilde{w} = 0$:

$$\tilde{w} = (H + \alpha I)^{-1} H w^*$$

$\alpha \to 0 \Rightarrow \tilde{w} \to w^*$ (no regularization).

- If H is **symmetric PSD**: $H = Q\Lambda Q^T$ (Q orthogonal, $QQ^T = Q^TQ = I$):

$$\tilde{w} = Q(\Lambda + \alpha I)^{-1}\Lambda Q^T w^* = QDQ^T w^*, \quad D = \operatorname{diag}\left(\frac{\lambda_1}{\lambda_1 + \alpha}, \ldots, \frac{\lambda_n}{\lambda_n + \alpha}\right)$$

- Interpretation: w^* is rotated by Q^T (into the Hessian eigenbasis), each component i is **scaled by** $\frac{\lambda_i}{\lambda_i + \alpha}$, then rotated back by Q.
  - $\lambda_i \gg \alpha$: factor ≈ 1 → component **retained**.
  - $\lambda_i \ll \alpha$: factor ≈ 0 → component **suppressed**.
  - Only **significant directions (large eigenvalues)** are retained.

$$\text{Effective parameters} = \sum_{i=1}^{n} \frac{\lambda_i}{\lambda_i + \alpha} < n$$

- All elements shrink, **some more than others** → only important features get high weights. ($\tilde{w}$ is a rotated/rescaled version of w^*.)

D. Formulas

Formula	Symbols	Conditions	Interpretation	Common mistake
$\tilde{w} = (H + \alpha I)^{-1} H w^*$	H Hessian at w^*	quadratic (2nd-order Taylor) approx, $\nabla\mathscr{L}(w^*) = 0$	regularized optimum in terms of unregularized	writing $(H + \alpha I)^{-1} w^*$
scale $= \lambda_i/(\lambda_i + \alpha)$	λ_i eigenvalue of H	H symmetric PSD	per-eigendirection shrinkage, in $[0, 1)$ for $\alpha > 0$	inverting: $\alpha/(\lambda_i + \alpha)$ is the *removed* fraction
eff. params $= \sum \lambda_i/(\lambda_i + \alpha)$			$< n$ for $\alpha > 0$; $\to n$ as $\alpha \to 0$; $\to 0$ as $\alpha \to \infty$	

E. Numerical (verified)

$\lambda = (10, 1, 0.1)$, $\alpha = 1$: factors $= 0.909,\ 0.5,\ 0.091$; effective parameters $= 1.5$ (out of 3). Shortcut: when $\lambda_i = \alpha$, factor $= 0.5$ exactly. Increasing α ⇒ every factor decreases ⇒ fewer effective parameters.

F. Traps

- Large eigenvalue = direction where loss curves sharply = **important** direction → kept. Small eigenvalue (flat direction) → shrunk toward 0.
- Shrinkage is **not uniform** (unlike simply multiplying w^* by a constant).
- The analysis is about **eigenvectors of the Hessian**, not of the data covariance (on slides).

8.5 Dataset augmentation

- Exploit the fact that **certain transformations do not change the label** (e.g. digit "2" rotated by 20°/65°, shifted vertically/horizontally, blurred, some pixels changed). Augmented data is **created using knowledge of the task**.
- More data = better learning (typically). Works well for **image classification / object recognition**; also shown to work for **speech**.
- Limitation: **for some tasks it may not be clear how to generate such data.**
- (Additional clarification) a transformation must be label-preserving for the task — e.g. large rotations can turn "6" into "9".

8.6 Parameter sharing and tying

	Parameter **Sharing**	Parameter **Tying**
Where	**CNNs**	**Autoencoders** (typically)
What	Same filter applied at different positions of the image / same weight matrix acts on different input neurons	**Encoder and decoder weights are tied**

- Both reduce effective number of free parameters → regularize. (Additional clarification: tied AE typically uses $W^* = W^T$.)

8.7 Adding noise to the inputs

- Noise process $P(\tilde{x} \mid x)$ — seen in (denoising) autoencoders. Can be viewed as **data augmentation**.
- **Result**: for a **simple input–output network** (linear: $\hat{y} = \sum_i w_i x_i$), adding Gaussian noise $\tilde{x}_i = x_i + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$:

$$\tilde{y} = \hat{y} + \sum_i w_i \varepsilon_i, \qquad E[(\tilde{y} - y)^2] = E[(\hat{y} - y)^2] + \sigma^2 \sum_{i=1}^{n} w_i^2$$

≡ L_2 **(weight decay) penalty** with strength proportional to σ^2.

- Assumptions used: ε_i independent of ε_j (cross terms vanish), ε_i independent of $(\hat{y} - y)$, zero-mean noise (middle term $= 0$).
- Numerical (verified): $w = (1, -2, 3)$, $\sigma^2 = 0.1$ → extra penalty $= 0.1 \times 14 = 1.4$.
- Trap: equivalence is shown for a **simple (linear) input-output network**, not proven on slides for arbitrary deep nets. More noise ($\sigma^2 \uparrow$) = stronger regularization.

8.8 Adding noise to the outputs (soft targets / label smoothing)

- **Hard targets** (10 classes, true class 2): $p = \{0, 0, 1, 0, \ldots, 0\}$; estimated distribution q.
- Intuition: **do not trust the true labels, they may be noisy** → use **soft targets**:

$$p = \left\{ \tfrac{\varepsilon}{9}, \tfrac{\varepsilon}{9}, 1 - \varepsilon, \tfrac{\varepsilon}{9}, \ldots \right\}, \quad \varepsilon = \text{small positive constant}$$

(true class gets $1 - \varepsilon$; remaining ε split equally among the other **9** classes; still sums to 1.)

- Generally (Additional clarification): with K classes, others get $\varepsilon/(K-1)$.
- Numerical (verified): $\varepsilon = 0.1$, 10 classes → true class 0.9, each other $0.1/9 \approx 0.0111$.

- ⚠ Slide note: slides write "minimize $\sum_{i=0}^{9} p_i \log q_i$"; the cross-entropy to minimize is $-\sum_i p_i \log q_i$ (sign missing).

8.9 Early stopping

A. Core Concept

- Track validation error; **patience parameter** p. At step k, if **no improvement in validation error in the previous** p **steps**, stop training and **return the model stored at step** $k-p$.
- Stops before training drives training error to 0 and blows up validation error.
- **Very effective and the most widely used form of regularization**; can be used **together with other regularizers** (e.g. l_2).

C. Why it regularizes

Intuition: SGD $w_{t+1} = w_t - \eta \nabla w_t = w_0 - \eta \sum_i \nabla w_i$. If τ = max gradient,

$$|w_{t+1} - w_0| \leq \eta t |\tau|$$

→ t controls how far w can go from w_0 — **controls the space of exploration**.

Math (quadratic approx, $\nabla \mathscr{L}(w) = H(w - w^*)$): $w_t = (I - \eta H) w_{t-1} + \eta H w^*$. With $H = Q \Lambda Q^T$ and $w_0 = 0$ (proved by induction in Appendix):

$$w_t = Q[I - (I - \eta \Lambda)^t] Q^T w^*$$

Compare with l_2: $\tilde{w} = Q[I - (\Lambda + \alpha I)^{-1} \alpha] Q^T w^*$. So $w_t = \tilde{w}$ **if** η, t, α **are chosen such that**

$$(I - \eta \Lambda)^t = (\Lambda + \alpha I)^{-1} \alpha \quad \text{i.e. per direction } (1 - \eta \lambda_i)^t = \frac{\alpha}{\lambda_i + \alpha}$$

- ⚠ Slide note: the slide writes ε in $(I - \varepsilon \Lambda)^t$ where it means the learning rate η (the appendix uses η). Also the sum index in $w_0 - \eta \sum_{i=1}^{t} \nabla w_i$ should run over $i = 0, \ldots, t$ — immaterial.
- **Things to remember:** early stopping allows only t updates. Important parameters (large $\partial \mathscr{L} / \partial w$) move far; unimportant ones (small gradient) get small updates and **cannot grow large in** t **steps** → early stopping **shrinks parameters in less important directions — same as weight decay**.

E. Numerical (verified)

- Bound: $\eta = 0.01$, $t = 100$, $|\tau| = 5$ → $|w_{t+1} - w_0| \leq 5$.
- Equivalence: $\lambda = 1$, $\alpha = 1$, $\eta = 0.1$ → need $(0.9)^t = 0.5$ → $t = \ln 0.5 / \ln 0.9 \approx 6.58$ steps.
- (Additional clarification) for small $\eta \lambda_i$, roughly $\alpha \approx 1/(\eta t)$: **more steps ⇔ weaker regularization**.

F. Traps

- Return the model at step $k - p$, not at step k.
- Equivalence to L_2 requires: quadratic approximation of loss, $w_0 = 0$, and appropriate η, t, α — not unconditional.
- Patience p (early stopping) ≠ retention probability p (dropout).

8.10 Ensemble methods

A. Core Concept

- **Combine outputs of different models to reduce generalization error.** Models can be different classifiers (Logistic Regression, SVM, Naive Bayes → y_{final}), or different instances of the same classifier trained with **different hyperparameters, different features, or different samples of training data**.
- **Bagging**: ensemble of different instances of the **same** classifier; from a dataset construct $T_1, \ldots, T_k$ by **sampling with replacement**; train ith instance on T_i.

D. Error analysis

k models (e.g. LR), model i makes error ε_i on a test example, ε_i from a **zero-mean multivariate normal**; $E[\varepsilon_i^2] = V$, $E[\varepsilon_i \varepsilon_j] = C$ ($i \neq j$). Ensemble error $= \frac{1}{k}\sum_i \varepsilon_i$:

$$\text{mse} = E\left[\left(\frac{1}{k}\sum_i \varepsilon_i\right)^2\right] = \frac{1}{k^2}\left(kV + k(k-1)C\right) = \frac{1}{k}V + \frac{k-1}{k}C$$

Case	Condition	mse	Meaning
Perfectly correlated	$C = V$	V	**Bagging does not help** (as bad as individual)
Uncorrelated/independent	$C = 0$	V/k	Error falls as $1/k$ (inversely with k)
General	$0 < C < V$	between V/k and V	

- On average, the ensemble performs **at least as well as its individual members**.
- (Additional clarification) as $k \to \infty$, mse $\to C$: correlation sets the floor.

E. Numerical (verified)

$V = 4, C = 1, k = 5$: mse $= 0.8 + 0.8 = 1.6$. With $C = 0$: 0.8. With $C = V = 4$: 4. $k = 1000$, $C = 1$: ≈ 1.003.

8.11 Dropout

A. Core Concept

- Ensembles (model averaging) typically help, but training several large NNs is **prohibitively expensive** (option 1: different architectures; option 2: same network on different samples) and **combining many models at test time is infeasible in real-time applications**. Dropout addresses **both** issues: trains many networks with no significant overhead + efficient **approximate** way of combining exponentially many networks.
- **Dropout** = temporarily remove a node and all its incoming/outgoing connections → **thinned network**. Each node is **retained** with fixed probability p **(typically 0.5 for hidden nodes, 0.8 for visible/input nodes)**.
- n nodes → 2^n **possible thinned networks** (too many to train separately).
- **Trick:** (1) **share weights across all networks**; (2) **sample a different thinned network for each training instance (or mini-batch)**.

Training

- Per instance/mini-batch: apply dropout → thinned net → loss → backprop. **Only active parameters are updated.**
- A weight active in both of two instances gets 2 updates; active in only one gets 1.
- Each thinned network gets trained **rarely (or even never)**, but **parameter sharing ensures no model has untrained or poorly trained parameters**.

Test time

- Impossible to aggregate 2^n thinned networks → use the **full network** and **scale the output of each node by the fraction of times it was on during training** (i.e. outgoing weights $w_i \to pw_i$). Node is "present with probability p" at training; "always present" at test with weights $pw_1, pw_2, \ldots$.
- (Additional clarification) many libraries use "inverted dropout": scale by $1/p$ during training instead, leaving test time unchanged.

C. Intuition

- Dropout applies a **masking noise** to hidden units; **prevents co-adaptation** — a unit cannot rely too much on others since they may be dropped.
- Each unit must be robust to random dropouts → **redundancy**: e.g. if h_i detects a face by firing on a nose, dropping h_i erases "nose exists" → model learns another unit that redundantly encodes nose, or detects face via other features.

E. Numerical (verified)

$n = 10$ nodes → $2^{10} = 1024$ thinned networks. Weight $w = 2$, $p = 0.8$ → test-time weight 1.6.

F. Traps

- p = probability of **retaining**, not dropping (on slides).
- Dropout at test time: **no** random dropping; full network with scaled weights.
- Number of thinned networks is 2^n, not $n!$ or n^2.
- Dropout is an approximate **ensemble** of thinned networks with **shared** parameters (Additional clarification: bagging trains models with independent parameters, each on its own bootstrap sample).

G. Quick Revision Summary — Lecture 8

- $E[(y - \hat{f})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$; simple = high bias/low variance; complex = low bias/high variance.
- Train error is optimistic (noise used in fitting ⇒ covariance $\neq 0$); test error ≈ true error up to a constant. Stein: covariance term $= \frac{\sigma^2}{n} \sum \partial \hat{f}(x_i)/\partial y_i$ ↔ model complexity ⇒ minimize $\mathscr{L}_{train} + \Omega(\theta)$.
- l_2: $\tilde{w} = Q(\Lambda + \alpha I)^{-1} \Lambda Q^T w^*$; scale $\lambda_i/(\lambda_i + \alpha)$; effective params $< n$.
- Input Gaussian noise ⇒ $+\sigma^2 \sum w_i^2$ (linear net). Output noise ⇒ soft targets $1 - \varepsilon$, $\varepsilon/9$.
- Early stopping: patience p, return step $k - p$; $\equiv l_2$ when $(1 - \eta\lambda_i)^t = \alpha/(\lambda_i + \alpha)$, $w_0 = 0$.
- Ensemble mse $= V/k + \frac{k-1}{k} C$; $C = 0 \Rightarrow V/k$; $C = V \Rightarrow V$.
- Dropout: retain w.p. p (0.5 hidden, 0.8 input); 2^n thinned nets with shared weights; test: scale by p.

Week 8 · Lecture 9: Greedy Layerwise Pre-training, Better Activations, Better Initialization, Batch Normalization

9.1 Recap: training deep neural networks

- Single sigmoid neuron (squared error): $\nabla w = (f(x) - y)\, f(x)(1 - f(x))\, x$; wider: $\nabla w_i = (f(x) - y) f(x)(1 - f(x)) x_i$.
- Deep chain ($a_i = w_i h_{i-1}$, $h_i = \sigma(a_i)$, $h_0 = x$): $\frac{\partial \mathscr{L}}{\partial w_1} = \frac{\partial \mathscr{L}}{\partial y} \frac{\partial y}{\partial a_3} \frac{\partial a_3}{\partial h_2} \cdots \frac{\partial a_1}{\partial w_1}$; in general $\nabla w_i = \frac{\partial \mathscr{L}}{\partial y} \cdots h_{i-1}$.
- **Key fact used later: the gradient w.r.t. a parameter is proportional to the input to that parameter (h_{i-1}).**
- Deep & wide nets: chain rule summed over multiple paths.
- "Training NNs is a **game of gradients**"; the gradient = responsibility of a parameter towards the loss.
- History: backprop popularized by **Rumelhart et al. (1986)**, but for very deep nets it wasn't successful — **till 2006** very deep networks were hard to train (training often did not converge even after many epochs).

9.2 Unsupervised (greedy layerwise) pre-training

A. Core Concept

- Field revived by **Hinton & Salakhutdinov (2006)**, *Reducing the dimensionality of data with neural networks*, Science. Introduced with **RBMs**; the course explains it with **autoencoders**.
- Procedure:
    1. Train weights between x and h_1 with an **unsupervised** objective — reconstruct x from h_1: $\min \frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{n} (\hat{x}_{ij} - x_{ij})^2$. (Unsupervised: uses only x, not y.) h_1 becomes an abstract representation of x.
    2. **Fix layer-1 weights**; repeat for layer 2 (reconstruct h_1 from h_2: $\min \frac{1}{m} \sum\sum (\hat{h}_{1ij} - h_{1ij})^2$).
    3. Continue **till the last hidden layer** (layer before output).
    4. **Add the output layer** and train the **whole network** with the task-specific (supervised) objective $\min_\theta \frac{1}{m} \sum_i (y_i - f(x_i))^2$ → **fine-tuning**.
- In effect: weights **initialized** by the greedy unsupervised objective, then **fine-tuned** with the supervised objective.

C. Why does it work? (Erhan et al. 2009; Larochelle et al. 2009)

Hypothesis	Question	Evidence on slides
Better optimization	Without pre-training, can we not drive $\mathscr{L}(\theta)$ to 0 even on training data?	Error surface is highly non-convex (hills, plateaus, valleys), but given large capacity of DNNs it's still easy to reach a 0-error region. **Larochelle et al.: if the last layer has large capacity, $\mathscr{L}(\theta) \to 0$ even without pre-training; if capacity is small, pre-training helps.**
Better regularization	Does it constrain weights to good regions?	Regularization constrains weights to regions of parameter space (L_1: most weights 0; L_2: prevents large weights). Unsupervised objective $\Omega(\theta) = \frac{1}{m} \sum\sum (x_{ij} - \hat{x}_{ij})^2$ acts as an **additional constraint**: weights lie where **characteristics of the data are captured well**; learning is **not greedy w.r.t. the supervised objective**.

Hypothesis	Question	Evidence on slides
Better initialization		Pre-training is **more robust to random initializations**; accepted hypothesis: pre-training leads to **better weight initializations** (layers capture internal characteristics of the data).

- Since 2006–2009, deep learning evolved via: **better optimization algorithms, better regularization methods, better activation functions, better weight initialization strategies** (+ later batch norm).

F. Traps

- Pre-training is **greedy** (layer by layer, earlier layers fixed) and **unsupervised**; **fine-tuning** is supervised and updates **all** layers.
- "Pre-training always needed to reach zero training error" → **False** (not if last layer has large capacity).
- Original paper used **RBMs**, not autoencoders.

9.3 Better activation functions

A. Why non-linearity

- Replacing every sigmoid with a linear map: $y = w_4(w_3(w_2(w_1 x)))$ is just a **linear transformation of** x → only **linear decision boundaries**. A deep **non-linear** network can learn arbitrary boundaries (Universal Approximation Theorem). (Material largely from Karpathy's cs231n notes.)

Sigmoid — problems

$\sigma(x) = \frac{1}{1+e^{-x}} \in [0,1]$, $\sigma'(x) = \sigma(x)(1-\sigma(x))$.

1. **Saturated neurons cause gradients to vanish.** Saturation: $\sigma(x) = 1$ or $0 \Rightarrow$ gradient $= 0$. In the chain rule a factor $\partial h_3 / \partial a_3 = \sigma(a_3)(1-\sigma(a_3))$ appears. Neurons saturate e.g. when **weights are initialized to very high values** ($\sum w_i x_i$ large) → gradients vanish, training stalls.
2. **Not zero-centred.** For $a_3 = w_1 h_{21} + w_2 h_{22}$, $\nabla w_1 = (\text{common term}) \cdot h_{21}$, $\nabla w_2 = (\text{common term}) \cdot h_{22}$, and $h_{21}, h_{22} \in [0,1]$ are positive ⇒ **all gradients at a layer are either all positive or all negative** ⇒ update directions restricted to two quadrants ⇒ reaching an optimum in a "not possible" quadrant requires a **zig-zag path**.
3. **Computationally expensive** (because of $\exp(x)$).

Activation table

Function	Formula	Derivative	Range	Zero-centred	Saturates	Dies	Cost
Sigmoid	$\frac{1}{1+e^{-x}}$	$\sigma(1-\sigma)$ (max 0.25 at 0)	$[0,1]$	No	Yes (both sides)	–	expensive (exp)
tanh	$\tanh(x)$	$1-\tanh^2(x)$ (max 1 at 0)	$[-1,1]$	**Yes**	Yes (gradient still vanishes at saturation)	–	expensive
ReLU	$\max(0,x)$	0 if $x<0$; 1 if $x>0$	$[0,\infty)$	No	**No (in positive region)**	**Yes**	cheap
Leaky ReLU	$\max(0.01x, x)$	0.01 if $x<0$; 1 if $x>0$	$(-\infty,\infty)$	close to zero-centred	No	**No**	cheap

Function	Formula	Derivative	Range	Zero-centred	Saturates	Dies	Cost
Parametric ReLU	$\max(\alpha x, x)$, α learned by backprop	α / 1			No	No	cheap
ELU	x if $x > 0$; $ae^x - 1$ if $x \leq 0$ (slide)	1 / ae^x		close to zero-centred	No in +ve region ("all benefits of ReLU")	No ("small gradient flows")	**expensive** (exp)
Maxout	$\max(w_1^T x + b_1,\ w_2^T x + b_2)$	gradient flows to active (max) piece			**No**	**No**	**doubles #parameters**

ReLU details

- $\max(0, x)$ is **non-linear**. Two ReLUs combine to give a piecewise-linear approximation of sigmoid: $f(x) = \max(0, x+1) - \max(0, x-1)$ (equals 0 for $x \leq -1$, $x+1$ on $[-1, 1]$, 2 for $x \geq 1$).
- **Advantages:** no saturation in positive region; computationally efficient; **converges much faster** than sigmoid/tanh in practice (Krizhevsky, Sutskever, Hinton 2012 — ImageNet/AlexNet).
- **Caveat — dead ReLU:** if a large gradient pushes bias b to a **large negative** value, then $w_1 x_1 + w_2 x_2 + b < 0$ for all inputs → neuron outputs 0 **and** $\partial h_1 / \partial a_1 = 0 \rightarrow w_1, w_2, b$ never updated (zero term in chain rule) → **dead forever**.
  - A **large fraction** of ReLUs can die if the **learning rate is too high**.
  - Remedies: **initialize bias to a positive value (0.01)**; use ReLU variants.
- Leaky ReLU: $0.01x$ ensures at least a small gradient flows → won't die.
- Maxout **generalizes ReLU and Leaky ReLU** (Additional clarification: $w_1 = 0, b_1 = 0$ gives ReLU).

Things to remember (slide)

- Sigmoids are bad. **ReLU is more or less the standard unit for CNNs.** Can explore Leaky ReLU/Maxout/ELU. **tanh/sigmoids are still used in LSTMs/RNNs.**

E. Numerical (verified)

x	$\sigma(x)$	$\sigma'(x)$	$\tanh'(x)$
0	0.5	0.25	1
2	0.881	0.105	0.0707
5	0.9933	0.00665	0.00018

- Chain of 5 sigmoid derivatives at best: $0.25^5 \approx 0.00098$ (vanishing).
- Leaky ReLU at $x = -3$: -0.03, slope 0.01. ELU ($a = 1$) at $x = -1$: $e^{-1} - 1 = -0.632$, slope $e^{-1} = 0.368$.
- ⚠ Slide note: ELU written as "$ae^x - 1$"; the standard ELU is $a(e^x - 1)$. They agree for $a = 1$; the slide's version is discontinuous at 0 unless $a = 1$.

F. Traps

- tanh fixes **zero-centring** but **not** saturation.
- ReLU does not saturate for $x > 0$ but has **zero gradient for** $x < 0$ (dead neurons).
- ReLU is non-linear (piecewise linear ≠ linear).
- Maxout cost: **doubles parameters**; ELU cost: **exp**.

9.4 Better initialization strategies

Zero / equal initialization — symmetry breaking problem

- All weights 0 ⇒ $a_{11} = w_{11}x_1 + w_{12}x_2 = 0 = a_{12} \Rightarrow h_{11} = h_{12}$ (all neurons in layer 1 get the same activation).
- Backprop: $\nabla w_{11} = \frac{\partial \mathscr{L}}{\partial y}\frac{\partial y}{\partial h_{11}}\frac{\partial h_{11}}{\partial a_{11}}x_1$ and ∇w_{21} is identical ⇒ **same update, remain equal; symmetry never breaks** during training. Same for w_{12}, w_{22} and all layer-2 weights.
- This is the **symmetry breaking problem**; it happens whenever **all weights are initialized to the same value** (not only zero).
- ⚠ Slide note: slide writes "$a_{12} = a_{12}$"; it means $a_{11} = a_{12}$.

Experiment setup

Input: 1000 points ∈ $\mathbb{R}^{500}$ from unit Gaussian; 5 layers × 500 neurons; forward prop with different inits.

Init	Code on slide	tanh activations	sigmoid activations	Effect on gradients
Small random	`randn(fan_in, fan_out) * 0.01`	collapse to **≈0** in deeper layers	collapse to **≈0.5**	activations near 0 ⇒ $\nabla w \propto$ activation ≈ 0 ⇒ **vanishing gradient**
Large random	`randn(fan_in, fan_out)`	**saturate at ±1**	**saturate at 0/1**	saturated ⇒ gradients ≈ 0 ⇒ **vanishing gradient**
Scaled $1/\sqrt{n}$	`randn(fan_in, fan_out) / sqrt(fan_in)`	healthy spread in all layers	healthy spread	good
$1/\sqrt{n}$ with **ReLU**	`randn(fan_in, fan_out) / sqrt(fan_in)`	(ReLU net, 10 layers) activations **collapse towards 0** with depth	—	"does not work" for ReLU
He (ReLU)	`randn(fan_in, fan_out) / sqrt(fan_in/2)`	(ReLU net) distribution **stable across layers**	—	"better performance" for ReLU

Variance derivation (keep result + assumptions)

Assumptions: **zero-mean inputs and weights**; $\mathrm{Var}(x_i) = \mathrm{Var}(x)\ \forall i$; $\mathrm{Var}(w_{1i}) = \mathrm{Var}(w)\ \forall i$; (implicitly) weights independent of inputs and of each other; **non-linearity ignored** (pre-activation $s = \sum wx$ propagated linearly); weights in all layers have the same variance.

$$s_{11} = \sum_{i=1}^{n} w_{1i}x_i, \quad \mathrm{Var}(s_{1i}) = \big(n\mathrm{Var}(w)\big)\mathrm{Var}(x)$$

(using $\mathrm{Var}(wx) = E[w]^2\mathrm{Var}(x) + E[x]^2\mathrm{Var}(w) + \mathrm{Var}(x)\mathrm{Var}(w)$, first two terms 0 under zero means.)

$$\mathrm{Var}(s_{ki}) = [n\mathrm{Var}(w)]^k\,\mathrm{Var}(x)$$

- $n\mathrm{Var}(w) \gg 1$ ⇒ variance **blows up**; $n\mathrm{Var}(w) \to 0$ ⇒ variance **shrinks (vanishes)**.
- Want $n\mathrm{Var}(w) = 1$ ⇒ draw from **unit Gaussian and scale by** $1/\sqrt{n}$: $n\mathrm{Var}(z/\sqrt{n}) = n \cdot \frac{1}{n}\mathrm{Var}(z) = 1$ (using $\mathrm{Var}(az) = a^2\mathrm{Var}(z)$).
- (Additional clarification) this $1/\sqrt{n_{in}}$ scaling is commonly called **Xavier/Glorot** initialization; the slides just present it as "scale by $1/\sqrt{n}$".

- **Figures (slides 56–58):** with $1/\sqrt{n}$, tanh activations keep a healthy spread in all 5 layers (layer 1 broad, later layers bell-shaped); sigmoid likewise. With **ReLU +** $1/\sqrt{n}$ (10-layer plot) the activation histograms **collapse progressively towards 0** in deeper layers. With **ReLU +** $1/\sqrt{n/2}$ the distribution stays similar across all layers ("better performance").
- **ReLU:** $1/\sqrt{n}$ "does not work"; **He et al.**: a **factor of 2** is needed because ReLU's range is restricted to the **positive half** of the space ⇒ $W = \text{randn}/\sqrt{n/2}$, i.e. $\text{Var}(w) = 2/n$, std $= \sqrt{2/n}$.

E. Numerical (verified)

- $n = 500$: small init (0.01, Var 10^{-4}) ⇒ $n\text{Var}(w) = 0.05$ ⇒ after 5 layers factor $0.05^5 = 3.1 \times 10^{-7}$ (vanishes). Large init (Var 1) ⇒ $500^5 = 3.1 \times 10^{13}$ (explodes).
- $n = 500$: Xavier std $= 1/\sqrt{500} = 0.0447$ (Var 0.002, $n\text{Var} = 1$); He std $= \sqrt{2/500} = 0.0632$ (Var 0.004).
- $n = 256$: Xavier std $1/16 = 0.0625$; He std ≈ 0.0884.
- $n = 100$, Var$(w)=0.02$, Var$(x)=1$, 3 layers: $(100 \times 0.02)^3 = 8$.

F. Traps

- Zero init fails due to **symmetry**, not because zeros are "too small"; any constant init fails the same way.
- Both too-small and too-large init give **vanishing** gradients for tanh/sigmoid (small: activations ≈ 0 so $\nabla w \propto h \approx 0$; large: saturation).
- Here n = **fan-in** (number of inputs to the neuron).
- Variance grows/shrinks **exponentially** with depth k.

9.5 Batch Normalization

A. Core Concept

- Lets us be **less careful about initialization**.
- Intuition: with mini-batch training, if the **distribution of** h_3 **keeps changing across mini-batches**, learning the weights of the next layer is **hard**. (Additional clarification: this is often called *internal covariate shift*.)
- Fix: make pre-activations at each layer **unit Gaussian** by **standardizing** them, with mean/variance **computed from the mini-batch**:

$$\hat{s}_{ik} = \frac{s_{ik} - E[s_{ik}]}{\sqrt{\text{var}(s_{ik})}}$$

⇒ the distribution of inputs at different layers does not change across batches.

- Slide figure: the mini-batch X is an $N \times D$ matrix (N examples × D dimensions); mean and variance are computed **per column**, i.e. per dimension k over the N examples of the batch.
- **Legal?** Yes — the BN layer is **differentiable** (like tanh), so we can **backpropagate through it**.
- **Where:** network is …→ tanh → **BN** → tanh → **BN** → tanh …, i.e. BN normalizes the **pre-activation** feeding the non-linearity (input to the tanh layer).
- **Catch:** forcing unit-Gaussian input to tanh may not be best → let the network learn: add

$$y^{(k)} = \gamma^k \hat{s}_{ik} + \beta^{(k)}$$

γ^k, β^k are **additional learnable parameters** (per unit k). If the network learns $\gamma^k = \sqrt{\text{var}(x^k)}$, $\beta^k = E[x^k]$ it **recovers** s_{ik} (identity) if that is more favourable.

- MNIST, 2-layer comparison (slide figure): layer-1 activation histograms with and without BN both look roughly Gaussian on $[-3, 3]$. In the loss-vs-epochs plot, **the loss falls much faster in early epochs with BN**, and both curves reach a similar loss by about 90–100 epochs.

D. Formula (standard form)

(Additional clarification — the slide omits ε and the explicit batch estimates.) For mini-batch $\{s_1, \ldots, s_m\}$ of unit k:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} s_i, \quad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(s_i - \mu_B)^2, \quad \hat{s}_i = \frac{s_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{s}_i + \beta$$

ϵ = small constant for numerical stability (avoid division by 0). At test time, running averages of μ_B, σ_B^2 from training are used.

- ⚠ Slide note: the recovery slide writes $\gamma^k = \sqrt{var(x^k)}$, $\beta^k = E[x^k]$ — the x^k here refers to the un-normalized pre-activation s_{ik}.

E. Numerical (verified)

Batch $s = (1, 2, 3, 6)$: $\mu = 3$, $\sigma^2 = 3.5 \to \hat{s} = (-1.069, -0.535, 0, 1.604)$; with $\gamma = 2, \beta = 1$: $y = (-1.138, -0.069, 1, 4.207)$. Setting $\gamma = \sqrt{3.5}, \beta = 3$ recovers s.

F. Traps

- Mean/variance are from the **mini-batch**, not the whole dataset (during training).
- γ, β are **learned** (by backprop), not computed from data; BN can represent the **identity**.
- BN is applied per unit (per dimension k) to **pre-activations** (on slides).
- BN does not make init irrelevant; it makes training **less sensitive** to it.

G. Quick Revision Summary — Lecture 9

- Greedy layerwise unsupervised pre-training (Hinton & Salakhutdinov 2006, RBMs; here AEs): train layer by layer to reconstruct previous layer, fix, stack, then supervised fine-tuning. Works via better optimization (only when capacity is small), regularization (unsupervised objective constrains weights), better initialization.
- Sigmoid: saturation → vanishing grads; not zero-centred → same-sign gradients → zig-zag; exp cost. tanh: zero-centred but saturates. ReLU: no +ve saturation, cheap, fast convergence, but dead neurons (high LR; init bias 0.01). Leaky (0.01x), PReLU (α learned), ELU (exp cost), Maxout (2× params).
- Zero/equal init → symmetry never breaks. Small init → activations collapse; large → saturation; both → vanishing gradients.
- $\mathrm{Var}(s_k) = [n\mathrm{Var}(w)]^k \mathrm{Var}(x)$ ⇒ need $n\mathrm{Var}(w) = 1$ ⇒ $w \sim \mathcal{N}(0,1)/\sqrt{n}$; ReLU (He): $/\sqrt{n/2}$.
- BN: standardize pre-activations using mini-batch mean/var, then $\gamma\hat{s} + \beta$ (learnable, can recover identity); differentiable; less careful init.
- 2016–17 outlook: even better optimization, data-driven initialization, beyond batch normalization.

Overview: Lectures 10–11: Word Vectors and CNNs

Lectures covered / modules:

- **Lecture 10 – Learning Vectorial Representations of Words**: 10.1 One-hot representations · 10.2 Distributed representations (co-occurrence matrix, PMI/PPMI) · 10.3 SVD for word representations · 10.4 Continuous bag of words (CBOW) · 10.5 Skip-gram (+ negative sampling) · 10.6 Contrastive estimation · 10.7 Hierarchical softmax · 10.8 GloVe · 10.9 Evaluating word representations · 10.10 Relation between SVD and word2vec
- **Lecture 11 – CNNs, LeNet, AlexNet, ZF-Net, VGGNet, GoogLeNet, ResNet**: 11.1 The convolution operation · 11.2 Input size, output size and filter size · 11.3 Convolutional neural networks (sparse connectivity, weight sharing, pooling, LeNet-5, training) · 11.4 ImageNet success stories (AlexNet, ZF-Net, VGGNet) · 11.5 GoogLeNet and ResNet

---

Week 9 · Lecture 10: Learning Vectorial Representations of Words

Why this matters: an ML model $\hat{y} = f(x)$, for example sentiment of a review, needs each input word turned into a vector x. The lecture moves from **count-based** methods (one-hot, co-occurrence, PMI, SVD) to **(direct) prediction-based** methods (CBOW, skip-gram/word2vec, with negative sampling, contrastive estimation and hierarchical softmax), then **GloVe** (count + learn), evaluation, and finally the result that **word2vec implicitly factorizes a PMI matrix**, much like SVD.

10.1 One-hot representations

A. Core concept

- **Vocabulary** V: the set of all unique words across all input streams (sentences/documents) in the corpus.
- **One-hot vector** of word i: size $|V|$, with a 1 in position i and 0 in the other $|V| - 1$ positions.

C. Properties and limitations

Property	Value / consequence
Dimension	$\|V\|$, very large (≈50K for PTB, 13M for Google 1T corpus)
Sparsity	Only 1 non-zero out of $\|V\|$
Euclidean distance between ANY two distinct words	$\sqrt{2}$
Cosine similarity between ANY two distinct words	0
Similarity	**Not captured**: cat–dog are as far apart as cat–truck

- Ideally cat and dog (both domestic animals) should be closer than cat and truck. One-hot cannot do this because all vectors are mutually orthogonal and equidistant.

E. Numerical

- $\|e_i - e_j\| = \sqrt{1^2 + (-1)^2} = \sqrt{2} \approx 1.414$, and $\cos(e_i, e_j) = e_i^\top e_j = 0$, for any $i \neq j$, **independent of** $|V|$.
- Trap: increasing $|V|$ increases the dimension but does **not** change the distance ($\sqrt{2}$) or the cosine (0).

10.2 Distributed representations: co-occurrence matrix, PMI, PPMI

A. Core concept

- **Distributional hypothesis**: "You shall know a word by the company it keeps" (Firth, 1957). For example, *bank* is represented by its accompanying words (*financial, deposits, credit*).
- **Co-occurrence matrix**: a **terms × terms** matrix (also called the **word × context** matrix). Entry (i, j) is the number of times term j appears in the **context** of term i, where the context is a **window of k words** around the term. The toy example uses $k = 2$.
- The word and context sets may be the **same or different**.

- **Each row** gives the vector of the corresponding **word**, and **each column** gives the vector of the corresponding **context**.

B. Terminology

- **Window size** k: number of words on each side counted as context.
- **Stop words** (a, the, for…): very frequent, so their counts are very high and they dominate the vectors.

C. Problems and fixes (from the slides)

Problem	Type	Fix
Stop words have huge counts	fixable	**Sol 1:** ignore very frequent words
	fixable	**Sol 2:** threshold t (e.g. 100): $X_{ij} = \min(\text{count}(w_i, c_j), t)$
	fixable	**Sol 3:** use **PMI** instead of raw counts (then PMI_0 or PPMI to handle $-\infty$)
Very high dimensional ($\|V\|$), very sparse, grows with vocabulary	severe	**Dimensionality reduction (SVD)**, see 10.3

D. Formulas

$$\text{PMI}(w, c) = \log \frac{p(c \mid w)}{p(c)} = \log \frac{\text{count}(w, c) \cdot N}{\text{count}(c) \cdot \text{count}(w)}$$

- **Symbols:** N = total number of words; $\text{count}(w, c)$ = co-occurrence count; $\text{count}(w)$, $\text{count}(c)$ = individual counts.
- **Interpretation:** PMI > 0 means w and c co-occur more than chance; PMI = 0 means independent; PMI < 0 means less than chance.
- **Problem:** if $\text{count}(w, c) = 0$, then $\text{PMI} = \log 0 = -\infty$.

$$\text{PMI}_0(w, c) = \begin{cases} \text{PMI}(w, c) & \text{if count}(w, c) > 0 \\ 0 & \text{otherwise} \end{cases} \qquad \text{PPMI}(w, c) = \begin{cases} \text{PMI}(w, c) & \text{if PMI}(w, c) > 0 \\ 0 & \text{otherwise} \end{cases}$$

- **PMI_0** zeroes out only the **unseen** pairs, so negative PMI values survive. **PPMI** zeroes out **all non-positive** values, so the entries are ≥ 0.
- The toy PPMI table appears to use the **natural log** (inferred: human–machine = 2.944 = ln 19; the slides do not state the base or the exact N used).
- Toy table (k = 2) examples from the slides: count(human, machine) = 1, count(system, user) = 2. After PPMI: human–machine 2.944, human–for 2.25, system–for 1.15, system–user 1.84; the matrix is symmetric.

E. Numerical example (verified)

count(w,c) = 10, N = 1000, count(w) = 50, count(c) = 20, so PMI = ln(10·1000 / (50·20)) = ln 10 ≈ **2.303** ($\log_2$ gives 3.32). PPMI = 2.303. If count(w,c) = 0, then PMI = −∞, while PMI_0 = PPMI = 0.

F. Traps

- PMI_0 ≠ PPMI. A pair with count > 0 but PMI < 0 keeps a negative value in PMI_0 and becomes 0 in PPMI.
- The thresholding fix caps counts with **min**, not max.
- The co-occurrence matrix is $|V| \times |V|$ when words and contexts are the same set. That is still high-dimensional and sparse, even after PPMI.

10.3 SVD for learning word representations

A. Core concept

- Apply SVD to the PPMI co-occurrence matrix X ($m \times n$; written X for X_{PPMI}):

$$X_{m\times n} = U_{m\times k}\,\Sigma_{k\times k}\,V^\top_{k\times n} = \sigma_1 u_1 v_1^\top + \sigma_2 u_2 v_2^\top + \cdots + \sigma_k u_k v_k^\top$$

- Each term $\sigma_i u_i v_i^\top \in \mathbb{R}^{m\times n}$ is a **rank-1** matrix (an $m \times 1$ vector times a $1 \times n$ vector).
- Truncating after the first term gives the **best rank-1 approximation**, truncating after two terms gives the best rank-2 approximation, and so on (SVD theorem).
- u_1, v_1, σ_1 store the **most** information in X (akin to principal components). Each later term stores **less and less important** information.
- **Discovers latent semantics.** Colour-bits analogy: compressing 8 bits to 4 bits keeps the most important bits, and the latent similarity between shades of green becomes obvious.

C. Key insights

- **Rank-1 storage:** reconstructing from a rank-1 term needs only $m + n + 1$ numbers ($u \in \mathbb{R}^m, v \in \mathbb{R}^n, \sigma \in \mathbb{R}$) instead of $m \times n$.
- After the low-rank reconstruction $\hat{X}$, latent co-occurrences show up that were 0 in X: {system, machine} and {human, user}. Toy example: cosine sim(human, user) is **0.21** using $XX^\top$ rows and **0.33** using $\hat{X}\hat{X}^\top$.
- $XX^\top$**:** its ij-th entry is the dot product of row i and row j of X (word $i \cdot$ word j), so it **(roughly) captures the cosine similarity** between word$_i$ and word$_j$. Slide toy check: rows $(1, 2, 3)$ and $(1, 3, 5)$ give the entry $1 + 6 + 15 = 22$.
- (Additional clarification) The exact cosine is the normalized entry: $\cos(i, j) = \dfrac{(XX^\top)_{ij}}{\sqrt{(XX^\top)_{ii}(XX^\top)_{jj}}}$. With the slide's $\hat{X}\hat{X}^\top$ entries, $6.84/\sqrt{25.4 \times 17.11} = 0.33$ ✓. With the $XX^\top$ entries shown, $7.01/\sqrt{32.5 \times 28.3} = 0.23$; the slide says 0.21, presumably computed from the full matrix.
- **Why not use the rows of $\hat{X}$ as representations?** They are still n-dimensional (high-dimensional).
- **Wishlist:** low-dimensional representations whose dot products equal those of the rows of $\hat{X}$.

D. Key derivation result

$$\hat{X}\hat{X}^\top = (U\Sigma V^\top)(U\Sigma V^\top)^\top = U\Sigma V^\top V\Sigma^\top U^\top = U\Sigma\Sigma^\top U^\top = (U\Sigma)(U\Sigma)^\top \quad (\because V^\top V = I)$$

- **Conventionally:** $W_{word} = U\Sigma \in \mathbb{R}^{m\times k}$ gives the representations of the m words (row i = word i, now k-dimensional), and $W_{context} = V$ gives the representations of the contexts.
- **Condition:** V has orthonormal columns ($V^\top V = I$), which is why V drops out.

E. Numerical

- $m = n = 1000$ and rank $k = 100$: storage is $k(m + n + 1) = 100 \times 2001 = 200{,}100$ numbers instead of 10^6. A rank-1 approximation needs 2001.
- Shape check: $W_{word} = U\Sigma$ is $m \times k$. Each word gets a k-dimensional vector, where $k \ll n$.

F. Traps

- The word representation is $U\Sigma$, **not** U alone and **not** a row of $\hat{X}$.
- $W_{context} = V$, **not** $V\Sigma$.
- SVD gives the **best** rank-k approximation. The rank-1 term holds the **most** information, not the least.

- ⚠ Slide note: the toy $XX^\top$ / $\hat{X}\hat{X}^\top$ tables have a few inconsistent entries. For example, $XX^\top$(system, system) = 0 even though the system row is non-zero, and $\hat{X}\hat{X}^\top$ (for, system) = 0.96 while (system, for) = 18.03. Use them only for the qualitative point (0.21 → 0.33).

10.4 Continuous bag of words (CBOW)

A. Core concept

- **Count-based models** (SVD etc.) use co-occurrence counts. **(Direct) prediction-based models** learn representations directly by solving a prediction task.
- **Task:** predict the n-th word given the previous $n-1$ words (e.g. "he sat on a chair"). Training data is every n-word window in the corpus (e.g. all of Wikipedia), so it is easily available.
- Simplest case $n = 2$: predict the 2nd word from the 1st.
- **Model:** a feed-forward network.
  - **Input:** one-hot $x \in \mathbb{R}^{|V|}$ of the context word.
  - **Hidden:** $h \in \mathbb{R}^k$.
  - **Output:** softmax over $|V|$ classes (multi-class classification).
- **Parameters:** $W_{context} \in \mathbb{R}^{k \times |V|}$ and $W_{word} \in \mathbb{R}^{k \times |V|}$. This assumes the word and context vocabularies are the same set, each of size $|V|$.

B. Interpretation of the weight matrices

- $W_{context}x$ with one-hot x simply **selects the i-th column** of $W_{context}$. There is a one-to-one correspondence between words and columns, so **column i of $W_{context}$ is the representation of context i** (u_c).
- $P(\text{word} = i \mid \text{sat})$ depends on the i-th column of W_{word}, so **column i of W_{word} is the representation of word i** (v_w).
- This is analogous to SVD, where the rows of W_{word} were the word vectors. There the matrices were learned differently.

D. Formulas

$$h = W_{context}\, x_c = u_c, \qquad \hat{y}_w = P(w \mid c) = \frac{\exp(u_c \cdot v_w)}{\sum_{w' \in V} \exp(u_c \cdot v_{w'})}$$

$$\mathcal{L}(\theta) = -\log \hat{y}_w = -\log P(w \mid c) \quad \text{(cross-entropy)}$$

$$\nabla_{v_w} = -\big(u_c - \hat{y}_w\, u_c\big) = -u_c(1 - \hat{y}_w), \qquad v_w \leftarrow v_w - \eta \nabla_{v_w} = v_w + \eta\, u_c(1 - \hat{y}_w)$$

- **Notation:** c = index of the context word (sat), w = index of the correct output word (on).
- **Interpretation of the update:**
  - If $\hat{y}_w \to 1$ (already correct), v_w is **not** updated.
  - If $\hat{y}_w \to 0$, a fraction of u_c is **added** to v_w, which increases the cosine similarity between v_w and u_c.
  - The training objective therefore maximizes the cosine similarity between word (v_w) and context (u_c).
- **Transitivity intuition:** words w and w' that appear in similar contexts c both get high similarity with u_c, and hence (intuitively) with each other. The slides call this a reasonable intuition with **no formal proof**.

Window size d (multiple context words)

- With $d-1$ context words, the input is $x \in \mathbb{R}^{(d-1)|V|}$, a concatenation of one-hots. The weights are $[W_{context}, W_{context}]$, which means **stacking copies of the same matrix** (shared, not new parameters).

$$h = \sum_{i=1}^{d-1} u_{c_i} \qquad (\text{sum of the context columns})$$

- Slide example with 2 context words: the columns for "sat" $(0.5, -1, 1.7)$ and "he" $(2, -2, 3)$ sum to $h = (2.5, -3, 4.7)$.
- In practice you don't do the matrix multiplication; you just look up the columns and add them.
- During backprop, the loss depends on $\{W_{word}, u_{c_1}, \ldots, u_{c_{d-1}}\}$, so all of these get updated.

C. Problem

- The **softmax is expensive**: the denominator sums over **all** $|V|$ **words**. This motivates negative sampling, contrastive estimation and hierarchical softmax (10.5–10.7).

E. Numerical

- **Parameter count:** $2k|V|$. For $|V| = 10{,}000$ and $k = 300$, that is $2 \times 300 \times 10{,}000 = 6\text{M}$. Using a window of d words does **not** change it (the stacked copies are shared).
- **Update example:** $u_c = [1, 2]$, $\hat{y}_w = 0.2$, $\eta = 0.5$, so $\Delta v_w = 0.5 \cdot 0.8 \cdot [1, 2] = [0.4, 0.8]$.
- **Shapes:** x is $|V| \times 1$, h is $k \times 1$, output is $|V| \times 1$.

F. Traps

- CBOW predicts the **word from the context (bag of words)**. Skip-gram predicts the **context from the word**.
- The gradient factor is $(1 - \hat{y}_w)$. The update is **zero** when the prediction is already perfect.
- ⚠ Slide note: the slides write the output as $W_{word}h$ with $W_{word} \in \mathbb{R}^{k \times |V|}$. Dimensionally this should be $W_{word}^{\top}h$ to give $|V|$ scores.
- ⚠ Slide note: on the window-size slide the figure labels $W_{word} \in \mathbb{R}^{k \times 2|V|}$. The output layer should still be $k \times |V|$; only the input side doubles.
- ⚠ Slide note: one slide says "v_w is the column of W_{word} corresponding to **context** w". It means **word** w.
- (Additional clarification) The original word2vec CBOW **averages** the context vectors. The slides use a **sum**.

10.5 Skip-gram model (and negative sampling)

A. Core concept

- **Skip-gram** predicts **context words given an input word**. The roles of context and word are swapped relative to CBOW, and context words are typically predicted on **both sides**.
- With one context word, the update rule for u_c is the same as the one derived for v_w in CBOW.
- With multiple context words, the loss is a **sum of cross-entropies**:

$$\mathcal{L}(\theta) = -\sum_{i=1}^{d-1} \log \hat{y}_{w_i}$$

- **Same problem as CBOW:** the softmax is expensive. The three solutions are (1) negative sampling, (2) contrastive estimation, (3) hierarchical softmax.

Negative sampling (the famous word2vec = "skip-gram with negative sampling")

- D = the set of all **correct** (w, c) pairs in the corpus, e.g. (sat, on), (sat, chair), (on, sat)…
- D' = a set of **incorrect** (w, r) pairs, built by randomly sampling a context r that **never appeared** with w, e.g. (sat, oxygen), (chair, walking).
- **Model:** a binary classifier on the dot product

$$p(z = 1 \mid w, c) = \sigma(u_c^\top v_w) = \frac{1}{1 + e^{-u_c^\top v_w}}, \qquad p(z = 0 \mid w, r) = 1 - \sigma(u_r^\top v_w) = \sigma(-u_r^\top v_w)$$

- **Objective** (maximize the likelihood of both sets, then take logs):

$$\max_\theta \sum_{(w,c)\in D} \log \sigma(u_c^\top v_w) + \sum_{(w,r)\in D'} \log \sigma(-u_r^\top v_w)$$

- θ = all word vectors v_w and context vectors u_c.
- Pushes $u_c^\top v_w$ **up** for true pairs and $u_r^\top v_w$ **down** for random pairs.
- **Sampling:**
  - Mikolov et al. sample k **negative pairs for every positive pair**, so $|D'| = k|D|$.
  - r is drawn from a **modified unigram distribution**: $r \sim p(r)^{3/4}$, i.e. proportional to $\text{count}(r)^{3/4}$.

C. Properties

- It replaces the $|V|$-way softmax with $k + 1$ **sigmoid (binary) terms** per positive pair, so the normalization over V disappears.
- The $3/4$ power **flattens** the distribution. Frequent words are sampled relatively less and rare words relatively more than under the raw unigram.

E. Numerical (verified)

- Counts (100, 10, 1):
  - Unigram probabilities: (0.901, 0.090, 0.009).
  - With power 3/4: (0.827, 0.147, 0.026). The rare word's probability roughly triples.
- $k = 5$ negatives with $|D| = 10^6$ gives $|D'| = 5 \times 10^6$.
- One positive pair with $u_c^\top v_w = 2$ and one negative pair with $u_r^\top v_w = -1$: objective $= \log \sigma(2) + \log \sigma(1) = -0.127 + (-0.313) = -0.440$.

F. Traps

- Negative pair uses $\sigma(-u_r^\top v_w)$, i.e. $1 - \sigma(u_r^\top v_w)$. It is **not** $\sigma(u_r^\top v_w)$.
- The power is **3/4**, applied to counts. It is not 1/2 and not applied to probabilities after normalizing.
- $|D'| = k|D|$, so k is a multiplier on positives, not a fixed total.
- ⚠ Slide note: the slide writes $r \sim \text{count}(r)^{3/4}/N$. For a valid distribution the normalizer should be $\sum_{r'} \text{count}(r')^{3/4}$, not N.
- ⚠ Slide note: the combined-objective slide writes $v_c^\top v_w$ and $v_r^\top v_w$ where earlier slides used u_c, u_r for the context vectors. One line also mixes v_r into $1 - \frac{1}{1+e^{-v_r^\top v_w}}$. They mean the same thing.

10.6 Contrastive estimation

A. Core concept

- **Positive** window: "He sat **on** a chair". **Negative** (corrupted) window: "He sat **abracadabra** a chair".
- **Network:**
  - Concatenate v_c, v_w (size $2d$).
  - Hidden layer $W_h \in \mathbb{R}^{2d \times h}$.
  - Output $W_{out} \in \mathbb{R}^{h \times 1}$ gives a **score**: s for the true pair, s_c for the corrupted pair.
- **Goal:**
  - We want $s > s_c$, so maximize $s - s_c$.
  - We also want the difference to be at least a **margin** m.
  - If $s > s_c + m$, do nothing.
- Slide's final objective: "maximize $\max(0, s - (s_c + m))$".

F. Traps

- ⚠ Slide note: as written, "maximize $\max(0, s - (s_c + m))$" is inconsistent with "if $s > s_c + m$ don't do anything". The standard hinge (margin) form that matches the stated intent is: **minimize** $\max(0,\ m - (s - s_c))$ = $\max(0, s_c + m - s)$. (Additional clarification.)
- It is a **ranking/margin** loss on scores, **not** a probabilistic softmax. No normalization over V is needed.
- Notation switch: from contrastive estimation onward (and in hierarchical softmax) the slides write the **context** vector as v_c (earlier it was u_c). In hierarchical softmax, u_i denotes an **internal tree node** vector, not a context vector.

10.7 Hierarchical softmax

A. Core concept

- **Tree:**
  - Build a **binary tree with** $|V|$ **leaf nodes**, one per vocabulary word. There is a **unique path** from the root to each leaf.
  - $l(w_1), \ldots, l(w_p)$ are the nodes on the path to w.
  - $\pi(w)$ is a binary vector: $\pi(w)_k = 1$ if the path **branches left** at node $l(w_k)$, else 0.
  - Each **internal node** i has a vector u_i.
- **Parameters:** $W_{context}$ and $u_1, \ldots, u_{|V|}$. This is in effect the **same number of parameters as before**.
- $h = v_c$ (the context representation).

D. Formulas

$$P(w \mid v_c) = \prod_{k=1}^{|\pi(w)|} P(\pi(w)_k \mid v_c), \qquad P(\pi(w)_i = 1) = \frac{1}{1 + e^{-v_c^\top u_i}} = \sigma(v_c^\top u_i), \qquad P(\pi(w)_i = 0) = \frac{1}{1 + e^{v_c^\top u_i}}$$

- Example: $P(\text{on} \mid v_{sat}) = P(\pi(\text{on})_1 = 1) \cdot P(\pi(\text{on})_2 = 0) \cdot P(\pi(\text{on})_3 = 0)$.
- Predicting a word is the same as **predicting the correct unique path** from the root to it.
- v_c gets **high (low)** similarity with u_i if the path branches **left (right)** at u_i. Transitively, contexts that appear with the same words get similar representations.

C. Properties

- **Cost:** $|\pi(w)|$ computations instead of $|V|$ for softmax. For a balanced tree $|\pi(w)| \approx \log_2 |V|$ (additional clarification of "log|V|").
- **Tree construction:** even a **random arrangement** of words on the leaves does well in practice.
- (Additional clarification) The probabilities over all leaves automatically sum to 1, because left and right probabilities at each node sum to 1.

E. Numerical

- $|V| = 1024$: a balanced tree has depth $\log_2 1024 = 10$, so 10 sigmoid evaluations instead of a 1024-term softmax. $|V| = 50{,}000$ gives about $\lceil 15.6 \rceil = 16$.
- A binary tree with $|V|$ leaves has $|V| - 1$ internal nodes, hence about $k|V|$ parameters for the u_i, the same order as W_{word}.

10.8 GloVe representations

A. Core concept

- **Count-based** methods (SVD) rely on **global co-occurrence counts**. **Predict-based** methods learn from co-occurrence information, but locally (window by window). **GloVe combines the two** (count and learn).
- X_{ij} (symmetric, $X_{ij} = X_{ji}$) encodes **global** information about the co-occurrence of i and j. It is "global" because it is computed over the entire corpus.
- **Idea:** learn vectors whose dot product is faithful to the global $P(j \mid i)$.

D. Formulas

$$P(j \mid i) = \frac{X_{ij}}{\sum_j X_{ij}} = \frac{X_{ij}}{X_i}, \qquad v_i^\top v_j = \log P(j \mid i) = \log X_{ij} - \log X_i, \qquad v_j^\top v_i = \log X_{ij} - \log X_j$$

Adding the two gives $v_i^\top v_j = \log X_{ij} - \frac{1}{2}\log X_i - \frac{1}{2}\log X_j$. The $\log X_i, \log X_j$ terms depend only on the individual words, so they are replaced by **learned word-specific biases** b_i, b_j:

$$v_i^\top v_j + b_i + b_j = \log X_{ij}$$

$$\min_{v_i, v_j, b_i, b_j} \sum_{i,j} f(X_{ij}) \Big(\underbrace{v_i^\top v_j + b_i + b_j}_{\text{predicted}} - \underbrace{\log X_{ij}}_{\text{actual, from corpus}} \Big)^2$$

$$f(x) = \begin{cases} \left(\dfrac{x}{x_{max}}\right)^\alpha & x < x_{max} \\ 1 & \text{otherwise} \end{cases}$$

- **Why f:** without it the loss **weighs all co-occurrences equally**. The wishlist is that **neither rare nor frequent words are over-weighted**.
- f grows with x but is **capped at 1**. α is tuned per dataset.
- (Additional clarification) The GloVe paper uses $x_{max} = 100$, $\alpha = 3/4$. Also $f(0) = 0$, so pairs with $X_{ij} = 0$ (where $\log X_{ij}$ is undefined) drop out of the sum.

E. Numerical (verified)

With $x_{max} = 100$ and $\alpha = 0.75$: $f(10) = 0.1^{0.75} = 0.178$, $f(50) = 0.5^{0.75} = 0.595$, and $f(100) = f(500) = 1$.

F. Traps

- GloVe is a **weighted least-squares regression on log counts**. It is not a softmax/cross-entropy model.
- The biases absorb $\frac{1}{2}\log X_i$-type terms. They are **learned**, not computed.
- f caps frequent pairs at 1. It does **not** keep increasing.
- ⚠ Slide note: one GloVe slide writes $P(j|i) = X_{ij}/\sum X_i$. The intended form is X_{ij}/X_i with $X_i = \sum_j X_{ij}$. The illustrative X shown is the SVD-reconstructed toy matrix, which contains a negative entry (-0.13) whose log is undefined. Treat it as illustrative only.

10.9 Evaluating word representations

Task	How	Comparison
Semantic relatedness	Humans rate word pairs (S_{human}, e.g. cat–dog = 0.8). The model score is cosine $S_{model} = \frac{v_{cat}^\top v_{dog}}{\|v_{cat}\|\|v_{dog}\|}$ (e.g. 0.7)	Model 1 > Model 2 if corr(S_{model1}, S_{human}) > corr(S_{model2}, S_{human}) over many pairs
Synonym detection	Given a term and 4 candidates, pick $\arg\max_{c\in C} \cos(v_{term}, v_c)$ (e.g. levied → imposed / believed / requested / correlated)	Accuracy
Analogy (semantic)	brother : sister :: grandson : ? → nearest neighbour of $v_{sister} - v_{brother} + v_{grandson}$	Accuracy
Analogy (syntactic)	work : works :: speak : ? → nearest neighbour of $v_{works} - v_{work} + v_{speak}$	Accuracy

Which is best?

- **Baroni et al. 2014** (slide spells it "Boroni"): **predict** models consistently outperform **count** models in all tasks.
- **Levy et al. 2015** (a more thorough analysis): **good old SVD does better** than prediction-based models on **similarity** tasks but **not on analogy** tasks.

Traps

- Relatedness compares models by the **correlation** with human judgments, not by raw cosine values.
- Analogy uses a **vector offset** ($b - a + c$) followed by a nearest-neighbour search. Watch the order: for a : b :: c : ?, the query is $v_b - v_a + v_c$.
- Synonym detection picks the candidate with the **largest cosine**, and models are compared by **accuracy**.
- ⚠ Slide note: the synonym-detection candidate list prints "unposed". This is a typo for **imposed** (the intended synonym of *levied*).

10.10 Relation between SVD and word2vec

- SVD does an explicit **matrix factorization** of the co-occurrence (PPMI) matrix.
- **Levy et al. [2015]**: word2vec (skip-gram with negative sampling) **also implicitly does a matrix factorization**:

$$M = W_{context} * W_{word}, \qquad M_{ij} = \mathrm{PMI}(w_i, c_j) - \log k$$

where k = **number of negative samples**.

- So word2vec factorizes a **shifted PMI matrix**, very similar to what SVD does. This is why "good old SVD does just fine".
- **Numerical:** with $k = 5$ the shift is $\log 5 = 1.609$ (natural log). With $k = 1$ there is no shift ($\log 1 = 0$).
- ⚠ Slide note: with $W_{context}, W_{word} \in \mathbb{R}^{k \times |V|}$, the product must be $W_{context}^{\top} W_{word}$ ($|V| \times |V|$) to be dimensionally valid. Also, the symbol k is overloaded: it is the embedding dimension elsewhere and the number of negative samples here. The slide writes $M_{ij} = \text{PMI}(w_i, c_i) - \log(k)$; the context index should be j.

G. Lecture 10 – Quick Revision Summary

- **One-hot:** dimension $|V|$, sparse; any two distinct words have distance $\sqrt{2}$ and cosine 0, so there is **no similarity**.
- **Co-occurrence** (word × context, window k) → fixes for frequent words: ignore, $\min(\text{count}, t)$, PMI → PMI_0/PPMI → still $|V|$-dimensional and sparse → **SVD**.
- **SVD:** best rank-k approximation. $W_{word} = U\Sigma$ ($m \times k$), $W_{context} = V$, because $\hat{X}\hat{X}^{\top} = (U\Sigma)(U\Sigma)^{\top}$.
- **CBOW:**
  - $h = W_{context} x = u_c$ (column lookup), $\hat{y}_w = \text{softmax}(u_c \cdot v_w)$, loss $-\log \hat{y}_w$.
  - Update $v_w \mathrel{+}= \eta u_c (1 - \hat{y}_w)$.
  - For $d - 1$ context words, $h = \sum u_{c_i}$ (shared $W_{context}$).
- **Skip-gram:**
  - Predicts the context from the word. Loss = sum of cross-entropies.
  - The softmax over $|V|$ is expensive, so use negative sampling / contrastive estimation / hierarchical softmax.
- **Negative sampling:**
  - $\max \sum_D \log \sigma(u_c^{\top} v_w) + \sum_{D'} \log \sigma(-u_r^{\top} v_w)$.
  - k negatives per positive; $r \sim \text{count}^{3/4}$.
- **Contrastive estimation:** score with a margin m: true s vs corrupted s_c.
- **Hierarchical softmax:** binary tree with $|V|$ leaves; $P(w) = \prod$ sigmoids along the path; cost $|\pi(w)| \approx \log |V|$; a random tree works fine.
- **GloVe:** $\min \sum f(X_{ij})(v_i^{\top} v_j + b_i + b_j - \log X_{ij})^2$, with $f = (x/x_{max})^{\alpha}$ capped at 1.
- **Evaluation:** relatedness (correlation with humans), synonym detection, analogy (vector offset). SVD wins on similarity, predict models win on analogy (Levy 2015).
- **word2vec ≈ factorization** of $\text{PMI} - \log k$.

Likely MCQ takeaways

1. Cosine between any two one-hot vectors is 0 and the Euclidean distance is $\sqrt{2}$.
2. $W_{word} = U\Sigma$ (not U). Each $\sigma_i u_i v_i^{\top}$ has rank 1, and the rank-1 approximation needs $m + n + 1$ numbers.
3. CBOW update factor $(1 - \hat{y}_w)$: no update when the prediction is perfect.
4. Negative sampling uses $\sigma(-u_r^{\top} v_w)$ for negatives, k negatives per positive, and the 3/4 power.
5. Hierarchical softmax reduces cost from $|V|$ to path length ($\sim \log |V|$) with about the same number of parameters.

Week 10 · Lecture 11: Convolutional Neural Networks, LeNet, AlexNet, ZF-Net, VGGNet, GoogLeNet, ResNet

11.1 The convolution operation

A. Core concept: 1D

- Motivation: tracking an aeroplane with a **noisy** laser sensor. To get a less noisy estimate, average several measurements, giving more weight to recent ones (a weighted average):

$$s_t = \sum_{a=0}^{\infty} x_{t-a}\, w_{-a} = (x * w)_t$$

- **Terms:** x = **input**, w = **filter** (weight array), s = output of the convolution.
- In practice, sum only over a **small window** (slide: $a = 0..6$, 7 weights $w_{-6..0}$ = 0.01, 0.01, 0.02, 0.02, 0.04, 0.4, 0.5). **Slide the filter** over the input and compute s_t from a window around x_t.
- Verified: $s_6 = 1.90(0.5) + 1.80(0.4) + 1.70(0.04) + 1.40(0.02) + 1.20(0.02) + 1.10(0.01) + 1.00(0.01) = 1.811 \approx 1.80$ (as on the slide). The following outputs are 1.967 and 2.112 (slide: 1.96, 2.11), also matching.
- ⚠ Slide note: the slide's expansion "$s_6 = x_6w_0 + x_5w_{-1} + x_4w_{-2} + x_3w_{-3} + x_4w_{-4} + x_5w_{-5} + x_6w_{-6}$" has index typos. It should end with $+x_2w_{-4} + x_1w_{-5} + x_0w_{-6}$. On recomputation, the last three S values on the slide (2.16, 2.28, 2.42) do not match exactly (≈2.27, 2.41, 2.57). They are illustrative only.

2D convolution

$$S_{ij} = (I * K)_{ij} = \sum_{a=0}^{m-1}\sum_{b=0}^{n-1} I_{i-a,\,j-b}\, K_{a,b} \quad \text{(preceding neighbours)}$$

$$S_{ij} = \sum_{a=0}^{m-1}\sum_{b=0}^{n-1} I_{i+a,\,j+b}\, K_{a,b} \quad \text{(used in practice: succeeding neighbours)}$$

$$S_{ij} = \sum_{a=\lfloor -m/2 \rfloor}^{\lfloor m/2 \rfloor}\ \sum_{b=\lfloor -n/2 \rfloor}^{\lfloor n/2 \rfloor} I_{i-a,\,j-b}\, K_{\frac{m}{2}+a,\,\frac{n}{2}+b} \quad \text{(kernel centred on the pixel of interest; used for the rest of the lectu}$$

- **Toy example:**
  - A 3×4 input (a…ℓ) and a 2×2 kernel (w, x, y, z) give a 2×3 output.
  - First output = $aw + bx + ey + fz$, next = $bw + cx + fy + gz$, and so on.
  - This matches the $W_1 - F + 1$ rule in each dimension.
- **Standard kernels on slides:**

Kernel (3×3)	Effect
all 1s	**blurs** the image
[0 −1 0; −1 5 −1; 0 −1 0]	**sharpens** the image
[1 1 1; 1 −8 1; 1 1 1]	**detects edges**

- **Feature maps:**

- Slide the kernel over the image. Each position gives one output value, and the full output is a **feature map**.
- **Multiple filters give multiple feature maps.**

3D (volumes)

- For an RGB input the filter is also 3D (a **volume**). In this lecture the **filter always extends to the full depth** of the input.
- The filter moves only along the **height and width, not depth**. So this is effectively a **2D convolution on a 3D input**, and the output of each filter is **2D**.
- K filters give K feature maps, so the output depth is K.

F. Traps

- 1D filter on 1D input, 2D filter on 2D input. For a 3D input with a full-depth filter, the per-filter output is **2D, not 3D**.
- The "convolution" used in CNNs (the $I_{i+a,j+b}$ form) does not flip the kernel; it is technically cross-correlation (additional clarification). Since the kernel is learned, this makes no difference.

11.2 Relation between input size, output size and filter size

B. Quantities

Symbol	Meaning
W_1, H_1, D_1	width, height, depth of the input
F	spatial extent of each filter (filter is $F \times F \times D_1$; **depth = input depth**)
S	stride (interval at which the filter is applied)
P	padding (rows/columns of zeros added on each side)
K	number of filters
W_2, H_2, D_2	output dimensions

A. Building up the formula

1. **No padding, stride 1:** the kernel can't be placed at the borders/corners because it would cross the input boundary. So $W_2 = W_1 - F + 1$, and the output **shrinks more as F grows** (a 5×5 kernel shrinks it more than 3×3).
2. **Padding:** add zero rows and columns at the top, bottom, left and right. For example, $P = 1$ with a 3×3 kernel keeps the output the same size as the input. Then $W_2 = W_1 - F + 2P + 1$.
3. **Stride:** $S = 2$ skips every second position, giving an even smaller output.

D. Final formulas (HIGH PRIORITY)

$$W_2 = \frac{W_1 - F + 2P}{S} + 1, \qquad H_2 = \frac{H_1 - F + 2P}{S} + 1, \qquad D_2 = K$$

- **Conditions:** the formula assumes the filter fits an integral number of times. If the result is fractional, frameworks usually take the **floor** (additional clarification).

- **"Same" output size (S = 1):** $P = (F-1)/2$, e.g. $F = 3 \to P = 1$ and $F = 5 \to P = 2$ (additional clarification, consistent with the $P = 1$, 3×3 slide example).
- **Parameters of a conv layer** (each filter is $F \times F \times D_1$):

$$\#\text{params} = F \cdot F \cdot D_1 \cdot K \text{ (weights only, as on the slides)} \qquad \text{or} \qquad (F \cdot F \cdot D_1 + 1) \cdot K \text{ (with one bias per filt}$$

- **Parameters do NOT depend on** W_1, H_1, S, P (weight sharing). The **output size** does.
- **Computation** (multiplications) of a conv layer ≈ $W_2 \cdot H_2 \cdot K \cdot (F \cdot F \cdot D_1)$. The slides state $O(F \times F \times D)$ per output element.

E. Worked examples (all verified with Python)

#	Input $W_1 \times H_1 \times D_1$	F	S	P	K	Output	Params (no bias / with bias)
1 (slide)	227×227×3	11	4	0	96	(227−11)/4+1 = **55** → 55×55×96	34,848 / 34,944
2 (slide)	32×32×1	5	1	0	6	(32−5)/1+1 = **28** → 28×28×6	150 / 156
3	32×32×3	3	1	1	64	(32−3+2)/1+1 = **32** → 32×32×64 ("same")	1,728 / **1,792**
4	64×64×64	4	2	1	128	(64−4+2)/2+1 = **32** → 32×32×128	131,072 / 131,200
5	28×28×256	1	1	0	64	**28×28×64** (1×1 conv reduces depth)	16,384 / **16,448**
6	14×14×512	3	1	1	512	14×14×512	2,359,296 / 2,359,808
7 (trap)	224×224×3	7	2	3	64	(224−7+6)/2+1 = 112.5 → **floor 112**	9,408 / 9,472

- **How to recognise the question:**
  - "Output size / dimension" → the W_2 formula, plus $D_2 = K$.
  - "Number of parameters / weights" → $F^2 D_1 K$ (check whether biases are wanted).
  - "Number of computations" → output volume × $F^2 D_1$.
- **Effect of changing inputs:**
  - ↑ F → output shrinks and params grow ∝ F^2.
  - ↑ P → output grows by $2P/S$.
  - ↑ S → output shrinks about S-fold per dimension.
  - ↑ K → depth and params grow linearly; the spatial size is unchanged.
  - ↑ D_1 → params grow linearly; the output size is unchanged.

F. Traps

- Forgetting the **depth** D_1 in the param count. A 5×5 filter on a 96-deep input has $5 \cdot 5 \cdot 96$ weights, not 25.
- Output depth is K (number of filters), **not** D_1.
- Padding adds $2P$ (both sides), not P.
- The pooling output uses the same formula (usually $P = 0$), but its depth = **input depth** and it has **0 params**.

11.3 Convolutional Neural Networks

A. From handcrafted to learned features

- **Classic pipeline: static feature extraction (no learning)** using raw pixels, edge detectors or SIFT/HOG, followed by a **learned classifier** (car, bus, monument, flower).

- **CNN idea:** instead of handcrafted kernels (e.g. an edge detector), **learn multiple meaningful kernels/filters, and multiple layers of them**, in addition to the classifier weights. Treat the kernels as **parameters** and learn them with **backpropagation**. Such a network is a **Convolutional Neural Network**.

C. CNN vs regular feed-forward network

Aspect	Feed-forward (dense)	CNN
Connectivity	Dense: all 16 input pixels feed h_{11} (4×4 image example)	**Sparse connectivity**: only local pixels (1, 2, 5, 6 for a 2×2 kernel) feed h_{11}
Parameters	Separate weight per connection	**Weight sharing**: the same kernel is applied at all locations
Exploits	nothing about structure	image structure: interactions between neighbouring pixels are more interesting

- **Sparse connectivity:**
  - **Reduces the number of parameters.**
  - Doesn't it lose interactions between pixels? **Not really.** x_1 and x_5 don't interact in layer 1 but **indirectly** contribute to g_3 in a deeper layer (Goodfellow et al. 2016). Deeper units have a **larger receptive field**.
- **Weight sharing:**
  - An edge detector should be the same everywhere in the image, so the "orange and pink kernels should be the same".
  - This makes learning easier: you don't relearn the same kernel at different locations.
  - It does **not** mean only one kernel. You can have **many kernels, each shared by all locations**.

Pooling

- **Max pooling:** take the max over each $F \times F$ window. **Average pooling** is an alternative.
- Pooling has **no parameters** and acts on each feature map independently, so **depth is unchanged**.
- Output size: $(W_1 - F)/S + 1$.
- **Slide example** (input 4×4 = [1 4 2 1; 5 8 3 4; 7 6 4 5; 1 3 1 2]), verified:
  - 2×2 max pool, **stride 2** → [8 4; 7 5] (2×2).
  - 2×2 max pool, **stride 1** → [8 8 4; 8 8 5; 7 6 5] (3×3).
  - (Computed) 2×2 **average** pool, stride 2 → [4.5 2.5; 4.25 3.0].

LeNet-5 (handwritten character recognition): table as on the slides

Layer	Config	Output	Params (slide)	Check
Input	—	32×32 (×1)	—	
Conv1	$F = 5, S = 1, K = 6, P = 0$	28×28×6	**150**	$5 \cdot 5 \cdot 1 \cdot 6$ (no bias)
Pool1	$F = 2, K = 6, P = 0$	14×14×6	**0**	
Conv2	$F = 5, S = 1, K = 16, P = 0$	10×10×16	**2400**	$5 \cdot 5 \cdot 6 \cdot 16$ (no bias)
Pool2	$F = 2, K = 16, P = 0$	5×5×16	**0**	
FC1	120 units	120	**48,120**	$400 \cdot 120 + 120$ (with bias; $5 \cdot 5 \cdot 16 = 400$)
FC2	84 units	84	**10,164**	$120 \cdot 84 + 84$
Output	10 units	10	**850**	$84 \cdot 10 + 10$

- Alternates convolution and pooling layers.

- ⚠ Slide note: the pooling layers are labelled $S = 1$, but 28→14 and 10→5 require $S = 2$: $(28 - 2)/2 + 1 = 14$. With $S = 1$ you would get 27.
- ⚠ Slide note: the conv parameter counts **exclude biases** (with biases: 156 and 2,416) while the FC counts **include biases**.

Training a CNN

- A CNN can be implemented as a **feed-forward network in which only a few weights (coloured) are active and the rest (grey) are zero**, with the active weights **shared** across positions.
- So a CNN is trained with **ordinary backpropagation**, treating it as a feed-forward network with sparse connections.
- (Additional clarification) The gradient for a shared kernel weight is the **sum** of the gradients from every position where it is used.

F. Traps

- Sparse connectivity and weight sharing are **different** ideas. Sparse connectivity = each output sees a local patch. Weight sharing = the same weights at every patch. Both reduce parameters.
- Pooling has **zero** learnable parameters (both max and average).
- "Sparse connectivity loses all interactions between distant pixels" is **false**: they interact in deeper layers.

11.4 CNNs: success stories on ImageNet (AlexNet, ZF-Net, VGGNet)

ILSVRC history (top-5 error %)

Year	Winner	Error	Depth
ILSVRC'10	(shallow)	28.2	shallow
ILSVRC'11	(shallow)	25.8	shallow
ILSVRC'12	**AlexNet**	16.4	8 layers
ILSVRC'13	**ZFNet**	11.7	8 layers
ILSVRC'14	**VGG**	7.3	19 layers
ILSVRC'14	**GoogLeNet**	6.7	22 layers
ILSVRC'15	**ResNet**	3.57	152 layers

AlexNet (slide version; per-layer table; verified)

Layer	Input	Config	Output	Params (slide)	Exact (no bias)
Conv1	227×227×3	$K = 96, F = 11, S = 4, P = 0$	**55×55×96**	(11×11×3)×96 = 34K	34,848
MaxPool1	55×55×96	$F = 3, S = 2$	**27×27×96**	0	0
Conv2	27×27×96	$K = 256, F = 5, S = 1, P = 0$	**23×23×256**	(5×5×96)×256 = 0.6M	614,400
MaxPool2	23×23×256	$F = 3, S = 2$	**11×11×256**	0	0
Conv3	11×11×256	$K = 384, F = 3, S = 1, P = 0$	**9×9×384**	(3×3×256)×384 = 0.8M	884,736

Layer	Input	Config	Output	Params (slide)	Exact (no bias)
Conv4	9×9×384	$K = 384, F = 3, S = 1, P = 0$	**7×7×384**	(3×3×384)×384 = 1.327M	1,327,104
Conv5	7×7×384	$K = 256, F = 3, S = 1, P = 0$	**5×5×256**	(3×3×384)×256 = 0.8M	884,736
MaxPool3	5×5×256	$F = 3, S = 2$	**2×2×256**	0	0
FC1	flatten 2×2×256 = **1024**	dense 4096	4096	(2×2×256)×4096 = 4M	4,194,304
FC2	4096	dense 4096	4096	4096×4096 = 16M	16,777,216
FC3	4096	dense 1000	1000	4096×1000 = 4M	4,096,000
Total				**27.55M**	**28,813,344 ≈ 28.8M**

- **FC layers:** the last conv/maxpool output is **stretched into a 1D vector** (2×2×256 = 1024) and densely connected, as in a regular FFN.
- **Observation:** the FC layers hold most of the parameters (≈25.1M of ≈28.8M).
- ⚠ Slide note: the slide total 27.55M comes from summing **rounded** values (34K + 0.6M + 0.8M + 1.327M + 0.8M + 4M + 16M + 4M). The exact sum (no biases) is ≈ 28.81M. Conv3/Conv5 are really 0.88M, not 0.8M. The slide's architecture is a simplified, no-padding version (the original AlexNet uses padding and has ~60M params, additional clarification). For the exam, follow the slide's numbers and method.

ZF-Net vs AlexNet (layer-by-layer differences, as on the slides)

Layer	Change from AlexNet	Param difference (slide)	Recomputed
1 (Conv1)	$F = 11 \to 7$ (slide still draws 55×55×96; see ⚠ below)	$((11^2 - 7^2) \times 3) \times 96 = 20.7K$ (fewer)	20,736 ✓
2 (MaxPool)	no difference	—	
3 (Conv2)	no difference	—	
4 (MaxPool)	no difference	—	
5 (Conv3)	$K = 384 \to 512$	$(3 \times 3 \times 256) \times (512 - 384) = 0.29M$	294,912 ✓
6 (Conv4)	$K = 384 \to 1024$ (input depth now 512)	slide: $3 \times 3 \times ((384 \times 384) - (512 \times 1024)) = 0.8M$	$9 \times (512 \cdot 1024 - 384 \cdot 384) = 3,391,488 \approx 3.39M$
7 (Conv5)	$K = 256 \to 512$ (input depth now 1024)	slide: $3 \times 3 \times ((384 \times 256) - (1024 \times 512)) = 0.36M$	$9 \times (1024 \cdot 512 - 384 \cdot 256) = 3,833,856 \approx 3.83M$
8 (MaxPool)	no difference	—	
9, 10 (FC)	no difference	—	
Total difference		**1.45M**	conv layers: **≈7.50M** (7,499,520) more than AlexNet

- ⚠ Slide note: the Layer 6 and 7 numbers on the slide are arithmetically wrong. The bracket is also written in reverse order, which would make the difference negative. The slide's "1.45M" total is just $0.29 + 0.8 + 0.36 = 1.45$; the 20.7K *reduction* in layer 1 is ignored by the rounding (it would give ≈1.43M).
- ⚠ Slide note: the slide only changes F (11 → 7) in layer 1 and still draws a 55×55×96 output. With the AlexNet settings $S = 4, P = 0$, a 7×7 filter gives $(227 - 7)/4 + 1 = 56$, not 55. So the drawn shapes are not consistent with the formula. (Additional clarification: the actual ZF-Net uses 7×7 filters with **stride 2**.) The slide also shows the layer-8 maxpool output as 2×2×**256** for ZF-Net even though layer 7 outputs

depth 512. Pooling cannot change depth, so FC1 would actually see 2×2×512 and gain another ≈4.19M params.

- **Conceptual takeaway:** ZF-Net = AlexNet with a **smaller first-layer filter (7×7 instead of 11×11)** and **more filters in the middle conv layers (512, 1024, 512)**.

VGGNet

- **Kernel size is 3×3 throughout.** Conv blocks with max-pool after each block. The slide figure shows:

Stage	Output
Input	224×224×3
Conv ×2	224×224×64
maxpool	112×112×64
Conv ×2	112×112×128
maxpool	56×56×128
Conv ×3	56×56×256
maxpool	28×28×256
Conv ×3	28×28×512
maxpool	14×14×512
Conv ×3	14×14×512
maxpool	7×7×512
fc 4096 → fc 4096 → softmax 1000	

- Spatial size is preserved by the convs (3×3, $P = 1$, $S = 1$) and halved by each 2×2/stride-2 max pool (additional clarification of the figure).
- **Total params in non-FC layers ≈ 16M.** Recomputed for the 13-conv (VGG-16) figure: 14.71M.
- **FC params** = $(512 \times 7 \times 7 \times 4096) + (4096 \times 4096) + (4096 \times 1000) \approx 122\text{M}$ (slide). Recomputed: 102.76M + 16.78M + 4.10M = **123.6M**.
- **Most parameters are in the first FC layer (≈102M).**
- ⚠ Slide note: the slide writes the last FC as 4096×1024. It should be 4096×1000 (1000 classes). The figure shows 13 conv + 3 FC = **16 weight layers (VGG-16)**, while the ILSVRC timeline says **19 layers** (VGG-19).
- **Why stack small 3×3 filters?** (Additional clarification; the slides only state "3×3 throughout".)
  - Two stacked 3×3 layers have a **5×5 receptive field**, and three have **7×7** ($n(F - 1) + 1$).
  - They use **fewer params**: $3 \cdot (9C^2) = 27C^2$ vs $49C^2$ for one 7×7 layer. With $C = 64$: 110,592 vs 200,704.
  - They give **more non-linearities**.

F. Traps

- In AlexNet (slide version) and VGG, the FC layers dominate the parameter count. In VGG the **first FC** layer alone is ≈102M.
- Max-pool layers have 0 params, but they change the spatial size, e.g. 55 → 27 with $F = 3, S = 2$.
- When flattening for FC1, multiply all three dimensions: 2×2×256 = 1024 (AlexNet), 7×7×512 = 25,088 (VGG).

11.5 GoogLeNet and ResNet

GoogLeNet: motivation and 1×1 convolutions

- After a layer you could apply a max-pool, a 1×1, a 3×3 or a 5×5 conv. **Why choose? Apply all of them at once and concatenate the feature maps.**
- **Naive cost is large:**
  - Convolving a $W \times H \times D$ input with an $F \times F \times D$ filter ($P = 0, S = 1$) gives a $(W - F + 1)(H - F + 1)$ output.
  - Each output element costs $O(F \times F \times D)$.
- **1×1 convolution:**
  - It **aggregates along the depth**.
  - D_1 1×1 filters ($D_1 < D$) on a $D \times W \times H$ input give a $D_1 \times W \times H$ output ($S = 1, P = 0$). This **reduces the dimension** (depth).
  - Subsequent 3×3/5×5 filters then cost $O(F \times F \times D_1)$ **instead of** $O(F \times F \times D)$.

Inception module

- **Four parallel branches**, then **filter concatenation** along the depth:
  1. 1×1 convolutions.
  2. 1×1 reduction (D_1 filters) → 3×3 convolutions (on the reduced input).
  3. 1×1 reduction (D_2 filters) → 5×5 convolutions (on the reduced input).
  4. 3×3 max-pooling → 1×1 convolutions.
- Different reductions (D_1, D_2) can be used before the 3×3 and 5×5 filters.
- Output depth = sum of the depths of all branches. The spatial size is the same across branches.

Inception configs from the slides (verified: depth = 1×1 + 3×3 + 5×5 + pool-proj)

Module	In (H×W×D)	#1×1	3×3 reduce → #3×3	5×5 reduce → #5×5	pool → #1×1 proj	Out depth
3a	28×28×192	64	96 → 128	16 → 32	32	**256**
3b	28×28×256	128	128 → 192	32 → 96	64	**480**
4a	14×14×480	192	96 → 208	16 → 48	64	**512**
4b	14×14×512	160	112 → 224	24 → 64	64	**512**
4c	14×14×512	128	128 → 256	24 → 64	64	**512**
4d	14×14×512	112	144 → 288	32 → 64	64	**528**
4e	14×14×528	256	160 → 320	32 → 128	128	**832**
5a	7×7×832	256	160 → 320	32 → 128	128	**832**
5b	7×7×832	384	192 → 384	48 → 128	128	**1024**

GoogLeNet pipeline (slide):

- Stem: Input 229×229 → Conv 112×112×64 → maxpool 56×56×64 → Conv 56×56×192 → maxpool 28×28×192.
- Stage 3: Inception 3a (28×28×256) → 3b (28×28×480) → maxpool 14×14×480.
- Stage 4: Inception 4a, 4b, 4c (14×14×512) → 4d (14×14×528) → 4e (14×14×832) → maxpool 7×7×832.
- Stage 5: Inception 5a (7×7×832) → 5b (7×7×1024).
- Head: **avgpool** 1×1×1024 → **dropout (40%)** → fc 1000 → softmax.

(Additional clarification: the original paper uses a 224×224 input. The slide shows 229.)

Important trick: global average pooling instead of FC

- The last layer outputs 7×7×1024. Flattening it and adding an FC layer with 1000 nodes would need **$7\times7\times1024\times1000 \approx 49$M (exactly 50,176,000) params**, with $W \in \mathbb{R}^{50176\times1000}$.
- Instead, use **average pooling of size 7×7 on each of the 1024 feature maps**. This gives a **1024-dim** vector, and $W \in \mathbb{R}^{1024\times1000}$ has 1,024,000 params, **49× fewer** for this layer.
- **Overall: 12× fewer parameters than AlexNet, but 2× more computations.**

E. Computation-savings numerical (Additional clarification; the classic inception-3a 5×5 branch; verified)

Setting	Multiplications
Input 28×28×192 → 32 filters of 5×5 directly: $28 \cdot 28 \cdot 32 \cdot (5 \cdot 5 \cdot 192)$	**120,422,400 ≈ 120M**
With 16 1×1 reduction filters first: $28 \cdot 28 \cdot 16 \cdot 192 + 28 \cdot 28 \cdot 32 \cdot (5 \cdot 5 \cdot 16)$	2,408,448 + 10,035,200 = **12,443,648 ≈ 12.4M**
Saving	**≈9.7× fewer**

- Same idea for the 3×3 branch (192 → 96 reduce → 128 3×3): 173.4M → 101.2M (≈1.7×).
- **Recipe:** cost = $W_2H_2K \cdot F^2 \cdot D_{in}$. The 1×1 reduction replaces $D_{in} = D$ by D_1 and adds a small $W_2H_2D_1 \cdot D$ term.

ResNet

- **Problem (degradation):**
  - Intuitively, if a shallow network trains well, a deeper version (extra layers) should do at least as well by learning **identity functions** in the new layers. The solution space of the shallow network is a **subset** of the deep network's.
  - **In practice this doesn't happen.** On CIFAR-10 plain nets, the 56-layer net has **higher** error than the 20-layer net (both training and test curves are shown; the deeper nets are worse).
- **Residual block:** for two stacked layers, instead of learning $H(x)$ directly, learn the **residual** $F(x)$ and add an **identity (skip) connection**:

$$H(x) = F(x) + x$$

- **Why it helps:**
  - The identity connection lets the network **retain a copy of the input**. If identity is optimal, the block only needs $F(x) \to 0$.
  - This made it possible to **train really deep networks** (152 layers).
- **Results (ResNet-152):**
  - 1st place in **all five main tracks**.
  - ImageNet classification: "ultra-deep" 152-layer nets (3.57% top-5).
  - ImageNet detection: 16% better than the 2nd best. ImageNet localization: 27% better.
  - COCO detection: 11% better. COCO segmentation: 12% better.
- **Bag of tricks:**
  - **Batch Normalization after every conv layer.**
  - **Xavier/2 initialization** (He et al.).
  - **SGD + momentum (0.9).**
  - **Learning rate 0.1, divided by 10 when validation error plateaus.**

- **Mini-batch size 256.**
- **Weight decay 1e-5.**
- **No dropout.**

F. Traps (11.5)

- The 1×1 conv reduces **depth**, not width/height (with $S = 1, P = 0$).
- In the inception module, the max-pool branch keeps the depth. It is the **1×1 conv after it** that controls its depth contribution.
- GoogLeNet has **fewer** parameters than AlexNet (12×) but **more** computation (2×). Don't swap the two.
- The GAP trick removes the large FC layer: 49M → ~1M for the classifier.
- ResNet's degradation problem is about deeper plain nets having **higher training and test error**. It is not simply overfitting (additional clarification: training error is also higher in the figure).
- ResNet uses BN and **no dropout**. GoogLeNet uses dropout (40%) before the FC.

G. Lecture 11 – Quick Revision Summary

- **Convolution:**
  - 1D: $s_t = \sum_a x_{t-a} w_{-a}$, a weighted average with a sliding filter.
  - 2D: $S_{ij} = \sum_a \sum_b I_{i+a,j+b} K_{a,b}$, or the centred form.
  - The filter spans the full input depth, so the output per filter is 2D and K filters give K maps.
- **Size:** $W_2 = (W_1 - F + 2P)/S + 1$, H_2 the same, $D_2 = K$.
- **Params:** $F^2 D_1 K$ (+K biases). They do not depend on the input size, stride or padding.
- **CNN vs FFN: sparse connectivity** (local receptive field, fewer params, distant pixels still interact deeper) + **weight sharing** (the same kernel everywhere; many kernels allowed).
- **Pooling:** max/avg, **0 params**, depth unchanged, reduces spatial size.
- **Training:** a CNN is an FFN with mostly-zero, shared weights, trained with standard backprop.
- **LeNet-5:** 32×32 → 28×28×6 → 14×14×6 → 10×10×16 → 5×5×16 → 120 → 84 → 10. Params 150, 0, 2400, 0, 48120, 10164, 850.
- **AlexNet (slide):** 227 → 55 → 27 → 23 → 11 → 9 → 7 → 5 → 2 (×256) → 4096 → 4096 → 1000. Total ≈ 27.55M (slide) / 28.8M (exact). The FC layers dominate.
- **ZF-Net:** F 11→7 in layer 1; K 384→512, 384→1024, 256→512 in conv layers 3–5.
- **VGG:** 3×3 kernels throughout. ~16M conv params + ~122M FC params, with the first FC ≈ 102M.
- **GoogLeNet:** inception modules (parallel 1×1 / 3×3 / 5×5 / pool with 1×1 reductions, concatenated); 1×1 conv = depth reduction; global average pooling instead of FC (49M → 1M); 12× fewer params than AlexNet, 2× more compute; 22 layers.
- **ResNet:** $H(x) = F(x) + x$; fixes degradation; 152 layers, 3.57%; BN, no dropout, SGD+momentum 0.9, lr 0.1/10, batch 256, weight decay 1e-5.

Likely MCQ takeaways

1. Compute output size with $(W - F + 2P)/S + 1$ and params with $F^2 D_1 K (+K)$, including the input depth.
2. Pooling has zero parameters. Pool $F = 3, S = 2$: 55 → 27, 23 → 11, 5 → 2.
3. 1×1 convolutions reduce depth and computation (≈10× in the 5×5 branch example).
4. GoogLeNet: average pooling replaces FC; 12× fewer params than AlexNet, 2× compute.
5. ResNet skip connections let deep nets learn identity easily. Deeper plain nets showed higher error.

Overview: Lectures 12–13: Visualizing CNNs, Deep Dream/Art, Fooling CNNs · RNNs, BPTT, Vanishing/Exploding Gradients

Lectures covered / modules:

- **Lecture 12: Visualizing CNNs, Guided Backpropagation, Deep Dream, Deep Art, Fooling CNNs**
  - 12.0 Setup: the CNN used throughout and the **receptive field** of a neuron (numerical)
  - 12.1 Visualizing patches which maximally activate a neuron
  - 12.2 Visualizing filters of a CNN
  - 12.3 Occlusion experiments
  - 12.4 Finding influence of input pixels using backpropagation
  - 12.5 Guided backpropagation
  - 12.6 Optimization over images
  - 12.7 Creating images from embeddings
  - 12.8 Deep Dream
  - 12.9 Deep Art (neural style transfer)
  - 12.10 Fooling deep convolutional neural networks
- **Lecture 13: Sequence learning problems, RNNs, BPTT, vanishing/exploding gradients, truncated BPTT**
  - 13.1 Sequence learning problems
  - 13.2 Recurrent neural networks
  - 13.3 Backpropagation through time (BPTT)
  - 13.4 The problem of exploding and vanishing gradients (+ truncated BPTT)
  - 13.5 Some gory details (shapes, the $\partial^+ s_k / \partial W$ tensor)

Week 10 · Lecture 12 — Visualizing CNNs, Guided Backprop, Deep Dream, Deep Art, Fooling CNNs

(Slides acknowledge Andrej Karpathy's video lecture on visualization / Deep Dream / neural style / adversarial examples.)

12.0 Setup: the CNN used in the slides & the receptive field of a neuron

A. Core Concept

All modules use a trained AlexNet-like ConvNet (same as Lecture 11). The slide figure gives these sizes (no padding, $P = 0$):

Layer	Filter / window	Stride (implied)	Output volume
Input	—	—	$227 \times 227 \times 3$
Conv1	11×11, 96 filters	4	$55 \times 55 \times 96$
MaxPool1	3×3	2	$27 \times 27 \times 96$
Conv2	5×5, 256 filters	1	$23 \times 23 \times 256$
MaxPool2	3×3	2	$11 \times 11 \times 256$
Conv3	3×3, 384	1	$9 \times 9 \times 384$
Conv4	3×3, 384	1	$7 \times 7 \times 384$
Conv5	3×3, 256	1	$5 \times 5 \times 256$
MaxPool5 ("pool5")	3×3	2	$2 \times 2 \times 256$
FC6, FC7, FC8	dense	—	4096, 4096, 1000

Output size: $W_2 = \frac{W_1 - F + 2P}{S} + 1$ (from Lecture 11). Pooling layers have **0 parameters** (slide annotation "S = 2, P = 0, Parameters: 0").

Receptive field (Additional clarification — the slides only show it pictorially as "trace back to the patch in the image which causes these neurons to fire"): the receptive field of a neuron is the region of the **input image** that can affect its value. A neuron in layer 1 sees $F \times F$ pixels; neurons in deeper layers see progressively larger patches because each looks at a window of neurons that themselves look at windows.

D. Mathematical Formulas (receptive field — Additional clarification)

$$r_\ell = r_{\ell-1} + (k_\ell - 1)\, j_{\ell-1}, \qquad j_\ell = j_{\ell-1}\, s_\ell, \qquad r_0 = 1,\ j_0 = 1$$

- r_ℓ: receptive field (side length, in input pixels) of a neuron in layer ℓ; k_ℓ: filter/pool window size; s_ℓ: stride; j_ℓ: "jump" = distance in input pixels between adjacent neurons of layer ℓ (product of all strides so far).
- Conditions: square filters; padding does **not** change the receptive field size (it changes only where windows sit).
- Special case — stride 1 everywhere: $r_L = 1 + \sum_\ell (k_\ell - 1)$. L stacked 3×3 stride-1 convs → $r = 2L + 1$ (two 3×3 → 5×5; three 3×3 → 7×7).
- Common mistakes: forgetting pooling layers also enlarge the receptive field; multiplying filter sizes; forgetting that earlier strides multiply the contribution of later filters.

E. Numerical Problem-Solving

Worked example (slide architecture, verified in python):

Layer	k	s	out size	jump j	receptive field r
Conv1	11	4	55	4	11
Pool1	3	2	27	8	$11 + 2 \cdot 4 = 19$
Conv2	5	1	23	8	$19 + 4 \cdot 8 = 51$
Pool2	3	2	11	16	$51 + 2 \cdot 8 = 67$
Conv3	3	1	9	16	$67 + 2 \cdot 16 = 99$
Conv4	3	1	7	16	131
Conv5	3	1	5	16	163
Pool5	3	2	2	32	$163 + 2 \cdot 16 = 195$

So a **pool5 neuron "sees" a** 195×195 **patch** of the 227×227 image — which is why the pool5 neurons in 12.1 respond to whole object parts (faces, flowers) and why we can trace back a patch.

Other quick ones: 3×3 (s=2) then 3×3 (s=1) → $r = 3 + 2 \cdot 2 = 7$; 3×3 conv, 2×2 pool (s=2), 3×3 conv → $r = 3 + 1 + 2 \cdot 2 = 8$.

Recognise: question gives a stack of layers with sizes/strides and asks "how many input pixels influence a neuron in layer L" → receptive field recursion. Result grows with depth, kernel size and (multiplicatively) stride.

12.1 Visualizing patches which maximally activate a neuron

A. Core Concept

- Pick some neurons in a given layer of a trained CNN.
- Feed many images; identify the images that cause these neurons to **fire** (high activation).
- **Trace back** to the patch in the image (the neuron's receptive field) responsible for the firing.
- Experiment by **Girshick et al., 2014** (slide spells "Grishick"): 6 neurons in the **pool5** layer; each neuron fires for a consistent concept — **people faces, dog faces, flowers, numbers, houses, shiny surfaces**.

C. Properties

- Shows that individual deep neurons act as detectors of semantically meaningful patterns.
- Data-driven: it only shows patterns present in the dataset, not what the neuron "ideally" wants (contrast with 12.6).

12.2 Visualizing filters of a CNN

A. Core Concept

- Recall autoencoders: the input that maximally excites hidden neuron $h_1 = w_1^T x$ under a norm constraint:

$$\max_x \; w^T x \quad \text{s.t.} \quad \|x\|^2 = x^T x = 1 \quad \Rightarrow \quad x = \frac{w_1}{\sqrt{w_1^T w_1}} = \frac{W}{\|W\|}$$

- A CNN is a feed-forward network with **sparse connections and weight sharing** (figure: 16 inputs = 4×4 image, 2×2 filter, neurons $h_{11}, h_{12}, \ldots, h_{14}$). So the same result holds: the input patch that maximally activates a conv neuron is $W/\|W\|$, where W is the **filter** (2×2 in the example).
- Hence **filters are pattern detectors**: we can plot the $K \times K$ filter weights as images and see the patterns they detect (figure: AlexNet conv1 filters — oriented edges, colour blobs, gratings).

C. Properties

- Plotting filters is **only interpretable for the first convolution layer** (slide asks "Why?").
  - (Additional clarification) First-layer filters act directly on pixel (RGB) space, so they can be shown as images; deeper filters act on multi-channel activation maps of previous layers (e.g., 96 or 256 channels), which are not in pixel space.
- The maximizer is **parallel to** w; the maximum activation equals $w^T \frac{w}{\|w\|} = \|w\|$. (Follows from Cauchy–Schwarz; Additional clarification.)

D/E. Formula & numerical

- $x^* = w/\|w\|$, max value $= \|w\|$. Ex: $w = [3, 4] \rightarrow x^* = [0.6, 0.8]$, max $w^T x = 5$. $w = [1, -2, 2] \rightarrow x^* = [1/3, -2/3, 2/3]$, max $= 3$.
- Trap: the answer is the **normalized** weight vector, not w itself, and not $w^T w$; bias does not change the maximizer.

12.3 Occlusion experiments

A. Core Concept

- Goal: which **portions of the image** are responsible for the probability of a certain class?
- **Occlude (gray out)** different patches of the image (slide a gray square over the image) and record the **predicted probability of the correct class** for each occluder position → a **heat map**.
- Example (Zeiler & Fergus figure): True label Pomeranian — occluding the **dog's face** gives the **maximum drop** in probability. Similar for Car Wheel and Afghan Hound.
- Figure detail: column (a) is the input image with the gray square; column (b) is labelled **"Layer 5, strongest feature map"**, i.e. a heat map over occluder positions. In the original Zeiler & Fergus figure this panel plots that feature map's total activation; the slide text reads the heat map as the drop in the correct class's probability.

C. Properties

- Model-agnostic, needs only forward passes (no gradients); one forward pass per occluder position → expensive.
- Low/blue region in the heat map = occluding there hurts the correct class a lot = important region.

E. Numerical (Additional clarification)

Number of forward passes = number of occluder positions $= \left(\frac{N-F}{S} + 1\right)^2$. E.g., 224×224 image, 32×32 occluder, stride 16 → $13^2 = 169$ forward passes.

12.4 Finding influence of input pixels using backpropagation

A. Core Concept

- Treat the image as $m \times n$ inputs $x_0, x_1, \ldots, x_{mn}$ (flattened). Influence of x_i on a neuron h_j: if a small change in x_i causes a large change in h_j, x_i has high influence → use the **gradient** $\partial h_j / \partial x_i$.
  - $\partial h_j / \partial x_i = 0$ → no influence; large → high influence; small → low influence.
- Compute the partial derivatives w.r.t. **all** inputs and **visualize this gradient matrix as an image** (a saliency-type map).
- How? A CNN can be represented as a feed-forward network, so ordinary **backpropagation** gives these gradients; e.g.

$$\frac{\partial h_{32}}{\partial x_2} = \sum_{i=1}^{3} \frac{\partial h_{32}}{\partial h_{1i}} \frac{\partial h_{1i}}{\partial x_2}, \qquad h_{1i} = \sum_{j=1}^{4} w_{ji} x_j \;\Rightarrow\; \frac{\partial h_{1i}}{\partial x_2} = w_{2i}$$

- ⚠ Slide note: the slide writes $\partial h_{1i} / \partial x_2 = w_{12}$; with $h_{1i} = \sum_j w_{ji} x_j$ it should be w_{2i} (index typo).

C. Properties

- The network weights are fixed; we differentiate w.r.t. the **input**, not the weights.
- Plain gradient images are **noisy — "do not show very sharp influences"** → motivates guided backpropagation (Springenberg et al.).

12.5 Guided backpropagation (Springenberg et al.)

A. Core Concept

Procedure:

1. Feed an input image, do a **forward pass**.
2. Consider **one neuron** in some feature map at some layer; **retain this neuron and set all other neurons in the layer to zero** (only its gradient is propagated).
3. Backpropagate all the way to the **input**, with a modified ReLU backward rule.
   - Slide figure: input image f^0 → forward pass through $f^1, \ldots, f^L$ → feature map $\begin{bmatrix} 1 & 0 \\ 3 & 2 \end{bmatrix}$. Only one neuron is retained, giving $\begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix}$. This is passed backward through $R^L, \ldots, R^1$, and the result is the **reconstructed image** R^0, a visualization in pixel space.

ReLU rules:

- Forward: ReLU passes positive values and clamps negatives to 0.
- Normal backprop: **no gradient flows through "dead" ReLUs** (positions where the forward input was negative).
- **Guided backprop: additionally, any negative gradients flowing from the upper layer are set to 0.**

So in guided backprop the gradient at a ReLU is passed only where **both** the forward activation was positive **and** the incoming gradient is positive:

$$g_{\text{in}}^{\text{guided}} = g_{\text{out}} \cdot 1[x > 0] \cdot 1[g_{\text{out}} > 0]$$

(compact form — Additional clarification; the slides state it in words and via the example below).

- **Intuition:** neglect all the negative influences (gradients) and focus only on the positive influences → gives a better (sharper) picture of the true influence of the input.

E. Worked example (slide, verified)

Forward input $x = \begin{bmatrix} 1 & -1 & 5 \\ 2 & -5 & -7 \\ -3 & 2 & 4 \end{bmatrix}$ → ReLU output $\begin{bmatrix} 1 & 0 & 5 \\ 2 & 0 & 0 \\ 0 & 2 & 4 \end{bmatrix}$. Upper-layer gradient $g = \begin{bmatrix} -2 & 3 & -1 \\ 6 & -3 & 1 \\ 2 & -1 & 3 \end{bmatrix}$.

Method	Result	Rule
Backprop	$\begin{bmatrix} -2 & 0 & -1 \\ 6 & 0 & 0 \\ 0 & -1 & 3 \end{bmatrix}$	zero where $x < 0$
Guided backprop	$\begin{bmatrix} 0 & 0 & 0 \\ 6 & 0 & 0 \\ 0 & 0 & 3 \end{bmatrix}$	zero where $x < 0$ **or** $g < 0$

Trap: entries like $g = 3$ at position (1,2) are zeroed by **both** methods (forward input -1 was negative) even though the gradient is positive; entries like -2 at (1,1) survive normal backprop but not guided.

F. MCQ Traps

- Guided backprop ≠ "only zero negative gradients" — it zeros negative gradients **and** keeps the usual ReLU masking from the forward pass.
- It changes only the **backward** pass; the forward pass is ordinary.
- It is a visualization technique, not a training method; weights are not updated.

12.6 Optimization over images

A. Core Concept

- Want an image that looks like a "dumbbell" (or ostrich, car...): an image I that, passed through a **trained** ConvNet, maximizes the score of class c:

$$\arg\max_I \ \big(S_c(I) - \lambda\,\Omega(I)\big)$$

- $S_c(I)$ = score for class c **before softmax**; $\Omega(I)$ = some regularizer to ensure that I looks like an image; λ = regularization weight.
- Think of the **image (pixels $i_0, \ldots, i_{mn}$) as the parameters**; keep the **network weights fixed**; adjust pixels to maximize the class score.

Algorithm (slide):

1. Start with a **zero image**.
2. Set the score vector to $[0, 0, \ldots, 1, \ldots, 0, 0]$ (one-hot at class c — only S_c is backpropagated).
3. Compute the gradient $\partial S_c(I)/\partial i_k$.
4. Update each pixel $i_k = i_k - \eta\,\partial S_c(I)/\partial i_k$.
5. Do a forward pass again. 6. Go to step 2.

- ⚠ Slide note: since we **maximize** S_c, the update should be gradient **ascent** $i_k = i_k + \eta\, \partial S_c / \partial i_k$ (Deep Dream's slide uses "+"); the "−" sign is consistent only if we minimize $-S_c$.
- Results: class visualizations for dumbbell, cup, dalmatian, bell pepper, lemon, husky (recognisable fragments of the class repeated in the image).

Any neuron, not just class scores: Repeat: feed image → set activations in the layer of interest to zero except the neuron of interest → backprop to image → $i_k = i_k - \eta\, \partial A(I)/\partial i_k$, $A(I)$ = activation of the ith neuron in some layer (same sign caveat).

- **Different initializations** (instead of a zero image) give different insights: 4 images obtained for one neuron in layer 8 from different initializations; similar analysis for layer 7, etc.

C/F. Properties & traps

- Optimized variable: pixels; fixed: weights (opposite of training).
- Score **before** softmax is used. (Additional clarification: maximizing the softmax probability can be achieved by pushing other classes' scores down rather than making the image look like class c.)
- Without Ω, the result need not look like a natural image (cf. fooling images, 12.10).

12.7 Creating images from embeddings

A. Core Concept

- Treat the **fc7** layer (4096-d) as an **embedding** of the image. Question: given an embedding, can we reconstruct the image? Pose as optimization: find an image such that (i) **its embedding is similar to a given embedding**, (ii) **it looks natural** (prior/regularization).
- Algorithm: ϕ_0 = embedding of image of interest; X = random image (say zero image). Repeat: forward pass with $X \to \phi(x)$; compute

$$\mathcal{L}(i) = \|\phi(x) - \phi_0\|^2 + \lambda\, \|\phi(x)\|_6^6, \qquad i_k = i_k - \eta\, \frac{\partial \mathcal{L}(i)}{\partial i_k}$$

- Here "−" is correct (we **minimize** $\mathcal{L}$).
- ⚠ Slide note: the update on the slide is written $i_k = i_k - \eta\, \frac{\mathscr{L}(i)}{\partial i_k}$, with the ∂ missing from the numerator. It means $\partial \mathcal{L}(i)/\partial i_k$.
- ⚠ Slide note: the regularizer is written on $\phi(x)$; for the stated purpose ("image looks natural") the 6-norm prior is normally on the **image** x, i.e., $\lambda \|x\|_6^6$ (Mahendran & Vedaldi) — flag if an option uses either form.
- Results (monkey image), reconstructing from each layer: Conv-1, Relu-1, Mpool-1, Norm-1, … Conv-5, Mpool-5, FC-6, Relu-6, FC-7, Relu-7, FC-8. Reconstruction from Conv-1 is nearly identical to the original; deeper layers give progressively more abstract, blurrier, colour-shifted reconstructions (FC-8 only retains rough layout/parts).

C. Properties

- Deeper representations discard pixel-level detail while preserving semantic content.

12.8 Deep Dream

A. Core Concept

- Instead of a zero image, **start with an actual image** (e.g., clouds). Focus on some layer; change the image so those neurons **fire even more**.
- For a neuron h_{ij} in some layer:

$$\max_I \mathcal{L}(I), \quad \mathcal{L}(I) = h_{ij}^2, \qquad \frac{\partial \mathcal{L}(I)}{\partial i_{mn}} = \frac{\partial \mathcal{L}(I)}{\partial h_{ij}} \frac{\partial h_{ij}}{\partial i_{mn}} \left(= 2h_{ij} \frac{\partial h_{ij}}{\partial i_{mn}} \right)$$

- Update $i_{mn} = i_{mn} + \frac{\partial \mathcal{L}(I)}{\partial i_{mn}}$ (gradient **ascent**), feed back; target neurons fire even more; iterating makes the image more and more like the patterns that cause the neuron to fire.
- Results on the slides (image-only pages): a **sky with clouds** turns into a sky full of hallucinated **towers/pagoda-like buildings**. A **lake landscape** gets dog- and animal-like creatures all over its hills. A **photo of people** comes back covered in dog-like faces and eyes on the faces, clothes and background. The original scene stays recognisable, with the network's favourite patterns painted over it.
- Why ("Inceptionism", Google 2015): network trained to detect patterns frequent in ImageNet (dogs, cats, birds); "if a cloud looks a little bit like a bird, the network will make it look more like a bird" → feedback loop until a detailed bird appears out of nowhere.

F. Distinctions

	Class-score optimization (12.6)	Deep Dream
Start	zero/random image	real image
Objective	$S_c(I) - \lambda\Omega(I)$	h_{ij}^2 (amplify existing activations of a layer)
Output	canonical image of a class	real image with hallucinated patterns

---

12.9 Deep Art (Neural style transfer)

A. Core Concept

Content image (Gandalf) + style image (Picasso) = new image with content of the first and style of the second. Define two quantities and generate the new image by optimizing over its pixels (network fixed).

Content target: activations of the layers for the content image; we want the new image's activations close to them. With $\vec{p}, \vec{x}$ = activations of content image and new image:

$$\mathcal{L}_{content}(\vec{p}, \vec{x}) = \sum_{ijk} (\vec{p}_{ijk} - \vec{x}_{ijk})^2$$

Style: if $V \in \mathbb{R}^{64 \times (256 \times 256)}$ is the activation at a layer (64 channels, each flattened 256×256 map), then the **Gram matrix** (slide: $V^T V$) $\in \mathbb{R}^{64 \times 64}$ captures the style of the image. **Deeper layers capture more of this style information.** In the slide figure, the style image (Van Gogh's *Starry Night*) is rebuilt from the Gram matrices of successively deeper layers. Shallow layers give fine texture, and deeper layers reproduce larger-scale strokes and swirls. (Additional clarification: Gram entry (i, j) = inner product of feature maps i and j = channel co-activation statistics, independent of spatial position — hence "style", not "content".)

- ⚠ Slide note: with $V \in \mathbb{R}^{64 \times (256 \cdot 256)}$, $V^T V$ would be 65536×65536; the 64×64 Gram matrix is $V V^T$ (or $V^T V$ if V is stored as $(256 \cdot 256) \times 64$).

Per-layer style loss and total style loss:

$$E_\ell = \sum_{ij} \left(G^\ell_{ij} - A^\ell_{ij}\right)^2, \qquad \mathcal{L}_{style}(\vec{a}, \vec{x}) = \sum_{\ell=0}^{L} w_\ell E_\ell$$

G^ℓ and A^ℓ are the style Gram matrices at layer ℓ for the **style image and new image** respectively (as stated on the slide), w_ℓ = layer weights.

- ⚠ Slide note: slide writes G^ℓ_j (missing i) and $\bar{x}$ in $\mathcal{L}_{style}(\vec{a}, \bar{x})$; also labels the style image "Content Image" on that slide. (In Gatys et al. G is for the generated image and A for the style image — the squared difference is symmetric so the loss is the same.)

Total loss:

$$\mathcal{L}_{total}(\vec{p}, \vec{a}, \vec{x}) = \alpha\, \mathcal{L}_{content}(\vec{p}, \vec{x}) + \beta\, \mathcal{L}_{style}(\vec{a}, \vec{x})$$

- α, β trade off content vs style: larger α/β → output closer to content image; larger β/α → more stylized.

E. Numerical

- Gram matrix size = (#channels)^2, independent of spatial size: 64 channels → $64 \times 64 = 4096$ entries, even though each map has $256 \times 256 = 65536$ positions.
- Gram matrix is symmetric (Additional clarification) → only $C(C+1)/2$ distinct entries.

F. Traps

- Content loss compares **activations directly**; style loss compares **Gram matrices** (correlations), not activations.
- What is optimized: pixels of the new image; the CNN weights are fixed.

12.10 Fooling deep convolutional neural networks

A. Core Concept

- Same idea of **optimizing over the input** can fool ConvNets.
- Feed an image (bus); instead of maximizing log-likelihood of the correct class, **maximize some incorrect class (ostrich)**. With **minimal changes** to the image (via backprop) the ConvNet is convinced it is an ostrich.
- **Szegedy et al., 2013 ("Intriguing properties of neural networks")**: original image + small perturbation (2nd image) = 3rd image; images 1 and 3 are **indistinguishable to humans**, yet the ConvNet predicts **ostrich**. This is shown for six quite different originals (e.g. a school bus, a temple, a dog), and every one becomes "ostrich", so the target class can be chosen freely.
- **Nguyen, Yosinski, Clune, 2014 ("Neural Networks are Easily Fooled")**: start from **random images** and optimize them to predict a class (robin, cheetah, armadillo, lesser panda, centipede, peacock, jackfruit, bubble) — classifier is **99.6% confident** though images are unrecognizable noise.

C. Intuitive explanation (slide)

- Images are extremely high-dimensional objects ($\mathbb{R}^{227\times227}$); there are very many points in this space, of which only a few are natural images (some seen during training).

- Training fits decision boundaries using these images, but in doing so also takes decisions about the many unseen points → large regions (green/red in the figure) containing **no training points** are still confidently assigned to a class; a noise image can fall deep inside the "cheetah" region.

F. Traps

- Adversarial perturbations are imperceptible but change the prediction; fooling images are unrecognizable but get high confidence — two different phenomena, both from optimizing the input.
- Cause per slides: high dimensionality + decision boundaries extrapolated to regions without training data (not "the network is under-trained").

L12 — Comparison of visualization / input-optimization techniques

Technique	What is computed	Needs gradients?	Starts from	Changes image?
Max-activating patches	images/patches that fire a neuron most	No	dataset	No
Filter visualization	plot W (max input $= W/\|W\|$)	No	—	No (only layer 1 interpretable)
Occlusion	prob. of true class vs occluder position	No (forward only)	given image	Only grayed patches
Gradient w.r.t. input	$\partial h_j/\partial x_i$ as image	Yes (backprop)	given image	No
Guided backprop	gradient with negative grads & dead ReLUs zeroed	Yes (modified)	given image	No
Class/neuron maximization	$\arg\max_I S_c(I) - \lambda\Omega(I)$	Yes	zero/random image	Yes
Embedding inversion	min $\|\phi(x) - \phi_0\|^2$ + prior	Yes	random/zero	Yes
Deep Dream	max h_{ij}^2	Yes	real image	Yes
Deep Art	min $\alpha\mathcal{L}_c + \beta\mathcal{L}_s$	Yes	(new image)	Yes
Fooling	max wrong-class score	Yes	real or random image	Yes (tiny)

G. Quick Revision Summary — Lecture 12

- Receptive field grows with depth: $r_\ell = r_{\ell-1} + (k_\ell - 1)\prod_{m<\ell} s_m$; slide AlexNet pool5 neuron → 195×195.
- Input that maximally activates a neuron/filter (unit-norm constraint) $= W/\|W\|$; filter plots interpretable only for layer 1.
- Occlusion: gray patches, watch true-class probability; face of the dog → max drop.
- Gradient $\partial h_j/\partial x_i$ = influence; noisy → guided backprop zeros **negative gradients** in addition to **dead-ReLU positions**.
- Optimization over images: weights fixed, pixels are parameters; class score **before softmax** + regularizer Ω.
- Deep Dream: real image, maximize h_{ij}^2 → amplifies patterns. Deep Art: $\alpha\mathcal{L}_{content} + \beta\mathcal{L}_{style}$, style = Gram matrix ($C \times C$).
- Fooling: tiny perturbations (Szegedy 2013) / random images at 99.6% confidence (Nguyen 2014); reason = high-dim space, few real images.

Likely MCQ takeaways: (1) guided backprop zeroes both; (2) Gram matrix size = channels^2; (3) only first-layer filters interpretable; (4) S_c before softmax; (5) receptive field arithmetic.

Week 11 · Lecture 13 — Sequence Learning Problems, RNNs, BPTT, Vanishing/Exploding Gradients

13.1 Sequence learning problems

A. Core Concept

In feed-forward and convolutional networks:

- the **input size was always fixed** (e.g., 32×32 images for classification; a fixed window of k words in word2vec);
- each input was **independent** of previous/future inputs (outputs for two successive images are completely independent).

In many applications neither holds. Example — **auto-completion** (character level): given 'd' predict 'e', then 'e', 'p', ⟨stop⟩ for "deep". Observations:

1. **Successive inputs are not independent** (to predict 'e' you need the previous input as well as the current one).
2. **Length of inputs and number of predictions is not fixed** ("learn", "deep", "machine" differ in length).
3. **Each network (orange-blue-green block) performs the same task** (input: character, output: character).

Sequence learning problems: look at a sequence of (dependent) inputs and produce an output (or outputs); **each input corresponds to one time step**.

Examples on slides:

Task	Input	Output	Output at every step?
Auto-completion	characters	next character	Yes
POS tagging ("man is a social animal")	words	tag per word (noun, verb, article, adjective, noun)	Yes — after "social" (adjective) the next is almost surely a noun
Sentiment of a movie review ("The movie was boring and long")	words	+/− only at the end	No — intermediate outputs "don't care"; prediction depends on earlier words, not only the last
Activity recognition in video ("Surya Namaskar")	sequence of frames (images)	activity	Only at end

F. Traps

- Sequences need not be words (video = sequence of images).
- Even when only the final output matters, the network performs the same task at each step — we just ignore intermediate outputs.

13.2 Recurrent Neural Networks

A. Core Concept

Wishlist for a sequence model:

1. Account for **dependence** between inputs.
2. Account for **variable number of inputs**.
3. Make sure the **function executed at each time step is the same**.

Step 1 (same function): at each time step, $s_i = \sigma(Ux_i + b)$, $y_i = \mathcal{O}(Vs_i + c)$; **share the same network/parameters at every time step**. Parameter sharing also makes the network **agnostic to the length of the input** — just create copies of the network (unroll) for as many steps as needed.

Infeasible way to model dependence: **feed all previous inputs at each time step** → $y_1 = f_1(x_1)$, $y_2 = f_2(x_1, x_2)$, $y_3 = f_3(x_1, x_2, x_3)$ — violates the other two wishes: function differs per time step, and network is sensitive to sequence length (length 10 needs $f_1..f_{10}$, length 100 needs $f_1..f_{100}$).

Solution — recurrent connection:

$$s_i = \sigma(Ux_i + Ws_{i-1} + b), \qquad y_i = \mathcal{O}(Vs_i + c) \quad \text{or} \quad y_i = f(x_i, s_{i-1}, W, U, V, b, c)$$

- s_i: **state** of the network at time step i; s_0: constant starting state.
- Parameters W, U, V, b, c are **shared across time steps**; the same network computes $y_1..y_{10}$ or y_{100}.
- Compact (folded) representation: one block with a self-loop W.

B. Terminology

- U: input→state; W: state→state (recurrent); V: state→output; b, c: biases; σ: state non-linearity (sigmoid/tanh); $\mathcal{O}$: output function (softmax for classification).

D. Shapes (slide)

$x_i \in \mathbb{R}^n$, $s_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}^k$ (k classes);

$$U \in \mathbb{R}^{n \times d}, \quad V \in \mathbb{R}^{d \times k}, \quad W \in \mathbb{R}^{d \times d}$$

- ⚠ Slide note: with $s_i = \sigma(Ux_i + \dots)$ written as matrix–vector product, U must be $d \times n$ and V must be $k \times d$; the slide lists the transposes ($n \times d$, $d \times k$). The **number of parameters is the same either way.** W is always square $d \times d$.

E. Numerical — parameter count

$$\#\text{params} = nd + d^2 + dk + d + k \quad (U, W, V, b, c)$$

- **Independent of sequence length** T (parameter sharing).
- Ex: $n = 100, d = 50, k = 10$: $U = 5000$, $W = 2500$, $V = 500$, $b = 50$, $c = 10$ → **8060**.
- Ex: $n = 300, d = 128, k = 4$: $38400 + 16384 + 512 + 128 + 4 = 55428$ (55296 without biases).
- Effect of doubling d: W term ×4, U, V terms ×2.

C. Properties

- Recurrence (Ws_{i-1}) handles dependence; sharing handles same-function and variable length.

- The state s_i summarizes $x_1..x_i$.

13.3 Backpropagation through time (BPTT)

A. Setup (auto-completion example)

- Vocabulary of 4 characters {d, e, p, ⟨stop⟩}; predict one at each step. Output function: **softmax**; loss: **cross entropy**.
- Parameters $\theta = \{U, V, W, b, c\}$.

Total loss = sum over time steps:

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \mathcal{L}_t(\theta), \qquad \mathcal{L}_t(\theta) = -\log(y_{tc})$$

y_{tc} = predicted probability of the **true** character at step t; T = number of time steps.

Numerical (slide example, inputs d,e,e,p; targets e,e,p,⟨stop⟩): predicted true-class probabilities $0.7, 0.7, 0.7, 0.1 \rightarrow \mathcal{L} = -3\ln 0.7 - \ln 0.1 = 3(0.357) + 2.303 = 3.373$ (natural log).

- ⚠ Slide note: the predicted vectors $[0.2, 0.7, 0.1, 0.1]$ sum to **1.1**, so they aren't valid softmax outputs; also the next slide changes the 4th input/target (input 'e', target 'p'), under which $\mathcal{L} = -4\ln 0.7 = 1.427$. Treat as illustrative.

B. Gradient w.r.t. V

$$\frac{\partial \mathcal{L}(\theta)}{\partial V} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t(\theta)}{\partial V}$$

(V is a matrix, so strictly $\nabla_V \mathcal{L}$.) Each term is just the derivative w.r.t. output-layer weights — ordinary backprop; no recursion through time is needed since V affects $\mathcal{L}_t$ only via y_t.

C. Gradient w.r.t. W — paths through time

$$\frac{\partial \mathcal{L}(\theta)}{\partial W} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t(\theta)}{\partial W}$$

$\partial \mathcal{L}_t / \partial W$ = sum of gradients along **all paths** from $\mathcal{L}_t$ to W. For $\mathcal{L}_4$: $\mathcal{L}_4$ depends on s_4; s_4 depends on s_3 **and** W; s_3 on s_2 and W; ... s_1 on s_0 and W (s_0 constant).

This is an **ordered network**: each state variable is computed one at a time in a specified order (s_1, then s_2, ...).

$$\frac{\partial \mathcal{L}_4}{\partial W} = \frac{\partial \mathcal{L}_4}{\partial s_4} \frac{\partial s_4}{\partial W}$$

With $s_4 = \sigma(W s_3 + b)$ we **cannot treat s_3 as a constant** (it also depends on W). The total derivative has two parts:

- **Explicit** $\frac{\partial^+ s_4}{\partial W}$: treating all other inputs as constant.
- **Implicit**: summing over all indirect paths from s_4 to W.

$$\frac{\partial s_4}{\partial W} = \underbrace{\frac{\partial^+ s_4}{\partial W}}_{\text{explicit}} + \underbrace{\frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial W}}_{\text{implicit}}$$

Expanding recursively and "short-circuiting" paths ($\frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial s_2} = \frac{\partial s_4}{\partial s_2}$, $\frac{\partial s_4}{\partial s_4} = I$):

$$\frac{\partial s_4}{\partial W} = \sum_{k=1}^{4} \frac{\partial s_4}{\partial s_k}\frac{\partial^+ s_k}{\partial W} \qquad \Longrightarrow \qquad \boxed{\frac{\partial \mathcal{L}_t(\theta)}{\partial W} = \frac{\partial \mathcal{L}_t(\theta)}{\partial s_t}\sum_{k=1}^{t} \frac{\partial s_t}{\partial s_k}\frac{\partial^+ s_k}{\partial W}}$$

BPTT = backpropagate over all previous time steps.

- ⚠ Slide note: slides write $s_4 = \sigma(W s_3 + b)$ (and later $a_k = W s_{k-1} + b$), omitting $U x_4$; harmless for $\partial/\partial W$ since Ux does not depend on W.
- (Additional clarification) U and b also enter every state, so their gradients need the same through-time sum; V and c do not.

E. Counting

- $\partial \mathcal{L}_t/\partial W$ has t **terms** ($k = 1..t$); term k contains the Jacobian product $\partial s_t/\partial s_k$ with $t-k$ **factors**. Total over the sequence: $\sum_t t = T(T+1)/2$ terms (e.g., $T = 4 \rightarrow 10$).

13.4 The problem of exploding and vanishing gradients

A. Core Concept

Focus on $\partial s_t/\partial s_k$:

$$\frac{\partial s_t}{\partial s_k} = \frac{\partial s_t}{\partial s_{t-1}}\frac{\partial s_{t-1}}{\partial s_{t-2}}\cdots\frac{\partial s_{k+1}}{\partial s_k} = \prod_{j=k}^{t-1}\frac{\partial s_{j+1}}{\partial s_j}\left(= \prod_{j=k+1}^{t}\frac{\partial s_j}{\partial s_{j-1}}\right)$$

One factor: with $a_j = W s_{j-1} + b$ (slide writes $W s_j + b$ — ⚠ typo), $s_j = \sigma(a_j)$ element-wise:

$$\frac{\partial s_j}{\partial a_j} = \operatorname{diag}(\sigma'(a_j)) \;\; \text{(off-diagonals 0 since } s_{jp} \text{ depends only on } a_{jp}\text{)}, \qquad \frac{\partial s_j}{\partial s_{j-1}} = \frac{\partial s_j}{\partial a_j}\frac{\partial a_j}{\partial s_{j-1}} = \operatorname{diag}(\sigma'(a_j))\, W$$

If this is small (large), $\partial s_t/\partial s_k$ and hence $\partial \mathcal{L}_t/\partial W$ will vanish (explode).

D. The bound

$$\left\|\frac{\partial s_j}{\partial s_{j-1}}\right\| = \|\operatorname{diag}(\sigma'(a_j))W\| \le \|\operatorname{diag}(\sigma'(a_j))\|\,\|W\| \le \gamma\|W\| \le \gamma\lambda$$

$$\left\|\frac{\partial s_t}{\partial s_k}\right\| = \left\|\prod_{j=k+1}^{t}\frac{\partial s_j}{\partial s_{j-1}}\right\| \le \prod_{j=k+1}^{t}\gamma\lambda = (\gamma\lambda)^{t-k}$$

- σ bounded (sigmoid, tanh) ⇒ σ' bounded: $\gamma = 1/4$ **for logistic**, $\gamma = 1$ **for tanh**.
  - (Additional clarification) What the bound actually uses is $\max \sigma'$. The logistic gives $\sigma' = \sigma(1-\sigma) \le 1/4$, with the maximum at $a = 0$. Tanh gives $1 - \tanh^2 \le 1$, also at $a = 0$. Since $\operatorname{diag}(\sigma'(a_j))$ is diagonal, its (spectral) norm is $\max_p \sigma'(a_{jp}) \le \gamma$. A saturated unit has $\sigma' \approx 0$, which makes vanishing even worse.
- λ: bound on $\|W\|$ (implicit on the slide: $\|W\| \le \lambda$).

- $\gamma\lambda < 1$ → **gradient will vanish;** $\gamma\lambda > 1$ → **gradient could explode.**

Field	
Purpose	explains why long-range dependencies are hard to learn with BPTT
Conditions	sub-multiplicative norm; bounded activation derivative
Interpretation	contribution of state s_k to $\mathcal{L}_t$ decays/grows **exponentially in the gap** $t-k$
MCQ relevance	compute $(\gamma\lambda)^{t-k}$; pick vanish vs explode; logistic vs tanh γ
Common mistakes	saying $\gamma\lambda > 1$ **guarantees** explosion (it's only an upper bound → "could"); using $\gamma = 1/2$ for sigmoid; exponent t instead of $t-k$

E. Numerical (verified)

- Logistic ($\gamma = 0.25$), $\|W\| \le \lambda = 2$: $\gamma\lambda = 0.5$, gap $t-k = 10$ → bound $0.5^{10} \approx 9.77 \times 10^{-4}$ → vanishes.
- tanh ($\gamma = 1$), $\lambda = 1.5$, gap 10 → $1.5^{10} \approx 57.7$ → could explode.
- Logistic with $\lambda = 4$: $\gamma\lambda = 1$ → bound stays 1 (borderline).
- Logistic needs $\|W\| > 4$ for the bound to even allow explosion; tanh needs $\|W\| > 1$.
- Effect of inputs: larger gap $t-k$ → exponentially stronger effect; smaller $\|W\|$ or saturating activations (small σ') → vanishing.

Truncated BPTT (the remedy discussed in L13)

- "One simple way of avoiding this is to use **truncated backpropagation** where we restrict the product to τ $(< t-k)$ terms" — i.e., backpropagate only through the last τ time steps (figure: loss $\mathcal{L}_t$ box covering only $x_3..x_n$).
- Trade-off (Additional clarification): bounds the number of Jacobian factors (so less vanishing/exploding and cheaper computation), but the model cannot get gradient signal about dependencies longer than τ steps.
- (Additional clarification) Other remedies — gradient clipping for explosion; gated architectures (LSTM/GRU, next lecture) for vanishing — are not part of these L13 slides.

13.5 Some gory details

D. Shapes in the BPTT formula

$$\underbrace{\frac{\partial \mathcal{L}_t(\theta)}{\partial W}}_{\in \mathbb{R}^{d\times d}} = \underbrace{\frac{\partial \mathcal{L}_t(\theta)}{\partial s_t}}_{\in \mathbb{R}^{1\times d}} \sum_{k=1}^{t} \underbrace{\frac{\partial s_t}{\partial s_k}}_{\in \mathbb{R}^{d\times d}} \underbrace{\frac{\partial^+ s_k}{\partial W}}_{\in \mathbb{R}^{d\times d\times d}}$$

- $\partial \mathcal{L}_t/\partial s_t$: scalar w.r.t. vector (from ordinary backprop); $\partial s_t/\partial s_k$: vector w.r.t. vector (Jacobian); $\partial^+ s_k/\partial W$: vector $\in \mathbb{R}^d$ w.r.t. matrix $\in \mathbb{R}^{d\times d}$ → **3-D tensor** $\in \mathbb{R}^{d\times d\times d}$.

Element of the tensor

$\frac{\partial^+ s_{kp}}{\partial W_{qr}}$ is the (p,q,r)-th element. With $a_k = W s_{k-1} + b$, $a_{kp} = \sum_{i=1}^{d} W_{pi} s_{k-1,i}$, $s_{kp} = \sigma(a_{kp})$:

$$\frac{\partial s_{kp}}{\partial W_{qr}} = \sigma'(a_{kp}) \frac{\partial a_{kp}}{\partial W_{qr}} = \begin{cases} \sigma'(a_{kp})\, s_{k-1,r} & \text{if } p = q \\ 0 & \text{otherwise} \end{cases}$$

(slide writes "if $p = q$ and $i = r$" — the $i = r$ just picks the matching term of the sum.)

E. Numerical

- $d = 50$: tensor has $50^3 = 125000$ entries but only those with $p = q$ are nonzero → $d^2 = 2500$ nonzero (very sparse).
- Row p of W only affects state component p directly (explicitly).

G. Quick Revision Summary — Lecture 13

- Sequence problems: variable-length input, dependent inputs, same task at each step; output at every step (POS, autocomplete) or only at the end (sentiment, video activity).
- RNN: $s_i = \sigma(Ux_i + Ws_{i-1} + b)$, $y_i = \mathcal{O}(Vs_i + c)$; W, U, V, b, c shared → length-agnostic; #params $= nd + d^2 + dk + d + k$, independent of T.
- Loss: $\mathcal{L} = \sum_t -\log y_{tc}$ (softmax + cross-entropy).
- BPTT: $\frac{\partial \mathcal{L}_t}{\partial W} = \frac{\partial \mathcal{L}_t}{\partial s_t} \sum_{k=1}^{t} \frac{\partial s_t}{\partial s_k} \frac{\partial^+ s_k}{\partial W}$; explicit ($\partial^+$) vs implicit (through earlier states).
- $\frac{\partial s_j}{\partial s_{j-1}} = \text{diag}(\sigma'(a_j))W$; $\|\partial s_t / \partial s_k\| \leq (\gamma\lambda)^{t-k}$; $\gamma = 1/4$ (logistic), 1 (tanh); < 1 vanish, > 1 could explode.
- Truncated BPTT: restrict product to τ terms.

Likely MCQ takeaways: (1) parameters don't grow with T; (2) $W \in \mathbb{R}^{d \times d}$; (3) γ values and $(\gamma\lambda)^{t-k}$; (4) "will vanish" vs "could explode"; (5) $\partial^+ s_k / \partial W$ is a $d \times d \times d$ tensor.

Overview: Lectures 14–15: LSTM, GRU, Encoder–Decoder Models, Attention

Lectures covered / modules:

- **Lecture 14 – LSTMs and GRUs**
  - 14.1 Selective Read, Selective Write, Selective Forget: the whiteboard analogy
  - 14.2 Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs)
  - 14.3 How LSTMs avoid the problem of vanishing gradients (and why they do not solve exploding gradients)
- **Lecture 15 – Encoder–Decoder Models and the Attention Mechanism**
  - 15.1 Introduction to Encoder–Decoder Models (language modelling, then image captioning)
  - 15.2 Applications of Encoder–Decoder Models (entailment, MT, transliteration, image QA, summarization, video captioning and classification, dialog)
  - 15.3 Attention Mechanism (with interpreting attention heat maps)
  - 15.4 Attention over Images
  - 15.5 Hierarchical Attention

Notation used on the slides throughout: s_t = RNN state (in an LSTM, the **cell state**), h_t = LSTM output/hidden vector, x_t = input, σ = logistic sigmoid, $\odot$ = element-wise product. W multiplies the previous state/hidden vector, U multiplies the input, V is the output layer, and b, c are biases. In Lecture 15 decoder states are s_t and encoder states are h_j (one slide calls them c_j).

Week 11 · Lecture 14: Long Short-Term Memory Cells (LSTMs), Gated Recurrent Units (GRUs)

14.1 Selective Read, Selective Write, Selective Forget: The Whiteboard Analogy

A. Core Concept

- The RNN state s_i records information from **all previous time steps**. At every step the old information gets **morphed** by the current input, so after t steps the information stored at step $t-k$ may be morphed so much that it can no longer be recovered.
- The same problem shows up in the **backward** direction. It is hard to assign responsibility (that is, gradients) for the error at step t to events at step $t-k$. This is the vanishing-gradient problem from Lecture 13.
- **Analogy.** The state is a **fixed-size memory**, like a fixed-size whiteboard. If you keep writing on it at every step, you eventually cannot tell how the information at $t-k$ contributed to the state at t.
- **Strategy for using a small board well:**
    1. **Selective write:** do not write every step of the derivation. Skip some.
    2. **Selective read:** when writing the next step, read only the relevant earlier steps.
    3. **Selective forget:** when the board is full, erase the **least useful** (obsolete) information.
- Since an RNN's state is also finite ($s_t \in \mathbb{R}^n$), we want the RNN to selectively read, write and forget. This "wishlist" is what the LSTM implements.

Whiteboard example (on the slides)

$a=1, b=3, c=5, d=11$. Compute $ac(bd+a)+ad$. The board holds **only 3 statements**.

Step	Board contents after the step
write ac, bd	$ac=5$, $bd=33$
read bd → write $bd+a$	$ac=5$, $bd=33$, $bd+a=34$ (board full)
forget bd → write $ac(bd+a)$	$ac=5$, $ac(bd+a)=170$, $bd+a=34$
forget $bd+a$ → write ad	$ac=5$, $ac(bd+a)=170$, $ad=11$
forget ac → write result	$ad+ac(bd+a)=181$, $ac(bd+a)=170$, $ad=11$

- Selective write: we choose which derivation steps to write at all.
- Selective read: at step 3 ($bd+a$), the information from step 2 (bd) is what matters.
- Selective forget: once $bd+a$ has been used, bd is obsolete and is erased.
- Another analogy from the slides: the brain stores a finite number of facts and selectively reads, writes and forgets them.

Concrete RNN example: sentiment of a review

"The first half of the movie was dry but the second half really picked up pace. The lead actor delivered an amazing performance." The RNN reads left to right, updates s_t after every word, and predicts +/− at the end. By the end, information from the first few words is lost. **Ideally** the RNN should:

- **forget** the information added by stop words (a, the, ...),
- **selectively read** the information added by earlier sentiment-bearing words (awesome, amazing, ...),
- **selectively write** new information from the current word to the state.

C. Insights

- Vanilla RNN: $s_t = \sigma(W s_{t-1} + U x_t)$. The **whole** of s_{t-1} is passed on and overwritten at every step.
- The strictest form of selection would be **binary** (keep entries 1 and 3, delete the rest). A more sensible choice is a **fraction in** $[0, 1]$ **per element**. This is why gates are **sigmoid** vectors applied element-wise.

14.2 LSTM and GRU

A. Building the LSTM step by step (slide order)

We have s_{t-1} and the new input x_t, and we want to compute s_t while keeping only important information.

1. Selective write (output gate). Instead of passing s_{t-1} as it is, we pass only a fraction of each element:

$$o_{t-1} = \sigma(W_o h_{t-2} + U_o x_{t-1} + b_o), \qquad h_{t-1} = o_{t-1} \odot \sigma(s_{t-1})$$

- Each element of o_{t-1} is in $(0, 1)$ (because of the sigmoid) and multiplies the corresponding element of $\sigma(s_{t-1})$.
- o_t is called the **output gate** because it decides how much to pass (write) to the next time step.
- W_o, U_o, b_o are **learned** along with W, U, V. The network learns what fraction to pass on.

2. Selective read (input gate). First compute a temporary state from h_{t-1} and x_t:

$$\tilde{s}_t = \sigma(W h_{t-1} + U x_t + b)$$

(W, U, b play the same role as in a vanilla RNN.) $\tilde{s}_t$ captures all the information from the previous state (h_{t-1}) and the current input. We may not want all of it, so we read selectively through the **input gate**:

$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i), \qquad \text{selectively read: } i_t \odot \tilde{s}_t$$

3. Selective forget (forget gate). The simplest way to combine the old and new is $s_t = s_{t-1} + i_t \odot \tilde{s}_t$. But we may want to forget parts of s_{t-1}, so we add a **forget gate**:

$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f), \qquad s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$$

D. Full LSTM equations (slide 24, exact form)

$$\begin{array}{ll} \textbf{Gates:} & o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o) \\ & i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i) \\ & f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f) \\ \textbf{States:} & \tilde{s}_t = \sigma(W h_{t-1} + U x_t + b) \\ & s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t \\ & h_t = o_t \odot \sigma(s_t) \quad \text{and} \quad rnn_{out} = h_t \end{array}$$

Symbol	Meaning	Shape (state dim d, input dim n)
s_t	cell state (the "whiteboard")	$\mathbb{R}^d$

Symbol	Meaning	Shape (state dim d, input dim n)
h_t	output/hidden vector, the selectively written view of s_t	$\mathbb{R}^d$
$\tilde{s}_t$	candidate (temporary) state	$\mathbb{R}^d$
o_t, i_t, f_t	output, input and forget gates, entries in $(0,1)$	$\mathbb{R}^d$
W, W_o, W_i, W_f	recurrent weights (act on h_{t-1})	$d \times d$
U, U_o, U_i, U_f	input weights	$d \times n$
b, b_o, b_i, b_f	biases	$\mathbb{R}^d$

- All the gates and $\tilde{s}_t$ are computed from h_{t-1} **and** x_t, **not** from s_{t-1}.
- The LSTM carries **two** vectors across time (h_t and s_t). In Lecture 15's compact notation this is $h_t, s_t = \mathrm{LSTM}(h_{t-1}, s_{t-1}, x_t)$.
- The output of the cell (what goes to V/softmax) is h_t.
- The note on slide 25: the LSTM has **many variants** with different numbers and arrangements of gates. The one shown is one of the most popular. The GRU is another, equally popular variant.
- ⚠ Slide note: the slides use σ for $\tilde{s}_t$ and in $h_t = o_t \odot \sigma(s_t)$. (Additional clarification: most textbooks and libraries use $\tanh$ in these two places and σ only for the gates. Answer using the slide form unless the question says otherwise.)
- ⚠ The numbers in the slide figures ($s_{t-1} = [-1.4, -0.4, 1, \ldots, -2]$, $o_{t-1} = [0.2, 0.34, 0.9, \ldots, 0.29]$, $\tilde{s}_t = [0.4, 0.6, 0.1, \ldots, 0.2]$, $i_t = [0.8, 0.66, 0.1, \ldots, 0.71]$, $f_t = [0.9, 0.7, 0.9, \ldots, 0.8]$) are illustrative only. Recomputing (python) $f_t \odot s_{t-1} + i_t \odot \tilde{s}_t = [-0.94, 0.12, 0.91, \ldots, -1.46]$, which only roughly matches the drawn $s_t = [-0.9, 0.2, 1, \ldots, -1.9]$ (slide 23); slide 24 then redraws the first entry as -1.5. Likewise $o_{t-1} \odot \sigma(s_{t-1}) = [0.04, 0.14, 0.66, \ldots, 0.035]$, not the drawn $h_{t-1} = [0.5, 0.36, 0.9, \ldots, 0.6]$. Do not use the figure numbers as a worked example. (In the GRU figure, slide 26, the drawn $1 - i_t = [0.2, 0.34, 0.9, \ldots, 0.29]$ *is* consistent with i_t.)

D. GRU equations (slide 26, exact form)

$$
\begin{aligned}
\textbf{Gates:} \quad & o_t = \sigma(W_o s_{t-1} + U_o x_t + b_o) \\
& i_t = \sigma(W_i s_{t-1} + U_i x_t + b_i) \\
\textbf{States:} \quad & \tilde{s}_t = \sigma(W(o_t \odot s_{t-1}) + U x_t + b) \\
& s_t = (1 - i_t) \odot s_{t-1} + i_t \odot \tilde{s}_t
\end{aligned}
$$

- **No explicit forget gate.** The forget and input gates are **tied**: the forget fraction is $1 - i_t$.
- The gates depend **directly on** s_{t-1}, not on an intermediate h_{t-1} as in the LSTM. There is **no separate** h_t, and the state itself is the output.
- In the GRU, o_t is applied to s_{t-1} **inside** the candidate computation (a selective read of the old state). In the LSTM, o_t is applied to $\sigma(s_t)$ to produce h_t (a selective write).
- (Additional clarification: in the standard GRU literature o_t here is the **reset gate** r_t and i_t is the **update gate** z_t. Standard GRUs also use $\tanh$ for $\tilde{s}_t$.)
- ⚠ **Slide inconsistency (verified on both rendered slides).** Lecture 15 slide 8 (compact notation) writes the GRU as $s_t = i_t \odot s_{t-1} + (1 - i_t) \odot \tilde{s}_t$, which swaps the roles of i_t and $1 - i_t$ relative to Lecture 14 slide 26 ($s_t = (1 - i_t) \odot s_{t-1} + i_t \odot \tilde{s}_t$). The candidate $\tilde{s}_t = \sigma(W(o_t \odot s_{t-1}) + U x_t + b)$ is identical on both. Both are convex combinations and equally valid models; only the meaning of i_t flips ("fraction of new candidate" vs "fraction of old state kept"). Lecture 14 slide 26 is the dedicated GRU slide, so default to it, but check which convention a question states. (Additional clarification: both conventions also exist in the literature, e.g. Cho et al. 2014 / PyTorch use $z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$, others use $(1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$.)

F. LSTM vs GRU vs RNN

	Vanilla RNN	LSTM (slide form)	GRU (slide form)
State(s) carried	s_t	s_t (cell) **and** h_t	s_t only
Gates	none	3: output o, input i, forget f	2: o (reset), i (update)
Gates computed from	–	h_{t-1}, x_t	s_{t-1}, x_t
Forget mechanism	full overwrite	separate f_t	tied: $(1 - i_t)$
Weight blocks (W, U, b)	1	4 $(o, i, f, \tilde{s})$	3 $(o, i, \tilde{s})$
Compact form	$s_t = \mathrm{RNN}(s_{t-1}, x_t)$	$h_t, s_t = \mathrm{LSTM}(h_{t-1}, s_{t-1}, x_t)$	$s_t = \mathrm{GRU}(s_{t-1}, x_t)$

E. Numericals: parameter counts (Additional clarification: these are not on the slides but follow directly from the equations)

Let the state/hidden dim be d and the input dim be n. Each block of $(W : d \times d,\ U : d \times n,\ b : d)$ has $d^2 + dn + d = d(d + n + 1)$ parameters.

Model	# recurrent-cell parameters
RNN	$d(d + n + 1)$
GRU	$3\,d(d + n + 1)$
LSTM	$4\,d(d + n + 1)$

The output layer $V \in \mathbb{R}^{k \times d}$ plus bias $c \in \mathbb{R}^k$ adds $kd + k$ for k output classes or vocabulary words, the same for all three models.

Worked examples (python-verified):

- $d = 256, n = 100$: per block $= 256 \cdot 357 = 91{,}392$. RNN $91{,}392$, GRU $274{,}176$, LSTM $365{,}568$.
- $d = 100, n = 50$: per block $15{,}100$. GRU $45{,}300$, LSTM $60{,}400$. Adding a softmax over $k = 1000$ words ($kd + k = 101{,}000$) gives LSTM $161{,}400$ and GRU $146{,}300$ in total.
- $d = 10, n = 5$: RNN 160, GRU 480, LSTM 640.
- Shortcut: **LSTM : GRU : RNN = 4 : 3 : 1** (cell parameters only, with the same d, n).
- Traps: forgetting the biases ($4d$ in an LSTM); using $n \times d$ vs $d \times n$ (same count, so no matter); counting V inside the cell; doubling for the h and s vectors (wrong, because s_t adds no extra weights).

E. Numerical: one LSTM/GRU step (scalar, $d = 1$)

Given $s_{t-1} = 2,\ f_t = 0.5,\ i_t = 0.25,\ \tilde{s}_t = 0.8,\ o_t = 1$:

- LSTM: $s_t = 0.5 \cdot 2 + 0.25 \cdot 0.8 = 1.2$, and $h_t = o_t\,\sigma(1.2) = 0.7685$.
- GRU with update gate $i_t = 0.25$, Lecture 14 convention: $s_t = (1 - 0.25) \cdot 2 + 0.25 \cdot 0.8 = 1.7$. With the Lecture 15 slide 8 convention: $0.25 \cdot 2 + 0.75 \cdot 0.8 = 1.1$.
- Limits: $f_t \to 1, i_t \to 0$ gives $s_t = s_{t-1}$ (memory preserved exactly). $f_t \to 0$ erases the old state.

14.3 How LSTMs Avoid the Problem of Vanishing Gradients

A. Intuition

- During **forward** propagation the gates control the flow of information. They stop irrelevant information being written to the state.
- During **backward** propagation they control the flow of gradients. The gradients get **multiplied by the gates**.
- If s_{t-1} did not contribute much to s_t (i.e. $\|f_t\| \to 0$ and $\|o_{t-1}\| \to 0$), the gradients flowing into s_{t-1} vanish. **That is fine**: since s_{t-1} did not contribute to s_t, we should not "hold it responsible for the crimes of s_t".
- **Key difference from vanilla RNNs:** the gates make the gradients vanish **only when they should**, that is, when s_{t-1} did not contribute much to s_t.

C. The "paths" argument (examinable logic)

- Recall the vanilla RNN gradient (Lecture 13), where the product term causes vanishing:

$$\frac{\partial \mathcal{L}_t(\theta)}{\partial W} = \frac{\partial \mathcal{L}_t(\theta)}{\partial s_t} \sum_{k=1}^{t} \Big(\prod_{j=k}^{t-1} \frac{\partial s_{j+1}}{\partial s_j} \Big) \frac{\partial^+ s_k}{\partial W}$$

If $\mathcal{L}_4$ was high because W computed s_1 badly, that information does not reach W along this long path.

- **General principle (very examinable):**
  - The gradient of $\mathcal{L}_t(\theta)$ w.r.t. θ_i **vanishes** only when the gradients along **each and every path** from $\mathcal{L}_t(\theta)$ to θ_i vanish.
  - It **explodes** when the gradient along **at least one path** explodes.
- So to show "no vanishing", it is enough to show that there is **at least one path** along which gradients flow effectively. To show "exploding is possible", it is enough to find one path that can explode.
- **Dependency graph** at step k: s_{k-1} and h_{k-1} feed into $o_k, i_k, f_k, \tilde{s}_k$ (all from h_{k-1}). Then s_k depends on $f_k, s_{k-1}, i_k, \tilde{s}_k$, and h_k depends on o_k, s_k. This recursion continues to step t, where $h_t \to \mathcal{L}_t(\theta)$. The parameters ($W, W_o, W_i, W_f, U, \ldots$) are drawn on the edges.
- **Goal:** does the gradient reach, say, W_f through s_k for an arbitrary k? It is **sufficient to show** $\partial \mathcal{L}_t / \partial s_k$ **does not vanish**, because W_f can then be reached through s_k.
- There are **many paths** from $\mathcal{L}_t$ to s_k: through s_{k+1}, through h_k, and h_k itself has many outgoing arrows.

D. Path t_0 (through the cell states): no vanishing

$$t_0 = \frac{\partial \mathcal{L}_t(\theta)}{\partial h_t} \frac{\partial h_t}{\partial s_t} \frac{\partial s_t}{\partial s_{t-1}} \cdots \frac{\partial s_{k+1}}{\partial s_k}$$

- $\dfrac{\partial \mathcal{L}_t}{\partial h_t}$ is fine. h_t is directly connected to the loss with no intermediate nodes.
- $\dfrac{\partial h_t}{\partial s_t}$: since $h_t = o_t \odot \sigma(s_t)$, h_{ti} depends only on o_{ti} and s_{ti}. So this is a **square diagonal matrix** in $\mathbb{R}^{d \times d}$, written $\mathcal{D}(o_t \odot \sigma'(s_t))$.
- $\dfrac{\partial s_t}{\partial s_{t-1}}$: $s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$, and $\tilde{s}_t$ **also** depends on s_{t-1} (via h_{t-1}), so this is an ordered network with an **explicit + implicit** term. **Worst-case assumption:** the implicit term vanishes. The explicit term (treating $\tilde{s}_t$ as constant) is $\mathcal{D}(f_t)$.
- Result:

$$t_0 = \mathcal{L}_t'(h_t)\,\mathcal{D}(o_t \odot \sigma'(s_t))\,\mathcal{D}(f_t)\cdots\mathcal{D}(f_{k+1}) = \mathcal{L}_t'(h_t)\,\mathcal{D}(o_t \odot \sigma'(s_t))\,\mathcal{D}\big(\bigodot_{i=k+1}^{t} f_i\big)$$

- **Interpretation:** the first two factors do not vanish. The last factor is a **product of forget gates**. The forget gates regulate gradient flow according to the **explicit contribution of s_t to s_{t+1}**.
  - If s_t contributed little to s_{t+1} in the forward pass ($f_t \to 0$), the gradient does not reach s_t in the backward pass. This is **fair**, because f_t applies the same regulation forward and backward.
  - So there **exists one path** along which the gradient does not vanish when it shouldn't. If the gradient reaches W_f through s_k along t_0, we are fine.
- (Additional clarification) Why this beats the vanilla RNN: along t_0 there is **no repeated multiplication by a weight matrix** W and no repeated σ' (≤ 0.25) factor. Only the gate values, which the network learns, are multiplied.

E. Numerical: forget-gate product vs vanilla-RNN factor (python-verified)

Over 10 steps:

- with all $f_i = 0.9$: $0.9^{10} = 0.349$, so the gradient survives;
- with $f_i = 0.5$: $0.5^{10} \approx 9.8 \times 10^{-4}$, which vanishes, but only because the network chose to forget;
- with a vanilla RNN factor bounded by $\sigma' \leq 0.25$ (Lecture 13; taking $\|W\| \approx 1$): $0.25^{10} \approx 9.5 \times 10^{-7}$. (Additional clarification: the comparison is illustrative; the slides make the argument symbolically.)

Also $0.99^{100} = 0.366$: a forget gate near 1 keeps gradients alive over very long spans.

D. Path t_1 (through the output gates): exploding is still possible

LSTMs **do not** solve exploding gradients. Take one term of $\partial\mathcal{L}_t/\partial h_{k-1}$ along the path $h_{k-1} \to o_k \to h_k \to \cdots \to o_t \to h_t \to \mathcal{L}_t$:

$$t_1 = \frac{\partial \mathcal{L}_t(\theta)}{\partial h_t}\Big(\frac{\partial h_t}{\partial o_t}\frac{\partial o_t}{\partial h_{t-1}}\Big)\cdots\Big(\frac{\partial h_k}{\partial o_k}\frac{\partial o_k}{\partial h_{k-1}}\Big) = \mathcal{L}_t'(h_t)\,\big(\mathcal{D}(\sigma(s_t)\odot o_t')W_o\big)\cdots\big(\mathcal{D}(\sigma(s_k)\odot o_k')W_o\big)$$

$$\|t_1\| \leq \|\mathcal{L}_t'(h_t)\|\,\big(\|K\|\,\|W_o\|\big)^{t-k+1}$$

- Here W_o is multiplied repeatedly. **Depending on the norm of W_o, $\partial\mathcal{L}_t/\partial h_{k-1}$ may explode**. W_i, W_f and W can similarly cause explosion.
- There are $t-k+1$ factors, one for each step $k, \ldots, t$. (Additional clarification: K is not defined on the slide; read $\|K\|$ as a bound on the norms of the diagonal factors $\mathcal{D}(\sigma(s_m)\odot o_m')$. Since $\sigma(\cdot) < 1$ and $o_m' = \sigma'(\cdot) \leq 0.25$, $\|K\| \leq 0.25$, so explosion needs $\|W_o\|$ large.) E.g. if $\|K\|\|W_o\| = 1.5$ over 10 factors, the bound grows as $1.5^{10} \approx 57.7$; if $\|K\|\|W_o\| < 1$ the bound shrinks.
- Since **explosion along one path is enough**, the LSTM can still explode.

Gradient clipping (fix for exploding gradients)

- While backpropagating, **if the norm of the gradient exceeds a threshold, it is scaled down** to keep the norm within that threshold. **The direction is retained and only the norm is scaled.** (Pascanu, Mikolov & Bengio, ICML 2013, "On the difficulty of training recurrent neural networks.")
- (Additional clarification, standard form) If $\|g\| > \tau$: $g \leftarrow \tau\,\dfrac{g}{\|g\|}$.
- Example: $g = [3, 4]$ ($\|g\| = 5$), $\tau = 1$ gives $[0.6, 0.8]$. $g = [6, 8]$ ($\|g\| = 10$), $\tau = 5$ gives $[3, 4]$. If $\|g\| \leq \tau$, g is unchanged.

F. MCQ Traps & Conceptual Distinctions (Lecture 14)

Statement	Verdict
"LSTMs solve vanishing **and** exploding gradients"	**False.** They address vanishing only. Exploding needs gradient clipping.
"In an LSTM, gradients never vanish"	**False.** They vanish when the forget gates are ≈0, which is intended or "fair".
"Gradient vanishes if it vanishes along at least one path"	**False.** It must vanish along **every** path.
"Gradient explodes if it explodes along at least one path"	**True.**
"$\partial h_t / \partial s_t$ is a full matrix"	**False.** It is diagonal, $\mathcal{D}(o_t \odot \sigma'(s_t))$.
"In the t_0 analysis the implicit term was assumed to vanish"	**True** (worst-case assumption).
"LSTM gates are computed from s_{t-1}"	**False** for the LSTM (h_{t-1}). **True** for the GRU.
"GRU has a separate forget gate"	**False.** It is tied to the input gate as $1 - i_t$.
"Gradient clipping changes the gradient direction"	**False.** Only the norm is rescaled.
"Output gate decides how much of the state to **write** to the next step"	**True** (slide terminology: selective write = output gate).
"Input gate implements selective read; forget gate implements selective forget"	**True** (slide mapping).

- Mapping to remember: **write** ↔ **output gate** o, **read** ↔ **input gate** i, **forget** ↔ **forget gate** f.
- Gate values lie in $(0, 1)$, not $\{0, 1\}$. A binary decision was explicitly rejected in favour of fractions.
- The W, U, b used in $\tilde{s}_t$ are "similar to the parameters used in the RNN" (slide 20). The extra LSTM parameters are $W_o, U_o, b_o, W_i, U_i, b_i, W_f, U_f, b_f$, all learned along with W, U, V.

G. Quick Revision Summary (Lecture 14)

- A finite state gets morphed and overloaded, so we need **selective write/read/forget**, implemented by sigmoid gates.
- LSTM: $o_t, i_t, f_t = \sigma(W_* h_{t-1} + U_* x_t + b_*)$; $\tilde{s}_t = \sigma(W h_{t-1} + U x_t + b)$; $s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$; $h_t = o_t \odot \sigma(s_t)$.
- GRU: o_t, i_t from s_{t-1}; $\tilde{s}_t = \sigma(W(o_t \odot s_{t-1}) + U x_t + b)$; $s_t = (1 - i_t) \odot s_{t-1} + i_t \odot \tilde{s}_t$.
- Parameters: LSTM $4d(d + n + 1)$, GRU $3d(d + n + 1)$, RNN $d(d + n + 1)$.
- Vanishing: the gradient along the cell-state path is $\propto \mathcal{D}(\bigodot f_i)$, so it vanishes only if the forget gates close. Exploding: the path through o repeatedly multiplies by W_o, giving $\|t_1\| \leq \|\mathcal{L}'_t\| (\|K\| \|W_o\|)^{t-k+1}$. Fix with **gradient clipping**.
- Likely MCQ takeaways: (1) the gate ↔ operation mapping; (2) GRU's tied gates and its dependence on s_{t-1}; (3) the "every path / at least one path" rule; (4) LSTM ≠ fix for exploding gradients; (5) the 4:3:1 parameter ratio.

Week 12 · Lecture 15: Encoder–Decoder Models, Attention Mechanism

15.1 Introduction to Encoder–Decoder Models

A. Language modelling with an RNN (the starting point)

- Given $y_1, \ldots, y_{t-1}$, predict the t-th word: $y^* = \arg\max P(y_t \mid y_1, \ldots, y_{t-1})$. Shorthand: $P(y_t \mid y_1^{t-1})$. (Slide example: `<GO>` → "I am at home today" → `<stop>`, each predicted word fed back as the next input.)
- RNN model, with $j \in \mathcal{V}$ (the vocabulary):

$$P(y_t = j \mid y_1^{t-1}) = P(y_t = j \mid s_t) = \text{softmax}(Vs_t + c)_j, \qquad s_t = \sigma(Ws_{t-1} + Ux_t + b)$$

The recurrent connections ensure that s_t contains information about y_1^{t-1}.

- **Data:** all sentences from any large corpus (e.g. Wikipedia). **Parameters:** U, V, W, b, c.
- **Loss:** $\mathcal{L}(\theta) = \sum_{t=1}^{T} \mathcal{L}_t(\theta)$, with $\mathcal{L}_t(\theta) = -\log P(y_t = \ell_t \mid y_1^{t-1})$, where ℓ_t is the **true word** at step t. This is cross-entropy summed over time.
- **Input at each step** = the word predicted at the previous step: $x_t = e(v_j)$, where j is the index of the max-probability word at $t-1$. $e(v_j)$ is a one-hot vector. **In practice a pre-trained word embedding** of the j-th word is used instead of one-hot. The first input is `<GO>` and generation ends at `<stop>`.
- s_0 **is not computed.** It is randomly initialised and **learned** along with the other parameters (of the RNN, LSTM or GRU).

Compact notation (slide 8), used from here on

Model	Equations	Compact
RNN	$s_t = \sigma(Ux_t + Ws_{t-1} + b)$	$s_t = \text{RNN}(s_{t-1}, x_t)$
GRU	$\tilde{s}_t = \sigma(W(o_t \odot s_{t-1}) + Ux_t + b)$, $s_t = i_t \odot s_{t-1} + (1 - i_t) \odot \tilde{s}_t$ ⚠	$s_t = \text{GRU}(s_{t-1}, x_t)$
LSTM	$\tilde{s}_t = \sigma(Wh_{t-1} + Ux_t + b)$, $s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$, $h_t = o_t \odot \sigma(s_t)$	$h_t, s_t = \text{LSTM}(h_{t-1}, s_{t-1}, x_t)$

⚠ Slide note: the GRU line here swaps i_t and $1 - i_t$ compared with Lecture 14 slide 26. See 14.2.

Image captioning: conditioning on an image

- We now want $P(y_t \mid y_1^{t-1}, I)$, where I is an image. This is again a conditional distribution. Model it as $P(y_t = j \mid s_t, fc_7(I))$, where $fc_7(I)$ is the representation from the **fc7 layer** of a CNN.
- **Option 1:** set $s_0 = fc_7(I)$. Then s_0, and hence every later s_t, depends on $fc_7(I)$.
- **Option 2 (more explicit):** $s_t = \text{RNN}(s_{t-1}, [x_t, fc_7(I)])$. The image encoding is fed (concatenated) at **every** step.
- Other ways of conditioning are possible.
- **Encoder–decoder architecture:** a **CNN encodes** the image and an **RNN decodes** (generates) the sentence. **Both** the encoder and the decoder are neural networks. The encoder output can go only to s_0 or alternatively to every decoder step.

15.2 Applications of Encoder–Decoder Models

For every application the slides ask four questions: (1) the encoder, (2) the decoder, (3) the parameters, (4) the loss. The training **algorithm** is always **gradient descent with backpropagation**.

Generic sequence-output template, with T the input length and $\hat{y}_{t-1}$ the previous prediction:

$$\text{Encoder: } h_t = \text{RNN}(h_{t-1}, x_{it}); \quad \text{Decoder: } s_0 = h_T,\ s_t = \text{RNN}(s_{t-1}, e(\hat{y}_{t-1})),\ P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t +$$

$$\mathcal{L}(\theta) = \sum_t \mathcal{L}_t(\theta) = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^{t-1}, x)$$

- **Option 1:** $s_0 = h_T$ only. **Option 2:** also feed h_T at every step, $s_t = \text{RNN}(s_{t-1}, [h_T, e(\hat{y}_{t-1})])$. Option 2 has the same parameter list on the slides.

Task	Data $\{x_i, y_i\}$	Encoder	Decoder	Parameters (slide)	Loss
Image captioning	image, caption	$s_0 = \text{CNN}(x_i)$	$s_t = \text{RNN}(s_{t-1}, e(\hat{y}_{t-1}))$, softmax(Vs_t+b)	$U_{dec}, V, W_{dec}, W_{conv}, b$	$-\sum_t \log P(y_t = \ell_t \mid y_1^{t-1}, I)$
Textual entailment (e.g. "It is raining outside" → "The ground is wet")	premise, hypothesis	RNN over premise; $s_0 = h_T$	RNN generator (Opt 1 or 2)	$U_{dec}, V, W_{dec}, U_{enc}, W_{enc}, b$	sum of CE over output words
Machine translation ("I am going home" → "Mein ghar ja raha hoon")	source, target	RNN over source	RNN over target words (Opt 1 or 2)	same as above	same
Transliteration ("INDIA" → इंडिया)	src word, tgt word (**character** sequences)	RNN over source chars	RNN over target chars; Opt 2: $[e(\hat{y}_{t-1}), h_T]$	same	same
Image question answering ("What is the bird's color?" → "White")	$\{I, q\}$, answer	$\hat{h}_I = \text{CNN}(I)$, $\tilde{h}_t = \text{RNN}(\tilde{h}_{t-1}, q_{it})$, $s = [\tilde{h}_T; \hat{h}_I]$	**single** softmax: $P(y \mid q, I) = \text{softmax}(Vs + b)$	$V, b, U_q, W_q, W_{conv}, b$	$-\log P(y = \ell \mid I, q)$ (one term)
Document summarization	document, summary	RNN over document words	RNN generator, $s_0 = h_T$	$U_{dec}, V, W_{dec}, U_{enc}, W_{enc}, b$	sum of CE
Video captioning ("A man walking on a rope")	video, description	$h_t = \text{RNN}(h_{t-1}, \text{CNN}(x_{it}))$ (CNN per frame, then RNN)	RNN generator, $s_0 = h_T$	$U_{dec}, W_{dec}, V, b, W_{conv}, U_{enc}, W_{enc}, b$	sum of CE

Task	Data $\{x_i, y_i\}$	Encoder	Decoder	Parameters (slide)	Loss
Video classification ("Surya Namaskar")	video, activity	same CNN+RNN encoder	$s = h_T$; $P(y \mid I) = \text{softmax}(Vs + b)$	$V, b, W_{conv}, U_{enc}, W_{enc}, b$	$-\log P(y = \ell \mid \text{Video})$
Dialog ("How are you" → "I am fine")	utterance, response	RNN over utterance	RNN generator, $s_0 = h_T$	$U_{dec}, V, W_{dec}, U_{enc}, W_{enc}, b$	sum of CE

Key distinctions:

- **Sequence output** (captioning, entailment, MT, transliteration, summarization, video captioning, dialog): an RNN decoder, with the loss summed over output time steps.
- **Single label output** (image QA, video classification): the "decoder" is just a **softmax layer** on the encoding, and the loss is a single $-\log P$ term.
- **Two encoders** (image QA): a CNN for the image and an RNN for the question, with their outputs concatenated.
- **CNN inside the RNN encoder** (video tasks): the per-frame CNN output is the RNN input.
- W_{conv} appears whenever a CNN is part of the model, since the CNN parameters are also learned.
- ⚠ Slide notes: (i) the loss is written $\sum_{i=1}^{T} \mathcal{L}_t(\theta)$, where the index should be t. (ii) Image QA and the video tasks list b twice (two different biases, encoder/decoder). (iii) Video classification writes $P(y \mid I)$ although the input is a video. (iv) The slides use T for both input and output length.
- Encoder–decoder models can be made more expressive by adding **attention**.

15.3 Attention Mechanism

A. Motivation (illustrated with MT: "Main ghar ja raha hoon" → "I am going home")

- The vanilla encoder **reads the sentence only once** and encodes it. At each step the decoder uses **this same embedding** to produce a new word. It is fed either once (at s_0) or the same encoding at every step.
- Humans don't translate this way. They produce each output word by **focusing only on certain input words**. Effectively, at each output step there is a **distribution over input words** that says how much attention to pay to each.

Output step	Attention over (Main, ghar, ja, raha, hoon)
t_1: I	[1, 0, 0, 0, 0]
t_2: am	[0, 0, 0, 0, 1]
t_3: going	[0, 0, 0.5, 0.5, 0]
t_4: home	[0, 1, 0, 0, 0]

- Ideally the decoder should be fed only the **relevant** information (the encodings of the relevant words) at each step. This is a **weighted average** of the word representations, e.g. at $t = 3$ a weighted average of 'ja' and 'raha'.
- This should work better because the decoder is **not overloaded with irrelevant information** about words that do not matter at that step. (Additional clarification: this is often described as removing the *fixed-length vector bottleneck*, since the whole input no longer has to be squeezed into one h_T.)

D. Attention equations

- With no oracle, the machine must **learn** the weights from data:

$$e_{jt} = f_{ATT}(s_{t-1}, c_j) \quad \text{(importance of the } j\text{-th input word for decoding the } t\text{-th output word)}$$

$$\alpha_{jt} = \frac{\exp(e_{jt})}{\sum_{j=1}^{M} \exp(e_{jt})} \quad \text{(softmax over input positions } j\text{; } M = \#\text{ input words)}$$

- α_{jt} is the **probability of focusing on the j-th input word to produce the t-th output word**. So $\alpha_{jt} \geq 0$ and $\sum_j \alpha_{jt} = 1$ for every t.
- One (among many) parametric choices, with decoder state s_t and encoder state c_j (slide 38):

$$e_{jt} = V_{att}^T \tanh(U_{att} s_{t-1} + W_{att} c_j), \qquad V_{att} \in \mathbb{R}^d,\ U_{att} \in \mathbb{R}^{d \times d},\ W_{att} \in \mathbb{R}^{d \times d}$$

These **additional parameters** are learned **along with** the encoder and decoder parameters.

- **Context vector** fed to the decoder: $c_t = \sum_{j=1}^{T} \alpha_{jt} h_j$, a weighted average of encoder states.

Full MT model with attention (slide 43)

$$\begin{aligned}
&\textbf{Encoder: } h_t = \text{RNN}(h_{t-1}, x_t), \quad s_0 = h_T \\
&\textbf{Decoder: } e_{jt} = V_{attn}^T \tanh(U_{attn} h_j + W_{attn} s_t) \\
&\alpha_{jt} = \text{softmax}(e_{jt}), \qquad c_t = \sum_{j=1}^{T} \alpha_{jt} h_j \\
&s_t = \text{RNN}(s_{t-1}, [e(\hat{y}_{t-1}), c_t]), \qquad \ell_t = \text{softmax}(V s_t + b)
\end{aligned}$$

- **Parameters:** $U_{dec}, V, W_{dec}, U_{enc}, W_{enc}, b, U_{attn}, V_{attn}$. **Loss and algorithm remain the same** (CE summed over time, GD with backprop).
- Slide 44: an attention component can be added in the same way to **all** the other encoder–decoder models above (captioning, entailment, summarization, video, dialog, ...), answering the same questions (data, encoder, decoder, loss, algorithm).
- **How c_t enters the decoder:** it is **concatenated with the previous output's embedding** as the decoder RNN input at step t. Unlike vanilla Option 2, which feeds the same h_T every step, c_t is **different at every step**.
- ⚠ Slide inconsistencies to be aware of:
  1. Slide 38 uses s_{t-1} and calls encoder states c_j. Slide 43 uses h_j for encoder states, c_t **for the context vector**, and writes $W_{attn} s_t$. Using s_t is circular, since s_t needs c_t, which needs e_{jt}. The consistent form uses s_{t-1}.
  2. The U/W roles swap between slides 38 and 43 (slide 38: U on the decoder state, W on the encoder state; slide 43: U on h_j, W on s_t).
  3. The parameter list on slide 43 omits W_{attn}.
  4. The softmax sum runs to M on slide 36 but the context sum runs to T. Both mean the number of input positions.
  5. ℓ_t on slide 43 denotes the output distribution, while earlier ℓ_t was the true word.
  6. Heat-map slides write α_{tj} while the equations use α_{jt}. Same quantity: input j, output t.

C. Training without true alignments

- If the true α's were given (e.g. $\alpha^{true} = [0, 0, 0.5, 0.5, 0]$ vs $\alpha^{pred} = [0.1, 0.1, 0.35, 0.35, 0.1]$), we could minimise $\mathcal{L}(\alpha^{true}, \alpha^{pred})$ **in addition to** $\mathcal{L}(\theta)$.

- But α^{true} is very hard to get. Someone would have to manually annotate which source words contribute to every target word.
- **Why it still works:** it is a **better modelling choice**, a more informed model that approaches the problem in a more natural way. Given enough data it should learn the attention weights, and **in practice attention models work better than vanilla encoder–decoder models**. The attention weights are learned **without direct supervision**, only through the end-task loss.

E. Numerical: computing α and c_t (python-verified)

Given scores for 4 input positions, $e_{\cdot t} = [2, 1, 0, 1]$, and encoder states $h_1 = [1, 0], h_2 = [0, 1], h_3 = [1, 1], h_4 = [2, -1]$:

- $\exp(e) = [7.389, 2.718, 1, 2.718]$, sum $= 13.825$, so $\alpha_{\cdot t} = [0.5344, 0.1966, 0.0723, 0.1966]$ (sums to 1).
- $c_t = 0.5344[1, 0] + 0.1966[0, 1] + 0.0723[1, 1] + 0.1966[2, -1] = [1.000,\ 0.0723]$.

Shortcuts and traps:

- **Equal scores** give a **uniform** $\alpha = 1/M$, so c_t is the plain mean of the h_j. Adding the same constant to all e_{jt} does not change α.
- If $e = [\ln 1, \ln 2, \ln 1]$, then $\alpha \propto [1, 2, 1]$, i.e. $[0.25, 0.5, 0.25]$.
- The softmax is over **input positions** j (for a fixed output step t), not over output steps and not over vocabulary words.
- Shapes: $e_{\cdot t}, \alpha_{\cdot t} \in \mathbb{R}^M$; $c_t \in \mathbb{R}^d$ (same dimension as h_j); e_{jt} is a **scalar** because $V_{att}^T(\cdot)$ is a dot product.
- **Attention parameter count** (Additional clarification, derived from the slide-38 dims, which assume encoder and decoder states are both d-dimensional): $|V_{att}| + |U_{att}| + |W_{att}| = d + 2d^2$. For $d = 256$ that is $131{,}328$; for $d = 100$, $20{,}100$; for $d = 4$, 36. It does **not** depend on the input length M.

Interpreting attention (heat maps)

- To check whether the model learns something meaningful (does it focus on the most relevant input words at step t?), **plot the attention weights as a heat map**.
- The heat map shows a **soft alignment** between the input and the generated output. **Each cell is** α_{tj}: the importance of the j-th input word for predicting the t-th output word, as determined by the model.
- Examples on the slides: attention-based **summarization** (Rush et al. 2015; input "russian defense minister ivanov called sunday for the creation of a joint front for combating global terrorism" → output "russia calls for joint front against terrorism", each output word attends to its source counterpart(s)), **neural MT** English→French (Cho et al. 2015; the alignment is near-diagonal with local reorderings, e.g. "European Economic Area" ↔ "zone économique européenne", where the order is reversed), and **video captioning** (Yao et al. 2015; bars under each frame show the attention weight per frame, and the captions with attention (+Local+Global) are more specific than the Basic model's).
- (Additional clarification) Reading a heat map: rows = output words t, columns = input words j (or vice versa); a bright/dark cell = large α_{tj}; each output row sums to 1. Off-diagonal bright cells = reordering.

15.4 Attention over Images

- For text there is a representation for **every location (time step)** of the input. For images we typically use a **fully connected (fc7) representation**, which has **no location information**. So what should attention attend over?

- **Answer:** use the output of a **convolutional layer**, which keeps spatial information. For example, the output of the **5th convolutional layer of VGGNet** is a $14 \times 14 \times 512$ feature map, treated as **196 locations, each a 512-dimensional vector**.
- The model learns attention $\alpha_{t1}, \ldots, \alpha_{t196}$ over these locations, which correspond to actual image regions. The context is $c_t = \sum_{j=1}^{196} \alpha_{tj} a_j \in \mathbb{R}^{512}$. (Additional clarification: the c_t formula follows the text case and a_j is just a label for the location feature.)
- The VGG figure on the slide: 224×224 input → … → 28×28×512 → maxpool → **14×14×512** (the conv5 block) → maxpool → 7×7×512 → fc 4096 → fc 4096 → softmax 1000. If a question uses the **7×7×512** map instead, there are **49 locations**.
- Examples (Kyunghyun Cho et al. 2015 on the slide, image captioning): the model attends to the correct object (white = attended region) for the underlined caption word, e.g. "A woman is throwing a **frisbee** in a park", "A **dog** is standing on a hardwood floor", "A **stop** sign is on a road…", "A little **girl** sitting on a bed with a teddy bear", "A group of **people** sitting on a boat…", "A giraffe standing in a forest with **trees** in the background".
- Numerics: number of locations $= H \times W$ (not $H \times W \times C$). Each location's feature dimension $= C$. The total $14 \cdot 14 \cdot 512 = 100{,}352$ numbers.

15.5 Hierarchical Attention

A. Sequence of sequences

- **Dialog:** the context is a sequence of **utterances** (U: "Can you suggest a good movie?" / B: "Yes, sure. How about Logan?" / U: "Okay, who is the lead actor?"), and each utterance is a sequence of **words**. The response is "Hugh Jackman, of course".
- **Two-level hierarchical RNN encoder:** a level-1 RNN runs over the words of each utterance to give one representation per utterance. A level-2 RNN runs over this sequence of utterance representations to give a single representation. The decoder generates the response conditioned on it.
- **Document classification/summarization:** a document is a sequence of sentences, and each sentence is a sequence of words.

D. Hierarchical encoder (no attention), document classification

$$\textbf{Word level (1):}\ h^1_{ij} = \mathrm{RNN}(h^1_{ij-1}, w_{ij}), \quad s_i = h^1_{iT_i}\ \ (T_i = \text{length of sentence } i)$$
$$\textbf{Sentence level (2):}\ h^2_i = \mathrm{RNN}(h^2_{i-1}, s_i), \quad s = h^2_K\ \ (K = \#\text{sentences})$$
$$\textbf{Decoder:}\ P(y \mid \text{document}) = \mathrm{softmax}(Vs + b)$$

Params: $W^1_{enc}, U^1_{enc}, W^2_{enc}, U^2_{enc}, V, b$. Loss: cross-entropy. Algorithm: GD with backprop.

D. Hierarchical Attention Network (Yang et al.)

Attention is needed at **two levels**: first attend to the important (most informative) **words in a sentence**, then to the important **sentences in a document**.

$$\textbf{Word level:}\ h_{ij} = \mathrm{RNN}(h_{ij-1}, w_{ij}), \quad u_{ij} = \tanh(W_w h_{ij} + b_w), \quad \alpha_{ij} = \frac{\exp(u_{ij}^T u_w)}{\sum_t \exp(u_{it}^T u_w)}, \quad s_i = \sum_j \alpha_{ij} h_{ij}$$

$$\textbf{Sentence level:}\ h_i = \mathrm{RNN}(h_{i-1}, s_i), \quad u_i = \tanh(W_s h_i + b_s), \quad \alpha_i = \frac{\exp(u_i^T u_s)}{\sum_i \exp(u_i^T u_s)}, \quad s = \sum_i \alpha_i h_i$$

$$\textbf{Decoder:}\ P(y \mid \text{document}) = \mathrm{softmax}(Vs + b)$$

- **Parameters (slide):** $W_w, W_s, V, b_w, b_s, b, u_w, u_s$. Loss: cross-entropy. Algorithm: GD with backprop.
- ⚠ Slide note: this list omits the word-level and sentence-level RNN encoder weights, which are also learned.
- With attention, the sentence vector s_i is an **attention-weighted sum of word states**, not the last state h_{iT_i}. The document vector s is an attention-weighted sum of sentence states, not h_K^2.
- In the α_{ij} denominator, t is just a dummy index over the words of sentence i (not a time step of a decoder); in α_i the sum runs over all sentences.
- The HAN figure (Yang et al.) stacks: word encoder → word attention (with u_w) → sentence vector s_i → sentence encoder → sentence attention (with u_s) → document vector s → softmax. (Additional clarification: the figure's encoders are bidirectional, i.e. forward and backward states; the slide equations simply write RNN.)
- (Additional clarification) u_w and u_s are **learned word-level and sentence-level context ("query") vectors**. Unlike MT attention, the query is **not a decoder state**, because for classification there is no decoder RNN.

F. MCQ Traps & Conceptual Distinctions (Lecture 15)

Confusable	Correct understanding
Encoder vs decoder in image captioning	Encoder = **CNN** (fc7), decoder = **RNN**
Option 1 vs Option 2 conditioning	Opt 1: encoding only as s_0. Opt 2: encoding concatenated to the input at **every** step
Vanilla Option 2 vs attention	Both feed something every step, but Opt 2 feeds the **same** h_T while attention feeds a **step-specific** c_t
s_0 in a language model	randomly initialised **and learned**, not computed
Image QA / video classification "decoder"	a single softmax, not an RNN. The loss has one term
Softmax for α_{jt}	normalises over **input positions** j, for each output step t
α supervision	**not** required. It is learned end-to-end (true alignments are hard to get)
Image attention input	conv feature map (spatial), **not** fc7 (no location information)
VGG conv5 map	$14 \times 14 \times 512$ means **196** locations of 512-d
Hierarchical attention levels	word-level (words in a sentence) then sentence-level (sentences in a document)
Attention parameters	$V_{att} \in \mathbb{R}^d, U_{att}, W_{att} \in \mathbb{R}^{d \times d}$, independent of sequence length

- "Attention requires annotated alignments to train" is **False**.
- "Attention weights for a given output step sum to 1" is **True**.
- "The context vector has dimension equal to the number of input words" is **False**. It has the dimension of an encoder state. The **α vector** has length M.
- "Every encoder–decoder model in the lecture is trained with gradient descent with backprop and a cross-entropy-type loss" is **True**.
- "In the language model the input x_t at test time is the word predicted at $t-1$" is **True** (slide 6).

G. Quick Revision Summary (Lecture 15)

- Language model: $P(y_t = j \mid y_1^{t-1}) = \text{softmax}(V s_t + c)_j$; $\mathcal{L} = -\sum_t \log P(y_t = \ell_t \mid y_1^{t-1})$.
- Encoder–decoder: encode the input(s) (CNN/RNN/hierarchical), decode with an RNN (sequence) or a softmax (label). Same loss family and same algorithm.

- Attention: $e_{jt} = V_{att}^T \tanh(U_{att} s_{t-1} + W_{att} h_j)$, $\alpha_{jt} = \text{softmax}_j(e_{jt})$, $c_t = \sum_j \alpha_{jt} h_j$, $s_t = \text{RNN}(s_{t-1}, [e(\hat{y}_{t-1}), c_t])$.
- Image attention: conv5 of VGG gives $14 \times 14 \times 512$, i.e. 196 regions. Hierarchical attention (HAN): word attention with u_w, then sentence attention with u_s.
- Likely MCQ takeaways: (1) compute α / c_t from given scores; (2) identify the encoder/decoder/loss for an application; (3) why attention helps (no overloading with irrelevant information, step-specific context); (4) attention is learned without alignment labels; (5) number of image locations = spatial size of the conv map.

Part II · High-Yield Revision

Use this for last-day revision. It brings together the most testable facts and the cross-lecture comparisons. Each point is explained in full, with its derivation, in Part I (Lecture Notes). Formulas are in Part III (Formula Sheet).

---

1. Week 1–2: MP neuron → Perceptron → Sigmoid neuron

	MP neuron	Perceptron	Sigmoid neuron
Inputs	Boolean	Real	Real
Weights	**None** (all equal); inhibitory input = absolute veto	Real, **learned** (PLA)	Real, learned (GD)
Output	0/1 (threshold θ)	0/1 ($\mathbf{w}^T\mathbf{x} \geq 0$)	Smooth, in (0, 1)
Learning	None (θ set by hand)	PLA: converges **iff linearly separable**	Gradient descent
Decision boundary	Linear	Linear (hyperplane, $\mathbf{w}$ is normal to it)	Linear (at 0.5), soft

- **Boolean functions of n inputs:** 2^{2^n}. For n = 2, 14 of 16 are linearly separable; **XOR and XNOR are not**.
- **Any** boolean function of n inputs can be built with **one hidden layer of** 2^n **perceptrons** plus one output perceptron. This size is **sufficient, not necessary**, and it grows exponentially.
- **PLA convergence theorem:** if the data are linearly separable, PLA makes a **finite** number of corrections. The proof shows that cos β grows like $\sqrt{k}$, and since cos β ≤ 1, k is bounded. On non-separable data (XOR), PLA never converges.
- **Sigmoid:** $\sigma(0) = 0.5$ and $\sigma' = \sigma(1-\sigma) \leq 0.25$. A larger w makes the transition steeper (closer to a step). The bias b moves the transition point to $x = -b/w$.
- **GD:** $w \leftarrow w - \eta \nabla w$. Moving **opposite** to the gradient gives the largest first-order decrease (β = 180°). The first-order Taylor argument needs η to be small.
- **Universal Approximation Theorem:** an FFN with **one hidden layer** of enough sigmoid neurons can approximate **any continuous** function to **any desired precision** ε. It is an **existence** result: it says nothing about how to learn the weights or how many neurons are needed. In 1-D a "tower" needs 2 neurons; in 2-D it needs 4 neurons plus a sigmoid.

2. Week 3: FFN and Backpropagation

- **Parameters per layer:** $d_{i-1}d_i + d_i$. Remember to add the biases of every non-input layer.
- **Output and loss pairing:** real-valued outputs use a **linear output with squared error**. Classification uses **softmax with cross-entropy**, and the loss is $-\log \hat{y}_\ell$.
- **Key gradients** (each gradient has the **same shape** as the quantity it is taken with respect to):
  - $\nabla_{\mathbf{a}_L} \mathscr{L} = \hat{\mathbf{y}} - \mathbf{e}(\ell)$
  - $\nabla_{\mathbf{h}_i} = W_{i+1}^T \nabla_{\mathbf{a}_{i+1}}$
  - $\nabla_{\mathbf{a}_i} = \nabla_{\mathbf{h}_i} \odot g'(\mathbf{a}_i)$
  - $\nabla_{W_i} = \nabla_{\mathbf{a}_i} \mathbf{h}_{i-1}^T$
  - $\nabla_{\mathbf{b}_i} = \nabla_{\mathbf{a}_i}$
- **Activation derivatives:** $\sigma' = \sigma(1-\sigma)$, maximum 0.25. $\tanh' = 1 - \tanh^2$, maximum 1.

3. Week 4: Optimizers (comparison)

Method	Update idea	Key property / weakness
GD (batch)	$w \leftarrow w - \eta \nabla w$ over all N points	1 update per epoch; slow on gentle slopes
Momentum	$u_t = \gamma u_{t-1} + \eta \nabla w_t$, $w \leftarrow w - u_t$	Fast on gentle slopes; **overshoots and oscillates** near the minimum, but still converges faster than GD
NAG	gradient at the look-ahead $w_t - \gamma u_{t-1}$	"Look before you leap"; **smaller oscillations** than momentum
SGD	update per data point	N updates per epoch; noisy (greedy) steps
Mini-batch	update per batch of B	N/B updates per epoch; less noisy than SGD
AdaGrad	$v_t = v_{t-1} + g^2$; step $\eta / \sqrt{v_t + \epsilon}$	Larger effective learning rate for **sparse features**; v_t only grows, so the **lr decays too aggressively**
RMSProp	$v_t = \beta v_{t-1} + (1-\beta) g^2$	EMA instead of a sum, so the decay is not so aggressive
Adam	momentum (m_t, β_1=0.9) + RMSProp (v_t, β_2=0.999), with **bias correction** $\hat{m} = m / (1 - \beta_1^t)$	Bias correction fixes the zero-initialization bias in early steps

- **Epoch vs step:** one epoch is one pass over the data; one step is one parameter update.
- **Learning-rate schedules:**
  - Step decay: halve η every 5 epochs, or halve it when the validation error is higher than at the end of the previous epoch.
  - Exponential decay: $\eta_0 e^{-kt}$ (⚠ the slide prints it as η_0^{-kt}).
  - 1/t decay: $\eta_0 / (1 + kt)$.
- **Line search:** try several values of η at each step and keep the best. It converges in fewer steps than vanilla GD, but each step costs more computation.

4. Week 5: Linear algebra, PCA, SVD

- **Eigenvectors:** $A\mathbf{x} = \lambda \mathbf{x}$, defined only for **square** matrices. Distinct eigenvalues give linearly independent eigenvectors. A **symmetric** matrix has **orthogonal** eigenvectors (and real eigenvalues).
- **Repeated multiplication:** applying A again and again aligns any vector with the **dominant eigenvector**. If $|\lambda_d| > 1$ it explodes, if < 1 it vanishes, and if $= 1$ it reaches a steady state, as with stochastic matrices. This is the same idea as vanishing and exploding gradients in RNNs.
- **EVD:** $A = U \Lambda U^{-1}$ needs n linearly independent eigenvectors. For a symmetric matrix this becomes $A = U \Lambda U^T$.
- **PCA:** project onto the eigenvectors of the covariance matrix $\frac{1}{m} X^T X$ (with X centred). PCA:
  - **maximises the variance** along the retained directions;
  - **minimises the reconstruction error**;
  - makes the new dimensions **uncorrelated**. The eigenvalue of a direction is the variance along it. The fraction of variance kept is $\sum_{i \le k} \lambda_i / \sum \lambda_i$.
- **SVD:** $A = U \Sigma V^T$, which exists for **any** matrix, rectangular included.
  - V = eigenvectors of $A^T A$; U = eigenvectors of $A A^T$.
  - $\sigma_i = \sqrt{\lambda_i (A^T A)} \ge 0$.
  - Truncating gives the **best rank-k approximation** in the Frobenius norm.

5. Week 6: Autoencoders

Variant	Mechanism	Prevents / gives
Undercomplete ($\dim h < \dim x$)	Bottleneck	Learns compressed, important characteristics
Overcomplete ($\dim h \geq \dim x$)	–	**Can learn the trivial identity** (copy x to h). Needs regularization
Regularized (L2 / tied weights $W^* = W^T$)	Penalty, or fewer free parameters	Reduces overfitting
Denoising AE	Corrupt the input ($\tilde{x}$: zero with probability q, or add Gaussian noise); reconstruct the **clean** x	Cannot copy, so it learns robust features
Sparse AE	KL penalty pushes the average activation $\hat{\rho}_l$ toward ρ (small, e.g. 0.005)	Most neurons are inactive (sigmoid output ≈ 0)
Contractive AE	Penalty $\|J_x(h)\|_F^2$	Insensitive to small input variations; the reconstruction loss pulls the other way, so only important variations are captured

- **Loss pairing:** binary inputs use a **sigmoid decoder with cross-entropy**. Real inputs use a **linear decoder with squared error**.
- **AE ≡ PCA** when all of these hold: a **linear encoder**, a **linear decoder**, **squared-error loss**, and **normalised inputs** $\frac{1}{\sqrt{m}}(x - \bar{x})$.

6. Week 7: Regularization (comparison)

Technique	Key idea / result
Bias–variance	Simple models have high bias and low variance; complex models have low bias and high variance. Expected error = Bias² + Variance + σ² (irreducible)
Train vs test error	Training error **underestimates** the true error, because the covariance term (Stein's lemma) grows with model complexity. Test error ≈ true error up to a constant
L2	Shrinks weights along each Hessian eigenvector by $\frac{\lambda_i}{\lambda_i+\alpha}$. Large-λ (important) directions are kept; small-λ directions shrink toward 0. The effective number of parameters is less than n
Dataset augmentation	Label-preserving transforms (rotation, shift, noise) to get more data
Parameter sharing / tying	CNN filters share weights; AE tied weights $W^* = W^T$
Input noise	For a linear model, Gaussian input noise ≡ L2 with strength σ^2
Output noise (label smoothing)	Soft targets: $1 - \varepsilon$ for the true class and $\varepsilon/(K-1)$ for each other class
Early stopping	Patience p: stop when validation error hasn't improved for p steps and return the model from step $k - p$. Under a quadratic approximation it is ≈ L2, with more steps ≈ smaller α
Ensembles (bagging)	Error = $V/k + \frac{k-1}{k}C$. Uncorrelated errors give V/k; perfectly correlated errors (C = V) give no gain
Dropout	Drop each node with probability 1 − p. Gives 2^n thinned networks that share weights. At test time, scale weights by p (the retain probability). It acts like a cheap ensemble and stops hidden units co-adapting

7. Week 8: Training deep nets better

- **Greedy layer-wise unsupervised pre-training:** train one AE layer at a time, then fine-tune the whole network with supervision. It helps through better optimization (mainly when the last layer has low capacity), regularization, and better initialization.
- **Activations:**
  - **Sigmoid** saturates (gradient ≈ 0), is not zero-centred, and needs exp.
  - **tanh** is zero-centred but still saturates.
  - **ReLU** does not saturate for x > 0 and is cheap, but it can **die**: if a large negative bias makes the input always negative, the unit never recovers.
  - **Leaky ReLU** ($0.01x$) and **PReLU** (with a learned α) fix dying.
  - **ELU** is ≈ zero-centred but needs exp.
  - **Maxout** generalizes ReLU and Leaky ReLU and needs 2× the parameters.
- **Initialization:**
  - All-zero (or all-equal) weights cause **symmetry**: every unit gets the same gradient and stays the same forever.
  - Small random weights make activations vanish in deep nets.
  - Large random weights saturate the units (sigmoid/tanh).
  - The output variance after k layers is $[n\mathrm{Var}(w)]^k\mathrm{Var}(x)$, so choose $\mathrm{Var}(w) = 1/n$ (tanh) or $2/n$ (He, ReLU).
- **Batch normalization:** standardize each unit's pre-activation using the **mini-batch** mean and variance, then apply a learnable scale γ and shift β. The network can learn to undo it ($\gamma = \sqrt{var}$, $\beta = \mathrm{mean}$). It makes training less sensitive to initialization.

8. Week 9: Word representations

Method	Type	Notes
One-hot	–	Dimension |V|, sparse. Every pair is at distance $\sqrt{2}$ with cosine 0, so it carries **no similarity**
Co-occurrence → PMI / PPMI → SVD	**Count-based**	$W_{word} = U\Sigma$; truncated SVD gives dense vectors
CBOW	Prediction	Predicts the centre word from the context; h = sum of the context columns u_c
Skip-gram	Prediction	Predicts the context words from the centre word
Negative sampling	Softmax alternative	Binary task: real pair → $\sigma(u_c^T v_w)$, random pair → $\sigma(-u_r^T v_w)$. Negatives are drawn from $\mathrm{count}^{3/4}$
Hierarchical softmax	Softmax alternative	Binary tree, cost ≈ $\log_2\|V\|$ instead of $\|V\|$
GloVe	Hybrid	Weighted least squares to $\log X_{ij}$, combining **global** counts with a prediction-style model

- word2vec with negative sampling implicitly factorises a **shifted PMI** matrix, $\mathrm{PMI} - \log k$. This links it to SVD.
- **Evaluation:** semantic relatedness (correlation with human similarity scores), synonym detection, and analogies ($v_b - v_a + v_c$).

9. Week 10: CNNs and visualization

- **Output size:** $\frac{W-F+2P}{S}+1$, and the **depth equals K** (the number of filters). Each filter is $F \times F \times D_{in}$.
- **Parameters:** $(F^2 D_{in}+1)K$ including biases. ⚠ The slides' worked numbers (150, 34K, 0.6M) are $F^2 D_{in} K$ **without** biases, so check which one the options match. Parameters do not depend on the input size, stride or padding. **Pooling has 0 parameters.**
- **CNN vs FFN:** sparse connectivity and weight sharing, plus pooling. Training is still ordinary backprop, with shared weights. (Additional clarification: pooling gives some tolerance to small translations.)
- **Architectures:**
  - **LeNet-5**
  - **AlexNet** (the FC layers dominate the parameters; additional clarification: it uses ReLU and dropout, which the L11 slides do not list)
  - **ZF-Net** (first-layer filter 7×7 instead of 11×11, more filters)
  - **VGG** (only 3×3 filters; the first FC layer has ≈102M parameters)
  - **GoogLeNet** (inception modules; 1×1 convs for depth reduction; global average pooling instead of FC; 12× fewer parameters than AlexNet but 2× the compute)
  - **ResNet** ($H(x)=F(x)+x$; skip connections fix degradation; 152 layers)
- **Stacking 3×3 filters** (additional clarification): two give the receptive field of one 5×5, and three give 7×7, with fewer parameters and more non-linearity.
- **Visualization:**
  - **Filter visualization:** the input that maximises a neuron's response is $W/\|W\|$.
  - **Occlusion maps:** slide a grey patch over the image and watch the probability drop.
  - **Gradient of a neuron w.r.t. the input pixels** shows each pixel's influence.
  - **Guided backprop:** at each ReLU, zero both the **negative gradients** and the positions where the **forward input was negative**.
  - **Class-model visualization:** maximise the **pre-softmax** class score with respect to the image, plus a regularizer.
  - **Deep Dream:** start from a real image and amplify a layer's activations.
  - **Deep Art:** loss = $\alpha\,\mathscr{L}_{content}$ (activations) + $\beta\,\mathscr{L}_{style}$ (Gram matrices).
  - **Fooling:** a tiny, imperceptible perturbation can cause a confident misclassification, and even random-noise images can be classified with very high confidence (99.6% on the slides).
  - Filter plots are interpretable only for the **first** layer, because only those filters act directly on the image.

10. Week 11: RNN, BPTT, LSTM, GRU

- **RNN:** $s_t=\sigma(Ux_t+Ws_{t-1}+b)$, $y_t=\mathcal{O}(Vs_t+c)$. The **same parameters are used at every time step**, so the number of parameters does not depend on T.
- **BPTT:** $\frac{\partial \mathscr{L}_t}{\partial W}=\frac{\partial \mathscr{L}_t}{\partial s_t}\sum_{k=1}^{t}\frac{\partial s_t}{\partial s_k}\frac{\partial^+ s_k}{\partial W}$. The term $\partial s_t/\partial s_k$ is a product of $t-k$ Jacobians, bounded by $(\gamma\lambda)^{t-k}$:
  - if $\gamma\lambda<1$ the gradient **vanishes**;
  - if $\gamma\lambda>1$ it **may explode**;
  - γ = 1/4 for the logistic sigmoid and 1 for tanh.
- **Fixes:**
  - Truncated BPTT: backpropagate through only τ steps.
  - Gradient clipping for exploding gradients: rescale the norm and keep the direction.
  - LSTM/GRU for vanishing gradients.
- **LSTM:**

    - Three gates: output, input, forget. Each gate is σ(·) of (h_{t-1}, x_t).
    - Cell update $s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$ is **additive**, so along the cell path the gradient is a product of forget gates (no repeated W). Vanishing happens there only when the forget gates are ≈ 0, which is "fair".
    - **LSTMs can still explode** along the output-gate path, through repeated W_o.
- **GRU:**
    - Two gates. In the slides' notation o_t acts as the reset gate and i_t as the update gate.
    - **No separate forget gate**: the forget fraction is $1 - i_t$. That is the L14 form, $s_t = (1 - i_t) \odot s_{t-1} + i_t \odot \tilde{s}_t$. ⚠ L15 slide 8 swaps the two, so check which form the question uses.
    - **No separate** h_t.
- **Parameter ratio:** LSTM : GRU : RNN = 4 : 3 : 1, each unit being $d(d + n + 1)$.
- ⚠ **Slide notation:** the slides use σ (not tanh) for $\tilde{s}_t$ and inside h_t. Follow whatever form the question uses.

11. Week 12: Encoder–decoder and attention

- **Encoder–decoder:** the encoder (an RNN or CNN) compresses the input into a representation. The decoder (usually an RNN) generates the output. The loss is the **sum of cross-entropies over time**.

Task	Encoder	Decoder
Image captioning	CNN (fc7)	RNN
MT / transliteration / summarization	RNN	RNN
Image QA	CNN + RNN (question)	softmax over answers
Video captioning	CNN per frame → RNN	RNN
Video classification	CNN per frame → RNN	softmax (single label)
Document classification (hierarchical)	word-level RNN → sentence-level RNN	softmax

- **Why attention:** a single fixed-length encoding is a **bottleneck** for long inputs. Attention computes a **different context vector at each decoder step**:
    - $e_{jt} = V_{att}^T \tanh(U_{att} s_{t-1} + W_{att} h_j)$
    - $\alpha_{jt} = \mathrm{softmax}_j(e_{jt})$, so $\sum_j \alpha_{jt} = 1$
    - $c_t = \sum_j \alpha_{jt} h_j$
- **Learning attention:** the weights are learned **without** true alignment labels, trained end-to-end with the rest of the model. The attention maps show which input each output step focused on.
- **Image attention:** a conv feature map such as VGG's **14×14×512** gives **196 location vectors** of dimension 512.
- **Hierarchical attention:** attention at the **word level** produces a sentence vector, and attention at the **sentence level** produces a document vector.

Top cross-course MCQ traps

1. "The perceptron can learn XOR" → false. A single perceptron cannot; a network of perceptrons can.
2. "UAT says one hidden layer is enough to *learn* any function efficiently" → false. It only guarantees that an approximation exists.
3. "Momentum never overshoots" → false. It does overshoot; NAG reduces the oscillations.

4. "AdaGrad works well because its effective learning rate never decays" → false. It decays too fast; RMSProp fixes that.
5. "Eigen-decomposition exists for every square matrix" → false. It needs n linearly independent eigenvectors. **SVD** exists for every matrix.
6. "An overcomplete AE always learns useful features" → false. It can learn the identity unless it is regularized.
7. "Dropout at test time drops nodes randomly" → false. At test time all nodes are used and the weights are scaled by p.
8. "Zero initialization is fine with ReLU" → false. The symmetry problem occurs regardless of the activation.
9. "Pooling layers have learnable parameters" → false.
10. "The number of RNN parameters grows with the sequence length" → false.
11. "LSTMs solve both vanishing and exploding gradients" → false. They address vanishing gradients only; exploding gradients need clipping.
12. "Attention weights are normalised over the output time steps" → false. The softmax is over the **input positions j**.
13. "Guided backprop zeroes only the negative gradients" → false. It also zeroes positions where the forward ReLU input was negative.
14. "Class visualization maximises the softmax probability" → false. It maximises the **unnormalised score** S_c.

Part III · Formula Sheet

Every important formula from the 15 lectures, grouped by week and topic. Derivations are in Part I (Lecture Notes). ⚠ marks the most likely mistake or a known slide typo.

Notation (Khapra's slides):

- $W_i, \mathbf{b}_i$: weights and bias of layer i.
- $\mathbf{a}_i$: pre-activation; $\mathbf{h}_i$: activation, with $\mathbf{h}_0 = \mathbf{x}$ and $\mathbf{h}_L = \hat{\mathbf{y}}$.
- L: number of layers, counting the output layer.
- η: learning rate; σ: logistic sigmoid; $\odot$: element-wise product.
- RNN/LSTM: s_t is the state (the cell state in an LSTM) and h_t is the LSTM output.

Quick formula finder: question cue → formula

If the question asks...	Use	Week
how many boolean functions of n inputs	2^{2^n}	1
how many hidden perceptrons are enough for any boolean function of n inputs	2^n (one hidden layer, plus 1 output)	1
for the perceptron weight update	$\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}$ (positive misclassified), $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{x}$ (negative misclassified)	1
for the GD step, or σ'(·)	$w \leftarrow w - \eta \nabla w$; $\sigma' = \sigma(1-\sigma) \le 0.25$	2
for the number of parameters of an FFN	$\sum_i (d_{i-1} d_i + d_i)$	3
for the gradient at a softmax+CE output	$\nabla_{\mathbf{a}_L} \mathscr{L} = \hat{\mathbf{y}} - \mathbf{e}(\ell)$	3
for a weight gradient or its shape	$\nabla_{W_k} \mathscr{L} = \nabla_{\mathbf{a}_k} \mathscr{L}\, \mathbf{h}_{k-1}^T$ $(d_k \times d_{k-1})$	3
for the number of updates per epoch	batch 1 · SGD N · mini-batch N/B	4
for one step of momentum, NAG, AdaGrad, RMSProp or Adam	the L5 update-rule tables (note bias correction $1-\beta^t$)	4
for eigenvalues, % variance explained, or PCA directions	$\det(A - \lambda I) = 0$; $\lambda_i / \sum \lambda$; eigenvectors of $X^T X$	5
for the shapes in an SVD, or singular values	$A_{m\times n} = U_{m\times m} \Sigma_{m\times n} V^T_{n\times n}$; $\sigma_i = \sqrt{\lambda_i (A^T A)}$	5
for the sparse-AE penalty	$\sum_l \rho \log \frac{\rho}{\hat{\rho}_l} + (1-\rho) \log \frac{1-\rho}{1-\hat{\rho}_l}$	6
for the L2-shrink factor along an eigen-direction	$\lambda_i / (\lambda_i + \alpha)$	7
for the ensemble error with correlation	$V/k + \frac{k-1}{k} C$	7
for the number of dropout sub-networks, or test-time scaling	2^n; $w \to pw$	7
for the variance after k layers, or the init scale	$[n \operatorname{Var}(w)]^k \operatorname{Var}(x)$; $1/\sqrt{n}$ (tanh), $\sqrt{2/n}$ (ReLU)	8
for PMI, PPMI, or # word2vec params	$\log \frac{\operatorname{count}(w,c) N}{\operatorname{count}(w) \operatorname{count}(c)}$; \$2k	V
for conv output size, or # conv params	$\frac{W-F+2P}{S} + 1$; $(F^2 D_1 + 1) K$ (slides often omit the bias)	10
for the cost saved by a 1×1 reduction, or # FC params after flattening	multiply out the MACs; $W \cdot H \cdot D \cdot n_{out}$	10
for a receptive field	stride-1 stack: $1 + \sum(k-1)$; general: $r_\ell = r_{\ell-1} + (k_\ell - 1) j_{\ell-1}$	10
for # RNN params	$nd + d^2 + dk + d + k$ (independent of T)	11

If the question asks...	Use	Week
for the vanishing/exploding bound	$\|\partial s_t/\partial s_k\| \le (\gamma\lambda)^{t-k}$	11
for # LSTM or GRU params	$4\,d(d+n+1)$ / $3\,d(d+n+1)$	11
for gradient clipping	$g \leftarrow \tau g/\|g\|$ if $\|g\| > \tau$	11
for attention weights or the context vector	$\alpha_{jt} = \mathrm{softmax}_j(e_{jt})$; $c_t = \sum_j \alpha_{jt} h_j$	12

Weeks 1–2 · Lectures 1–3

MP neuron and perceptron (L2) · Week 1

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
MP aggregation	$g(\mathbf{x}) = \sum_{i=1}^{n} x_i$	$x_i \in \{0,1\}$	No weights	Counts the active excitatory inputs	Adding weights (MP has none)
MP decision (thresholding logic)	$y = 1$ if $g(\mathbf{x}) \ge \theta$, else 0; $y = 0$ if any inhibitory input is 1	θ = threshold (hand-set)	Boolean I/O	θ = 1 gives OR, θ = n gives AND, θ = 0 gives always ON	Treating an inhibitory input as weight −1 (it is an absolute veto)
Perceptron	$y = 1$ if $\sum_{i=0}^{n} w_i x_i \ge 0$, else 0	$x_0 = 1$, $w_0 = -\theta$ (bias)	Real inputs allowed	Hyperplane $\mathbf{w}^T\mathbf{x} = 0$ splits the space into two halves	Sign of θ ($w_0 = -\theta$). A point on the boundary is classified as **1**
Angle with **w**	$\cos\alpha = \dfrac{\mathbf{w}^T\mathbf{x}}{\|\mathbf{w}\|\|\mathbf{x}\|}$	α = angle between **w** and **x**	–	On the line α = 90°; positive side α < 90°; negative side α > 90°	Thinking **w** is parallel to the boundary (it is normal to it)
PLA update	$\mathbf{x} \in P,\ \mathbf{w}^T\mathbf{x} < 0:\ \mathbf{w} \leftarrow \mathbf{w} + \mathbf{x};\ \mathbf{x} \in N,\ \mathbf{w}^T\mathbf{x} \ge 0:\ \mathbf{w} \leftarrow \mathbf{w} - \mathbf{x}$	P = label 1, N = label 0	Only on mistakes	Adding **x** raises $\mathbf{w}^T\mathbf{x}$ by $\|\mathbf{x}\|^2$; subtracting lowers it	Updating on correct points; adding a learning rate (none in the slides' PLA)
Convergence: numerator	$\mathbf{w}^* \cdot \mathbf{w}_{t+1} \ge \mathbf{w}^* \cdot \mathbf{w}_0 + k\delta$	$\delta = \min_i \mathbf{w}^* \cdot \mathbf{p}_i > 0$; k = number of corrections	$\|\mathbf{p}\| = \|\mathbf{w}^*\| = 1$; N negated into P′	Grows linearly in k	k is the number of corrections, not the number of time steps t
Convergence: denominator	$\|\mathbf{w}_{t+1}\|^2 \le \|\mathbf{w}_0\|^2 + k$	–	Uses $\mathbf{w}_t \cdot \mathbf{p}_i \le 0$ at a correction	The norm grows like $\sqrt{k}$	–
Convergence: bound	$\cos\beta \ge \dfrac{\mathbf{w}^* \cdot \mathbf{w}_0 + k\delta}{\sqrt{\|\mathbf{w}_0\|^2 + k}} \propto \sqrt{k}$	β = angle between $\mathbf{w}^*$ and $\mathbf{w}_{t+1}$	Finite, linearly separable data	cos β ≤ 1, so k is bounded and PLA converges. (Additional clarification) With $\mathbf{w}_0 = 0$, $k \le 1/\delta^2$	Applying it to non-separable data such as XOR

Boolean-function counting (L2) · Week 1

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Number of boolean functions	2^{2^n}	n = number of inputs	–	2^n truth-table rows, each output 0 or 1. n = 2 gives 16; n = 3 gives 256	Writing 2^n (that is the number of rows)

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Linearly separable, n = 2	14 of 16	–	–	All except XOR and XNOR	–
Linearly separable, n = 3	104 of 256	–	(Additional clarification; not on the slides)	–	–
Hidden perceptrons for any boolean function	2^n hidden + 1 output	Inputs ±1; hidden weights ±1; hidden bias $-n$	One hidden layer	Each hidden unit fires for exactly one input; output weight w_i handles row i	"Necessary": it is only **sufficient**. The size grows exponentially
XOR via network	$w_1 < w_0,\ w_2 \geq w_0,\ w_3 \geq w_0,\ w_4 < w_0$	w_0 = output threshold (fire if $\sum w_i h_i \geq w_0$)	4 hidden units	No contradiction, so XOR is representable	Confusing this w_0 (a threshold) with the bias $-\theta$

Sigmoid neuron and ML set-up (L3) · Week 2

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Logistic sigmoid neuron	$y = \frac{1}{1+e^{-(w_0+\sum_{i=1}^{n} w_i x_i)}} = \frac{1}{1+e^{-(\mathbf{w}\cdot\mathbf{x}+b)}}$	$b = w_0 = -\theta$	–	Smooth output in (0,1), read as a probability; σ(0) = 0.5	Treating the output as binary
Transition point	$x = -b/w$	–	1-D σ(wx+b)	Large w gives a step; b shifts the transition	Thinking b controls steepness
Squared-error loss	$\mathscr{L}(w,b) = \frac{1}{2}\sum_{i=1}^{N}(y_i - f(x_i))^2$	$f(x) = \sigma(wx+b)$	½ is optional (it appears on some slides, not others)	Minimizing it fits the sigmoid through the data	Forgetting the ½ when the question includes it
Toy loss value	$\mathscr{L}(0.5, 0) = 0.073$	Data (0.5, 0.2) and (2.5, 0.9)	With ½	The optimum is near w ≈ 1.79, b ≈ −2.28	–

Gradient descent (L3) · Week 2

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Parameter step	$\boldsymbol{\theta}_{new} = \boldsymbol{\theta} + \eta\,\Delta\boldsymbol{\theta}$	$\boldsymbol{\theta} = [w, b]$	Small η	A conservative move along Δθ	–
First-order Taylor	$\mathscr{L}(\boldsymbol{\theta} + \eta\mathbf{u}) \approx \mathscr{L}(\boldsymbol{\theta}) + \eta\,\mathbf{u}^T\nabla_{\boldsymbol{\theta}}\mathscr{L}(\boldsymbol{\theta})$	$\mathbf{u} = \Delta\boldsymbol{\theta}$	η small, so the η^2, η^3 terms → 0	Loss decreases iff $\mathbf{u}^T\nabla\mathscr{L} < 0$	–
Best direction	$\mathbf{u}^T\nabla\mathscr{L} = k\cos\beta$ with $k = \|\mathbf{u}\|\|\nabla\mathscr{L}\|$, range $[-k, k]$	β = angle between **u** and ∇	–	Most negative at β = 180°: move opposite to the gradient	Choosing β = 90° (no change) or β = 0° (ascent)
GD update	$w_{t+1} = w_t - \eta\nabla w_t, \quad b_{t+1} = b_t - \eta\nabla b_t$	Gradients evaluated at (w_t, b_t)	–	Steepest descent	Writing + instead of −
Gradient wrt w (sigmoid + squared error)	$\nabla w = \sum_i (f(x_i) - y_i)\,f(x_i)(1-f(x_i))\,x_i$	–	Loss with ½	Error × sigmoid slope × input	Dropping x_i
Gradient wrt b	$\nabla b = \sum_i (f(x_i) - y_i)\,f(x_i)(1-f(x_i))$	–	–	Same as ∇w without x_i	Including x_i

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Sigmoid derivative	$\frac{\partial}{\partial w}\sigma(wx+b) = \sigma(1-\sigma)\,x$	–	–	Max of σ(1−σ) is 0.25 at σ = 0.5. It is ≈ 0 when the neuron is saturated, which slows learning	–

Representation power (L3) · Week 2

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
UAT (Cybenko 1989, Hornik 1991)	There exists g with $\lvert g(\mathbf{x}) - f(\mathbf{x})\rvert < \epsilon$	$f : \mathbb{R}^n \to \mathbb{R}^m$ continuous	One hidden layer with enough sigmoid neurons	Approximation to any precision; existence only	"Exact" or "any function"; thinking it says how to find the weights
1-D tower	$\sigma(wx+b_1) - \sigma(wx+b_2)$	Large w; $b_1 \neq b_2$	–	A bump between $-b_1/w$ and $-b_2/w$. 2 neurons per tower	–
2-D tower	Two open towers (along x_1 with $w_2 = 0$, along x_2 with $w_1 = 0$), added, then passed through a sigmoid	4 first-layer neurons	–	The final sigmoid removes the elevated base	Thinking one difference of 2-D steps is already a tower (it is open on two sides)
2-D tower, slide values	First layer $(w_1, w_2, b) = (100, 1, \pm 200)$ and $(1, 100, \pm 200)$; final sigmoid $\sigma(50s - 100)$	s = sum of the two open towers (≈1 on the base, ≈2 in the tower)	Sharp steps	Base: σ(−50) ≈ 0; tower centre: σ(0) = 0.5	–
Neurons per tower in n-D	$2n$ first-layer sigmoids (+ combining/threshold units)	n = input dimension	(Additional clarification; the slide leaves it as "Think")	1-D: 2, 2-D: 4	Answering 2^n (that is the boolean-network count)

Weeks 3–4 · Lectures 4–5

L4: Feedforward network · Week 3

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Pre-activation	$\mathbf{a}_i = \mathbf{b}_i + W_i \mathbf{h}_{i-1}$	$\mathbf{h}_0 = \mathbf{x}$	$1 \leq i \leq L$	linear part of layer i	using W_{i-1}
Activation	$\mathbf{h}_i = g(\mathbf{a}_i)$	g element-wise	$1 \leq i \leq L-1$	non-linearity	applying g at output (output uses O)
Output	$\hat{\mathbf{y}} = f(\mathbf{x}) = \mathbf{h}_L = O(\mathbf{a}_L)$	O = softmax/linear	—	prediction	—
Params in a layer	$d_{i-1}d_i + d_i$	d = layer widths	input layer has no bias	weights + biases	forgetting output biases
Slide network total	$(L-1)(n^2+n) + nk + k$	n inputs & hidden width, k outputs, $L-1$ hidden layers	—	—	treating L as # hidden layers

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Weight shapes (slide)	$W_i \in \mathbb{R}^{n\times n}$, $W_L \in \mathbb{R}^{n\times k}$, $\mathbf{b}_L \in \mathbb{R}^k$	—	—	—	⚠ for $W_L\mathbf{h}_{L-1}$ to be k-dim, W_L is really $k \times n$

L4: Output & loss functions · Week 3

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Squared error	$\frac{1}{N}\sum_{i=1}^{N}\sum_j(\hat{y}_{ij}-y_{ij})^2$	N examples	real-valued outputs, linear O	deviation of prediction	using sigmoid output for unbounded y
Linear output	$O(\mathbf{a}_L) = W_O\mathbf{a}_L + \mathbf{b}_O$	—	$y \in \mathbb{R}$	unbounded output	—
Softmax	$\hat{y}_j = \frac{e^{a_{L,j}}}{\sum_{i=1}^{k} e^{a_{L,i}}}$	k classes	—	outputs a probability distribution	forgetting normalisation
Cross-entropy	$-\sum_{c=1}^{k} y_c \log \hat{y}_c = -\log \hat{y}_\ell$	ℓ true class	one-hot $\mathbf{y}$	min CE = max log-likelihood	thinking non-true-class $\hat{y}$ enter the loss directly

L4: Backpropagation (softmax + cross-entropy) · Week 3

Name / purpose	Expression	Shape	Interpretation	⚠ Likely mistake
Grad w.r.t. output	$\frac{\partial \mathscr{L}}{\partial \hat{y}_i} = -\frac{1_{(i=\ell)}}{\hat{y}_\ell}$; $\nabla_{\hat{\mathbf{y}}}\mathscr{L} = -\frac{1}{\hat{y}_\ell}\mathbf{e}(\ell)$	k	only true-class entry nonzero	—
Softmax derivative	$\frac{\partial \hat{y}_\ell}{\partial a_{L,i}} = \hat{y}_\ell(1_{(\ell=i)} - \hat{y}_i)$; general Jacobian $\frac{\partial \hat{y}_j}{\partial a_{L,i}} = \hat{y}_j(\delta_{ij} - \hat{y}_i)$	$k \times k$ (symmetric)	every $a_{L,i}$ affects $\hat{y}_\ell$; diagonal $\hat{y}_i(1-\hat{y}_i) > 0$, off-diagonal $-\hat{y}_i\hat{y}_j < 0$	ignoring denominator dependence; ⚠ slide 36 line 3 typo: denominator $\sum_{i'}\exp(\mathbf{a}_L)_\ell$ should be $\sum_{i'}\exp(\mathbf{a}_L)_{i'}$
Grad w.r.t. output pre-activation	$\nabla_{\mathbf{a}_L}\mathscr{L} = -(\mathbf{e}(\ell) - \hat{\mathbf{y}}) = \hat{\mathbf{y}} - \mathbf{e}(\ell)$	k	true class entry $\hat{y}_\ell - 1$, others $\hat{y}_i$	sign flip
Multi-path chain rule	$\frac{\partial p}{\partial z} = \sum_m \frac{\partial p}{\partial q_m}\frac{\partial q_m}{\partial z}$	—	sum over all paths	taking a single path
Grad w.r.t. hidden unit	$\frac{\partial \mathscr{L}}{\partial h_{ij}} = \sum_m \frac{\partial \mathscr{L}}{\partial a_{i+1,m}} W_{i+1,m,j}$; $\nabla_{\mathbf{h}_i}\mathscr{L} = W_{i+1}^T \nabla_{\mathbf{a}_{i+1}}\mathscr{L}$	d_i	j-th **column** of W_{i+1}	using W_{i+1} (no transpose) or W_i
Grad w.r.t. hidden pre-activation	$\nabla_{\mathbf{a}_i}\mathscr{L} = \nabla_{\mathbf{h}_i}\mathscr{L} \odot [\dots, g'(a_{ij}), \dots]$	d_i	element-wise	matrix-multiplying instead of $\odot$
Grad w.r.t. weights	$\frac{\partial \mathscr{L}}{\partial W_{kij}} = \frac{\partial \mathscr{L}}{\partial a_{ki}} h_{k-1,j}$; $\nabla_{W_k}\mathscr{L} = \nabla_{\mathbf{a}_k}\mathscr{L}\,\mathbf{h}_{k-1}^T$	$d_k \times d_{k-1}$	outer product (destination error × source activation)	$\mathbf{h}_{k-1}\nabla_{\mathbf{a}_k}^T$ (transposed)
Grad w.r.t. biases	$\nabla_{\mathbf{b}_k}\mathscr{L} = \nabla_{\mathbf{a}_k}\mathscr{L}$	d_k	since $\partial a_{ki}/\partial b_{ki} = 1$	—
Sigmoid derivative	$\sigma'(z) = \sigma(z)(1-\sigma(z))$	scalar	max 0.25 at $z = 0$	$\sigma(1+\sigma)$
tanh derivative	$\tanh'(z) = 1 - \tanh^2(z)$	scalar	max 1 at $z = 0$	$1 - \tanh z$
GD on θ	$\theta_{t+1} = \theta_t - \eta\nabla\theta_t$	same as θ	—	gradient has same shape as parameter

L5: GD and momentum family · Week 4

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Taylor / descent condition	$\mathscr{L}(\theta+\eta u)=\mathscr{L}(\theta)+\eta u^T\nabla\mathscr{L}+\frac{\eta^2}{2!}u^T\nabla^2\mathscr{L}u+\ldots$; need $u^T\nabla\mathscr{L}<0$; $u^T\nabla\mathscr{L}=\lvert u\rvert\lvert\nabla\mathscr{L}\rvert\cos\beta$	$u=\Delta\theta$	η small (drop η^2 terms)	most negative at $\beta=180^\circ$	—
GD (recap, L3)	$w_{t+1}=w_t-\eta\nabla w_t$	η lr	Taylor 1st order, small η	move opposite to gradient	moving along gradient
Toy loss (code)	$\mathscr{L}(w,b)=\frac{1}{2}\sum_i(y_i-f(x_i))^2$	data $(0.5,0.2),(2.5,0.9)$	slide 4 omits the $\frac{1}{2}$	$\mathscr{L}(0.5,0)=0.073$	—
Toy sigmoid-neuron grads	$\nabla w=\sum_i(f(x_i)-y_i)f(x_i)(1-f(x_i))x_i$; $\nabla b=\sum_i(f(x_i)-y_i)f(x_i)(1-f(x_i))$	$f=\sigma(wx+b)$, $\mathscr{L}=\frac{1}{2}\sum(y-f)^2$	—	$\nabla w=0$ whenever $x_i=0$ (sparse)	dropping x_i in ∇w
Momentum	$u_t=\gamma u_{t-1}+\eta\nabla w_t$; $w_{t+1}=w_t-u_t$; $u_0=0$	γ momentum (0.9 in code)	—	EMA-like accumulation of history	sign / forgetting η
Momentum unrolled	$u_t=\sum_{\tau=1}^{t}\gamma^{t-\tau}\eta\nabla w_\tau$	—	—	older grads down-weighted	⚠ slide typo: 2nd term written with ∇w_1
Momentum steady state (add. clarification)	$u_\infty=\frac{\eta g}{1-\gamma}$	constant g	—	10× GD step for $\gamma=0.9$	—
NAG	$w_{la}=w_t-\gamma u_{t-1}$; $u_t=\gamma u_{t-1}+\eta\nabla w_{la}$; $w_{t+1}=w_t-u_t$	—	—	look before you leap	evaluating gradient at w_t

L5: Stochastic / mini-batch · Week 4

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Steps per epoch	Batch: 1; SGD: N; Mini-batch: N/B	N points, B batch size	—	1 epoch = 1 pass, 1 step = 1 update	confusing epochs and steps
Total updates	(steps/epoch) × #epochs	—	—	—	—
Mini-batch rule (code)	update when `num_points_seen % mini_batch_size == 0`, then reset `dw, db`	`mini_batch_size`=2 in the demo	—	$B=1\Rightarrow$ SGD, $B=N\Rightarrow$ batch GD	—

L5: Learning-rate & momentum schedules · Week 4

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Step decay	halve η every 5 epochs, or when val. error rises vs previous epoch	—	—	piecewise constant	—

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Exponential decay	slide: $\eta = \eta_0^{-kt}$ (standard: $\eta_0 e^{-kt}$)	η_0, k hyperparams, t step	—	decays with steps	⚠ slide form grows for $\eta_0 < 1$
$1/t$ decay	$\eta = \frac{\eta_0}{1+kt}$	as above	—	$\eta_0/2$ when $kt = 1$	—
Initial lr search	log scale $10^{-4}, \dots, 1$ then finer search (e.g. 0.05, 0.2, 0.3 around 0.1)	—	heuristic	—	—
Sutskever momentum schedule	$\gamma_t = \min(1 - 2^{-1-\log_2(\lfloor t/250 \rfloor + 1)}, \gamma_{max}) = \min\left(1 - \frac{1}{2(\lfloor t/250 \rfloor + 1)}, \gamma_{max}\right)$	$\gamma_{max} \in \{0.999, 0.995, 0.99, 0.9, 0\}$	—	γ grows: 0.5, 0.75, 0.833, 0.875…	—
Line search	try $\eta \in \{0.1, 0.5, 1, 5, 10\}$, keep w with lowest loss	—	—	best η per step	ignoring extra cost

L5: Adaptive methods · Week 4

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
AdaGrad	$v_t = v_{t-1} + (\nabla w_t)^2$; $w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t$	$\epsilon = 10^{-8}$ (code), $\eta = 0.1$ (code)	per parameter	rare (sparse) params get larger effective lr	v_t never decreases ⇒ lr → 0
RMSProp	$v_t = \beta v_{t-1} + (1-\beta)(\nabla w_t)^2$; $w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t$	$\beta = 0.9$ (code)	—	EMA ⇒ less aggressive decay	confusing with AdaGrad sum
Adam moments	$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla w_t$; $v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla w_t)^2$	$\beta_1 = 0.9, \beta_2 = 0.999$	$m_0 = v_0 = 0$	1st & 2nd moment EMAs	swapping β_1/β_2
Adam bias correction	$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$, $\hat{v}_t = \frac{v_t}{1-\beta_2^t}$	t = step	—	removes bias toward 0 at small t	using β^t in numerator
Adam update	$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$	$\epsilon = 10^{-8}$	—	first step magnitude ≈ η	using m_t instead of $\hat{m}_t$
EMA unrolled	$m_t = (1-\beta)\sum_{i=1}^{t} \beta^{t-i} g_i$	—	$m_0 = 0$	weights sum to $1 - \beta^t$	—
Bias of EMA	$\mathbb{E}[m_t] = \mathbb{E}[g](1-\beta^t) \Rightarrow \mathbb{E}[\hat{m}_t] = \mathbb{E}[g]$	—	all g_i i.i.d. ($\mathbb{E}[g_i] = \mathbb{E}[g]$)	correction exact in expectation	forgetting the assumption
ε placement	slides: $\sqrt{v + \epsilon}$; elsewhere often $\sqrt{v} + \epsilon$	—	—	numerical safety only	—

Weeks 5–6 · Lectures 6–7

L6: Eigenvalues and dynamics · Week 5

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Eigen equation	$Ax = \lambda x$	A square, $x \neq 0$	square A only	the direction is unchanged, only scaled	rectangular matrices have no eigenvectors
Dominant eigenvalue	$\lvert\lambda_1\rvert \geq \lvert\lambda_i\rvert$, $i = 2..n$	—	—	largest in absolute value	ignoring the sign (−5 dominates 3)

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Stochastic matrix	entries > 0, each **column** sums to 1	e.g. $M = \begin{bmatrix} p & 1-q \\ 1-p & q \end{bmatrix}$	—	dominant eigenvalue = 1, giving a steady state	rows vs columns
Markov update	$v^{(n)} = M^n v^{(0)}$	$v^{(0)} = [k_1, k_2]^T$	—	converges to a multiple of e_d	—
Power behaviour	$A^{p+n} x_0 = k\lambda_d^n e_d$	λ_d dominant	dominant eigenvalue exists	$\lvert\lambda_d\rvert > 1$ explodes, < 1 vanishes, $= 1$ steady	"vanish" is not "steady"

L6: Bases · Week 5

Name	Expression	Conditions	Interpretation	⚠
Linear independence	$\sum c_i v_i = 0 \Rightarrow c_i = 0 \; \forall i$	—	no vector is a combination of the others	—
Coordinates, general basis	solve $U\alpha = z$	u_i independent	Gaussian elimination, $O(n^3)$	—
Coordinates, orthonormal basis	$\alpha_i = z^T u_i = \lvert z\rvert \cos\theta$	$u_i^T u_j = \delta_{ij}$	$O(n)$ each, $O(n^2)$ total	only valid for an orthonormal basis
Theorem 1	distinct eigenvalues give independent eigenvectors	—	sufficient, not necessary	—
Theorem 2	symmetric A gives orthogonal eigenvectors	$A = A^T$	convenient basis (also real eigenvalues, add'l)	—

L6: EVD · Week 5

Name	Expression	Conditions	Interpretation	⚠
Eigen matrix form	$AU = U\Lambda$	columns of U = eigenvectors	—	—
EVD	$A = U\Lambda U^{-1}$; $U^{-1}AU = \Lambda$ (diagonalisation)	n linearly independent eigenvectors	—	not every matrix has one
Symmetric EVD	$A = U\Lambda U^T$, $U^T U = I$	symmetric, normalised eigenvectors	inverse = transpose	the slide writes $U\Sigma U^T$
Rayleigh max/min	$\max_{\lVert x\rVert=1} x^T Ax = \lambda_{\max}$; min $= \lambda_{\min}$	symmetric A	from the Lagrangian $x^T Ax - \lambda(x^T x - 1)$, giving $Ax = \lambda x$	the critical points are eigen**vectors** (the slide says "eigenvalues")
Eigenbasis action	$x = \sum \alpha_i v_i \Rightarrow Ax = \sum \alpha_i \lambda_i v_i$	eigenvectors form a basis	multiplication becomes scaling	—

L6: PCA · Week 5

Name	Expression	Variables	Conditions	Interpretation	⚠
Transform	$\alpha_{ij} = x_i^T p_j$; $\hat{X} = XP$	$X \in \mathbb{R}^{m\times n}$ (rows = points), P orthonormal $n \times n$	—	new coordinates	—

Name	Expression	Variables	Conditions	Interpretation	⚠
Zero mean preserved	$1^T X = 0 \Rightarrow 1^T \hat{X} = 0$	—	centred X	—	—
Covariance (definition)	$C_{ij} = \frac{1}{m}\sum_{k=1}^{m}(X_{ki} - \mu_i)(X_{kj} - \mu_j)$	μ_i = mean of column i	any data	covariance of features i and j	slides divide by m, not $m-1$
Covariance (centred)	$\Sigma = C = \frac{1}{m}X^T X$, $C_{ij} = \frac{1}{m}\sum_k X_{ki}X_{kj}$	—	zero-mean columns	$n \times n$, symmetric	not XX^T
Correlation	$\rho_{yz} = \frac{\sum_i (y_i-\bar{y})(z_i-\bar{z})}{\sqrt{\sum_i (y_i-\bar{y})^2}\sqrt{\sum_i (z_i-\bar{z})^2}}$	two columns	—	$\lvert\rho\rvert \approx 1$ means a redundant column	—
Transformed covariance	$\frac{1}{m}\hat{X}^T\hat{X} = P^T\Sigma P = D$	D diagonal	—	uncorrelated new dimensions, so P = eigenvectors of Σ	—
Reconstruction error	$e = \sum_i \lVert x_i - \hat{x}_i\rVert^2 = \sum_i\sum_{j>k}\alpha_{ij}^2 = \sum_{j>k} p_j^T(mC)p_j$	k kept dimensions	orthonormal P	minimised by discarding the smallest-eigenvalue eigenvectors; $e_{\min} = m\sum_{j>k}\lambda_j(C)$	the slide's index typo $\alpha_{ik}p_k$
Variance along PC i	$\frac{1}{m}\hat{X}_i^T\hat{X}_i = \frac{\lambda_i}{m}$ (λ_i of X^TX) $= \lambda_i(C)$	$\hat{X}_i = Xp_i$	—	eigenvalue = variance	forgetting $1/m$
Variance explained (add'l)	$\sum_{i\le k}\lambda_i / \sum_i \lambda_i$; total $= \operatorname{tr}(C)$	—	—	used to choose k	—
Eigenfaces storage	$k\cdot n$ (once) $+ m\cdot k$ coefficients	$n = 10^4$	—	vs $m\cdot n$ raw	—

L6: SVD · Week 5

Name	Expression	Shapes	Interpretation	⚠
Singular pairs	$Av_i = \sigma_i u_i$	$v_i \in \mathbb{R}^n$, $u_i \in \mathbb{R}^m$	maps the input basis to the output basis	—
Compact SVD	$A_{m\times n}V_{n\times k} = U_{m\times k}\Sigma_{k\times k}$	$k = \operatorname{rank}(A)$	—	—
Full SVD	$A = U\Sigma V^T$; $U^TAV = \Sigma$	$U_{m\times m}$, $\Sigma_{m\times n}$, $V_{n\times n}$	U, V orthogonal	Σ is rectangular
Link to EVD	$A^TA = V\Sigma^2V^T$; $AA^T = U\Sigma^2U^T$	—	V = eigvecs(A^TA) (right), U = eigvecs(AA^T) (left)	swapping U and V
Singular values	$\sigma_i = \sqrt{\lambda_i(A^TA)} \ge 0$	—	—	not the eigenvalues of A
Outer-product form	$A = \sum_{i=1}^{k}\sigma_i u_i v_i^T$	—	sum of rank-1 matrices	—
Best rank-k	$\arg\min_{\operatorname{rank}B=k}\lVert A-B\rVert_F^2 = U_{:,k}\Sigma_{k,k}V_{k,:}^T$	—	minimises reconstruction error; error $= \sum_{i>k}\sigma_i^2$ (add'l)	slide omits the constraint rank$(B)\le k$ (else $B = A$)
Frobenius norm	$\lVert A\rVert_F^2 = \sum_{ij}a_{ij}^2 = \sum\sigma_i^2$	—	—	—

L7: Autoencoder basics · Week 6

Name	Expression	Conditions	Interpretation	⚠
Encoder	$h = g(Wx_i + b)$	g typically sigmoid	hidden code	—

Name	Expression	Conditions	Interpretation	⚠
Decoder	$\hat{x}_i = f(W^* h + c)$	f is logistic (binary inputs) or linear (real inputs)	reconstruction	tanh/logistic for real inputs is wrong
Under/overcomplete	$\dim h < \dim x$ / $\dim h \geq \dim x$	—	overcomplete can learn the identity	"=" counts as overcomplete
Squared-error loss	$\frac{1}{m}\sum_i \sum_j (\hat{x}_{ij} - x_{ij})^2$	real inputs	—	—
Squared-error gradient	$\partial \mathscr{L} / \partial h_2 = 2(\hat{x}_i - x_i)$	per example	—	sign and factor 2
Cross-entropy loss	$-\sum_j [x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij})]$	binary inputs, sigmoid decoder	minimised at $\hat{x}_{ij} = x_{ij}$	—
Cross-entropy gradient	$\frac{\partial \mathscr{L}}{\partial h_{2j}} = -\frac{x_{ij}}{\hat{x}_{ij}} + \frac{1-x_{ij}}{1-\hat{x}_{ij}}$; $\frac{\partial h_{2j}}{\partial a_{2j}} = \sigma(a_{2j})(1 - \sigma(a_{2j}))$	—	product $= \hat{x}_{ij} - x_{ij}$ (add'l)	—
Chain rule	$\frac{\partial \mathscr{L}}{\partial W} = \frac{\partial \mathscr{L}}{\partial h_2} \frac{\partial h_2}{\partial a_2} \frac{\partial a_2}{\partial h_1} \frac{\partial h_1}{\partial a_1} \frac{\partial a_1}{\partial W}$	—	—	—

L7: AE and PCA · Week 6

Name	Expression	Conditions	Interpretation	⚠
Normalisation	$\hat{x}_{ij} = \frac{1}{\sqrt{m}}\left(x_{ij} - \frac{1}{m}\sum_k x_{kj}\right)$	—	makes $X^T X$ the covariance matrix	not division by the standard deviation
AE as matrix factorisation	$\min_{W^*,H} \lVert X - HW^* \rVert_F^2$	linear decoder, squared error, biases ignored	—	—
Optimum	$HW^* = U_{\cdot,\leq k}\Sigma_{k,k}V^T_{\cdot,\leq k}$; $H = U_{\cdot,\leq k}\Sigma_{k,k} = XV_{\cdot,\leq k}$; $W^* = V^T_{\cdot,\leq k}$	—	encoder $W = V_{\cdot,\leq k}$ = PCA P	needs all 4 conditions (linear encoder, linear decoder, squared error, normalisation)

L7: Regularized autoencoders · Week 6

Name	Expression	Conditions	Interpretation	⚠
L2 / weight decay	$\Omega = \lambda \lVert\theta\rVert^2$; adds λW to $\partial \mathscr{L} / \partial W$	—	small weights	strictly $2\lambda W$
Tied weights	$W^* = W^T$	—	reduces capacity	—
DAE (masking)	$P(\tilde{x}_{ij} = 0 \mid x_{ij}) = q$; $P(\tilde{x}_{ij} = x_{ij} \mid x_{ij}) = 1 - q$	—	target is the **clean** x	q = probability of zeroing
DAE (Gaussian)	$\tilde{x}_{ij} = x_{ij} + \mathcal{N}(0, 1)$	—	—	—
Max-activating input	$\max W_1^T x$ s.t. $\lVert x\rVert = 1 \Rightarrow x = W_1 / \sqrt{W_1^T W_1}$	normalised inputs, bias ignored	filter visualisation	—
Average activation	$\hat{\rho}_l = \frac{1}{m}\sum_{i=1}^m h(x_i)_l$	sigmoid hidden units	—	average over examples
Sparsity penalty	$\Omega = \sum_{l=1}^k \rho \log \frac{\rho}{\hat{\rho}_l} + (1 - \rho) \log \frac{1-\rho}{1-\hat{\rho}_l}$	$\rho \approx 0.005$	KL; minimum 0 at $\hat{\rho}_l = \rho$	minimum is not at $\hat{\rho}_l = 0$
Sparsity gradient	$\frac{\partial \Omega}{\partial \hat{\rho}_l} = -\frac{\rho}{\hat{\rho}_l} + \frac{1-\rho}{1-\hat{\rho}_l}$; $\frac{\partial \hat{\rho}_l}{\partial W_{jl}} = \frac{1}{m}\sum_i g'(W_{\cdot,l}^T x_i + b_l) x_{ij}$	$W \in \mathbb{R}^{n \times k}$ here	gradient has the sign of $(\hat{\rho}_l - \rho)$; Ω itself is always ≥ 0	the slide's matrix form $x_i(g'(W^T x_i + b))^T$ drops $\frac{1}{m}\sum_i$ and is the $n \times k$ matrix for all l

Name	Expression	Conditions	Interpretation	⚠
Total sparse loss	$\hat{\mathscr{L}} = \mathscr{L} + \Omega$; $\partial\hat{\mathscr{L}}/\partial W = \partial\mathscr{L}/\partial W + \partial\Omega/\partial W$	—	—	—
Contractive penalty	$\Omega = \lVert J_x(h)\rVert_F^2 = \sum_{j=1}^{n}\sum_{l=1}^{k}(\partial h_l/\partial x_j)^2$	J is $k \times n$	insensitivity vs reconstruction gives only the important variations	Jacobian w.r.t. **input**, of the **encoder**
Sigmoid Jacobian (add'l)	$\partial h_l/\partial x_j = h_l(1-h_l)W_{lj}$	$h = \sigma(Wx+b)$	—	—

Weeks 7–8 · Lectures 8–9

L8 · Bias, variance, true error · Week 7

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Bias	$\mathrm{Bias}(\hat{f}(x)) = E[\hat{f}(x)] - f(x)$	E over models trained on different samples	fixed x	avg model vs truth; simple model → high	forgetting it is the *average* model
Variance	$\mathrm{Var}(\hat{f}(x)) = E[(\hat{f}(x) - E[\hat{f}(x)])^2]$			spread of models across samples; complex → high	
MSE decomposition	$E[(y-\hat{f}(x))^2] = \mathrm{Bias}^2 + \mathrm{Var} + \sigma^2$	σ^2 noise variance	unseen (x,y), $y = f(x)+\varepsilon$	σ^2 irreducible	bias not squared
Train / test error	$\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{f}(x_i))^2$; $\frac{1}{m}\sum_{i=n+1}^{n+m}(y_i-\hat{f}(x_i))^2$	n train, m test		train error optimistic for complex models	
True vs empirical error	$E[(\hat{f}-f)^2] = E[(\hat{y}-y)^2] - E[\varepsilon^2] + 2E[\varepsilon(\hat{f}-f)]$	$\varepsilon \sim \mathcal{N}(0,\sigma^2)$		last term = Cov$(\varepsilon,\hat f-f)$	
Test case	true err = emp. test err − $\frac{1}{m}\sum\varepsilon_i^2$ + 0		test pts not used in fitting ⇒ $\varepsilon \perp \hat{f}$	test err ≈ true err (up to const)	slide says "+ small constant"
Train case	covariance term $\neq 0$		ε used in fitting	train err < true err	
Stein's lemma	$\frac{1}{n}\sum\varepsilon_i(\hat{f}(x_i)-f(x_i)) = \frac{\sigma^2}{n}\sum\frac{\partial\hat{f}(x_i)}{\partial y_i}$		Gaussian noise	sensitivity to observations ↔ complexity	
Train–test gap (figure)	test err − train err ≈ $\frac{\sigma^2}{n}\sum_i\frac{\partial\hat{f}(x_i)}{\partial y_i}$			gap (optimism) grows with complexity	
True error ↔ complexity	true err = emp. train err + small const + Ω(model complexity)			basis of regularization	
Regularized objective	$\min_\theta \mathscr{L}_{train}(\theta) + \Omega(\theta)$	Ω high for complex models		l_1, l_2, early stopping, input noise are instances	

L8 · l_2 regularization · Week 7

Name	Expression	Variables	Conditions	Interpretation	⚠
Loss	$\widetilde{\mathscr{L}}(w) = \mathscr{L}(w) + \frac{\alpha}{2}\lVert w\rVert^2$	α strength			

Name	Expression	Variables	Conditions	Interpretation	⚠
Gradient / update	$\nabla\widetilde{\mathscr{L}} = \nabla\mathscr{L} + \alpha w$; $w_{t+1} = w_t - \eta\nabla\mathscr{L}(w_t) - \eta\alpha w_t$			weight decay	dropping the η on αw
Quadratic approx	$\mathscr{L}(w) = \mathscr{L}(w^*) + \frac{1}{2}(w - w^*)^T H(w - w^*)$; $\nabla\mathscr{L}(w) = H(w - w^*)$	w^* unregularized optimum	$\nabla\mathscr{L}(w^*) = 0$		
Regularized optimum	$\tilde{w} = (H + \alpha I)^{-1} H w^*$			$\alpha \to 0 \Rightarrow \tilde{w} \to w^*$	
Eigen form	$\tilde{w} = Q(\Lambda + \alpha I)^{-1}\Lambda Q^T w^* = QDQ^T w^*$	$H = Q\Lambda Q^T$	H symmetric PSD	rotate, scale, rotate back	
Scale factor	$D_{ii} = \frac{\lambda_i}{\lambda_i + \alpha}$	λ_i Hessian eigenvalue		$\lambda_i \gg \alpha \to 1$ kept; $\lambda_i \ll \alpha \to 0$ removed; $\lambda_i = \alpha \to 0.5$	confusing with $\alpha/(\lambda_i + \alpha)$
Effective params	$\sum_{i=1}^{n} \frac{\lambda_i}{\lambda_i + \alpha} < n$		$\alpha > 0$	↓ as α ↑	

L8 · Noise, early stopping, ensembles, dropout · Week 7

Name	Expression	Variables	Conditions	Interpretation	⚠
Input noise ≡ L_2	$E[(\tilde{y} - y)^2] = E[(\hat{y} - y)^2] + \sigma^2 \sum_i w_i^2$	$\tilde{x}_i = x_i + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, $\hat{y} = \sum w_i x_i$	simple (linear) I/O network; $\varepsilon_i \perp \varepsilon_j$, $\varepsilon \perp (\hat{y} - y)$	more noise = stronger weight decay	applies to linear model on slides
Soft targets	$p = \{\frac{\varepsilon}{9}, \ldots, 1 - \varepsilon, \ldots, \frac{\varepsilon}{9}\}$	10 classes, ε small		don't trust noisy labels	K classes → $\varepsilon/(K - 1)$; slide's "minimize $\sum p \log q$" lacks minus sign
Early stopping rule	stop at step k if no val. improvement in last p steps; return model at $k - p$	p patience			returning step k
Exploration bound	$\lvert w_{t+1} - w_0\rvert \le \eta t \lvert\tau\rvert$	τ max gradient		t limits how far w moves	
GD under quadratic	$w_t = (I - \eta H)w_{t-1} + \eta H w^*$				
Closed form	$w_t = Q[I - (I - \eta\Lambda)^t]Q^T w^*$		$w_0 = 0$		slide writes ε for η
ES ≡ L_2	$(I - \eta\Lambda)^t = (\Lambda + \alpha I)^{-1}\alpha$, i.e. $(1 - \eta\lambda_i)^t = \frac{\alpha}{\lambda_i + \alpha}$		quadratic approx, $w_0 = 0$	more steps ≈ smaller α	
Ensemble MSE	$E[(\frac{1}{k}\sum \varepsilon_i)^2] = \frac{V}{k} + \frac{k-1}{k}C$	$V = E[\varepsilon_i^2]$, $C = E[\varepsilon_i \varepsilon_j]$	zero-mean errors	$C = 0 \to V/k$; $C = V \to V$ (no help); $k \to \infty \to C$	swapping V, C coefficients
Dropout #thinned nets	2^n	n nodes		weights shared across all	
Dropout test-time	weight $w \to pw$	p retain prob (0.5 hidden, 0.8 input)	standard (non-inverted) dropout	output scaled by fraction of time node was on	p = retain, not drop

L9 · Activations · Week 8

Name	Expression	Derivative	Range / notes	⚠
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma(x)(1-\sigma(x))$, max 0.25	$[0,1]$; saturates; not zero-centred; exp	
tanh	$\tanh x$	$1-\tanh^2 x$, max 1	$[-1,1]$; zero-centred; still saturates; exp	
ReLU	$\max(0,x)$	0 ($x<0$), 1 ($x>0$)	no +ve saturation; cheap; can die	
2-ReLU sigmoid approx	$\max(0,x+1) - \max(0,x-1)$		0 → 2 piecewise linear	
Leaky ReLU	$\max(0.01x, x)$	0.01 / 1	doesn't die; ~zero-centred	
PReLU	$\max(\alpha x, x)$	α / 1	α learned by backprop	
ELU	x ($x>0$); $ae^x - 1$ ($x \leq 0$) [slide]	1 / ae^x	~zero-centred; exp cost	standard form $a(e^x - 1)$
Maxout	$\max(w_1^T x + b_1, w_2^T x + b_2)$		generalizes ReLU/Leaky; no saturation/death; 2× params	
Gradient ∝ input	$\nabla w_i = \frac{\partial \mathscr{L}}{\partial y} \cdots h_{i-1}$			why activations≈0 ⇒ grads≈0

L9 · Initialization & BN · Week 8

Name	Expression	Variables	Conditions	Interpretation	⚠
Var of product	$\mathrm{Var}(wx) = E[w]^2\mathrm{Var}(x) + E[x]^2\mathrm{Var}(w) + \mathrm{Var}(w)\mathrm{Var}(x)$		$w \perp x$	→ $\mathrm{Var}(w)\mathrm{Var}(x)$ for zero means	
One layer	$\mathrm{Var}(s_{1i}) = n\mathrm{Var}(w)\mathrm{Var}(x)$	n fan-in	zero-mean w, x; iid; linear		
k layers	$\mathrm{Var}(s_{ki}) = [n\mathrm{Var}(w)]^k\mathrm{Var}(x)$		same Var(w) all layers	>1 explodes, <1 vanishes	exponential in k
Scaling rule	$\mathrm{Var}(az) = a^2\mathrm{Var}(z)$				
$1/\sqrt{n}$ (Xavier/Glorot name = addl.)	$w = z/\sqrt{n}$, $z \sim \mathcal{N}(0,1)$ ⇒ $n\mathrm{Var}(w) = 1$		tanh/sigmoid	std $= 1/\sqrt{n}$	
He (ReLU)	$w = z/\sqrt{n/2}$ ⇒ $\mathrm{Var}(w) = 2/n$		ReLU	factor 2: ReLU uses only +ve half	std $= \sqrt{2/n}$ not $2/\sqrt{n}$
BN standardize (slide)	$\hat{s}_{ik} = \frac{s_{ik} - E[s_{ik}]}{\sqrt{\mathrm{var}(s_{ik})}}$	stats from mini-batch ($N \times D$ batch; per column/dimension k)		pre-activation → unit Gaussian	
BN scale/shift	$y^{(k)} = \gamma^k \hat{s}_{ik} + \beta^{(k)}$	γ, β learned		$\gamma = \sqrt{\mathrm{var}}, \beta = E$ ⇒ recovers s_{ik}	
BN standard (addl.)	$\hat{s} = \frac{s-\mu_B}{\sqrt{\sigma_B^2+\epsilon}}$, $\mu_B = \frac{1}{m}\sum s_i$, $\sigma_B^2 = \frac{1}{m}\sum(s_i - \mu_B)^2$	ϵ small		numerical stability	slide omits ϵ

Weeks 9–10 · Lectures 10–11

L10 – Count-based word representations · Week 9

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
One-hot distance / similarity	$\|e_i - e_j\| = \sqrt{2}$, $\cos(e_i, e_j) = 0$	e_i = one-hot of word i, dim $\|V\|$	$i \neq j$	No notion of similarity	Thinking the distance depends on $\|V\|$
Thresholded counts	$X_{ij} = \min(\text{count}(w_i, c_j), t)$	t = cap (e.g. 100)	—	Limits the dominance of stop words	Using max instead of min
PMI	$\text{PMI}(w, c) = \log \frac{p(c\|w)}{p(c)} = \log \frac{\text{count}(w,c)\, N}{\text{count}(c)\, \text{count}(w)}$	N = total words	count(w,c) > 0 (else $-\infty$)	>0: co-occur more than chance	Forgetting N in the numerator
PMI_0	PMI if count > 0, else 0	—	—	Fixes $-\infty$ only	Confusing with PPMI (PMI_0 keeps negatives)
PPMI	PMI if PMI > 0, else 0	—	—	All entries ≥ 0	—
SVD	$X_{m\times n} = U_{m\times k}\Sigma_{k\times k}V^\top_{k\times n} = \sum_{i=1}^{k} \sigma_i u_i v_i^\top$	σ_i singular values	—	Truncation gives the best rank-k approximation	Each $\sigma_i u_i v_i^\top$ is $m \times n$ with rank 1
Rank-1 storage	$m + n + 1$ (rank k: $k(m + n + 1)$)	—	—	Compression vs mn	—
Similarity matrix	$(XX^\top)_{ij} = X[i, :] \cdot X[j, :]$	—	—	≈ (unnormalized) cosine similarity of words i, j	—
Cosine from $XX^\top$ (additional)	$\cos(i, j) = \frac{(XX^\top)_{ij}}{\sqrt{(XX^\top)_{ii}(XX^\top)_{jj}}}$	—	—	Slide: sim(human, user) 0.21 with X → 0.33 with $\hat{X}$	Using the raw dot product as the cosine
SVD word vectors	$\hat{X}\hat{X}^\top = (U\Sigma)(U\Sigma)^\top \Rightarrow W_{word} = U\Sigma \in \mathbb{R}^{m\times k}$, $W_{context} = V$	—	$V^\top V = I$	Low-dim vectors with the same dot products as rows of $\hat{X}$	Taking U alone or rows of $\hat{X}$

L10 – Prediction-based (CBOW / skip-gram) · Week 9

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Hidden layer (CBOW, 1 context)	$h = W_{context} x_c = u_c$	$W_{context} \in \mathbb{R}^{k\times\|V\|}$, one-hot x	—	Column lookup	—
Hidden layer ($d-1$ contexts)	$h = \sum_{i=1}^{d-1} u_{c_i}$	stacked $[W_{context}, W_{context}]$ (shared)	—	Sum of the context columns	Counting stacked copies as extra params
Softmax output	$\hat{y}_w = \frac{\exp(u_c \cdot v_w)}{\sum_{w' \in V} \exp(u_c \cdot v_{w'})}$	v_w = column of W_{word}	—	Denominator costs $O(\|V\|)$	—
Loss	$\mathcal{L} = -\log \hat{y}_w$	—	—	Cross-entropy	—
Gradient / update	$\nabla_{v_w} = -u_c(1 - \hat{y}_w)$; $v_w \leftarrow v_w + \eta u_c(1 - \hat{y}_w)$	η = learning rate	—	No update if $\hat{y}_w = 1$; pulls v_w toward u_c	Sign error (+ in the update)
Skip-gram loss	$\mathcal{L} = -\sum_{i=1}^{d-1} \log \hat{y}_{w_i}$	—	—	Sum of cross-entropies over the context words	—

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Param count	$2k\|V\|$	k = embedding dim	—	e.g. $k = 300$, $\|V\| = 10^4$ gives 6M	—

L10 – Softmax alternatives · Week 9

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Neg. sampling: positive	$p(z = 1 \mid w, c) = \sigma(u_c^\top v_w)$	$(w, c) \in D$	—	True pair should get a high score	—
Neg. sampling: negative	$p(z = 0 \mid w, r) = 1 - \sigma(u_r^\top v_w) = \sigma(-u_r^\top v_w)$	$(w, r) \in D'$	—	Random pair should get a low score	Writing $\sigma(u_r^\top v_w)$
Neg. sampling objective	$\max_\theta \sum_D \log \sigma(u_c^\top v_w) + \sum_{D'} \log \sigma(-u_r^\top v_w)$	—	—	Binary classification replaces the softmax	—
Negatives	$\|D'\| = k\|D\|$; $r \sim \text{count}(r)^{3/4}$ (normalized)	k = negatives per positive	—	3/4 power flattens the distribution (rare words up)	⚠ the slide normalizes by N (should be $\sum \text{count}^{3/4}$)
Contrastive estimation	want $s > s_c + m$; hinge: minimize $\max(0, m - (s - s_c))$	s true score, s_c corrupted, m margin	—	Ranking loss	⚠ the slide writes "maximize max(0, s−(s_c+m))"
Hierarchical softmax	$P(w \mid v_c) = \prod_{k=1}^{\|\pi(w)\|} P(\pi(w)_k \mid v_c)$; $P(\pi_i = 1) = \sigma(v_c^\top u_i)$, $P(\pi_i = 0) = \sigma(-v_c^\top u_i)$	$\pi(w)_k = 1$ if the path branches left; u_i = internal node vector	Binary tree with $\|V\|$ leaves	Cost $\|\pi(w)\| \approx \log_2 \|V\|$ instead of $\|V\|$; same #params	Thinking the tree must be semantically built (a random one works)

L10 – GloVe and the SVD link · Week 9

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Conditional prob.	$P(j \mid i) = X_{ij}/X_i$, $X_i = \sum_j X_{ij}$	$X_{ij} = X_{ji}$	—	Global co-occurrence statistic	—
GloVe target	$v_i^\top v_j + b_i + b_j = \log X_{ij}$	b_i, b_j learned biases (absorb $\frac{1}{2} \log X_i$)	—	Dot product is faithful to log counts	—
GloVe objective	$\min \sum_{i,j} f(X_{ij})(v_i^\top v_j + b_i + b_j - \log X_{ij})^2$	—	—	Weighted least squares	Calling it cross-entropy
Weighting fn	$f(x) = (x/x_{max})^\alpha$ if $x < x_{max}$, else 1	α tuned	—	Neither rare nor frequent pairs over-weighted	Forgetting the cap at 1
Analogy	a : b :: c : ? → NN of $v_b - v_a + v_c$	—	—	e.g. $v_{sister} - v_{brother} + v_{grandson}$	Wrong sign/order
Relatedness eval	compare corr(S_{model}, S_{human}), S_{model} = cosine	—	—	Higher correlation = better model	—

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
word2vec as factorization	$M = W_{context}^{(\top)} W_{word}$, $M_{ij} = \mathrm{PMI}(w_i, c_j) - \log k$	k = # negative samples	SGNS	word2vec ≈ factorizing shifted PMI (like SVD)	k here ≠ embedding dim; the slide writes c_i (typo)
Synonym detection	$\arg\max_{c \in C} \cos(v_{term}, v_c)$	C = 4 candidates	—	Compare models by accuracy	—

L11 – Convolution · Week 10

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
1D convolution	$s_t = \sum_{a=0}^{\infty} x_{t-a} w_{-a} = (x * w)_t$	x input, w filter	In practice a small window	Weighted average of recent inputs	—
2D convolution (in practice)	$S_{ij} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I_{i+a,j+b} K_{a,b}$	$m \times n$ kernel	—	Looks at succeeding neighbours	—
2D centred	$S_{ij} = \sum_{a=\lfloor -m/2 \rfloor}^{\lfloor m/2 \rfloor} \sum_{b=\lfloor -n/2 \rfloor}^{\lfloor n/2 \rfloor} I_{i-a,j-b} K_{m/2+a,n/2+b}$	—	—	Kernel centred on the pixel	—
Output width/height	$W_2 = \frac{W_1 - F + 2P}{S} + 1$, $H_2 = \frac{H_1 - F + 2P}{S} + 1$	F filter size, P pad, S stride	Floor if not an integer	Spatial output size	P instead of $2P$; forgetting $+1$
Output depth	$D_2 = K$	K = # filters	Filter depth = D_1	One 2D map per filter	Taking $D_2 = D_1$
Conv params	$F \cdot F \cdot D_1 \cdot K$ (slides) / $(F \cdot F \cdot D_1 + 1)K$ with bias	—	—	Independent of W_1, H_1, S, P	Omitting D_1
Conv computations	$W_2 H_2 K \cdot F^2 D_1$ (each output $O(F^2 D_1)$)	—	—	Cost of the layer	—
"Same" padding	$P = (F-1)/2$	—	$S = 1$, odd F	Output = input size	—
Pooling output	$(W_1 - F)/S + 1$, depth unchanged	—	—	**0 params**	Adding params to pooling
FC after conv	params = $(W \cdot H \cdot D) \times n_{out}$ (+n_{out} biases)	flattened volume	—	e.g. 2×2×256=1024 → 4096: 4.19M	Forgetting to multiply the depth
Receptive field of stacked $F \times F$ (stride 1)	$n(F-1)+1$	n layers	additional	2×3×3 → 5×5, 3×3×3 → 7×7	—

L11 – Architectures (key numbers) · Week 10

Name / purpose	Expression	Interpretation	⚠ Likely mistake
LeNet-5	32 → 28×28×6 (150) → 14×14×6 (0) → 10×10×16 (2400) → 5×5×16 (0) → 120 (48,120) → 84 (10,164) → 10 (850)	conv counts without bias; FC with bias	the slide labels pooling S=1, but it must be S=2

Name / purpose	Expression	Interpretation	⚠ Likely mistake
AlexNet (slide)	227 → 55 → 27 → 23 → 11 → 9 → 7 → 5 → 2×2×256 → 4096 → 4096 → 1000	total 27.55M (slide) ≈ 28.8M exact	—
ZF-Net vs AlexNet	F: 11 → 7 (−20.7K); K: 384→512 (+0.29M), 384→1024, 256→512	slide total diff 1.45M (recomputed conv diff ≈ 7.5M)	slide arithmetic for layers 6–7 is wrong; the slide keeps the 55×55 output for F=7 (with S=4, P=0 it would be 56)
VGG FC params	$512\cdot7\cdot7\cdot4096+4096^2+4096\cdot1000\approx123.6$M (slide ~122M)	first FC ≈ 102M; non-FC ≈ 16M	slide writes 4096×1024
GoogLeNet GAP	FC on 7×7×1024 → 1000: $7\cdot7\cdot1024\cdot1000\approx49$M vs GAP → $1024\cdot1000\approx1$M	12× fewer params than AlexNet, 2× compute	—
Inception output depth	#1×1 + #3×3 + #5×5 + #pool-proj (e.g. 3a: 64+128+32+32 = 256)	concatenation along depth	adding the reduce-layer counts
1×1 reduction saving	direct: $HWKF^2D$; reduced: $HWD_1D + HWKF^2D_1$ (28×28×192, 32 5×5, D_1=16: 120.4M → 12.4M)	≈9.7× less	—
Residual block	$H(x) = F(x) + x$	layers learn the residual; identity is easy ($F \to 0$)	—

Weeks 10–11 · Lectures 12–13

L12 — Receptive field & CNN sizes · Week 10

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Conv/pool output size (from L11)	$W_2 = \frac{W_1 - F + 2P}{S} + 1$	F filter, P pad, S stride	integer result	slide AlexNet: 227→55→27→23→11→9→7→5→2	forgetting $+1$
Receptive field recursion (Additional clarification)	$r_\ell = r_{\ell-1} + (k_\ell - 1) j_{\ell-1}$, $j_\ell = j_{\ell-1} s_\ell$, $r_0 = j_0 = 1$	k window, s stride, j jump	padding irrelevant to size	input patch seen by one neuron; slide AlexNet pool5 → 195	ignoring pooling layers / earlier strides
Stride-1 stack	$r = 1 + \sum_\ell (k_\ell - 1)$; L × 3×3 → $2L + 1$		all strides 1	two 3×3 = 5×5, three 3×3 = 7×7	multiplying kernel sizes

L12 — Visualizing filters / neurons · Week 10

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Max-activating input	$\max_x w^T x$ s.t. $\|x\|^2 = 1 \Rightarrow x^* = \frac{w}{\sqrt{w^T w}} = \frac{W}{\|W\|}$	w filter/weights	unit-norm input	filter = pattern detector; max value $= \|w\|$	answering w (unnormalized)
Influence of pixel	$\partial h_j / \partial x_i$	x_i pixel, h_j neuron	weights fixed	0 → no influence; large → high	differentiating w.r.t. weights
Backprop to first layer	$\frac{\partial h_{32}}{\partial x_2} = \sum_{i=1}^{3} \frac{\partial h_{32}}{\partial h_{1i}} \frac{\partial h_{1i}}{\partial x_2}$, $h_{1i} = \sum_j w_{ji} x_j$			chain rule over paths	slide writes w_{12} (should be w_{2i})
Guided backprop at ReLU	$g_{in} = g_{out}\, 1[x > 0]\, 1[g_{out} > 0]$	x forward input, g_{out} upper gradient	only backward pass modified	keep only positive influences	zeroing only negative grads (forgetting dead ReLUs)

L12 — Optimization over images · Week 10

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Class visualization	$\arg\max_I \left(S_c(I) - \lambda \Omega(I)\right)$	S_c score **before softmax**, Ω image prior	weights fixed; start zero image	image the net thinks is class c	using softmax prob.; updating weights
Pixel update (slide)	$i_k = i_k - \eta\, \partial S_c(I) / \partial i_k$ (any neuron: $A(I)$)	η step			⚠ for maximization sign should be $+$
Embedding inversion	$\mathcal{L}(i) = \|\phi(x) - \phi_0\|^2 + \lambda \|\phi(x)\|_6^6$; $i_k = i_k - \eta\, \partial \mathcal{L} / \partial i_k$	ϕ_0 fc7 embedding of target	minimize	reconstruct image from embedding	⚠ prior normally on image x, i.e. $\|x\|_6^6$; slide's update also drops the ∂ in the numerator
Deep Dream	$\max_I h_{ij}^2$; $\frac{\partial \mathcal{L}}{\partial i_{mn}} = \frac{\partial \mathcal{L}}{\partial h_{ij}} \frac{\partial h_{ij}}{\partial i_{mn}}$; $= 2 h_{ij} \frac{\partial h_{ij}}{\partial i_{mn}}$; $i_{mn} = i_{mn} + \frac{\partial \mathcal{L}}{\partial i_{mn}}$ (ascent)	h_{ij} neuron	start from real image	amplifies patterns already "seen"	starting from zero image

L12 — Deep Art (style transfer) · Week 10

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Content loss	$\mathcal{L}_{content}(\vec{p}, \vec{x}) = \sum_{ijk} (\vec{p}_{ijk} - \vec{x}_{ijk})^2$	$\vec{p}$ content-image activations, $\vec{x}$ new-image activations		match activations	comparing pixels instead of activations
Gram (style) matrix	$V \in \mathbb{R}^{64 \times (256 \cdot 256)}$ → Gram $\in \mathbb{R}^{64 \times 64}$ (slide: $V^T V$)	V channel × flattened map		channel correlations = style; deeper layers more style	⚠ dims: VV^T gives 64×64; size = channels², not spatial
Layer style loss	$E_\ell = \sum_{ij} (G_{ij}^\ell - A_{ij}^\ell)^2$	G^ℓ, A^ℓ Gram of style & new image			slide typo G_j^ℓ
Total style loss	$\mathcal{L}_{style}(\vec{a}, \vec{x}) = \sum_{\ell=0}^{L} w_\ell E_\ell$	w_ℓ layer weights			
Total loss	$\mathcal{L}_{total} = \alpha \mathcal{L}_{content} + \beta \mathcal{L}_{style}$	α, β	optimize pixels only	α/β↑ → more content	swapping roles of α, β

L13 — RNN · Week 11

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
RNN state	$s_i = \sigma(Ux_i + Ws_{i-1} + b)$	s_0 constant	same params every step	state summarizes past	forgetting Ws_{i-1}
RNN output	$y_i = \mathcal{O}(Vs_i + c)$	$\mathcal{O}$ = softmax for classes			
Shapes (slide)	$x_i \in \mathbb{R}^n, s_i \in \mathbb{R}^d, y_i \in \mathbb{R}^k$; $U \in \mathbb{R}^{n\times d}, V \in \mathbb{R}^{d\times k}, W \in \mathbb{R}^{d\times d}$				⚠ for Ux_i as written, U is $d \times n$ (count same)
#params	$nd + d^2 + dk + d + k$			independent of T	multiplying by T
Total loss	$\mathcal{L}(\theta) = \sum_{t=1}^{T} \mathcal{L}_t$, $\mathcal{L}_t = -\log y_{tc}$	y_{tc} prob. of true char	softmax + CE		using prob. of predicted (argmax) class

L13 — BPTT · Week 11

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Grad w.r.t. V	$\frac{\partial \mathcal{L}}{\partial V} = \sum_t \frac{\partial \mathcal{L}_t}{\partial V}$			ordinary output-layer backprop	adding through-time sum
Total derivative of state	$\frac{\partial s_4}{\partial W} = \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial W}$	∂^+ = explicit	ordered network	explicit + implicit	treating s_3 as constant
BPTT	$\frac{\partial \mathcal{L}_t}{\partial W} = \frac{\partial \mathcal{L}_t}{\partial s_t}\sum_{k=1}^{t}\frac{\partial s_t}{\partial s_k}\frac{\partial^+ s_k}{\partial W}$; $\frac{\partial \mathcal{L}}{\partial W} = \sum_t \frac{\partial \mathcal{L}_t}{\partial W}$			t terms for $\mathcal{L}_t$	sum to T instead of t
Jacobian product	$\frac{\partial s_t}{\partial s_k} = \prod_{j=k}^{t-1}\frac{\partial s_{j+1}}{\partial s_j}$			$t-k$ factors	off-by-one
One factor	$\frac{\partial s_j}{\partial s_{j-1}} = \mathrm{diag}(\sigma'(a_j))W$	$a_j = Ws_{j-1} + b$	element-wise σ		slide typo $a_j = Ws_j + b$
Activation-derivative bound	$\sigma'(a) \le \gamma$: logistic $\sigma(1-\sigma) \le \frac{1}{4}$; tanh $1 - \tanh^2 \le 1$	γ	max at $a = 0$	$\lvert\mathrm{diag}(\sigma'(a_j))\rvert = \max_p \sigma'(a_{jp}) \le \gamma$	$\gamma = 1/2$ for sigmoid
Vanishing/exploding bound	$\lvert\frac{\partial s_t}{\partial s_k}\rvert \le (\gamma\lambda)^{t-k}$, $\lvert\mathrm{diag}(\sigma')\rvert \le \gamma$, $\lvert W\rvert \le \lambda$	$\gamma = 1/4$ logistic, 1 tanh		$\gamma\lambda < 1$ vanish; > 1 **could** explode	"> 1 will explode"; $\gamma = 1/2$
Truncated BPTT	restrict product to τ ($< t-k$) terms	τ window		caps gradient path length	
Shapes	$\frac{\partial \mathcal{L}_t}{\partial W} \in \mathbb{R}^{d\times d}$, $\frac{\partial \mathcal{L}_t}{\partial s_t} \in \mathbb{R}^{1\times d}$, $\frac{\partial s_t}{\partial s_k} \in \mathbb{R}^{d\times d}$, $\frac{\partial^+ s_k}{\partial W} \in \mathbb{R}^{d\times d\times d}$				calling the last one a matrix
Tensor element	$\frac{\partial s_{kp}}{\partial W_{qr}} = \sigma'(a_{kp})s_{k-1,r}$ if $p = q$, else 0			only d^2 nonzeros of d^3	

Weeks 11–12 · Lectures 14–15

Notation: s_t = state (cell state in an LSTM), h_t = LSTM output vector, x_t = input, σ = sigmoid, $\odot$ = element-wise product, d = state dim, n = input dim, $\mathcal{D}(v)$ = diagonal matrix with v on its diagonal.

L14: Gates and cells · Week 11

Name / purpose	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Vanilla RNN	$s_t = \sigma(W s_{t-1} + U x_t + b)$	$W : d \times d$, $U : d \times n$	–	overwrites the whole state each step	–
LSTM output gate (selective **write**)	$o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o)$	$o_t \in (0,1)^d$	from h_{t-1}, not s_{t-1}	fraction of the state passed on	calling it the read gate
LSTM input gate (selective **read**)	$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$			how much of $\tilde{s}_t$ to read	–
LSTM forget gate (selective **forget**)	$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$			how much of s_{t-1} to keep	thinking $f = 1$ means "forget all" (it means keep all)
LSTM candidate state	$\tilde{s}_t = \sigma(W h_{t-1} + U x_t + b)$	W, U, b as in the RNN	slides use σ (standard: $\tanh$)	new information from h_{t-1}, x_t	–
LSTM cell state	$s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$		element-wise	additive update (key for gradient flow)	using matrix multiplication
LSTM output	$h_t = o_t \odot \sigma(s_t)$; $rnn_{out} = h_t$		slides use $\sigma(s_t)$	selectively written view of the state	–
LSTM compact form	$h_t, s_t = \mathrm{LSTM}(h_{t-1}, s_{t-1}, x_t)$			carries two vectors	–
GRU gates	$o_t = \sigma(W_o s_{t-1} + U_o x_t + b_o)$, $i_t = \sigma(W_i s_{t-1} + U_i x_t + b_i)$		from s_{t-1} directly	$o \approx$ reset, $i \approx$ update	saying "from h_{t-1}"
GRU candidate	$\tilde{s}_t = \sigma(W(o_t \odot s_{t-1}) + U x_t + b)$			gated old state enters the candidate	–
GRU state (L14 slide 26)	$s_t = (1 - i_t) \odot s_{t-1} + i_t \odot \tilde{s}_t$		forget tied to input	no explicit forget gate	L15 slide 8 swaps this: $i_t \odot s_{t-1} + (1 - i_t) \odot \tilde{s}_t$ (verified on both slides; both conventions also exist in the literature)
Compact forms (L15 slide 8)	$s_t = \mathrm{RNN}(s_{t-1}, x_t)$; $s_t = \mathrm{GRU}(s_{t-1}, x_t)$; $h_t, s_t = \mathrm{LSTM}(h_{t-1}, s_{t-1}, x_t)$			LSTM carries two vectors	–

L14: Parameter counts (derived, not on the slides) · Week 11

Name	Expression	Interpretation
One (W, U, b) block	$d^2 + dn + d = d(d + n + 1)$	–
RNN / GRU / LSTM cell	$1\times$ / $3\times$ / $4\times$ $d(d + n + 1)$	ratio LSTM:GRU:RNN = 4:3:1
Output layer	$kd + k$ ($V \in \mathbb{R}^{k \times d}$, bias c)	same for all three models

Examples: $d = 256, n = 100$ gives LSTM 365,568 and GRU 274,176. $d = 10, n = 5$ gives LSTM 640 and GRU 480.

L14: Gradient flow · Week 11

Name	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
RNN BPTT (recall)	$\frac{\partial \mathcal{L}_t}{\partial W} = \frac{\partial \mathcal{L}_t}{\partial s_t} \sum_{k=1}^{t} \left(\prod_{j=k}^{t-1} \frac{\partial s_{j+1}}{\partial s_j} \right) \frac{\partial^+ s_k}{\partial W}$			the product causes vanishing	–
Path rule	vanishes ⇔ vanishes on **every** path; explodes ⇐ explodes on **at least one** path			one good path is enough to avoid vanishing	swapping "every" and "one"
$\partial h_t / \partial s_t$	$\mathcal{D}(o_t \odot \sigma'(s_t))$	$d \times d$ diagonal	h_{ti} depends only on o_{ti}, s_{ti}	–	treating it as a full matrix
$\partial s_t / \partial s_{t-1}$ (explicit)	$\mathcal{D}(f_t)$		implicit term assumed to vanish (worst case)	–	–
Path t_0 (cell path)	$t_0 = \mathcal{L}_t'(h_t)\, \mathcal{D}(o_t \odot \sigma'(s_t))\, \mathcal{D}(\bigodot_{i=k+1}^{t} f_i)$			vanishes only if the forget gates close, which is "fair"	–
Path t_1 (output-gate path)	$t_1 = \mathcal{L}_t'(h_t)\, (\mathcal{D}(\sigma(s_t) \odot o_t')W_o) \cdots (\mathcal{D}(\sigma(s_k) \odot o_k')W_o)$	one term of $\partial \mathcal{L}_t / \partial h_{k-1}$	factors ordered from step t down to k	repeated W_o can make it explode (W_i, W_f, W similarly)	–
Exploding bound	$\lvert t_1 \rvert \le \lvert \mathcal{L}_t'(h_t) \rvert (\lvert K \rvert \lvert W_o \rvert)^{t-k+1}$	K: not defined on slide; read as bound on the diagonal factors		grows if $\lvert K \rvert \lvert W_o \rvert > 1$	exponent is $t - k + 1$ (one factor per step $k..t$)
Gradient clipping (Pascanu et al. 2013)	if $\lvert g \rvert > \tau$: $g \leftarrow \tau\, g / \lvert g \rvert$	τ = threshold	only when the norm exceeds τ	keeps the direction, scales the norm	thinking it clips each element or changes direction

L15: Language model and encoder–decoder · Week 12

Name	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
LM objective	$y^* = \arg\max P(y_t \mid y_1^{t-1})$			predict the next word	–
RNN LM output	$P(y_t = j \mid y_1^{t-1}) = P(y_t = j \mid s_t) = \mathrm{softmax}(V s_t + c)_j$, $s_t = \sigma(W s_{t-1} + U x_t + b)$	$j \in \mathcal{V}$; params U, V, W, b, c	s_0 randomly initialised and learned	softmax over the vocabulary	thinking s_0 is computed
LM input	$x_t = e(v_j)$, $j = \arg\max$ word at $t-1$		one-hot, or in practice a pre-trained embedding	–	–
Sequence loss	$\mathcal{L}(\theta) = \sum_t \mathcal{L}_t = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^{t-1}, x)$	ℓ_t = true word		cross-entropy summed over time	slide writes the index as $\sum_{i=1}^{T}$
Single-label loss (Image QA, video classification)	$-\log P(y = \ell \mid \cdot)$			one term	–

Name	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Captioning Opt 1 / Opt 2	$s_0 = fc_7(I)$ / $s_t = \text{RNN}(s_{t-1}, [x_t, fc_7(I)])$			encoding at the start only / at every step	–
Seq2seq encoder–decoder	$h_t = \text{RNN}(h_{t-1}, x_{it})$; $s_0 = h_T$; $s_t = \text{RNN}(s_{t-1}, e(\hat{y}_{t-1}))$ [Opt 2: $[h_T, e(\hat{y}_{t-1})]$]; $P = \text{softmax}(Vs_t + b)$			MT, entailment, transliteration, summarization, dialog	–
Image QA encoder	$\hat{h}_I = \text{CNN}(I)$, $\tilde{h}_t = \text{RNN}(\tilde{h}_{t-1}, q_{it})$, $s = [\tilde{h}_T; \hat{h}_I]$; $P(y \mid q, I) = \text{softmax}(Vs + b)$			concatenates two encoders	–
Video encoder	$h_t = \text{RNN}(h_{t-1}, \text{CNN}(x_{it}))$	x_{it} = frame		per-frame CNN, then RNN	–

L15: Attention · Week 12

Name	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Score	$e_{jt} = f_{ATT}(s_{t-1}, c_j) = V_{att}^T \tanh(U_{att} s_{t-1} + W_{att} c_j)$	$V_{att} \in \mathbb{R}^d$; $U_{att}, W_{att} \in \mathbb{R}^{d \times d}$; c_j/h_j = encoder state j	slide 43 writes $V_{attn}^T \tanh(U_{attn} h_j + W_{attn} s_t)$	importance of input j for output t (a scalar)	using s_t (circular); confusing c_j with c_t
Weights	$\alpha_{jt} = \exp(e_{jt}) / \sum_{j=1}^{M} \exp(e_{jt})$	M = # input words	$\sum_j \alpha_{jt} = 1$	probability of focusing on word j at step t	normalising over t instead of j
Context	$c_t = \sum_{j=1}^{T} \alpha_{jt} h_j$		$c_t \in \mathbb{R}^d$	weighted average of encoder states	–
Decoder with attention	$s_t = \text{RNN}(s_{t-1}, [e(\hat{y}_{t-1}), c_t])$, $\ell_t = \text{softmax}(Vs_t + b)$			a step-specific context each step	–
Attention parameters (derived)	$d + 2d^2$		slide-38 dims ($V_{att} \in \mathbb{R}^d$, $U_{att}, W_{att} \in \mathbb{R}^{d \times d}$)	independent of sequence length	slide 43 parameter list omits W_{attn}
MT-with-attention parameters (slide 43)	$U_{dec}, V, W_{dec}, U_{enc}, W_{enc}, b, U_{attn}, V_{attn}$ (+ W_{attn})			loss and algorithm unchanged	–
Image attention context	$c_t = \sum_{j=1}^{196} \alpha_{tj} a_j$, $a_j \in \mathbb{R}^{512}$	a_j = feature of location j	conv map, not fc7	weighted average of region features	(formula is an additional clarification; slide shows it as a figure)

Name	Expression	Variables	Conditions	Interpretation	⚠ Likely mistake
Optional alignment loss	$\mathcal{L}(\alpha^{true}, \alpha^{pred}) + \mathcal{L}(\theta)$		only if true α's exist (rare)	not used in practice	thinking attention needs it
Image attention	conv map $H \times W \times C$ gives $H \cdot W$ locations of dim C	VGG conv5: $14 \times 14 \times 512$		196 locations	multiplying by C
Hierarchical encoder	$h^1_{ij} = \mathrm{RNN}(h^1_{ij-1}, w_{ij})$, $s_i = h^1_{iT_i}$; $h^2_i = \mathrm{RNN}(h^2_{i-1}, s_i)$, $s = h^2_K$; $P(y \mid doc) = \mathrm{softmax}(Vs + b)$	T_i = sentence length, K = # sentences	params $W^1_{enc}, U^1_{enc}, W^2_{enc}, U^2_{enc}, V, b$	sequence of sequences	–
HAN word attention	$u_{ij} = \tanh(W_w h_{ij} + b_w)$, $\alpha_{ij} = \frac{\exp(u_{ij}^T u_w)}{\sum_t \exp(u_{it}^T u_w)}$, $s_i = \sum_j \alpha_{ij} h_{ij}$	u_w = learned word context vector		important words per sentence	–
HAN sentence attention	$u_i = \tanh(W_s h_i + b_s)$, $\alpha_i = \frac{\exp(u_i^T u_s)}{\sum_i \exp(u_i^T u_s)}$, $s = \sum_i \alpha_i h_i$; $P(y \mid doc) = \mathrm{softmax}(Vs + b)$	u_s = learned sentence context vector		important sentences	the slide's parameter list omits the RNN weights

Part IV · Practice MCQs

This bank has 213 practice questions, roughly 8 to 14 per lecture. They mix conceptual, theory and numerical questions. Each question says whether it is single-correct or multiple-correct. Every numerical answer was checked by computation.

How to use it: cover the **Answer** line and attempt the question. When you get one wrong, re-read the matching section of Part I. Its traps section (F) usually explains the mistake.

These questions were written for practice from the lecture content. They are not past exam questions and make no prediction about what the exam will ask.

Weeks 1–2 · Lectures 1–3

Lecture 1: History of Deep Learning · Week 1

Q1. Who proposed that the nervous system is a single continuous network rather than a network of discrete cells (reticular theory)? (a) Santiago Ramón y Cajal (b) Joseph von Gerlach (c) Waldeyer-Hartz (d) Hubel and Wiesel **Answer: (b).** Gerlach proposed the reticular theory (1871–73) and Golgi supported it. Cajal proposed the neuron doctrine, and Waldeyer-Hartz coined the term "neuron".

Q2. Which technology finally confirmed the neuron doctrine in the 1950s? (a) Golgi staining (b) Electron microscopy (c) fMRI (d) X-ray crystallography **Answer: (b).** Electron microscopy showed that nerve cells are individual cells connected through synapses.

Q3. Match the correct pairs. *(Multiple correct)* (a) 1943: McCulloch and Pitts neuron (b) 1969: Minsky and Papert, *Perceptrons* (c) 1986: Werbos first used backprop for ANNs (d) 1847: Cauchy, gradient descent **Answer: (a), (b), (d).** Werbos used backprop for ANNs in **1982**. Rumelhart et al. popularized it in 1986.

Q4. The idea of unsupervised pre-training first appeared in (a) Hinton and Salakhutdinov, 2006 (b) Schmidhuber's "Very Deep Learner", 1991–1993 (c) LeCun, 1989 (d) Rumelhart, 1986 **Answer: (b).** The 2006 work popularized pre-training for deep autoencoders, but the idea dates back to 1991–93.

Q5. According to the slides, which ImageNet-winning network had 152 layers and an error of 3.6%? (a) VGGNet (b) GoogLeNet (c) MS ResNet (d) ZFNet **Answer: (c).** For reference: AlexNet 8 layers, 16.0%; ZFNet 8, 11.2%; VGG 19, 7.3%; GoogLeNet 22, 6.7%.

Q6. *(Numerical recall)* By how many layers does GoogLeNet exceed VGGNet, and by how many percentage points is its error lower? (a) 3 layers, 0.6 points (b) 14 layers, 4.5 points (c) 3 layers, 4.5 points (d) 19 layers, 0.6 points **Answer: (a).** 22 − 19 = 3 layers, and 7.3% − 6.7% = 0.6 percentage points.

Q7. Which statement about sequence models is correct? (a) A Jordan network feeds the hidden state to the next time step (b) An Elman network feeds the output state to the next time step (c) An Elman network feeds the hidden state to the next time step (d) The Hopfield network was introduced in 1997 **Answer: (c).** Jordan (1986) feeds back the output, Elman (1990) feeds back the hidden state, and Hopfield dates from 1982. LSTMs came in 1997.

Q8. Hochreiter et al. and Bengio et al. (1991–94) showed that RNNs are difficult to train because of (a) overfitting (b) exploding and vanishing gradients (c) lack of GPUs (d) non-convex losses only **Answer: (b).**

Q9. The Hubel and Wiesel experiment (1959) showed that (a) neurons fire in response to stimuli anywhere in the visual field (b) each neuron has a fixed receptive field (c) cats can recognize digits (d) the visual cortex is not hierarchical **Answer: (b).** This led to the Neocognitron (1980), LeCun's CNN (1989) and LeNet-5/MNIST (1998).

Q10. *(Numerical recall)* Ciresan et al.'s first superhuman visual pattern recognition result (IJCNN traffic signs, 2011) had an error rate of (a) 0.56% (b) 3.6% (c) 16.0% (d) 23.2% **Answer: (a).** The 16.0% and 23.2% figures are Dahl et al.'s speech relative error reductions (2010). 3.6% is ResNet.

Q11. Which of the following are listed as part of the "paradox of deep learning"? *(Multiple correct)* (a) High capacity (b) Numerical instability (c) Sharp minima (d) Guaranteed convexity of the loss **Answer: (a), (b), (c).** Non-robustness (the adversarial panda → gibbon example) is the fourth item. Convexity is not listed.

Lecture 2: MP Neuron, Perceptron, PLA, MLP · Week 1

Q1. An MP neuron has 4 excitatory inputs and θ = 2. For how many of the 16 possible input combinations is y = 1? (a) 6 (b) 10 (c) 11 (d) 5 **Answer: (c).** y = 1 when at least 2 inputs are 1: C(4,2) + C(4,3) + C(4,4) = 6 + 4 + 1 = 11.

Q2. In an MP neuron implementing NOR of x_1, x_2, the inputs are ___ and θ = ___. (a) excitatory, 0 (b) inhibitory, 0 (c) inhibitory, 1 (d) excitatory, 2 **Answer: (b).** Any active inhibitory input forces 0. When both inputs are 0, the sum is 0 ≥ 0, so the neuron fires.

Q3. A perceptron has $w_0 = -1, w_1 = 2, w_2 = -1$ (with $x_0 = 1$). What is the output for $(x_1, x_2) = (1, 1)$? (a) 0 (b) 1 (c) undefined (d) −1 **Answer: (b).** −1 + 2 − 1 = 0 ≥ 0 gives y = 1. A point on the boundary is classified as 1.

Q4. During PLA, $\mathbf{w} = [0.5, -1, 1]$ and a **positive** point $\mathbf{x} = [1, 2, 1]$ (with $x_0 = 1$) is picked. What is $\mathbf{w}$ after this step, and what is the new $\mathbf{w}^T\mathbf{x}$? (a) [−0.5,−3,0], −6.5 (b) [1.5,1,2], 5.5 (c) unchanged, −0.5 (d) [1.5,1,2], −0.5 **Answer: (b).** $\mathbf{w}^T\mathbf{x} = 0.5 - 2 + 1 = -0.5 < 0$, so the positive point is misclassified and $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x} = [1.5, 1, 2]$. The new dot product is 1.5 + 2 + 2 = 5.5, an increase of exactly $\|\mathbf{x}\|^2 = 6$.

Q5. For the OR function with $w_0 = -1$, $w_1 = 0.5$, $w_2 = 1.5$, how many of the 4 inputs are misclassified? (a) 0 (b) 1 (c) 2 (d) 3 **Answer: (b).** The sums for (0,0), (0,1), (1,0), (1,1) are −1, 0.5, −0.5, 1. Only (1,0) is wrong.

Q6. How many boolean functions of 3 inputs exist, and how many hidden perceptrons does the slides' construction need to represent any of them? (a) 8, 3 (b) 256, 8 (c) 64, 8 (d) 256, 3 **Answer: (b).** There are $2^{2^3} = 256$ functions, and the construction uses $2^3 = 8$ hidden perceptrons (each with bias −3) plus 1 output perceptron.

Q7. Of the 16 boolean functions of 2 inputs, how many are linearly separable? (a) 16 (b) 15 (c) 14 (d) 12 **Answer: (c).** All except XOR and XNOR (!XOR).

Q8. Which statements about the perceptron convergence proof are correct? *(Multiple correct)* (a) It assumes the sets P and N are finite and linearly separable (b) It bounds the number of corrections k (c) The lower bound on cos β grows proportionally to $\sqrt{k}$ (d) It shows that PLA finds the maximum-margin separator **Answer: (a), (b), (c).** Since cos β ≤ 1, k is finite. The proof says nothing about which separator is found, so (d) is false.

Q9. In the convergence proof, why are the points of N negated? (a) To normalize them (b) So that one condition, $\mathbf{w}^T\mathbf{p} \geq 0$ for all $\mathbf{p} \in P'$, and one update rule suffice (c) To make the data linearly separable (d) To remove the bias **Answer: (b).** $\mathbf{w}^T\mathbf{x} < 0 \iff \mathbf{w}^T(-\mathbf{x}) > 0$. Normalization ($\|\mathbf{p}\| = 1$) is a separate step.

Q10. In the 2-input network that represents any boolean function (True = +1, False = −1, hidden bias −2, hidden weights ±1), the input (1,−1) is fed to the hidden perceptron with weights (+1,+1). What does it output?

(a) 1 (b) 0 (c) −1 (d) 2 **Answer: (b).** The weighted sum is 1 − 1 = 0, and 0 − 2 < 0, so it does not fire. Only the unit with weights (+1,−1) fires for this input (sum 2 ≥ 2).

Q11. Which statements are true? *(Multiple correct)* (a) The weight vector **w** is perpendicular to the decision boundary $\mathbf{w}^T\mathbf{x} = 0$ (b) A single perceptron can implement XOR if the weights are learned long enough (c) MP neurons cannot learn their threshold (d) 2^n hidden perceptrons are necessary for every boolean function of n inputs **Answer: (a), (c).** (b) is false because XOR is not linearly separable. (d) is false because 2^n is sufficient, not necessary; AND needs only 1 perceptron.

Q12. In XOR's inequalities for a single perceptron ($w_0 < 0$, $w_2 \geq -w_0$, $w_1 \geq -w_0$, $w_1 + w_2 < -w_0$), which condition is contradicted? (a) The first by the fourth (b) The fourth by the second and third (c) The second by the third (d) None; XOR is separable **Answer: (b).** From the 2nd and 3rd, $w_1 + w_2 \geq -2w_0 > -w_0$, which contradicts the 4th.

Q13. In the lecture's illustration of hierarchical processing (visual cortex / Larochelle example), which ordering of what successive layers detect is correct? (a) Faces → feature groups → edges (b) Edges and corners → feature groups (nose, mouth, eyes) → high-level objects such as faces (c) Feature groups → edges → faces (d) All layers detect faces at different scales **Answer: (b).** Lower layers (V1) handle simple forms; higher layers (up to AIT) handle high-level objects.

Q14. *(Numerical)* Using the OR inequalities with $w_0 = -1$, which (w_1, w_2) gives **0** errors? (a) (−1, −1) (b) (1.5, 0) (c) (0.45, 0.45) (d) (1.1, 1.1) **Answer: (d).** Error counts are 3, 1, 3, 0 respectively. (0.45, 0.45) fails on (1,1) because −1 + 0.9 = −0.1 < 0.

Lecture 3: Sigmoid Neurons, GD, Representation Power · Week 2

Q1. For $\sigma(wx + b)$ with $w = 2$ and $b = -4$, at what x does the output equal 0.5? (a) −2 (b) 2 (c) 0.5 (d) 4 **Answer: (b).** The transition is at $x = -b/w = 4/2 = 2$.

Q2. With data (0.5, 0.2) and (2.5, 0.9), what is $\mathscr{L}(w, b) = \frac{1}{2}\sum(y_i - f(x_i))^2$ at $w = 0, b = 0$? (a) 0.25 (b) 0.125 (c) 0.073 (d) 0.5 **Answer: (b).** f = 0.5 at both points, so $\frac{1}{2}[(0.2 - 0.5)^2 + (0.9 - 0.5)^2] = \frac{1}{2}(0.09 + 0.16) = 0.125$.

Q3. Single point $x = 1$, $y = 0$, sigmoid neuron at $w = 0$, $b = 0$, loss $\frac{1}{2}(f(x) - y)^2$, η = 0.4. What is w after one GD step? (a) 0.05 (b) −0.05 (c) −0.125 (d) 0.125 **Answer: (b).** $f = 0.5$, so $\nabla w = (0.5 - 0)(0.5)(0.5)(1) = 0.125$ and $w = 0 - 0.4(0.125) = -0.05$.

Q4. Single point $x = 1$, $y = 1$, $w = 0.5$, $b = 0$. What is ∇w (to 3 decimals)? (a) −0.089 (b) 0.089 (c) −0.378 (d) −0.235 **Answer: (a).** $f = \sigma(0.5) = 0.6225$, so $(0.6225 - 1)(0.6225)(0.3775)(1) = -0.0887$. The gradient is negative, so GD increases w.

Q5. $\mathscr{L}(\boldsymbol{\theta}) = 2$ and $\nabla\mathscr{L}(\boldsymbol{\theta}) = [3, -1]$. Using the first-order Taylor approximation, estimate $\mathscr{L}$ after a step $\boldsymbol{\theta} - \eta\nabla\mathscr{L}$ with η = 0.01. (a) 2.10 (b) 1.97 (c) 1.90 (d) 1.99 **Answer: (c).** $\mathscr{L} + \eta\mathbf{u}^T\nabla\mathscr{L}$ with $\mathbf{u} = -\nabla\mathscr{L}$ gives $2 - 0.01(9 + 1) = 1.90$.

Q6. In the derivation of gradient descent, the move $\eta\mathbf{u}$ reduces the loss (to first order) if and only if (a) $\mathbf{u}^T\nabla\mathscr{L} > 0$ (b) $\mathbf{u}^T\nabla\mathscr{L} < 0$ (c) $\mathbf{u} \perp \nabla\mathscr{L}$ (d) $\|\mathbf{u}\| = 1$ **Answer: (b).** The decrease is largest when the angle β between **u** and the gradient is 180°. If **u** is perpendicular to the gradient, there is no first-order change.

Q7. Why can the second- and higher-order Taylor terms be ignored in the GD derivation? (a) The Hessian is always zero (b) η is small, so η^2, η^3, … → 0 (c) The loss is linear (d) The gradient is normalized **Answer: (b).**

Q8. Which properties does the sigmoid (logistic) neuron have and the perceptron lack? *(Multiple correct)* (a) Smooth (b) Continuous (c) Differentiable (d) Output restricted to {0,1} **Answer: (a), (b), (c).** The sigmoid's output lies in (0,1). The {0,1} output belongs to the perceptron.

Q9. In $\sigma(wx+b)$, increasing w (with b/w fixed) makes the curve ___, and changing b ___. (a) smoother; changes steepness (b) steeper, approaching a step; shifts the transition point (c) flatter; flips the sign (d) unchanged; scales the output **Answer: (b).**

Q10. Which statement correctly states the Universal Approximation Theorem as given in the lecture? (a) Any function can be represented exactly by a single hidden layer of perceptrons (b) A network with a single hidden layer of (sigmoid) neurons can approximate any continuous function $\mathbb{R}^n \to \mathbb{R}^m$ to any desired precision (c) Two hidden layers are necessary for any continuous function (d) The UAT gives the exact number of neurons needed **Answer: (b).** It is an existence result for "enough" neurons (Cybenko 1989, Hornik 1991). The 2-D tower construction uses two hidden layers only as a proof device.

Q11. In the tower-function construction, how many first-layer sigmoid neurons are needed per tower for 1-D input, and for 2-D input? (a) 1 and 2 (b) 2 and 4 (c) 2 and 3 (d) 4 and 8 **Answer: (b).** In 1-D, one step minus another step needs 2 neurons. In 2-D, two open towers need 4 neurons, followed by an add and a final sigmoid. *(Additional clarification)* The pattern gives 2n in n dimensions.

Q12. Approximating a 1-D function with 10 towers in the slides' construction needs how many neurons in the first hidden layer? (a) 10 (b) 20 (c) 40 (d) 11 **Answer: (b).** Each tower uses 2 sigmoid neurons, and 10 × 2 = 20.

Q13. Which are components of the "typical supervised ML setup" in Lecture 3? *(Multiple correct)* (a) Data $\{x_i, y_i\}$ (b) Model (c) Learning algorithm (d) Objective/loss function **Answer: (a), (b), (c), (d).** Parameters are also listed, e.g. **w** in $\hat{y} = \sigma(\mathbf{w}^T\mathbf{x})$, $\mathbf{w}^T\mathbf{x}$ or $\mathbf{x}^T W\mathbf{x}$.

Q14. *(Numerical)* In the slides' 2-D tower network, the final sigmoid is $\sigma(50s - 100)$, where s is the sum of the two open towers (sharp steps). What is its output at the centre of the tower ($s = 2$) and on the elevated base ($s = 1$)? (a) 1 and 0.5 (b) 0.5 and ≈0 (c) ≈1 and ≈0 (d) 0.5 and 0.5 **Answer: (b).** $\sigma(100 - 100) = \sigma(0) = 0.5$ and $\sigma(50 - 100) = \sigma(-50) \approx 0$. The final sigmoid removes the base.

Q15. In the slides' GD demo (data (0.5, 0.2), (2.5, 0.9); start $w = b = -2$; η = 1), the initial gradients are about $\nabla w \approx -0.0055$ and $\nabla b \approx -0.0077$. Why are they so small even though the loss (≈0.416) is high? (a) η is too large (b) The start lies on a flat (saturated) region of the error surface, where $f(1-f) \approx 0$ (c) The loss has no ½ factor (d) GD has already converged **Answer: (b).** At $w = b = -2$ both outputs are close to 0, so the sigmoid slope $f(1-f)$ is tiny. Progress is slow at first, but after 1000 epochs GD reaches $w \approx 1.79$, $b \approx -2.28$.

Weeks 3–4 · Lectures 4–5

Lecture 4: Feedforward Networks & Backpropagation · Week 3

Q1. (Numerical, single correct) A feedforward network has 100 inputs, two hidden layers of 50 neurons each and 10 output neurons (fully connected, with biases). Total number of learnable parameters?

- (a) 8000 (b) 8110 (c) 8100 (d) 8210

Answer: (b). Weights $100 \cdot 50 + 50 \cdot 50 + 50 \cdot 10 = 8000$; biases $50 + 50 + 10 = 110$. Option (a) forgets biases; the input layer has no biases.

Q2. (Numerical, single correct) In the slide's network: n-dimensional input, $L-1$ hidden layers of n neurons each, k output neurons. For $n=10$, $L=3$, $k=4$, the number of parameters is

- (a) 220 (b) 244 (c) 264 (d) 284

Answer: (c). $(L-1)(n^2+n)+nk+k = 2(110)+40+4 = 264$. $L=3$ means 2 hidden layers + output layer.

Q3. (Numerical, single correct) Softmax output with cross-entropy loss; output pre-activations $\mathbf{a}_L = [1, 2, 3]$, true class $\ell = 3$. $\nabla_{\mathbf{a}_L}\mathscr{L}$ is approximately

- (a) $[0.090, 0.245, 0.665]$ (b) $[-0.090, -0.245, 0.335]$ (c) $[0.090, 0.245, -0.335]$ (d) $[0, 0, -1.503]$

Answer: (c). $\hat{\mathbf{y}} = \mathrm{softmax}([1,2,3]) = [0.090, 0.245, 0.665]$; $\nabla_{\mathbf{a}_L} = \hat{\mathbf{y}} - \mathbf{e}(\ell)$. (b) has the sign flipped; (d) is $\nabla_{\hat{\mathbf{y}}} = -\mathbf{e}(\ell)/\hat{y}_\ell$.

Q4. (Numerical, single correct) A classifier outputs $\hat{\mathbf{y}} = [0.659, 0.242, 0.099]$ and the true class is the first one. The cross-entropy loss (natural log) is

- (a) 0.417 (b) 1.419 (c) 2.317 (d) 0.181

Answer: (a). $\mathscr{L} = -\log \hat{y}_\ell = -\ln 0.659 = 0.417$. Only the true-class probability matters for one-hot targets.

Q5. (Numerical, single correct) At layer k, $\nabla_{\mathbf{a}_k}\mathscr{L} = [0.2, -0.1]^T$ and $\mathbf{h}_{k-1} = [1, 2, 3]^T$. The entry in row 2, column 3 of $\nabla_{W_k}\mathscr{L}$ (with $\mathbf{a}_k = W_k\mathbf{h}_{k-1} + \mathbf{b}_k$) is

- (a) 0.6 (b) −0.3 (c) −0.2 (d) 0.3

Answer: (b). $\nabla_{W_k} = \nabla_{\mathbf{a}_k}\mathbf{h}_{k-1}^T$, entry $(i,j) = \frac{\partial \mathscr{L}}{\partial a_{ki}} h_{k-1,j} = (-0.1)(3) = -0.3$.

Q6. (Numerical, single correct) $W_k = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$, $\nabla_{\mathbf{a}_k}\mathscr{L} = [0.1, -0.2, 0.3]^T$ and $g'(\mathbf{a}_{k-1}) = [0.25, 0.16]^T$. Then $\nabla_{\mathbf{a}_{k-1}}\mathscr{L}$ is

- (a) $[1.0, 1.2]$ (b) $[0.25, 0.192]$ (c) $[0.025, -0.032]$ (d) $[0.4, 0.2]$

Answer: (b). $\nabla_{\mathbf{h}_{k-1}} = W_k^T \nabla_{\mathbf{a}_k} = [0.1 - 0.6 + 1.5,\ 0.2 - 0.8 + 1.8] = [1.0, 1.2]$; then $\odot g'$: $[0.25, 0.192]$. (a) stops before multiplying by g'.

Q7. (Numerical, multiple correct) Which are correct?

- (a) If $\sigma(z) = 0.8$ then $\sigma'(z) = 0.16$
- (b) If $\tanh(z) = 0.5$ then $\tanh'(z) = 0.5$
- (c) The maximum value of $\sigma'(z)$ is 0.25
- (d) The maximum value of $\tanh'(z)$ is 1

Answer: (a), (c), (d). $\sigma' = \sigma(1-\sigma) = 0.16$, max 0.25 at $z=0$; $\tanh' = 1 - \tanh^2 = 0.75$ (so (b) false), max 1 at $z = 0$.

Q8. (Conceptual, single correct) We want to predict three real-valued movie ratings (e.g. 7.5, 8.2, 7.7). The most appropriate output activation / loss pair is

- (a) Softmax / cross-entropy (b) Logistic / squared error (c) Linear / squared error (d) Linear / cross-entropy

Answer: (c). Real outputs → linear output (unbounded) + squared error. Logistic restricts outputs to (0,1); softmax/CE is for probability outputs.

Q9. (Theory, multiple correct) With $\mathbf{a}_k = \mathbf{b}_k + W_k\mathbf{h}_{k-1}$, $\mathbf{h}_k = g(\mathbf{a}_k)$, which are correct?

- (a) $\nabla_{\mathbf{b}_k}\mathscr{L} = \nabla_{\mathbf{a}_k}\mathscr{L}$
- (b) $\nabla_{\mathbf{h}_{k-1}}\mathscr{L} = W_k^T\nabla_{\mathbf{a}_k}\mathscr{L}$
- (c) $\nabla_{\mathbf{a}_{k-1}}\mathscr{L} = W_{k-1}^T\nabla_{\mathbf{h}_{k-1}}\mathscr{L}$
- (d) $\nabla_{W_k}\mathscr{L}$ has the same shape as W_k

Answer: (a), (b), (d). (c) is wrong: $\nabla_{\mathbf{a}_{k-1}} = \nabla_{\mathbf{h}_{k-1}} \odot g'(\mathbf{a}_{k-1})$ (element-wise, no weight matrix).

Q10. (Conceptual, single correct) Why does $\frac{\partial\mathscr{L}}{\partial h_{ij}}$ involve a **sum** $\sum_m \frac{\partial\mathscr{L}}{\partial a_{i+1,m}}W_{i+1,m,j}$?

- (a) Because the loss is a sum over training examples
- (b) Because h_{ij} influences the loss through every unit $a_{i+1,m}$ of the next layer (chain rule along multiple paths)
- (c) Because softmax normalises over all classes
- (d) Because biases are shared across units

Answer: (b). Multi-path chain rule: $\frac{\partial p}{\partial z} = \sum_m \frac{\partial p}{\partial q_m}\frac{\partial q_m}{\partial z}$ with $q_m = a_{i+1,m}$.

Q11. (Theory, single correct) Layer $k-1$ has 300 units and layer k has 100 units ($\mathbf{a}_k = W_k\mathbf{h}_{k-1} + \mathbf{b}_k$). The shapes of $\nabla_{W_k}\mathscr{L}$, $\nabla_{\mathbf{b}_k}\mathscr{L}$, $\nabla_{\mathbf{h}_{k-1}}\mathscr{L}$ are

- (a) 300×100, 300, 100 (b) 100×300, 100, 300 (c) 100×300, 300, 100 (d) 100×100, 100, 300

Answer: (b). $\nabla_{\mathbf{a}_k}\mathbf{h}_{k-1}^T$ is $(100 \times 1)(1 \times 300)$; bias gradient = size of layer k; $W_k^T\nabla_{\mathbf{a}_k}$ has size of layer $k-1$.

Q12. (Conceptual, single correct) For a k-class classification problem with one-hot label $\mathbf{y}$, which statement is FALSE?

- (a) Softmax makes $\hat{\mathbf{y}}$ a probability distribution
- (b) Minimising cross-entropy is the same as maximising $\log\hat{y}_\ell$ (log-likelihood)
- (c) Cross-entropy loss directly depends on the predicted probabilities of all wrong classes
- (d) $\hat{y}_\ell$ is a function of all parameters $W_1, \ldots, W_L, \mathbf{b}_1, \ldots, \mathbf{b}_L$

Answer: (c). With one-hot $\mathbf{y}$, $\mathscr{L} = -\log\hat{y}_\ell$; wrong-class probabilities affect it only indirectly through the softmax normalisation.

Lecture 5: Optimization Algorithms · Week 4

Q13. (Numerical, single correct) $N = 1000$ training points, mini-batch size $B = 50$, trained for 10 epochs. Number of parameter updates for SGD and mini-batch GD respectively:

- (a) 1000, 20 (b) 10000, 200 (c) 10, 200 (d) 10000, 500

Answer: (b). SGD: N steps/epoch → 10×1000; mini-batch: $N/B = 20$ per epoch → 200. (Batch GD would make 10.)

Q14. (Numerical, single correct) Momentum GD with $\eta = 0.1$, $\gamma = 0.9$, update$_0=0$, and the gradient equals 1 at every step. update$_3$ is

- (a) 0.3 (b) 0.271 (c) 0.19 (d) 0.243

Answer: (b). $0.1 \to 0.9(0.1) + 0.1 = 0.19 \to 0.9(0.19) + 0.1 = 0.271$. (Limit: $\eta/(1-\gamma) = 1$.)

Q15. (Numerical, single correct) NAG on $\mathscr{L}(w) = w^2$ with $\eta = 0.1$, $\gamma = 0.9$. At step t: $w_t = 0.8$, update$_{t-1}=0.2$. Then w_{t+1} is

- (a) 0.46 (b) 0.496 (c) 0.62 (d) 0.64

Answer: (b). $w_{la} = 0.8 - 0.9(0.2) = 0.62$; $\nabla = 2(0.62) = 1.24$; update$_t=0.18+0.124=0.304$; $w_{t+1} = 0.8 - 0.304 = 0.496$. Momentum (gradient at 0.8) would give 0.46.

Q16. (Numerical, single correct) Adam with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.1$, $\epsilon \approx 0$, $w_0 = 1$, first gradient $g_1 = 0.5$. After the first update $w_1 =$

- (a) 0.95 (b) 0.9 (c) 0.684 (d) 0.5

Answer: (b). $m_1 = 0.05$, $v_1 = 0.00025$; $\hat{m}_1 = 0.5$, $\hat{v}_1 = 0.25$; step $= 0.1 \cdot 0.5/0.5 = 0.1$. With bias correction the first step is $\eta \cdot \mathrm{sign}(g_1)$. Without correction the step would be 0.316 (→ (c)).

Q17. (Numerical, single correct) $1/t$ decay with $\eta_0 = 0.1$, $k = 0.1$. Learning rate at step $t = 10$:

- (a) 0.01 (b) 0.0368 (c) 0.05 (d) 0.1

Answer: (c). $\eta = \eta_0/(1+kt) = 0.1/2$. (0.0368 is $\eta_0 e^{-kt}$, the exponential decay.)

Q18. (Numerical, single correct) AdaGrad ($\eta = 0.1$, $\epsilon \approx 0$, $v_0 = 0$) on a parameter whose gradient is 1 at every step. The effective learning rate $\eta/\sqrt{v_t}$ at $t = 4$ is

- (a) 0.1 (b) 0.025 (c) 0.05 (d) 0.0707

Answer: (c). $v_4 = 1+1+1+1 = 4$; $0.1/2 = 0.05$. It keeps decaying as $1/\sqrt{t}$ (at $t = 100$: 0.01).

Q19. (Numerical, single correct) With $m_0 = 0$ and identically distributed gradients, $\mathbb{E}[m_t] = (1-\beta_1^t)\mathbb{E}[g]$. For $\beta_1 = 0.9$, $\mathbb{E}[m_2]$ equals

- (a) $0.1\,\mathbb{E}[g]$ (b) $0.19\,\mathbb{E}[g]$ (c) $0.81\,\mathbb{E}[g]$ (d) $\mathbb{E}[g]$

Answer: (b). $1 - 0.9^2 = 0.19$. Hence the correction $\hat{m}_t = m_t/(1-\beta_1^t)$ gives $\mathbb{E}[\hat{m}_t] = \mathbb{E}[g]$.

Q20. (Numerical, single correct) Using Sutskever et al.'s schedule $\gamma_t = \min(1 - 2^{-1-\log_2(\lfloor t/250 \rfloor + 1)}, \gamma_{max})$ with $\gamma_{max} = 0.99$, γ_t at $t = 750$ is

- (a) 0.75 (b) 0.875 (c) 0.99 (d) 0.5

Answer: (b). $\lfloor 750/250 \rfloor + 1 = 4$, $\log_2 4 = 2$, $1 - 2^{-3} = 0.875 < 0.99$.

Q21. (Theory, multiple correct) Which statements about stochastic gradient descent (one point per update) are correct?

- (a) Each update is guaranteed to decrease the total training loss
- (b) It makes N updates per epoch
- (c) Oscillations arise because each point greedily pushes parameters in its own favourable direction
- (d) Using mini-batches (e.g. k = 16, 32, 64) gives better gradient estimates and fewer oscillations

Answer: (b), (c), (d). (a) holds only for batch GD (true gradient); SGD uses an approximate gradient with no per-step guarantee.

Q22. (Conceptual, single correct) On a contour plot of the loss, contour lines that are very close together indicate

- (a) a gentle slope (b) a steep slope (c) a minimum (d) a saddle point

Answer: (b). Small distance between contours = steep slope; large distance = gentle slope.

Q23. (Theory, single correct) Why does AdaGrad eventually stall near convergence on the toy problem (stops moving along b)?

- (a) Its v_t is an exponential moving average that forgets old gradients
- (b) Its v_t is a cumulative sum of squared gradients, so the effective learning rate of frequently-updated (dense) parameters decays towards 0
- (c) It lacks bias correction for m_t
- (d) Its learning rate grows for sparse parameters until it diverges

Answer: (b). RMSProp fixes this by replacing the sum with an EMA $v_t = \beta v_{t-1} + (1-\beta) g_t^2$.

Q24. (Theory, multiple correct) Which are correct about Adam as presented in the lecture?

- (a) Typical values are $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
- (b) β_1 is used for the running average of squared gradients
- (c) It combines RMSProp-style scaling with a cumulative (momentum-like) history of gradients
- (d) For sequence generation, $\eta = 0.001$ or 0.0001 was observed to work best

Answer: (a), (c), (d). β_1 goes with m_t (gradients), β_2 with v_t (squared gradients).

Q25. (Conceptual, single correct) What is the main drawback of line search as described in the lecture?

- (a) It cannot handle sparse features
- (b) It requires many more computations (loss evaluations) per step
- (c) It always oscillates more than momentum
- (d) It never converges faster than vanilla GD

Answer: (b). It tries several η values (e.g. 0.1, 0.5, 1, 5, 10) per step and keeps the best; it converges faster than vanilla GD.

Q26. (Conceptual, single correct) In the toy experiment where $x = 0$ for ~80% of points, GD/momentum/NAG initially move mostly along the b axis because

- (a) b has a larger learning rate
- (b) $\nabla w \propto x$ is zero for most points, so w receives few updates while b's "input" is always 1
- (c) the loss surface is flat along b
- (d) momentum only acts on biases

Answer: (b). $\nabla w = (f(x) - y) f(x)(1 - f(x))\, x$; sparse x ⇒ sparse updates of w. Adaptive methods (AdaGrad etc.) give w a larger effective learning rate.

Q27. (Numerical, single correct) Softmax output $\hat{\mathbf{y}} = [0.2, 0.3, 0.5]$. What is $\partial \hat{y}_1 / \partial a_{L,2}$?

- (a) 0.06 (b) −0.06 (c) 0.16 (d) −0.15

Answer: (b). $\partial \hat{y}_j / \partial a_{L,i} = \hat{y}_j(\delta_{ij} - \hat{y}_i)$, so for $i \neq j$ it is $-\hat{y}_1\hat{y}_2 = -0.2 \times 0.3 = -0.06$. (c) is the diagonal entry $\hat{y}_1(1-\hat{y}_1) = 0.16$.

Q28. (Numerical, single correct) Toy sigmoid neuron $f(x) = \sigma(wx + b)$, data $(0.5, 0.2), (2.5, 0.9)$, loss $\frac{1}{2}\sum(y_i - f(x_i))^2$. At $(w, b) = (0.5, 0)$, $\nabla w \approx -0.0085$ and $\nabla b \approx 0.0679$. After one batch-GD step with $\eta = 1$, (w, b) is approximately

- (a) $(0.4915, 0.0679)$ (b) $(0.5085, -0.0679)$ (c) $(0.5085, 0.0679)$ (d) $(0.4915, -0.0679)$

Answer: (b). $w - \eta\nabla w = 0.5 + 0.0085$ and $b - \eta\nabla b = -0.0679$. The loss drops from 0.0731 to 0.0686.

Q29. (Numerical, single correct) Adam with $\beta_2 = 0.999$, $v_0 = 0$, first gradient $g_1 = 2$. What are v_1 and $\hat{v}_1$?

- (a) 0.004, 4 (b) 4, 4 (c) 0.002, 2 (d) 0.004, 0.004

Answer: (a). $v_1 = (1-\beta_2)g_1^2 = 0.001 \times 4 = 0.004$; $\hat{v}_1 = v_1/(1-0.999^1) = 4 = g_1^2$. Without the correction, v would be heavily biased towards 0 early on.

Weeks 5–6 · Lectures 6–7

Lecture 6: Eigenvalues, EVD, PCA, SVD · Week 5

Q1 (numerical, single correct). What is the dominant eigenvalue of $A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$? (a) 4 (b) 5 (c) 2 (d) 7 **Answer: (b).**

- The characteristic equation is $\lambda^2 - 7\lambda + 10 = 0$, so $\lambda = 5, 2$.
- The dominant eigenvalue has the largest $|\lambda|$, which is 5.
- Option (d) is the trace, and (a) is a diagonal entry.

Q2 (numerical, single correct). Each day 80% of the Chinese-restaurant students stay ($p = 0.8$), and 90% of the Mexican-restaurant students stay ($q = 0.9$). There are 300 students in total. In the steady state, how many eat Chinese food? (a) 150 (b) 200 (c) 100 (d) 240 **Answer: (c).**

- The steady state is proportional to $[1-q,\ 1-p] = [0.1, 0.2]$.
- So the Chinese count is $300 \times 0.1/0.3 = 100$, and the Mexican count is 200 (verified by iterating M^n).
- The other eigenvalue is $p + q - 1 = 0.7$.

Q3 (conceptual, single correct). A is square with dominant eigenvalue $\lambda_d = 0.9$. For a generic x_0, what happens to the sequence $x_0, Ax_0, A^2x_0, \ldots$? (a) It explodes (b) It reaches a non-zero steady state (c) It vanishes (d) It oscillates without bound **Answer: (c).** $A^{p+n}x_0 = k\lambda_d^n e_d$. With $|\lambda_d| < 1$ this tends to 0. A steady state needs $|\lambda_d| = 1$.

Q4 (theory, multiple correct). Which of the following are true? (a) Eigenvectors of a matrix with distinct eigenvalues are linearly independent. (b) A square matrix always has an eigenvalue decomposition $A = U\Lambda U^{-1}$. (c) If A is symmetric and its eigenvectors are normalised, then $U^{-1} = U^T$. (d) Having n distinct eigenvalues is a necessary condition for A to be diagonalisable. **Answer: (a), (c).**

- (b) is false: you need n linearly independent eigenvectors.
- (d) is false: distinct eigenvalues are only *sufficient* (e.g. the identity matrix is diagonalisable).

Q5 (numerical, single correct). In the orthonormal basis $u_1 = \frac{1}{\sqrt{2}}[1,1]^T$, $u_2 = \frac{1}{\sqrt{2}}[1,-1]^T$, the point $z = [2,4]^T$ is reconstructed using only u_1. What is the squared reconstruction error? (a) 0 (b) 1 (c) 2 (d) 4.24 **Answer: (c).**

- $\alpha_1 = z^T u_1 = 6/\sqrt{2}$, so $\hat{z} = \alpha_1 u_1 = [3,3]$.
- The error is $(2-3)^2 + (4-3)^2 = 2 = \alpha_2^2$, where $\alpha_2 = -\sqrt{2}$.

Q6 (numerical, single correct). For $A = \begin{bmatrix} 5 & 2 \\ 2 & 2 \end{bmatrix}$, what is $\max_{\|x\|=1} x^T A x$? (a) 5 (b) 6 (c) 7 (d) 1 **Answer: (b).** For a symmetric A the maximum is $\lambda_{\max}$. The eigenvalues are 6 and 1 (verified). Option (a) is what you get from $x = e_1$, which is not optimal.

Q7 (numerical, single correct). The covariance matrix of centred 2-D data is $\begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$. What fraction of the total variance does the first principal component capture? (a) 50% (b) 66.7% (c) 75% (d) 100% **Answer: (b).** The eigenvalues are 4 and 2, so the fraction is $4/(4+2) = 66.7\%$. The variance along a principal component equals its eigenvalue.

Q8 (theory, single correct). Data matrix $X \in \mathbb{R}^{m \times n}$ has one zero-mean data point per row. PCA projects the data onto: (a) the eigenvectors of XX^T with the smallest eigenvalues (b) the eigenvectors of $X^T X$ with the largest eigenvalues (c) the eigenvectors of $X^T X$ with the smallest eigenvalues (d) the rows of X with the largest norm **Answer: (b).**

- Retain the top-k eigenvectors of $X^T X$, which is $n \times n$ and proportional to the covariance.
- The smallest-eigenvalue eigenvectors are the ones discarded: they minimise the residual error term.

Q9 (theory, multiple correct). Which are valid interpretations or properties of PCA on the slides? (a) The covariance between the new dimensions is zero (the transformed covariance $P^T \Sigma P$ is diagonal). (b) The variance along the i-th new dimension equals λ_i / m, where λ_i is an eigenvalue of $X^T X$. (c) It minimises the reconstruction error for a given number k of retained dimensions. (d) It requires the principal directions to be non-orthogonal so that they can capture correlations. **Answer: (a), (b), (c).** The principal directions are orthogonal, being eigenvectors of the symmetric $X^T X$.

Q10 (numerical, single correct). What are the singular values of $A = \begin{bmatrix} 2 & 0 \\ 0 & -3 \end{bmatrix}$? (a) 2, −3 (b) 3, 2 (c) 9, 4 (d) $\sqrt{2}$, $\sqrt{3}$ **Answer: (b).**

- $A^T A = \mathrm{diag}(4, 9)$, and $\sigma_i = \sqrt{\lambda_i(A^T A)}$, giving 3 and 2.
- Singular values are non-negative, so (a) gives the eigenvalues instead. Option (c) is σ^2.

Q11 (numerical, single correct). What are the singular values of the 3×2 matrix $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$? (a) $\sqrt{70}$, 0 (b) 70, 0 (c) $\sqrt{14}$, $\sqrt{56}$ (d) 1, 2 **Answer: (a).**

- A has rank 1, so there is exactly one non-zero singular value.
- $\sigma_1^2 = \|A\|_F^2 = 1 + 4 + 9 + 4 + 16 + 36 = 70$, so $\sigma_1 = \sqrt{70} \approx 8.37$ (verified).

Q12 (theory, single correct). For $A \in \mathbb{R}^{5 \times 3}$ with full SVD $A = U \Sigma V^T$, which statement is correct? (a) U is 3×3 and holds the eigenvectors of $A^T A$ (b) V is 3×3 and holds the eigenvectors of $A^T A$ (c) Σ is 3×3 (d) V holds the eigenvectors of AA^T **Answer: (b).**

- U is 5×5 and holds the eigenvectors of AA^T (left singular vectors).
- V is 3×3 and holds the eigenvectors of $A^T A$ (right singular vectors).
- Σ is 5×3.

Q13 (theory, single correct). Computing all coordinates α_i of $z \in \mathbb{R}^n$ in an **orthonormal** basis costs: (a) $O(n)$ (b) $O(n^2)$ (c) $O(n^3)$ (d) $O(\log n)$ **Answer: (b).** Each $\alpha_i = z^T u_i$ costs $O(n)$, and there are n of them. A general basis needs Gaussian elimination, which is $O(n^3)$.

Q14 (numerical, single correct). Two features over $m = 4$ data points are $a = [1, 2, 3, 6]$ and $b = [2, 4, 5, 9]$. Using the slides' definition $C_{ij} = \frac{1}{m}\sum_k (X_{ki} - \mu_i)(X_{kj} - \mu_j)$, what is $\mathrm{Cov}(a, b)$? (a) 19 (b) 6.33 (c) 4.75 (d) 3.5 **Answer: (c).**

- The means are 3 and 5, so the products of deviations are $6 + 1 + 0 + 12 = 19$.
- Dividing by $m = 4$ gives 4.75. Option (b) divides by $m - 1$ (not the slides' convention); (a) forgets to divide.

Lecture 7: Autoencoders · Week 6

Q1 (conceptual, single correct). The inputs are binary, $x_{ij} \in \{0, 1\}$. Which decoder and loss pairing do the slides recommend? (a) Linear decoder + squared error (b) tanh decoder + squared error (c) Logistic decoder + cross-entropy (d) Linear decoder + cross-entropy **Answer: (c).** The logistic function restricts outputs to (0,1), so they can be read as probabilities. Cross-entropy is minimised when $\hat{x}_{ij} = x_{ij}$.

Q2 (conceptual, single correct). The inputs are real-valued, $x_{ij} \in \mathbb{R}$. The most apt decoder is: (a) $\hat{x} = \tanh(W^* h + c)$ (b) $\hat{x} = W^* h + c$ (c) $\hat{x} = \mathrm{logistic}(W^* h + c)$ (d) $\hat{x} = \mathrm{ReLU}(W^* h)$ **Answer: (b).** tanh and logistic restrict $\hat{x}$ to $[-1, 1]$ and $[0, 1]$ respectively, but we need $\hat{x} \in \mathbb{R}^n$. A linear decoder is paired with squared error.

Q3 (theory, multiple correct). The encoder of an autoencoder is equivalent to PCA if we: (a) use a linear encoder (b) use a sigmoid decoder (c) use a squared-error loss (d) normalise the inputs as $\frac{1}{\sqrt{m}}\left(x_{ij} - \frac{1}{m}\sum_k x_{kj}\right)$ **Answer: (a), (c), (d).** The fourth condition is a **linear** decoder; a sigmoid decoder breaks the equivalence. The normalisation makes $X^T X$ the covariance matrix.

Q4 (theory, single correct). An autoencoder with $\dim(h) = \dim(x_i)$ is: (a) undercomplete, and cannot learn the identity (b) overcomplete, and could learn the trivial identity mapping (c) equivalent to PCA (d) a sparse autoencoder **Answer: (b).** The slides define overcomplete as $\dim(h) \geq \dim(x_i)$. Such a network can copy $x \to h \to \hat{x}$, which motivates regularization.

Q5 (conceptual, single correct). In a denoising autoencoder, the loss compares $\hat{x}_i$ with: (a) the corrupted input $\tilde{x}_i$ (b) the original uncorrupted input x_i (c) the hidden representation h (d) Gaussian noise **Answer: (b).** The objective is still $\sum(\hat{x}_{ij} - x_{ij})^2$ with the clean x. Copying $\tilde{x}$ to the output therefore no longer minimises the loss.

Q6 (numerical, single correct). A sparse AE uses $\rho = 0.1$. A hidden neuron has average activation $\hat{\rho}_l = 0.2$. What is its sparsity penalty term (natural log)? (a) 0 (b) 0.0367 (c) 0.1 (d) −0.0367 **Answer: (b).** $0.1\ln(0.1/0.2) + 0.9\ln(0.9/0.8) = -0.0693 + 0.1060 = 0.0367$ (verified). The term is ≥ 0 (it is a KL divergence) and equals 0 only when $\hat{\rho}_l = \rho$.

Q7 (numerical, single correct). With $\rho = 0.05$ and $\hat{\rho}_l = 0.25$, what is $\partial\Omega/\partial\hat{\rho}_l$? (a) −0.2 (b) 1.067 (c) 1.267 (d) −1.067 **Answer: (b).** $-\rho/\hat{\rho}_l + (1-\rho)/(1-\hat{\rho}_l) = -0.2 + 0.95/0.75 = -0.2 + 1.2667 = 1.067$. It is positive because $\hat{\rho}_l > \rho$, so gradient descent pushes the activation down.

Q8 (numerical, single correct). A binary-input AE has a sigmoid decoder and cross-entropy loss. For one output, $x_{ij} = 1$ and $\hat{x}_{ij} = 0.25$. What is $\frac{\partial \mathscr{L}}{\partial h_{2j}} \cdot \frac{\partial h_{2j}}{\partial a_{2j}}$? (a) −4 (b) 0.1875 (c) −0.75 (d) 0.75 **Answer: (c).**

- $\partial\mathscr{L}/\partial h_{2j} = -1/0.25 = -4$.
- $\sigma(1-\sigma) = 0.25 \times 0.75 = 0.1875$.

- The product is $-0.75 = \hat{x} - x$.

Q9 (numerical, single correct). A real-input AE uses squared-error loss $(\hat{x} - x)^T(\hat{x} - x)$. With $x = [1, 2]^T$ and $\hat{x} = [1.5, 1]^T$, what is $\partial \mathscr{L} / \partial \hat{x}$? (a) $[0.5, -1]$ (b) $[1, -2]$ (c) $[-1, 2]$ (d) $[0.25, 1]$ **Answer: (b).** The gradient is $2(\hat{x} - x) = 2[0.5, -1] = [1, -2]$. Option (c) has the wrong sign and (a) is missing the factor 2.

Q10 (numerical, single correct). A contractive AE has input dimension $n = 784$ and $k = 64$ hidden units. How many terms are summed in $\|J_x(h)\|_F^2$, and what is the shape of J? (a) 848, 64×784 (b) 50176, 784×64 (c) 50176, 64×784 (d) 64, 64×1 **Answer: (c).** The Jacobian has rows indexed by hidden units l and columns by inputs j, so it is $k \times n = 64 \times 784$, with 50176 squared partials $(\partial h_l / \partial x_j)^2$.

Q11 (numerical, single correct). An AE has $n = 100$ inputs and $k = 20$ hidden units, with biases, and **tied weights** $W^* = W^T$. How many parameters does it have? (a) 4120 (b) 2120 (c) 2000 (d) 4000 **Answer: (b).** W (20×100) is 2000, plus b (20) and c (100), giving 2120. Untied weights would give 4120. Tying weights reduces capacity and so acts as a regularizer.

Q12 (numerical, single correct). A hidden neuron $h_1 = \sigma(W_1^T x)$ has $W_1 = [3, 4]^T$. Which unit-norm input maximally activates it? (a) $[3, 4]$ (b) $[0.6, 0.8]$ (c) $[0.8, 0.6]$ (d) $[1, 1]/\sqrt{2}$ **Answer: (b).** $x = W_1 / \sqrt{W_1^T W_1} = [3, 4]/5$.

Q13 (theory, multiple correct). Which statements about AE regularizers are correct? (a) The contractive penalty is the squared Frobenius norm of the Jacobian of the encoder with respect to the input. (b) The sparse AE penalty is minimised when every neuron has $\hat{\rho}_l = 0$. (c) In denoising AEs, increasing the masking noise q makes the learned filters wider (they rely on more neighbouring pixels). (d) A contractive AE trades off reconstruction (capture variations) against insensitivity (ignore variations), so it captures only the important variations. **Answer: (a), (c), (d).** (b) is false: the KL penalty is minimised (equal to 0) at $\hat{\rho}_l = \rho$, not at 0.

Q14 (numerical, single correct). For the AE ≡ PCA result, inputs are normalised as $\hat{x}_{ij} = \frac{1}{\sqrt{m}}\big(x_{ij} - \frac{1}{m}\sum_k x_{kj}\big)$. A feature column has values $[1, 3, 5, 7]$ ($m = 4$). What is the normalised value of the first entry? (a) −0.75 (b) −1.342 (c) −3 (d) −1.5 **Answer: (d).**

- The column mean is 4, so the centred value is $1 - 4 = -3$.
- Dividing by $\sqrt{m} = 2$ gives -1.5. Option (b) divides by the standard deviation $\sqrt{5}$ (a different normalisation); (a) divides by m; (c) only centres.

Weeks 7–8 · Lectures 8–9

Lecture 8: Regularization · Week 7

Q1. (Single correct) Compared with a degree-25 polynomial, a degree-1 model fitted to sinusoidal data typically has: A) Low bias, high variance B) High bias, low variance C) Low bias, low variance D) High bias, high variance **Answer: B.** Simple models trained on different samples agree with each other (low variance) but their average is far from the sinusoid (high bias → underfitting).

Q2. (Numerical) At a point x_0 the true value is $f(x_0) = 3$. Four models trained on different samples predict $2, 4, 3, 5$. The noise variance is $\sigma^2 = 0.5$. The expected squared error $E[(y - \hat{f}(x_0))^2]$ is: A) 1.25 B) 1.5 C) 2.0 D) 1.75 **Answer: C.** $E[\hat{f}] = 3.5$ → Bias² $= 0.25$; Variance $= (2.25 + 0.25 + 0.25 + 2.25)/4 = 1.25$; total $= 0.25 + 1.25 + 0.5 = 2.0$.

Q3. (Multiple correct) Which statements about estimating the true error $E[(\hat{f}(x_i) - f(x_i))^2]$ are correct? A) Empirical test error equals true error up to a small constant term B) Empirical training error underestimates the true error C) On training points, the covariance term $E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$ is zero D) By Stein's lemma the

covariance term is $\frac{\sigma^2}{n}\sum_i \partial\hat{f}(x_i)/\partial y_i$ **Answer: A, B, D.** C is false: noise on training points was used to fit $\hat{f}$, so $\varepsilon \not\perp \hat{f}$ and the term is non-zero (it is zero for test points).

Q4. (Single correct) $\partial\hat{f}(x_i)/\partial y_i$ is large for a model when: A) The model is simple B) A small change in an observation causes a large change in the estimate, as in complex models C) The noise variance is zero D) The model is evaluated on test data **Answer: B.** Complex models are more sensitive to changes in the observations, so $\Omega(\text{model complexity})$ is large.

Q5. (Numerical) With L_2 regularization ($\alpha = 3$), the Hessian at w^* has eigenvalues $9, 3, 1$. The scaling of the components of $Q^T w^*$ and the effective number of parameters are: A) $(0.25, 0.5, 0.75)$; 1.5 B) $(0.75, 0.5, 0.25)$; 1.5 C) $(0.75, 0.5, 0.25)$; 3 D) $(0.9, 0.5, 0.1)$; 1.5 **Answer: B.** Factor $= \lambda_i/(\lambda_i+\alpha)$: $9/12, 3/6, 1/4$. Sum $= 1.5 < 3$.

Q6. (Single correct) In the L_2 analysis $\tilde{w} = Q(\Lambda+\alpha I)^{-1}\Lambda Q^T w^*$, a direction with $\lambda_i \ll \alpha$: A) Is retained almost fully B) Is scaled by approximately 0 C) Is doubled D) Is unaffected because Q is orthogonal **Answer: B.** $\lambda_i/(\lambda_i+\alpha) \approx 0$. Only significant (large-eigenvalue) directions survive.

Q7. (Numerical) For a linear model $\hat{y} = \sum_i w_i x_i$ with $w = (2, 1, -1)$, Gaussian noise $\mathcal{N}(0, 0.5)$ is added independently to each input. The extra term added to the expected squared error is: A) 1.0 B) 3.0 C) 6.0 D) 1.5 **Answer: B.** $\sigma^2 \sum w_i^2 = 0.5 \times (4+1+1) = 3$. This equals an L_2 penalty.

Q8. (Numerical) With label smoothing on a 5-class problem and $\varepsilon = 0.2$, the soft target for the true class and for each other class are: A) 0.8, 0.05 B) 0.8, 0.2 C) 0.96, 0.01 D) 0.8, 0.04 **Answer: A.** True class $1-\varepsilon = 0.8$; the remaining 0.2 is shared equally over 4 classes → 0.05 each. (Slides: 10 classes → $\varepsilon/9$.)

Q9. (Single correct) Early stopping with patience $p = 5$ is triggered at step $k = 30$. Which model is returned? A) Step 30 B) Step 25 C) Step 35 D) Step 5 **Answer: B.** Return the model stored at step $k - p$.

Q10. (Multiple correct) Which statements about early stopping are correct (per the lecture)? A) $|w_{t+1} - w_0| \leq \eta t|\tau|$, so t limits the space of exploration B) Under a quadratic approximation and $w_0 = 0$, it is equivalent to L_2 when $(I-\eta\Lambda)^t = (\Lambda+\alpha I)^{-1}\alpha$ C) It cannot be combined with L_2 regularization D) Parameters with small gradients cannot grow large in t steps **Answer: A, B, D.** C is false: the slides say early stopping can be used together with other regularizers such as l_2.

Q11. (Numerical) An ensemble of $k = 4$ models has per-model error variance $V = 6$ and pairwise covariance $C = 2$. The expected squared error of the averaged prediction is: A) 1.5 B) 2.0 C) 3.0 D) 6.0 **Answer: C.** $V/k + \frac{k-1}{k}C = 1.5 + 0.75 \times 2 = 3.0$.

Q12. (Single correct) Bagging gives NO improvement over a single model when the models' errors are: A) Uncorrelated B) Perfectly correlated ($C = V$) C) Zero-mean D) Gaussian **Answer: B.** mse $= V/k + \frac{k-1}{k}V = V$. With $C = 0$ it would be V/k.

Q13. (Numerical / multiple correct) A network with 6 droppable nodes is trained with dropout, retention probability $p = 0.5$. Which are correct? A) There are 64 possible thinned networks B) At test time a weight of 0.6 is used as 0.3 C) At test time units are dropped randomly with probability 0.5 D) Every thinned network is guaranteed to be trained at least once **Answer: A, B.** $2^6 = 64$; test-time weight $pw = 0.3$. C: the full network is used at test time. D: some thinned networks are trained rarely or never; weight sharing makes this harmless.

Q14. (Single correct) The main reason dropout reduces overfitting, according to the lecture, is that it: A) Reduces the number of parameters permanently B) Prevents hidden units from co-adapting, forcing redundant and robust features C) Normalizes pre-activations D) Increases the learning rate **Answer: B.** Dropout is masking noise. A unit cannot rely on others that may be dropped.

Q15. (Numerical) Bagging with very many models ($k \to \infty$), per-model error variance $V = 5$ and pairwise covariance $C = 0.5$. The ensemble mse approaches: A) 0 B) 0.5 C) 2.75 D) 5 **Answer: B.** mse $= V/k + \frac{k-1}{k}C \to C = 0.5$. Correlation between the models' errors sets the floor.

Q16. (Single correct) On the error-vs-model-complexity plot, the gap between test error and training error corresponds to: A) σ^2 (irreducible error) B) Bias^2 C) $\frac{\sigma^2}{n}\sum_i \partial \hat{f}(x_i)/\partial y_i$, which grows with model complexity D) A constant independent of complexity **Answer: C.** By Stein's lemma the covariance term equals the sensitivity term $\frac{\sigma^2}{n}\sum_i \partial \hat{f}(x_i)/\partial y_i$. It is large for complex models, so the training error becomes more optimistic.

Lecture 9: Pre-training, Activations, Initialization, Batch Norm · Week 8

Q1. (Single correct) In greedy layerwise unsupervised pre-training (autoencoder view), after training layer 1 to reconstruct x: A) Layer 1 is discarded B) Layer-1 weights are fixed and layer 2 is trained to reconstruct h_1 C) All layers are immediately trained with the supervised loss D) The output layer is trained to reconstruct y **Answer: B.** You continue layer by layer up to the last hidden layer. Then you add the output layer and fine-tune the whole network with the supervised objective.

Q2. (Multiple correct) According to the studies cited in the lecture (Erhan et al., Larochelle et al.): A) If the last layer has large capacity, the training loss can reach 0 even without pre-training B) Pre-training helps optimization when network capacity is small C) The unsupervised objective acts like a regularizer constraining weights to regions that capture data characteristics D) Pre-training makes training more sensitive to random initialization **Answer: A, B, C.** D is the opposite: pre-training was found to be more robust to random initializations, which led to the "better initialization" hypothesis.

Q3. (Numerical) What is the largest possible value of the product of sigmoid derivatives $\prod_{l=1}^{4}\sigma'(a_l)$ along a 4-layer chain? A) 0.25 B) 0.0625 C) 0.0039 D) 1 **Answer: C.** $\max \sigma' = \sigma(0)(1-\sigma(0)) = 0.25$, and $0.25^4 \approx 0.0039$. This shows why gradients vanish.

Q4. (Single correct) Because sigmoid outputs are not zero-centred, for a neuron with $a_3 = w_1 h_{21} + w_2 h_{22}$: A) ∇w_1 and ∇w_2 always have opposite signs B) ∇w_1 and ∇w_2 always have the same sign, so updates follow a zig-zag path C) The gradients are always zero D) The gradients explode **Answer: B.** Both share a common factor multiplied by positive h's, so updates are restricted to the all-positive or all-negative quadrant.

Q5. (Multiple correct) Which are true of ReLU, $f(x) = \max(0, x)$? A) It is non-linear B) It does not saturate in the positive region C) A neuron whose bias becomes very negative can die permanently D) It is zero-centred **Answer: A, B, C.** ReLU outputs are non-negative, so it is not zero-centred. Dead neurons are more likely with a high learning rate; one remedy is a small positive bias such as 0.01.

Q6. (Single correct) Which activation doubles the number of parameters? A) Leaky ReLU B) ELU C) Maxout D) Parametric ReLU **Answer: C.** Maxout computes $\max(w_1^T x + b_1, w_2^T x + b_2)$, so each unit has two sets of weights. It generalizes ReLU and Leaky ReLU.

Q7. (Numerical) For ELU with $a = 1$, $f(-2)$ and $f'(-2)$ are approximately: A) $-0.865,\ 0.135$ B) $-2,\ 1$ C) $0,\ 0$ D) $-0.135,\ 0.865$ **Answer: A.** $e^{-2} - 1 = -0.8647$; derivative $ae^x = 0.1353$.

Q8. (Single correct) All weights are initialized to the same constant value $c \neq 0$. Then: A) Training works because $c \neq 0$ B) Hidden units in a layer compute identical activations and receive identical updates, so the symmetry never breaks C) Gradients explode D) Only biases learn **Answer: B.** This is the symmetry-breaking problem. It happens for any equal initialization, not only for zero.

Q9. (Numerical) Under the lecture's assumptions (zero-mean inputs and weights, i.i.d., linear propagation), what is $\mathrm{Var}(s_{3i})/\mathrm{Var}(x)$ after 3 layers with fan-in $n = 200$ and $\mathrm{Var}(w) = 0.01$? A) 2 B) 6 C) 8 D) 0.008 **Answer:**

C. $[n\mathrm{Var}(w)]^k = (200 \times 0.01)^3 = 2^3 = 8$, so the variance grows.

Q10. (Numerical) For a ReLU layer with fan-in $n = 200$, the He initialization standard deviation is, and for a tanh layer with fan-in 400 the $1/\sqrt{n}$ scaling gives: A) 0.1; 0.05 B) 0.0707; 0.05 C) 0.1; 0.0025 D) 0.01; 0.05 **Answer: A.** He: $\sqrt{2/200} = 0.1$ (`randn/sqrt(fan_in/2)`). The $1/\sqrt{400} = 0.05$ scaling gives $n\mathrm{Var}(w) = 1$.

Q11. (Single correct) With `randn(fan_in, fan_out)*0.01` in a 5-layer tanh network (500 units per layer), deeper-layer activations: A) Saturate at ±1 B) Collapse towards 0, giving vanishing weight gradients C) Stay unit Gaussian D) Explode **Answer: B.** $n\mathrm{Var}(w) = 500 \times 10^{-4} = 0.05 \ll 1$. Because $\nabla w \propto$ input activation, the gradients also vanish. Large unscaled `randn` would instead saturate the units.

Q12. (Numerical) A BN unit sees the mini-batch of pre-activations $(2, 4, 6, 8)$ with $\gamma = 2, \beta = 1$ (ignore ϵ). What is the BN output for the input 8? A) 1.342 B) 3.683 C) 2.342 D) 7.0 **Answer: B.** $\mu = 5$, $\sigma^2 = 5$, $\hat{s} = 3/\sqrt{5} = 1.342$, and $y = 2(1.342) + 1 = 3.683$.

Q13. (Multiple correct) Which statements about Batch Normalization (as presented) are correct? A) Mean and variance are computed per mini-batch B) BN is differentiable, so we can backpropagate through it C) With $\gamma = \sqrt{\mathrm{var}}$ and $\beta = E[\cdot]$, the network can recover the un-normalized pre-activation D) γ and β are fixed hyperparameters **Answer: A, B, C.** γ and β are learnable parameters of the network.

Q14. (Single correct) In the lecture's 10-layer ReLU experiment, weights are initialized as `randn(fan_in, fan_out)/sqrt(fan_in)`. What happens to the activations? A) They saturate at 1 B) They collapse towards 0 in deeper layers, so a factor of 2 is needed (`/sqrt(fan_in/2)`) C) They stay unit Gaussian in every layer D) They explode **Answer: B.** ReLU passes only the positive half of the space. He et al. therefore use $\mathrm{Var}(w) = 2/n$, and with that scaling the activation distribution stays stable across the layers.

Weeks 9–10 · Lectures 10–11

Lecture 10: Word representations · Week 9

Q1. (Single correct, numerical) The vocabulary has $|V| = 50{,}000$ words, each represented as a one-hot vector. What are the Euclidean distance and the cosine similarity between the vectors for "cat" and "truck"?

- A) 2 and 0
- B) $\sqrt{2}$ and 0
- C) $\sqrt{2}$ and 1
- D) $\sqrt{50000}$ and 0

Answer: B. Any two distinct one-hot vectors differ in exactly 2 positions, so the distance is $\sqrt{1+1} = \sqrt{2}$. They are orthogonal, so the cosine is 0. Neither value depends on $|V|$.

Q2. (Single correct, numerical) In a corpus with $N = 10{,}000$ words, count(w) = 40, count(c) = 100 and count(w, c) = 8. What is PMI(w, c) using the natural log?

- A) 0.693
- B) 1.386
- C) 2.996
- D) −2.996

Answer: C. $\ln \frac{8 \cdot 10000}{100 \cdot 40} = \ln 20 \approx 2.996$.

Q3. (Single correct) A word–context pair has count(w, c) = 3 and PMI(w, c) = −0.7. What are its PMI_0 and PPMI values?

- A) $\text{PMI}_0 = 0$, PPMI = 0
- B) $\text{PMI}_0 = -0.7$, PPMI = −0.7
- C) $\text{PMI}_0 = -0.7$, PPMI = 0
- D) $\text{PMI}_0 = 0$, PPMI = −0.7

Answer: C. PMI_0 zeroes an entry only when the count is 0, so it keeps −0.7. PPMI zeroes every non-positive PMI, so it gives 0.

Q4. (Single correct) The PPMI matrix is factorized by SVD as $X = U\Sigma V^\top$, truncated to rank k. What is conventionally used as the word representation matrix W_{word}?

- A) U
- B) $U\Sigma$
- C) V
- D) The rows of $\hat{X} = U\Sigma V^\top$

Answer: B. $\hat{X}\hat{X}^\top = (U\Sigma)(U\Sigma)^\top$, so the rows of $U\Sigma$ (size $m \times k$) have the same dot products as the rows of $\hat{X}$, but in lower dimension. V is used as $W_{context}$. The rows of $\hat{X}$ are still high-dimensional.

Q5. (Single correct, numerical) A 500×800 matrix is approximated by its rank-1 SVD term $\sigma_1 u_1 v_1^\top$. How many numbers must be stored?

- A) 400,000
- B) 1,300
- C) 1,301
- D) 801

Answer: C. $m + n + 1 = 500 + 800 + 1 = 1301$ (for u_1, v_1 and σ_1).

Q6. (Single correct, numerical) In CBOW with one context word, $u_c = [2, -1, 0]$, the predicted probability of the correct word is $\hat{y}_w = 0.75$, and $\eta = 0.4$. What change does the update make to v_w?

- A) $[0.6, -0.3, 0]$
- B) $[0.2, -0.1, 0]$
- C) $[-0.2, 0.1, 0]$
- D) No change

Answer: B. $v_w \leftarrow v_w + \eta u_c(1 - \hat{y}_w)$, so the change is $0.4 \times 0.25 \times [2, -1, 0] = [0.2, -0.1, 0]$.

Q7. (Multiple correct) Which statements about CBOW are true, as presented in the lecture?

- A) $W_{context}x$ for a one-hot x selects a column of $W_{context}$.
- B) With $d - 1$ context words, $h = \sum_i u_{c_i}$, and the stacked $[W_{context}, W_{context}]$ adds new independent parameters.
- C) If $\hat{y}_w \to 1$, v_w is essentially not updated.
- D) The softmax denominator requires a sum over the entire vocabulary.

Answer: A, C, D. B is false because the stacked matrices are copies of the same (shared) $W_{context}$, so no new parameters are added.

Q8. (Single correct, numerical) A CBOW model has $|V| = 20{,}000$ and embedding size $k = 100$. It uses a window of 4 context words. How many parameters are in $W_{context}$ and W_{word} combined?

- A) 2M
- B) 4M
- C) 8M
- D) 10M

Answer: B. $2k|V| = 2 \times 100 \times 20000 = 4 \times 10^6$. The window size does not change the count, because $W_{context}$ is shared across positions.

Q9. (Single correct) In skip-gram with negative sampling, which expression is $p(z = 0 \mid w, r)$ for a negative pair $(w, r) \in D'$?

- A) $\sigma(u_r^\top v_w)$
- B) $\sigma(-u_r^\top v_w)$
- C) $-\log \sigma(u_r^\top v_w)$
- D) $\mathrm{softmax}(u_r^\top v_w)$

Answer: B. $1 - \sigma(x) = \sigma(-x)$, and the objective maximizes $\sum_{D'} \log \sigma(-u_r^\top v_w)$.

Q10. (Single correct, numerical) A vocabulary has only two words, with counts 81 and 1. Negatives are drawn from the modified unigram distribution $\propto \mathrm{count}^{3/4}$. What is the probability of sampling the rare word?

- A) 0.012
- B) 0.036
- C) 0.10
- D) 0.50

Answer: B. $81^{3/4} = 27$ and $1^{3/4} = 1$, so the probability is $1/28 \approx 0.036$, compared with $1/82 \approx 0.012$ under the plain unigram. The 3/4 power boosts rare words.

Q11. (Single correct, numerical) Hierarchical softmax uses a balanced binary tree over $|V| = 4096$ words. How many sigmoid evaluations are needed to compute $P(w \mid v_c)$?

- A) 4096
- B) 2048
- C) 64
- D) 12

Answer: D. The path length is $\log_2 4096 = 12$, compared with 4096 terms for the full softmax. The number of parameters stays about the same: there are $|V| - 1$ internal node vectors.

Q12. (Single correct, numerical) GloVe uses $f(x) = (x/x_{max})^\alpha$ for $x < x_{max}$ and 1 otherwise, with $x_{max} = 100$ and $\alpha = 0.5$. What are $f(25)$ and $f(400)$?

- A) 0.25 and 4
- B) 0.5 and 1
- C) 0.5 and 2
- D) 0.0625 and 1

Answer: B. $(0.25)^{0.5} = 0.5$, and f is capped at 1 for $x \geq x_{max}$.

Q13. (Multiple correct) Which statements are consistent with the lecture?

- A) Baroni et al. (2014) found that predict models consistently outperform count models.
- B) Levy et al. (2015) found that SVD does better than prediction-based models on similarity tasks but not on analogy tasks.
- C) word2vec implicitly factorizes a matrix with $M_{ij} = \mathrm{PMI}(w_i, c_j) - \log k$, where k is the embedding dimension.
- D) In hierarchical softmax, even a random arrangement of words on the leaves works well in practice.

Answer: A, B, D. In C, k is the number of negative samples, not the embedding dimension.

Q14. (Single correct, numerical) For a co-occurrence matrix X, $(XX^\top)_{ii} = 36$, $(XX^\top)_{jj} = 25$ and $(XX^\top)_{ij} = 18$. What is the cosine similarity between word i and word j?

- A) 18
- B) 0.5
- C) 0.6
- D) 0.72

Answer: C. $18/\sqrt{36 \cdot 25} = 18/30 = 0.6$. The raw entry 18 is only an unnormalized dot product. D is the distractor $18/25$.

Q15. (Single correct) In the synonym-detection evaluation, how is the predicted synonym chosen and how are models compared?

- A) Smallest Euclidean distance; compared by correlation with human scores
- B) Largest cosine similarity with the term; compared by accuracy
- C) Largest PMI with the term; compared by accuracy
- D) Nearest neighbour of $v_b - v_a + v_c$; compared by correlation

Answer: B. Synonym $= \arg\max_{c \in C} \cos(v_{term}, v_c)$ over 4 candidates, and accuracy is compared. Correlation with humans is the semantic-relatedness task, and the vector offset is the analogy task.

Lecture 11: CNNs · Week 10

Q1. (Single correct, numerical) A $128 \times 128 \times 3$ input is convolved with 32 filters of size 5×5, using stride 1 and padding 2. What is the output volume?

- A) $124 \times 124 \times 32$
- B) $128 \times 128 \times 3$
- C) $128 \times 128 \times 32$
- D) $126 \times 126 \times 32$

Answer: C. $(128 - 5 + 4)/1 + 1 = 128$, and the output depth equals $K = 32$.

Q2. (Single correct, numerical) How many learnable parameters does the layer in Q1 have, including one bias per filter?

- A) 800
- B) 2,400
- C) 2,432
- D) 76

Answer: C. $(5 \cdot 5 \cdot 3 + 1) \times 32 = 76 \times 32 = 2432$. Without biases, the count is 2,400, which is the style used on the slides. The input depth of 3 must be included.

Q3. (Single correct, numerical) In the slide version of AlexNet, a $227 \times 227 \times 3$ input passes through Conv1 ($K = 96, F = 11, S = 4, P = 0$) and then max-pooling ($F = 3, S = 2$). What is the shape after the pooling layer?

- A) $55 \times 55 \times 96$
- B) $27 \times 27 \times 96$
- C) $26 \times 26 \times 96$
- D) $27 \times 27 \times 3$

Answer: B. Conv1 gives $(227 - 11)/4 + 1 = 55$, and the pool gives $(55 - 3)/2 + 1 = 27$. Pooling keeps the depth (96).

Q4. (Multiple correct) Which statements about pooling layers are true?

- A) A max-pooling layer has zero learnable parameters.
- B) Pooling changes the depth of the input volume.
- C) Average pooling is an alternative to max pooling.
- D) A 2×2 max pool with stride 1 on a 4×4 input gives a 3×3 output.

Answer: A, C, D. Pooling acts on each feature map independently, so the depth is unchanged and B is false. For D, $(4 - 2)/1 + 1 = 3$.

Q5. (Single correct, numerical) A $56 \times 56 \times 256$ volume is convolved with 64 filters of size 1×1 ($S = 1, P = 0$). What are the output shape and the weight count (no bias)?

- A) $56 \times 56 \times 64$ and 16,384
- B) $56 \times 56 \times 256$ and 64
- C) $56 \times 56 \times 64$ and 64
- D) $1 \times 1 \times 64$ and 16,384

Answer: A. A 1×1 convolution aggregates along the depth, reducing 256 to 64 while keeping the spatial size. The weights are $1 \cdot 1 \cdot 256 \cdot 64 = 16384$.

Q6. (Single correct, numerical) In GoogLeNet the last feature volume is $7 \times 7 \times 1024$. Compare a direct fully connected layer to 1000 classes with global 7×7 average pooling followed by an FC layer to 1000 classes. What are the two weight counts?

- A) 50.2M vs 1.02M
- B) 50.2M vs 7.2M
- C) 1.02M vs 50.2M
- D) 49K vs 1K

Answer: A. $7 \cdot 7 \cdot 1024 \cdot 1000 = 50{,}176{,}000$ (≈49M as stated on the slide), versus $1024 \cdot 1000 = 1{,}024{,}000$, a 49× reduction.

Q7. (Single correct, numerical) An inception module (3a) has 64 1×1 convolutions, 96 1×1 reductions followed by 128 3×3 convolutions, 16 1×1 reductions followed by 32 5×5 convolutions, and a 3×3 max-pool followed by 32 1×1 convolutions. What is the output depth?

- A) 192
- B) 256
- C) 368
- D) 480

Answer: B. Only the outputs of the branches are concatenated: $64 + 128 + 32 + 32 = 256$. Adding the reduction layers (96 and 16) gives the distractor 368.

Q8. (Single correct, numerical) A $14 \times 14 \times 480$ input is convolved with 48 filters of size 5×5 (same padding). Approximately how many times fewer multiplications are needed if 16 1×1 reduction filters are applied first?

- A) ≈3×
- B) ≈10×
- C) ≈21×
- D) ≈48×

Answer: C. Direct cost: $14\cdot14\cdot48\cdot25\cdot480 = 112.9$M. With reduction: $14\cdot14\cdot16\cdot480 + 14\cdot14\cdot48\cdot25\cdot16 = 1.51\text{M}+3.76\text{M}=5.27$M. The ratio is ≈21.4.

Q9. (Multiple correct) Which statements correctly distinguish a CNN from a fully connected feed-forward network?

- A) Sparse connectivity: each hidden unit depends only on a local patch of the input.
- B) Weight sharing forces a CNN layer to have only one kernel.
- C) Pixels that do not interact in layer 1 can still interact indirectly in deeper layers.
- D) A CNN can be trained with standard backpropagation, treating it as a feed-forward network with sparse, shared weights.

Answer: A, C, D. B is false: there can be many kernels, and each kernel is shared across all locations.

Q10. (Single correct, numerical) In LeNet-5, the second pooling layer outputs $5 \times 5 \times 16$, which feeds a fully connected layer of 120 units. How many parameters does this FC layer have, including biases (as on the slide)?

- A) 48,000
- B) 48,120
- C) 10,164
- D) 2,400

Answer: B. $400 \times 120 + 120 = 48120$. 10,164 is FC2 ($120 \cdot 84 + 84$), and 2,400 is Conv2.

Q11. (Single correct) Where are most of VGGNet's parameters located?

- A) In the 3×3 conv layers of the last block
- B) In the first fully connected layer (≈102M)
- C) In the softmax layer
- D) Evenly across all layers

Answer: B. $512\times7\times7\times4096 \approx 102.8$M. The conv layers total only about 16M (slide value).

Q12. (Multiple correct) Which statements about ResNet are true, as presented in the lecture?

- A) A residual block computes $H(x) = F(x) + x$ through an identity (skip) connection.
- B) Deeper plain networks (e.g. 56 layers) were observed to have higher error than shallower ones (20 layers).
- C) ResNet used dropout heavily together with batch normalization.
- D) The identity connection lets the network retain a copy of the input, making identity mappings easy to learn.

Answer: A, B, D. The bag of tricks used batch normalization after every conv layer and no dropout, so C is false.

Q13. (Single correct, numerical) A 7×7 input is convolved with a 3×3 filter, using stride 2 and no padding. What is the output width?

- A) 5
- B) 3
- C) 2.5
- D) 4

Answer: B. $(7 - 3 + 0)/2 + 1 = 3$.

Q14. (Single correct) How does ZF-Net differ from AlexNet, according to the slides?

- A) It replaces the first 11×11 filters with 7×7 filters and increases the number of filters in the middle conv layers (384→512, 384→1024, 256→512).
- B) It removes all fully connected layers.
- C) It uses inception modules.
- D) It uses residual connections.

Answer: A. The first-layer change saves $(11^2 - 7^2) \cdot 3 \cdot 96 = 20{,}736$ parameters, and the extra filters add parameters in conv layers 3–5.

Q15. (Single correct, numerical) In ZF-Net, the conv layer that takes the $11 \times 11 \times 256$ input ($F = 3$) uses 512 filters instead of AlexNet's 384. How many extra weights (no bias) does this add?

- A) 98,304
- B) 294,912
- C) 884,736

- D) 1,179,648

Answer: B. $(3\cdot3\cdot256)\times(512-384) = 2304\times128 = 294{,}912 \approx 0.29$M, as on the slide. C is AlexNet's whole Conv3 and D is ZF-Net's whole layer.

Weeks 10–11 · Lectures 12–13

Lecture 12 · Week 10

Q1. (Single correct, numerical) A CNN has (in order): conv 3×3 stride 1, conv 3×3 stride 1, conv 3×3 stride 1 (no pooling). What is the receptive field of a neuron in the third layer on the input? A) 3×3 B) 5×5 C) 7×7 D) 9×9 **Answer: C.** $r = 1 + 3 \cdot (3 - 1) = 7$.

Q2. (Single correct, numerical) Using the slide's network (conv1 11×11 s4 → maxpool 3×3 s2 → conv2 5×5 s1), what is the receptive field of a conv2 neuron on the 227×227 input? A) 19 B) 27 C) 51 D) 35 **Answer: C.** r: $11 \to 11 + 2 \cdot 4 = 19 \to 19 + 4 \cdot 8 = 51$ (jump after pool = $4 \cdot 2 = 8$).

Q3. (Single correct, numerical) A first-layer filter has weights $w = [1, -2, 2]$. Under the constraint $\|x\| = 1$, which input maximally activates the neuron, and what is the maximum value of $w^T x$? A) $[1, -2, 2]$, 9 B) $[1/3, -2/3, 2/3]$, 3 C) $[1/3, -2/3, 2/3]$, 1 D) $[1/9, -2/9, 2/9]$, 1 **Answer: B.** $x^* = w/\|w\|$, $\|w\| = 3$, max $= \|w\| = 3$.

Q4. (Single correct, numerical) At a ReLU, the forward inputs are $[2, -1, 3, -4]$ and the gradients coming from the upper layer are $[-5, 2, 4, 6]$. What gradient does guided backpropagation pass down? A) $[-5, 0, 4, 0]$ B) $[0, 2, 4, 6]$ C) $[0, 0, 4, 0]$ D) $[0, 0, 4, 6]$ **Answer: C.** Zero where forward input < 0 (positions 2,4) **and** where gradient < 0 (position 1). Plain backprop would give A.

Q5. (Multiple correct) Which statements about visualizing filters are true (per the lecture)? A) The input that maximally activates a neuron (unit-norm input) is $W/\|W\|$. B) Filters can be thought of as pattern detectors. C) Plotting filters as images is interpretable for all convolution layers equally. D) A CNN can be viewed as a feed-forward network with sparse connections and weight sharing. **Answer: A, B, D.** The slides state the plot is only interpretable for the first conv layer.

Q6. (Single correct) In occlusion experiments, what is measured as a gray patch is slid over the image? A) The gradient of the class score w.r.t. the patch pixels B) The predicted probability of the correct class C) The activation of all first-layer filters D) The Gram matrix of the last conv layer **Answer: B.** A large drop (e.g., occluding the dog's face) marks an important region; no gradients needed.

Q7. (Single correct) When generating an image that maximizes class c, the quantity maximized is $S_c(I) - \lambda\Omega(I)$, where $S_c(I)$ is: A) the softmax probability of class c B) the score for class c before softmax C) the cross-entropy loss for class c D) the activation of fc7 **Answer: B.** The slides define it as the pre-softmax score; Ω keeps I image-like; the network weights stay fixed.

Q8. (Single correct, numerical) In style transfer, a layer's activation V has 128 channels, each a 64×64 map. What is the size of the Gram (style) matrix? A) 64×64 B) 4096×4096 C) 128×128 D) 128×4096 **Answer: C.** Gram = channels × channels, independent of spatial size.

Q9. (Multiple correct) Regarding Deep Art, $\mathcal{L}_{total} = \alpha\mathcal{L}_{content} + \beta\mathcal{L}_{style}$: A) The content loss compares activations of the content image and the new image. B) The style loss compares Gram matrices across layers, weighted by w_ℓ. C) Increasing β/α makes the output more stylized. D) The CNN weights are updated during style transfer. **Answer: A, B, C.** Only the new image's pixels are optimized.

Q10. (Single correct) Deep Dream differs from class-score visualization mainly because it: A) starts from a real image and maximizes h_{ij}^2 of neurons in a chosen layer B) starts from a zero image and maximizes the softmax probability C) updates the network weights to hallucinate patterns D) uses guided backpropagation to zero negative gradients **Answer: A.** The image update is gradient ascent on $\mathcal{L}(I) = h_{ij}^2$, which amplifies patterns the network already "sees".

Q11. (Single correct) According to the lecture, why can a ConvNet classify a random-noise image as "cheetah" with 99.6% confidence? A) The network was trained for too few epochs B) Images live in a very high-dimensional space where most points are not natural images, but the learned decision boundaries still assign them to classes C) Softmax always outputs probabilities above 99% D) Max-pooling removes all noise **Answer: B.** Large regions with no training points still fall on one side of a boundary.

Q12. (Single correct) In the embedding-reconstruction experiment, which layer's activations are used as the image "embedding" ϕ_0? A) conv1 B) pool5 C) fc7 D) softmax **Answer: C.** Loss $\|\phi(x) - \phi_0\|^2$ plus a natural-image prior, minimized over pixels.

Q13. (Single correct, numerical) In Deep Dream with $\mathcal{L}(I) = h_{ij}^2$, the current activation is $h_{ij} = 3$ and $\partial h_{ij}/\partial i_{mn} = 0.5$ for a pixel whose value is $i_{mn} = 10$. Using the slide's update (no learning rate), what is the new pixel value? A) 8.5 B) 11.5 C) 13 D) 7 **Answer: C.** $\partial \mathcal{L}/\partial i_{mn} = 2h_{ij}\,\partial h_{ij}/\partial i_{mn} = 2 \cdot 3 \cdot 0.5 = 3$. This is gradient **ascent**, so $i_{mn} = 10 + 3 = 13$.

Q14. (Single correct) In the slide's guided-backprop example, the upper-layer gradient at one position is $+3$, but the forward input to the ReLU at that position was -1. What gradient flows down in normal backprop and in guided backprop, respectively? A) 3, 3 B) 0, 3 C) 3, 0 D) 0, 0 **Answer: D.** Both methods block gradients at dead ReLUs (forward input < 0). Guided backprop additionally blocks negative upper gradients.

Lecture 13 · Week 11

Q1. (Single correct, numerical) An RNN has input dimension $n = 100$, state dimension $d = 50$ and $k = 10$ output classes, with biases b, c. How many parameters does it have when unrolled over $T = 20$ time steps? A) 8060 B) 161200 C) 8000 D) 7560 **Answer: A.** $nd + d^2 + dk + d + k = 5000 + 2500 + 500 + 50 + 10 = 8060$. The parameters are shared, so the count does not depend on T.

Q2. (Single correct) With $x_i \in \mathbb{R}^n$, $s_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}^k$, the recurrent matrix W has shape: A) $n \times d$ B) $d \times k$ C) $d \times d$ D) $n \times k$ **Answer: C.** W maps $s_{i-1} \in \mathbb{R}^d$ to $\mathbb{R}^d$.

Q3. (Multiple correct) Which are properties of sequence learning problems (as listed in the lecture)? A) Successive inputs may depend on each other B) The length of the input can vary C) The same task (function) is performed at each time step D) An output must be produced at every time step **Answer: A, B, C.** For sentiment classification, only the final output matters.

Q4. (Single correct) Why is feeding all previous inputs $x_1..x_t$ at each time step a bad way to model dependence? A) It needs more memory than an RNN, but is otherwise fine B) The function differs at each time step ($f_1, f_2, \ldots$) and the network becomes sensitive to sequence length C) It cannot use backpropagation D) It makes the gradients vanish **Answer: B.** It violates two items on the wishlist.

Q5. (Single correct, numerical) In a 4-step character RNN, the predicted probabilities of the true characters are 0.7, 0.7, 0.7 and 0.1. What is the total cross-entropy loss (natural log)? A) 2.66 B) 3.37 C) 1.07 D) 2.30 **Answer: B.** $-3\ln 0.7 - \ln 0.1 = 1.070 + 2.303 = 3.373$.

Q6. (Single correct) In BPTT, $\frac{\partial s_4}{\partial W} = \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial W}$. The term $\frac{\partial^+ s_4}{\partial W}$ is: A) the implicit derivative through s_3 B) the explicit derivative, treating all other inputs (e.g., s_3) as constant C) always zero D) the derivative of the loss w.r.t.

W **Answer: B.** The second term is the implicit part.

Q7. (Single correct) For $\frac{\partial \mathcal{L}_t}{\partial W} = \frac{\partial \mathcal{L}_t}{\partial s_t} \sum_{k=1}^{t} \frac{\partial s_t}{\partial s_k} \frac{\partial^+ s_k}{\partial W}$ with $s_k \in \mathbb{R}^d$, $\frac{\partial^+ s_k}{\partial W}$ is a: A) d-dimensional vector B) $d \times d$ matrix C) $d \times d \times d$ tensor D) scalar **Answer: C.** It is the derivative of a vector in $\mathbb{R}^d$ w.r.t. a matrix in $\mathbb{R}^{d \times d}$.

Q8. (Single correct, numerical) An RNN uses the logistic activation, and $\|W\| \leq \lambda = 2$. What is the upper bound on $\|\partial s_t / \partial s_k\|$ for $t - k = 10$? A) 2^{10} B) $0.5^{10} \approx 9.8 \times 10^{-4}$ C) 0.25^{10} D) 1 **Answer: B.** $\gamma = 1/4$, so $\gamma\lambda = 0.5$ and the bound is $(\gamma\lambda)^{t-k} = 0.5^{10}$. The gradient vanishes.

Q9. (Single correct, numerical) With tanh activation, what is the smallest value of λ (the bound on $\|W\|$) above which the slide's bound allows gradients to explode? A) 0.25 B) 1 C) 4 D) 2 **Answer: B.** For tanh, $\gamma = 1$, so explosion is possible when $\gamma\lambda > 1$, i.e., $\lambda > 1$. For the logistic it would be $\lambda > 4$.

Q10. (Multiple correct) Which statements are correct? A) $\frac{\partial s_j}{\partial s_{j-1}} = \mathrm{diag}(\sigma'(a_j))\, W$ B) If $\gamma\lambda > 1$ the gradient will certainly explode C) If $\gamma\lambda < 1$ the gradient will vanish D) Truncated BPTT restricts the product of Jacobians to $\tau\ (< t - k)$ terms **Answer: A, C, D.** B is wrong because the bound is only an upper bound, so the gradient *could* explode.

Q11. (Single correct) Which gradient does NOT need a sum over earlier time steps through the recurrent states? A) $\partial \mathcal{L} / \partial W$ B) $\partial \mathcal{L} / \partial U$ C) $\partial \mathcal{L} / \partial V$ D) $\partial \mathcal{L} / \partial b$ **Answer: C.** V affects $\mathcal{L}_t$ only through y_t: $\partial \mathcal{L} / \partial V = \sum_t \partial \mathcal{L}_t / \partial V$, which is ordinary output-layer backprop. W, U, b enter every state.

Q12. (Single correct, numerical) In $\frac{\partial \mathcal{L}_6}{\partial W}$, how many Jacobian factors $\partial s_j / \partial s_{j-1}$ are multiplied in the term for $k = 2$? A) 2 B) 3 C) 4 D) 6 **Answer: C.** $\partial s_6 / \partial s_2$ has $t - k = 4$ factors.

Q13. (Single correct, numerical) For a sequence of $T = 5$ steps with $\mathcal{L} = \sum_t \mathcal{L}_t$, how many $\frac{\partial s_t}{\partial s_k} \frac{\partial^+ s_k}{\partial W}$ terms does the full BPTT gradient $\partial \mathcal{L} / \partial W$ contain in total? A) 5 B) 10 C) 15 D) 25 **Answer: C.** $\partial \mathcal{L}_t / \partial W$ has t terms ($k = 1..t$), so the total is $\sum_{t=1}^{5} t = 15$.

Q14. (Single correct, numerical) In the second version of the slide example, the predicted probability of the true character is 0.7 at each of the 4 time steps. What is the total loss (natural log)? A) 0.357 B) 1.427 C) 2.80 D) 0.24 **Answer: B.** $-4 \ln 0.7 = 4(0.3567) = 1.427$.

Weeks 11–12 · Lectures 14–15

Each question is marked **[Single]** (one correct option) or **[Multiple]** (one or more correct options). Every numerical answer was verified in python.

Lecture 14: LSTM, GRU, vanishing and exploding gradients · Week 11

Q1. [Single] In the slides' "selective read/write/forget" view of the LSTM, which gate implements **selective write**? A) Input gate i_t B) Forget gate f_t C) Output gate o_t D) Candidate state $\tilde{s}_t$ **Answer: C.** $h_{t-1} = o_{t-1} \odot \sigma(s_{t-1})$: the output gate decides what fraction of the state is passed (written) to the next step. The input gate does selective read and the forget gate does selective forget.

Q2. [Multiple] Which statements about the GRU (as on the Lecture 14 slides) are correct? A) It has no explicit forget gate; the forget fraction is tied to the input gate as $1 - i_t$ B) Its gates are computed directly from s_{t-1} rather than from an intermediate h_{t-1} C) It maintains a separate output vector $h_t = o_t \odot \sigma(s_t)$, like the LSTM D) Its candidate state is $\tilde{s}_t = \sigma(W(o_t \odot s_{t-1}) + Ux_t + b)$ **Answer: A, B, D.** The GRU carries only s_t. There is no separate h_t.

Q3. [Single] An LSTM (slide equations, including biases) has state dimension $d = 10$ and input dimension $n = 5$. How many parameters are in the recurrent cell (excluding the output layer V)? A) 480 B) 600 C) 640 D) 160 **Answer: C.** There are 4 blocks (W, U, b) each of size $d^2 + dn + d = 100 + 50 + 10 = 160$, so $4 \times 160 = 640$. Option B forgets the biases, A is the GRU count, and D is the RNN count.

Q4. [Single] A GRU with $d = 3$, $n = 2$ (with biases). How many cell parameters? A) 45 B) 54 C) 72 D) 18 **Answer: B.** $3 \times (9 + 6 + 3) = 54$. Option A has no biases, C is the LSTM count, D is the RNN count.

Q5. [Single] For a 1-dimensional LSTM: $s_{t-1} = 2$, $f_t = 0.5$, $i_t = 0.25$, $\tilde{s}_t = 0.8$. What is s_t? A) 1.0 B) 1.2 C) 1.7 D) 2.2 **Answer: B.** $s_t = f_t s_{t-1} + i_t \tilde{s}_t = 1.0 + 0.2 = 1.2$.

Q6. [Single] For the same numbers but a GRU using $s_t = (1 - i_t) \odot s_{t-1} + i_t \odot \tilde{s}_t$ (Lecture 14 form) with $i_t = 0.25$, what is s_t? A) 1.1 B) 1.2 C) 1.7 D) 0.7 **Answer: C.** $0.75 \times 2 + 0.25 \times 0.8 = 1.7$. (The swapped form on Lecture 15 slide 8 would give 1.1.)

Q7. [Single] According to the lecture, the gradient of $\mathcal{L}_t(\theta)$ with respect to a parameter θ_i **vanishes** when: A) the gradient along at least one path from $\mathcal{L}_t$ to θ_i vanishes B) the gradient along each and every path from $\mathcal{L}_t$ to θ_i vanishes C) the forget gate at time t equals 1 D) the gradient along the longest path vanishes **Answer: B.** It vanishes only if all paths vanish. It explodes if at least one path explodes.

Q8. [Single] In the analysis of path t_0, $\partial h_t / \partial s_t$ equals: A) W_o B) $\mathcal{D}(f_t)$ C) $\mathcal{D}(o_t \odot \sigma'(s_t))$ D) a dense $d \times d$ matrix $\sigma'(s_t) o_t^T$ **Answer: C.** h_{ti} depends only on o_{ti} and s_{ti}, so the Jacobian is diagonal.

Q9. [Single] Along the cell-state path, the only factors between s_t and s_k (worst case, implicit term ignored) are the forget gates. If every $f_i = 0.9$ and $t - k = 10$, by what factor is the gradient along this path scaled? A) ≈ 0.035 B) ≈ 0.349 C) $\approx 9.5 \times 10^{-7}$ D) 0.9 **Answer: B.** $\mathcal{D}(\bigodot_{i=k+1}^{t} f_i)$ gives $0.9^{10} \approx 0.349$. Option C is 0.25^{10}, the vanilla-RNN σ' worst case.

Q10. [Multiple] Which statements are correct? A) LSTMs solve both the vanishing and the exploding gradient problems B) In an LSTM, the gradient can explode along the path through the output gates because W_o is multiplied repeatedly C) Gradient clipping rescales the gradient's norm while retaining its direction D) In an LSTM, a vanishing gradient into s_{t-1} when $f_t \to 0$ is considered acceptable **Answer: B, C, D.** $\|t_1\| \leq \|\mathcal{L}'_t\| (\|K\| \|W_o\|)^{t-k+1}$, so the gradient can explode. Clipping (Pascanu et al. 2013) is the fix. Vanishing when s_{t-1} did not contribute is "fair".

Q11. [Single] A gradient $g = [6, 8]$ is clipped with norm threshold $\tau = 5$. What is the clipped gradient? A) $[5, 5]$ B) $[3, 4]$ C) $[0.6, 0.8]$ D) $[6, 8]$ **Answer: B.** $\|g\| = 10 > 5$, so $g \leftarrow 5 \cdot g / 10 = [3, 4]$. Option A clips element-wise, which changes the direction.

Q12. [Single] In the whiteboard example ($a = 1, b = 3, c = 5, d = 11$, compute $ac(bd + a) + ad$ with a board holding 3 statements), which is the final value, and which statement is erased first? A) 181; bd B) 181; ac C) 170; $bd + a$ D) 176; bd **Answer: A.** $5 \cdot 34 + 11 = 181$. Once $bd + a = 34$ is written, bd is obsolete and is the first item forgotten (selective forget).

Lecture 15: Encoder–decoder models and attention · Week 12

Q13. [Single] In the image-captioning encoder–decoder model of the lecture, the encoder and decoder are respectively: A) RNN, CNN B) CNN, RNN C) CNN, CNN D) RNN, softmax **Answer: B.** $s_0 = \mathrm{CNN}(x_i)$ (fc7), and an RNN generates the caption.

Q14. [Multiple] For which tasks in the lecture is the "decoder" just a single softmax layer, so the loss is a single $-\log P(y = \ell \mid \cdot)$ term? A) Image question answering B) Video classification C) Video captioning D)

Transliteration **Answer: A, B.** Both predict one label. C and D generate sequences with an RNN decoder and use a sum over time steps.

Q15. [Single] Attention scores for 3 input words at decoder step t are $e = [\ln 1, \ln 2, \ln 1]$ and the encoder states are $h_1 = [2, 0]$, $h_2 = [4, 2]$, $h_3 = [0, 6]$. What is c_t? A) $[2, 2.67]$ B) $[2.5, 2.5]$ C) $[4, 2]$ D) $[6, 8]$ **Answer: B.** $\alpha = [1, 2, 1]/4 = [0.25, 0.5, 0.25]$, so $c_t = 0.25[2, 0] + 0.5[4, 2] + 0.25[0, 6] = [2.5, 2.5]$. Option A is the unweighted mean and D is the unweighted sum.

Q16. [Single] Scores $e = [2, 1, 0, 1]$ over 4 input positions. What is α_{1t} (to 3 decimals)? A) 0.400 B) 0.500 C) 0.534 D) 0.731 **Answer: C.** $e^2/(e^2 + 2e + 1) = 7.389/13.825 = 0.534$.

Q17. [Single] Two sets of attention scores, $e = [1, 2, 3]$ and $e' = [11, 12, 13]$, are given for the same decoder step. The resulting attention weights are: A) identical, $\approx [0.090, 0.245, 0.665]$ B) e' gives a much sharper distribution C) e' gives a uniform distribution D) cannot be compared **Answer: A.** The softmax is invariant to adding the same constant to all scores.

Q18. [Single] The output of VGGNet's 5th convolutional layer, $14 \times 14 \times 512$, is used for attention over images. The number of locations attended over and the dimension of each location's representation are: A) 512 locations, 196-d B) 196 locations, 512-d C) 100352 locations, 1-d D) 49 locations, 512-d **Answer: B.** Locations $= 14 \times 14 = 196$, each 512-d. (D would apply to the $7 \times 7 \times 512$ map after the next max-pool.)

Q19. [Single] With $e_{jt} = V_{att}^T \tanh(U_{att} s_{t-1} + W_{att} c_j)$ and $V_{att} \in \mathbb{R}^d$, $U_{att}, W_{att} \in \mathbb{R}^{d \times d}$, how many attention parameters are there for $d = 4$ and an input sentence of 10 words? A) 36 B) 32 C) 360 D) 44 **Answer: A.** $d + 2d^2 = 4 + 32 = 36$. The count is independent of the sentence length.

Q20. [Single] A decoder assigns probabilities 0.5, 0.25 and 0.8 to the true words at steps 1, 2 and 3. What is the total loss $\mathcal{L}(\theta) = -\sum_t \log P(y_t = \ell_t \mid \cdot)$ (natural log)? A) 0.693 B) 1.550 C) 2.303 D) 0.100 **Answer: C.** $-\ln(0.5 \cdot 0.25 \cdot 0.8) = -\ln 0.1 = \ln 10 \approx 2.303$.

Q21. [Multiple] Which statements about the attention mechanism are correct? A) For each output step t, $\sum_j \alpha_{jt} = 1$ B) The model can only be trained if true alignments α^{true} are annotated C) Unlike vanilla "Option 2" decoders, the information fed to the decoder differs at each time step D) The attention parameters are learned jointly with the encoder and decoder parameters **Answer: A, C, D.** True α's are hard to obtain and are not needed. Attention is learned end-to-end because it is a better modelling choice.

Q22. [Single] Why does attention over images use a convolutional feature map instead of the fc7 representation? A) fc7 is too high-dimensional B) fc7 contains no location (spatial) information C) Conv features are not learned D) fc7 cannot be fed to an RNN **Answer: B.** Attention needs a representation per location, and the conv maps keep spatial layout.

Q23. [Single] In the language-modelling RNN of the lecture, s_0 is: A) computed from the `<GO>` token B) always the zero vector C) randomly initialised and learned along with the other parameters D) the fc7 feature of the input **Answer: C.** This is stated on the slide. (D is captioning Option 1, not language modelling.)

Q24. [Multiple] In the Hierarchical Attention Network (Yang et al.) for document classification: A) The sentence representation is $s_i = \sum_j \alpha_{ij} h_{ij}$ B) $\alpha_{ij} = \exp(u_{ij}^T u_w) / \sum_t \exp(u_{it}^T u_w)$, where u_w is a learned parameter vector C) The document representation is the last sentence-level state h_K D) Attention is applied at both the word level and the sentence level **Answer: A, B, D.** With attention, $s = \sum_i \alpha_i h_i$, not h_K. Using h_K^2 is the hierarchical encoder **without** attention.

Q25. [Single] In the exploding-gradient analysis of the LSTM, $\|t_1\| \le \|\mathcal{L}_t'(h_t)\| (\|K\| \|W_o\|)^{t-k+1}$. Take $\|\mathcal{L}_t'(h_t)\| = 1$, $\|K\| = 0.5$, $\|W_o\| = 3$, $t = 10$, $k = 1$. What is the bound? A) ≈ 38.4 B) ≈ 57.7 C) 59049 D) $\approx$

9.8×10^{-4} **Answer: B.** $\|K\|\|W_o\| = 1.5$ and there are $t - k + 1 = 10$ factors, so $1.5^{10} \approx 57.7$. A uses exponent $t - k = 9$; C is 3^{10} (ignores K); D is 0.5^{10} (ignores W_o).

Q26. [Multiple] About the attention heat maps shown in the lecture (summarization, English→French MT, video captioning): A) Each cell is α_{tj}, the importance of input word j for predicting output word t, as determined by the model B) The heat map shows a soft alignment between the input and the generated output C) The alignments in the heat map were annotated by humans and used as supervision D) In the MT example, "European Economic Area" aligns to "zone économique européenne" in reversed order **Answer: A, B, D.** The weights are learned without alignment supervision (C is false); the heat map is a way to check that the model learned something meaningful.

Q27. [Single] In the hierarchical encoder **without** attention for document classification, the sentence representation s_i and document representation s are: A) $s_i = \sum_j \alpha_{ij} h_{ij}$, $s = \sum_i \alpha_i h_i$ B) $s_i = h^1_{iT_i}$, $s = h^2_K$ C) $s_i = h^1_{i1}$, $s = h^2_1$ D) $s_i = \mathrm{softmax}(V h^1_{iT_i})$, $s = h^2_K$ **Answer: B.** The last word-level state of sentence i and the last sentence-level state. Option A is the HAN (with attention).

Q28. [Single] Lecture 15 slide 8 writes the GRU as $s_t = i_t \odot s_{t-1} + (1 - i_t) \odot \tilde{s}_t$. With $s_{t-1} = 2$, $\tilde{s}_t = 0.8$, $i_t = 0.25$ (scalar), s_t is: A) 1.1 B) 1.7 C) 1.2 D) 0.7 **Answer: A.** $0.25 \cdot 2 + 0.75 \cdot 0.8 = 1.1$. The Lecture 14 slide-26 form $(1 - i_t)s_{t-1} + i_t \tilde{s}_t$ gives 1.7 (B). Always check which convention the question states.

Appendix · Slide Errors and Ambiguities

These are discussed in detail in the relevant notes. If an exam question uses the slide's form, follow the question.

Lecture 2

- One bullet says $(w_1, w_2) = (-1, -1)$ gives 1 error, but the table on the same slide shows 3.
- The XOR network table uses 0/1 inputs, while the construction uses ±1.
- *(Found in the audit)* The MP-vs-perceptron comparison slide writes the MP sum starting at $i = 0$.
- *(Found in the audit)* The perceptron is described as discontinuous "at w_0", but its plot puts the jump at $-w_0$.

Lecture 3

- The ½ in the squared-error loss appears on some slides and not others.

Lecture 4

- The slide gives $W_L \in \mathbb{R}^{n \times k}$, but $W_L \mathbf{h}_{L-1}$ requires $k \times n$. The parameter count is unaffected.
- *(Found in the audit)* Slide 36 (p. 163) prints the softmax denominator as $\sum_{i'} \exp(a_L)_\ell$; it should be $\sum_{i'} \exp(a_L)_{i'}$.

Lecture 5

- Exponential decay is printed as η_0^{-kt}; the standard form is $\eta_0 e^{-kt}$.
- One term of the unrolled momentum sum is mis-indexed.
- The slides use $\sqrt{v + \epsilon}$.

Lecture 6

- "λ are the eigenvectors" should read eigenvalues.
- The covariance is written $X^T X$ without the $1/m$.
- Index typos in the PCA reconstruction-error derivation.
- *(Found in the audit)* The best rank-k slide omits the constraint $\mathrm{rank}(B) \leq k$. Without it, $B = A$ is the trivial solution.

Lecture 7

- The L2 gradient is written as λW; it is strictly $2\lambda W$.
- The matrix form of the sparse-AE gradient drops the $\frac{1}{m} \sum_i$.

Lecture 8

- The sign of the "small constant" in the true-error result is wrong.
- The label-smoothing objective is missing a minus sign.
- The early-stopping slide uses ε for η.

Lecture 9

- ELU is written $ae^x - 1$; the standard form is $a(e^x - 1)$.

- The slides omit the batch-norm ε.

Lecture 10

- The negative-sampling distribution is normalised by N.
- The sign and form of the contrastive-estimation hinge loss are wrong.
- Transposes are missing in the shape statements.
- *(Found in the audit)* In the word2vec ↔ PMI formula, M_{ij} uses c_i where it should be c_j.
- *(Found in the audit)* The synonym-detection candidate "unposed" should be "imposed".

Lecture 11

- AlexNet: the slide total of 27.55M comes from summing rounded values; the exact count is ≈28.81M.
- ZF-Net: the parameter differences for layers 6–7 are wrong.
- VGG: 4096×1024 should be 4096×1000.
- LeNet: pooling is labelled S=1 but must be S=2.
- *(Found in the audit)* ZF-Net layer 1 with $F = 7$, $S = 4$, $P = 0$ on a 227 input gives 56, not the 55 drawn.

Lecture 12

- Class-score maximization is written with $-\eta$ (descent) where ascent is meant.
- The Gram-matrix dimensions are wrong ($V^T V$ is written for $V V^T$).
- *(Found in the audit)* The embedding-inversion update (p. 104) is missing the ∂ in the numerator.

Lecture 13

- U and V are shown transposed.
- $a_j = W s_j + b$ should be $W s_{j-1}$, with the Ux term missing.

Lecture 14

- The slides use σ instead of tanh for $\tilde{s}_t$ and inside h_t.

Lecture 15

- The GRU convex combination is swapped relative to L14 (L15 slide 8 vs L14 slide 26).
- The attention score is written with s_{t-1} on one slide and s_t on another.